

Основы STM32

Изучаем FreeRTOS, libopencm3 и GCC

Уоррен Гей

ДМК Пресс
издательство

Уоррен Гей

Основы STM32

Beginning STM32

Developing with FreeRTOS, libopencm3, and GCC



Warren Gay

Apress®

Основы STM32

Изучаем FreeRTOS, libopencm3 и GCC



Уоррен Гей

ДМК Пресс
издательство

УДК 004.3’144:621.316.544.1STM32
ББК 32.971.32-04
Г29

Гей У.
Г29 Основы STM32 / пер. с англ. Ю. В. Ревича. – М.: ДМК Пресс, 2024. – 362 с.: ил.

ISBN 978-5-93700-294-5

В книге детально рассмотрены аппаратные возможности микроконтроллера STM32, включая порты GPIO и другие периферийные устройства, такие как USB и контроллеры шины CAN. Приведена настройка программного обеспечения для Windows. Используя FreeRTOS и библиотеку libopencm3 вместо программной среды Arduino, вы научитесь разрабатывать многозадачные приложения, выходящие за рамки возможностей Arduino.

Издание адресовано инженерам встраиваемых систем, разработчикам, желающим изучить архитектуру ARM, а также будет полезно студентам и радиолюбителям.

УДК 004.3’144:621.316.544.1STM32
ББК 32.971.32-04

First published in English under the title Beginning STM32: Developing with FreeRTOS, libopencm3, and GCC, by Warren Gay, edition: 2.

This edition has been translated and published under licence from APress Media, LLC, part of Springer Nature. APress Media, LLC, part of Springer Nature takes no responsibility and shall not be made liable for the accuracy of the translation.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 979-8-8688-0198-3 (англ.)
ISBN 978-5-93700-294-5 (рус.)

Copyright © 2024 by Warren Gay
© Перевод, оформление, издание, ДМК Пресс, 2024

Посвящается
моей жене Жаклин и моим детям

Содержание

От издательства 15
Об авторе 16

Глава 1 • Введение 17
Текстовые соглашения 18
STM32F103C8T6 18
FreeRTOS 20
libopencm3 21
Никаких Ардуино! 21
Никаких IDE 22
Фреймворк разработчика 23
Для кого написана эта книга? 23
Что вам потребуется? 23
Программатор ST-Link V2 24
Макетная плата 25
Провода DuPont (перемычки) 25
Развязывающие конденсаторы 0.1 мкФ 27
Адаптер последовательного порта USB-TTL 28
Источник питания 29
Мелочи 30
Итоги 31

Глава 2 • Настройка программного обеспечения 32
Используемые соглашения о каталогах 32
Операционное ПО 32
ПО для книги 33
Клонирование с помощью учетной записи github 33
Анонимная загрузка с github 33
libopencm3 33
Ядро FreeRTOS 34
Кросс-компилятор ARM 34
Создание программного обеспечения 37
Инструмент ST-Link 37
Установки для пользователей Windows 39
Учетная запись и пароль Linux 39
Обновление WSL 40
Запуск Ubuntu 40
Обновление Ubuntu 40
Установка зависимостей 41
Установка Windows usbipd 41
Утилита STM32 ST-LINK 43
Итоги 45

Глава 3 • Включаем питание и мигаем светодиодом....46
Питание....46
Стабилизатор +3.3 В....47
USB-питание и вывод +5 В....48
Питание +3.3 В....48
Правило одного источника питания....49
Общий провод....50
Сброс (RESET)....50
Быстрая проверка....50
ST-Link V2....51
Утилита st-flash....53
Чтение STM32....53
Запись flash....54
Стирание flash....55
Утилита Windows STM32 ST-LINK....55
Функция Info....55
Функция чтения Read....56
Функция записи Write....57
Функция стирания Erase....58
Итоги....58
Дополнительные источники....58

Глава 4 • GPIO....59
Сборка miniblink....59
Прошивка miniblink....60
Исходный код miniblink.c....60
API-интерфейс GPIO....63
Конфигурация GPIO....64
Входные режимы портов....66
Выходные режимы портов....67
Двухтактный выход....69
Выход с открытым стоком....69
GPIO_MODE_OUTPUT_*_MHZ....70
Выстраиваем уток в ряд....70
Входы GPIO....71
Цифровой выход, двухтактный....71
Цифровой выход, открытый сток....71
Характеристики GPIO....72
Пороги входного напряжения....73
Пороги выходного напряжения....73
Программные задержки....74
Проблемы с программной задержкой....74
Итоги....75
Упражнения....75

Глава 5 • FreeRTOS....77
Возможности FreeRTOS....77

Многозадачность 77
Очереди сообщений 78
Семафоры и мьютексы 79
Таймеры 79
Группы событий 80
Программа blinky2 80
Сборка и тестирование blinky2 84
Выполнение 84
Файл FreeRTOSConfig.h 84
Соглашение об именах FreeRTOS 86
Макроопределения FreeRTOS 87
Итоги 87
Упражнения 88

Глава 6 • USART 89
Периферийное устройство USART/UART 89
Асинхронные данные 89
USB-адаптеры последовательного порта 90
Подключение 91
Проект uart 93
Проект 96
Проект uart2 99
USART API 103
Включаемые файлы 105
Тактовые сигналы 105
Конфигурация 105
DMA 105
Прерывания 105
Статус ввод/вывод 106
Последовательность настроек 106
FreeRTOS 106
Задачи 106
Очереди 107
Итоги 108
Упражнения 108

Глава 7 • Последовательный порт USB 109
Проблема с USB на плате Blue Pill 109
Введение в USB 111
Каналы и конечные точки 111
Периферийное устройство последовательного порта USB 112
Последовательное USB-устройство в Linux 112
Последовательное USB-устройство в macOS 113
Последовательное USB-устройство в Windows 113
USB-выводы GPIO 114
Исходный демокод 114
Функция cdcacm_set_config() 115

Функция cdc_control_request() ... 116
Функция cdcacm_data_rx_cb() ... 117
Задача USB task ... 118
USB-прием ... 119
USB-отправка ... 119
Демопрограмма последовательного порта USB ... 120
Итоги ... 122
Дополнительные источники ... 122
Упражнения ... 123

Глава 8 • Флеш-память с интерфейсом SPI ... 124
Представление W25QXX ... 124
Шина последовательного периферийного интерфейса SPI ... 124
Сигнал chip select ... 126
Соединения и напряжение ... 126
Схема подключения SPI-флеш-памяти ... 126
Управление выводом NSS ... 127
Конфигурация STM32 SPI ... 129
Тактовая частота SPI ... 131
Режимы синхронизации SPI ... 132
Порядок битов и длина слова ... 134
Ввод-вывод через SPI ... 135
Чтение регистра SR1 ... 135
Ожидание готовности ... 136
Чтение ID производителя ... 136
Запись флеш-памяти ... 137
Стирание флеш-памяти ... 139
Чтение флеш-памяти ... 141
Демопрограмма ... 142
Запуск демоверсии ... 143
ID производителя ... 147
Режим Power Down ... 147
Итоги ... 147
Дополнительные источники ... 147
Упражнения ... 148

Глава 9 • Оверлеи ... 149
Проблема компоновки ... 149
Раздел MEMORY ... 150
ENTRY и EXTERN ... 152
Раздел SECTIONS ... 152
PROVIDE ... 155
Перемещение данных ... 155
Определение оверлея ... 156
Код оверлея ... 157
Заглушки оверлея ... 158
Диспетчер оверлеев ... 159
VMA и адреса загрузки ... 159

Использование в коде имен, созданных компоновщиком 161
Функция диспетчера оверлеев 162
Функция-заглушка 163
Демопрограмма 163
Извлечение оверлеев 164
Демопрограмма (продолжение) 168
Ловушка при изменении кода 170
Итоги 170
Дополнительные источники 171
Упражнения 171

Глава 10 • Часы реального времени (RTC) 172
Демонстрационные проекты 172
RTC с одним прерыванием 172
Конфигурация RTC 173
Прерывание и настройка 174
Процедура обработки прерываний 175
Уведомление о задаче 177
Мьютексы 179
Демопрограмма 179
Соединения UART1 182
Запуск демопрограммы 182
Прерывание rtc_alarm_isr() 184
EXTI-контроллер 185
Итоги 186
Упражнения 187

Глава 11 • Интерфейс I2C 188
Шина I2C 188
Ведущий и ведомый 188
Старт и стоп 189
Биты данных 189
I2C-адрес 190
I2C-транзакции 191
PCF8574: расширитель GPIO 192
Подключение через I2C 193
Линия $\overline{\text{INT}}$ микросхемы PCF8574 194
Конфигурация PCF8574 195
Подключение нагрузки к GPIO микросхемы PCF8574 196
Формирование фронтов 197
Демосхема 197
Программное обеспечение I2C 200
Проверка готовности I2C 201
Запуск I2C 202
Запись I2C 203
Чтение I2C 204
Перезапуск I2C 204
Демопрограмма 205

Демосессия....207
Итоги....209
Упражнения....209

Глава 12 • Дисплеи OLED....210
OLED-дисплей....210
Конфигурация....211
Подключение дисплея....212
Свойства дисплея....213
Схема демонстрационного примера....214
AFIO....214
Графика....216
Растровое изображение....217
Запись растра....219
Программное обеспечение изображения вольтметра....219
Главный модуль....221
Демопрограмма....223
Итоги....224
Упражнения....224

Глава 13 • OLED с использованием DMA....225
Проблемы....225
Схема....225
Операция прямого доступа к памяти....225
Выполнение прямого доступа к памяти....226
Сигналы запроса DMA....226
Демопрограмма....228
Инициализация DMA....230
Запуск прямого доступа к памяти....230
Задача управления OLED SPI/DMA....231
Процедура DMA ISR....234
Перезапуск передачи DMA....235
Выполнение демопримера....236
Дальнейшие проблемы....237
Итоги....237
Упражнения....238

Глава 14 • Аналого-цифровое преобразование....239
Ресурсы STM32F103C8T6....239
Демопрограмма....240
Аналоговые входы PA0 и PA1....240
Конфигурация периферийного устройства АЦП....240
Запуск демопрограммы....243
Чтение АЦП....243
Аналоговые напряжения....246
Итоги....248
Упражнения....248
Дополнительные источники....248

Глава 15 • Дерево тактовых сигналов 249
Основы 249
RC-генераторы 250
Кварцевые генераторы 250
Питание генераторов 250
Часы реального времени (RTCCLK) 252
Сторожевой таймер 252
Системный источник (SYSCLK) 252
SYSCLK и USB 254
Шина AHB 254
rcc_clock_setup_in_hse_8mhz_out_72mhz() 255
Периферийные устройства APB1 258
Периферийные устройства APB2 259
Таймеры 259
Функция rcc_set_mco() 259
Демопример HSI 260
Демопример HSE 261
Демонстрация частоты ФАПЧ/2 262
Итоги 262
Дополнительные источники 263
Упражнения 263

Глава 16 • Режим PWM таймера 2 264
PWM-сигналы 264
Таймер 2 265
PWM-цикл 268
Вычисление предварительного делителя таймера 269
Цикл 30 Гц 269
Подключение сервопривода 270
Запуск демопрограммы 271
PWM на PB3 271
Другие таймеры 272
Больше PWM-каналов 273
Итоги 273
Дополнительные источники 274
Упражнения 274

Глава 17 • PWM-вход с таймером 4 275
Сервосигнал 275
Напряжение сигнала 275
Демопроект 276
Конфигурация GPIO 276
Конфигурация таймера 4 276
Цикл task1 278
Процедура обработки прерывания ISR 278
Запуск демопрограммы 279
Вывод результатов сессии 281
Входы таймера 281

Итоги 283
Упражнения 283

Глава 18 • Шина CAN 284
Шина CAN 284
Дифференциальные сигналы 286
Доминантный/Рецессивный 287
Шинный арбитраж 288
Синхронизация 289
Формат сообщения 289
Ограничение STM32 290
Демопрограмма 290
Сборка программы 291
UART-интерфейс 291
Прошивка микроконтроллеров 292
Шина для демонстрации 292
Запуск демопрограммы 293
CAN-сообщения 296
Синхронизация 296
Итоги 296
Дополнительные источники 297

Глава 19 • Программное обеспечение CAN-шины 298
Инициализация 298
Функция can_init() 300
Приемные фильтры CAN 301
Прерывания приема CAN 302
Прием в приложении 305
Отправка CAN-сообщений 307
Итоги 308
Упражнения 308

Глава 20 • Прерывания UART 309
Основы 309
Подпрограммы обработки прерываний (ISR) 309
Демопрограмма 310
Разработка программы 310
Инициализация тактовых сигналов 312
Инициализация GPIO 312
Инициализация UART 312
Процедура обработки прерываний 313
Функция uart_getc() 314
Задача main_task() 315
Функция uart_putc() 315
Задача tx_task() 316
Запуск демопрограммы 317
Конфигурация 317
Запуск демопрограммы 318

Потеря данных 319
Итоги 320
Дополнительные источники 320
Упражнения 320

Глава 21 • Новые проекты 321
Создание проекта 321
Makefile 322
Дополнительные Makefile 324
Заголовки зависимостей 324
Параметры компиляции 324
Прошивка 128к 325
FreeRTOS 325
Модуль rtos/opencm3.c 325
Модуль rtos/heap_4.c 326
Необходимые модули 327
FreeRTOSConfig.h 327
Пользовательские библиотеки 329
Ошибки новичков 329
Итоги 330
Дополнительные источники 330
Упражнения 330

Глава 22 • Поиск неисправностей 331
Отладчик GNU (GDB) 331
GDB-сервер 331
Удаленный GDB 332
Текстовый пользовательский интерфейс GDB 335
Проблемы с выводами GPIO периферии 336
Альтернативная функция не работает 337
Сбой периферии 338
Сбой прерывания во FreeRTOS 338
Переполнение стека 338
Оценка размера стека 339
Когда отладчик не помогает 340
Push/Pull (двухтактный выход) или открытый сток 340
Дефекты периферии 341
Ресурсы 341
Библиотека libopencm3 342
Приоритеты задач FreeRTOS 343
Планирование в libopencm3 344
Итоги 345

Приложение А • Ответы на упражнения 346

Приложение Б • Выводы GPIO STM32F103C8T6 356

Предметный указатель 359

От издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг, мы будем очень благодарны, если вы сообщите о ней главному редактору по адресу dmkpress@gmail.com. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Об авторе

Уоррен Гей (Warren Gay) с детства увлекался электроникой и после школы часто таскал домой выброшенные телевизоры. В старшей школе он научился программировать IBM 1130, а затем продолжил карьеру в области разработки программного обеспечения в Политехническом институте Райерсона в Торонто. До выхода на пенсию он работал более 40 лет профессионально, в основном в области C/C++, Unix и Linux. Однако его любовь к электронике не угасала с момента создания самодельной системы Intel 8008 в 1970-х годах и до современных проектов на основе Raspberry Pi. Уоррен также имеет лицензию опытного радиолюбителя и в 1991 году получил возможность работать с космической станцией «Мир» (U2MIR) с использованием пакетной радиосвязи. Он является автором многих книг, в том числе «Sams Teach Yourself Linux in 24 Hours», «Linux Socket Programming by Example» и «Advanced Unix Programming».

Глава 1 • Введение

Сегодня существует значительный интерес к платформе ARM Cortex, поскольку устройства на основе технологий ARM можно встретить повсюду. Модули, содержащие ARM-основу, варьируются от небольших встроенных систем с микроконтроллерами до мобильных телефонов и крупных серверов под управлением Linux. Вскоре технологии ARM в больших количествах также будут присутствовать в центрах обработки данных. Все это веские причины ознакомиться с платформой ARM.

Даже несмотря на перспективы появления стандарта RISC-V, существуют причины, по которым платформа ARM будет продолжать оставаться важной. RISC-V – новая технология, многие расширения которой все еще находятся в разработке. Следовательно, производители будут пока стараться придерживаться курса ARM просто потому, что эти технологии проверены и известны затраты.

С учетом того что технологии варьируются от микроконтроллеров до полноценных серверов, естественно, возникает вопрос: «Зачем изучать программирование встроенных устройств? Почему бы не сосредоточиться на системах конечных пользователей под управлением Linux, таких как Raspberry Pi?»

Простой ответ заключается в том, что встроенные системы хорошо работают в сценариях, неудобных для более крупных систем. Они часто используются для взаимодействия с физическим миром, например для обмена данными между окружающей средой и настольной системой. В обычной клавиатуре используется специальный микроконтроллер (МК), который сканирует состояние клавиш и сообщает о событиях нажатия в настольную систему. Это не только уменьшает количество необходимых проводов, но и освобождает главный процессор от необходимости тратить свои высокопроизводительные вычисления на простую задачу отслеживания событий нажатия клавиш.

Другие приложения включают в себя встроенные системы в заводских цехах для систем контроля температуры, безопасности и обнаружения пожара. Нет смысла использовать полноценную настольную систему для такого типа приложений. Автономные встроенные системы экономят деньги и готовы к работе мгновенно. Наконец, небольшой размер МК делает его безальтернативным выбором для летающих дронов, где вес является решающим фактором.

Разработка встроенных систем традиционно требовала специалистов двух дисциплин:

- инженера по аппаратному обеспечению;
- разработчика программного обеспечения.

Часто одному человеку поручают разработать конечный продукт. Инженеры по аппаратному обеспечению специализируются на разработке электронных схем, но в конечном итоге для продукта требуется программное обеспечение. Это может быть проблемой, поскольку специалистам по программированию обычно не хватает знаний в области электроники, а инже-

нерам часто не хватает опыта работы с программным обеспечением. Из-за сокращения бюджета и сроков поставки инженер-электронщик часто становится одновременно и инженером-программистом.

Нет ничего плохого в том, чтобы один человек выполнял оба аспекта проектирования, если имеются необходимые навыки. Независимо от того, являетесь ли вы инженером-электронщиком, разработчиком программного обеспечения, любителем или производителем, нет ничего лучше настоящей реальной практики, которая поможет вам начать работу. Вот о чем эта книга.

Текстовые соглашения

В этой книге используется ряд текстовых соглашений.

`Код в тексте`: указывает служебные слова, имена переменных в тексте, имена функций, названия программных библиотек. Пример: «Введите `exit`, чтобы выйти из сеанса».

Блок кода или команд задается следующим образом:

```
$ cd build/Release
$ sudo make install
```

Имена файлов, имена папок, расширения файлов, пути файловой системы выделяются курсивом. Пример: «В результате у вас должен остаться загруженный файл с именем *master.zip*». Также выделяются курсивом названия разделов на страницах сайтов, например: «Вы можете найти ссылку на вкладке *Source Code/Downloads*».

Названия в меню или диалоговых окнах выделены жирным шрифтом с русским переводом в скобках. Например: «Нажмите кнопку **Start** (Пуск)».

Названия клавиш на клавиатуре и их сочетаний также выделяются жирным шрифтом и заключаются в угловые скобки: <**Ctrl**+**X**>.



Так обозначаются советы.



Так обозначаются примечания общего характера.



Так обозначаются предупреждения и предостережения.

STM32F103C8T6

Для этой книги выбрано устройство STMicroelectronics STM32F103C8T6. Такое обозначение компонента кажется очень сложным, поэтому давайте разберем его:

- STM32 (платформа STMicroelectronics);
- F1 (семейство устройств);
- 03 (подраздел семейства устройств);
- C8T6 (исполнение, указывающее на объем SRAM, флеш-памяти и т. д.).

Как следует из названия платформы, эти устройства основаны на 32-битных вычислениях, т. е. значительно более мощные, чем 8-битные устройства.

F103 (F1+03) – одна из ветвей платформы STM32. Название определяет возможности центрального процессора (ЦП) и периферийных устройств.

Наконец, суффикс C8T6 дополнительно определяет возможности устройства, такие как объем памяти и тактовая частота.

Устройство STM32F103C8T6 было выбрано для этой книги по следующим причинам:

- низкая стоимость (на eBay можно приобрести всего за 2.4 долл. США),
- доступность (eBay, Amazon, AliExpress и т. д.),
- расширенные возможности,
- малогабаритный форм-фактор.

STM32F103C8T6 в течение ближайшего времени, вероятно, останется самым дешевым пособием для изучения платформы ARM Cortex-M3 как для студентов, так и для любителей. Устройство легкодоступно и достаточно функционально. Наконец, форм-фактор небольшой печатной платы позволяет припаивать разъемы по краям и подключать устройство к макетной плате – наиболее удобный способ проведения широкого спектра экспериментов.

Микроконтроллер на синей печатной плате (рис. 1.1) ласково называют Blue Pill («синяя таблетка»), по мотивам фильма «Матрица». Есть несколько старых печатных плат красного цвета, которые назывались Red Pill («красная таблетка»). Есть и другие, черные, известные как Black Pill («черная таблетка»). В этой книге я буду предполагать, что у вас есть модель синего цвета, как на рисунке. Помимо некоторых особенностей USB, между ней и другими моделями не должно быть особых различий.

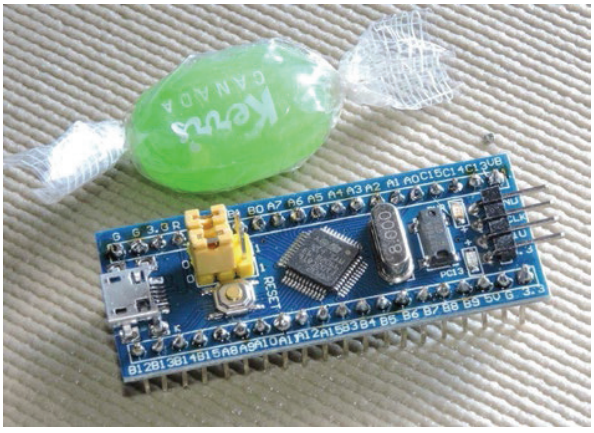


Рис. 1.1. Печатная плата STM32F103C8T6 с припаянными контактными разъемами (Blue Pill)

Низкая стоимость имеет еще одно преимущество – она позволяет вам иметь несколько устройств, например для проектов, связанных с передачей

данных по CAN. В этой книге рассматривается связь CAN с использованием трех устройств, соединенных общей шиной. Низкая стоимость означает, что вы останетесь в рамках студенческого бюджета.

Поддержка периферии STM32F103 просто удивительная, если учесть его цену. Периферийные устройства в комплекте включают:

- четыре 16-битных порта входов/выходов общего назначения GPIO (General Purpose Input/Output), большинство из них устойчивы к напряжению 5 В,
- три порта USART (Universal Synchronous/Asynchronous Receiver/Transmitter – универсальный синхронный/асинхронный приемник/передатчик),
- два интерфейса I2C,
- два интерфейса SPI,
- два аналого-цифрового преобразователя (АЦП),
- два контроллера прямого доступа к памяти (DMA),
- четыре таймера,
- сторожевой таймер,
- USB-контроллер,
- CAN-контроллер,
- генератор контрольной суммы CRC (Cyclic Redundancy Check, контрольный избыточный код),
- 20 Кбайт статической оперативной памяти,
- флеш-память 64 (или 128) Кбайт,
- процессор ARM Cortex M3, максимальная тактовая частота 72 МГц.

Однако есть некоторые ограничения. Например, контроллеры USB и CAN не могут работать одновременно. Другие периферийные устройства могут конфликтовать за использование контактов ввода-вывода. Большинство конфликтов контактов разрешаются с помощью конфигурации AFIO (Alternate Function Input Output, альтернативная функция ввода-вывода), позволяющей использовать разные контакты для функций периферийного устройства.

В конфигурации периферийных устройств можно индивидуально включать различные источники тактовых импульсов для настройки энергопотребления. Расширенные возможности этого микроконтроллера делают его удобным для изучения. То, что вы узнаете о семействе STM32F103, можно будет использовать в более продвинутых версиях, таких как STM32F407.

Официально указан объем флеш-памяти 64 Кбайта, но вы можете обнаружить, что устройство поддерживает 128 Кбайт. Эта особенность описана в главе 2 и позволяет размещать на устройстве приложения большого размера.

FreeRTOS

В отличие от популярного семейства чипов AVR (ныне принадлежащего Microchip[1]), семейство STM32F103 имеет достаточный объем статической

[1] В 2016 году официально было объявлено о приобретении создателя семейства AVR фирмы Atmel компанией Microchip. – *Здесь и далее в сносках примечания переводчика.*

оперативной памяти SRAM для комфортной работы операционной системы реального времени FreeRTOS (http://freertos.org). Доступ к RTOS обеспечивает ряд преимуществ, в том числе:

- вытесняющую многозадачность;
- очереди;
- мьютексы и семафоры;
- программные таймеры.

Особым преимуществом является возможность многозадачности. Это значительно облегчает бремя разработки программного обеспечения. Многие продвинутые проекты Arduino отягощены использованием конечных автоматов на основе модели цикла событий. Каждый раз при прохождении цикла программное обеспечение должно опрашивать, произошло ли событие, и определять, пришло ли время для какого-либо действия. Это требует управления переменными состояния, которое быстро усложняется и приводит к ошибкам программирования. Наоборот, вытесняющая многозадачность обеспечивает автономность задач управления, четко реализующих свои независимые функции. Это проверенная форма программной абстракции.

FreeRTOS обеспечивает вытесняющую многозадачность, которая автоматически распределяет время процессора между задачами. Однако независимость задач добавляет необходимость контроля безопасного взаимодействия между ними. Вот почему ядро FreeRTOS также предоставляет очереди сообщений, семафоры, мьютексы и многое другое для безопасного управления задачами. На протяжении всей книги мы будем изучать возможности RTOS.

libopencm3

Разработка кода для микроконтроллерных приложений может оказаться сложной задачей. Часть этой задачи заключается в программировании «голого железа» платформы. Сюда входят все специализированные периферийные регистры. Кроме того, многие периферийные устройства требуют определенного «танца с бубном», чтобы подготовить их к использованию.

Именно для этой цели пригодится библиотека libopencm3 (https://libopencm3.org/). Она не только определяет адреса памяти для адресов периферийных регистров, но также устанавливает макроопределения для специальных констант, которые необходимы. Наконец, библиотека включает проверенные C-функции для взаимодействия с периферийными ресурсами оборудования. Использование libopencm3 избавляет нас от необходимости делать все это с нуля.

Никаких Ардуино!

В этой книге не представлен код Arduino. Arduino хорошо служит своей цели, позволяя учащимся погрузиться в мир микроконтроллеров без предварительных знаний. Однако цель этой книги – выйти за рамки среды Arduino и использовать профессиональный способ разработки, независимый от инструментов Arduino.

Без Arduino не существует «цифрового порта 10». Вместо этого вы работаете напрямую с портом контроллера и – при необходимости – непосредственно с выводом. Например, плата Blue Pill, используемая в этой книге, имеет встроенный светодиод на порту C (как Arduino на контакте 13). Непосредственная работа с портами позволяет выполнять операции ввода-вывода на всех 16 выводах порта одновременно, если это необходимо приложению.

Никаких IDE

Сознательно было принято решение выбрать для этой книги среду разработки, нейтральную к выбранной платформе на настольном компьютере. Существует ряд сред IDE (Integrated Development Environment, интегрированная среда разработки) на базе Windows с различными лицензиями. Но IDE развиваются, лицензии меняются, а связанные с ними библиотеки меняются со временем. Преимущество выбранной IDE часто пропадает, когда обновляются IDE и операционная система, на которой она работает.

Использование подхода с открытым исходным кодом имеет то преимущество, что вы защищены от всех этих изменений и внезапной неработоспособности версий. Вы можете навсегда законсервировать весь свой код и инструменты поддержки, зная, что при необходимости их можно будет восстановить через десять лет. С другой стороны, использование лицензионного программного обеспечения делает вас уязвимым перед просроченными лицензиями или отключенными интернет-сайтами.

В этой книге разрабатываются проекты на основе следующих инструментов и библиотек с открытым исходным кодом:

- gcc/g++ (коллекция компиляторов GNU с открытым исходным кодом);
- *make* (утилита GNU binutils[2] с открытым исходным кодом);
- libopencm3 (библиотека с открытым исходным кодом);
- FreeRTOS (библиотека с открытым исходным кодом; бесплатна для коммерческого использования).

Благодаря этому фундаменту проекты, описанные в этой книге, останутся пригодными к использованию еще долгое время после того, как вы купите эту книгу. Кроме того, такой подход позволяет без ограничений участвовать пользователям Linux, FreeBSD и macOS. Теперь благодаря Microsoft WSL[3] (на самом деле WSL2) вы можете безболезненно запускать Ubuntu Linux со своего рабочего стола Windows, используя эту среду с открытым исходным кодом.

Во всех представленных проектах используется утилита GNU *make*, предоставляющая несколько функций сборки ПО с минимальными усилиями. Если сборки, представленные в этой книге, содержат ошибки, обязательно используйте команду GNU `make`, особенно во FreeBSD. Некоторые системы устанавливают GNU *make* как *gmake*.

---

[2] GNU Binary Utilities – набор инструментального ПО для обращения с объектным кодом в файлах различного формата.

[3] Microsoft WSL – Windows Subsystem for Linux, подсистема Windows для Linux. Подробнее см. https://habr.com/ru/companies/ruvds/articles/508014/.

Фреймворк разработчика

Хотя gcc, libopencm3 и FreeRTOS можно заставить работать вместе самостоятельно, это потребует немалой организации и усилий. Сколько стоит ваше время?

Вместо этой утомительной работы можно на github.com бесплатно загрузить среду разработки. Эта платформа интегрирует libopencm3 с FreeRTOS. Также предоставляются файлы *make*, необходимые для построения всего дерева проектов сразу или каждого проекта в отдельности. Наконец, есть некоторые библиотеки с открытым исходным кодом, которые могут сократить время разработки ваших новых приложений. Эту платформу можно загрузить с сайта *github.com* или вместе с исходным кодом книги.

Для кого написана эта книга?

Эта книга предназначена для аудитории, желающей выйти за рамки опыта работы с Arduino. Это относится к любителям, разработчикам или инженерам. Программное обеспечение, разработанное в этой книге, использует язык программирования C, поэтому свободное владение им будет полезно. Аналогично предполагаются некоторые базовые знания в области цифровой электроники, касающиеся периферийных интерфейсов, предоставляемых платформой. Дополнительную теорию можно найти, например, для таких областей, как шина CAN.

Платформа STM32 может представлять собой сложную задачу в настройке и правильной работе. Большая часть этой проблемы является следствием многочисленных возможностей настройки периферийных устройств. Каждый узел зависит от тактовых импульсов, которые необходимо включить и настроить делитель частоты. На некоторые устройства дополнительно влияют настройки входной тактовой частоты. Наконец, каждое периферийное устройство должно быть включено и настроено для использования. Вам не нужно быть экспертом, так как эти процедуры будут изложены и объяснены в последующих главах.

Любителям и разработчикам книга не должна показаться сложной. Даже когда они сталкиваются с трудностями, они все равно должны быть в состоянии собрать и выполнить каждый из экспериментов проекта. По мере роста знаний и уверенности каждый читатель может углубляться в изучаемые темы. Для такого углубления всем читателям предлагается изменить представленные проекты и провести расширенные эксперименты. Такая структура книги позволит вам создавать новые проекты с минимальными усилиями.

Что вам потребуется?

Давайте кратко рассмотрим некоторые предметы, которые вам желательно приобрести. Конечно, номером один в списке является устройство Blue Pill (см. рис. 1.1). Я рекомендую вам приобретать устройства, которые включают

в себя установленные разъемы, чтобы вы могли легко использовать устройство на макетной плате (или их необходимо припаять самостоятельно, если хотите).

Чтобы найти предложения по продаже, просто используйте для поиска название STM32F103C8T6[4]. В главах 18 и 19 используются три таких устройства, взаимодействующих друг с другом через шину CAN. Если вы захотите провести и эти эксперименты тоже, обязательно приобретите как минимум три экземпляра. В остальных демонстрационных проектах одновременно задействовано только одно устройство, однако на всякий случай всегда рекомендуется иметь запасное.

Программатор ST-Link V2

Следующим важным компонентом оборудования является внутрисхемный программатор. Просто найдите «ST-Link V2». Обязательно приобретите программатор «V2», так как использовать более старый вариант не имеет смысла.

В большинстве случаев в комплект поставки входят четыре съемных провода-перемычки для подключения программатора к устройству STM32. На рис. 1.2 показан USB-программатор ST-Link V2, который можно использовать в Windows, Raspberry Pi, Linux, macOS и FreeBSD[5].

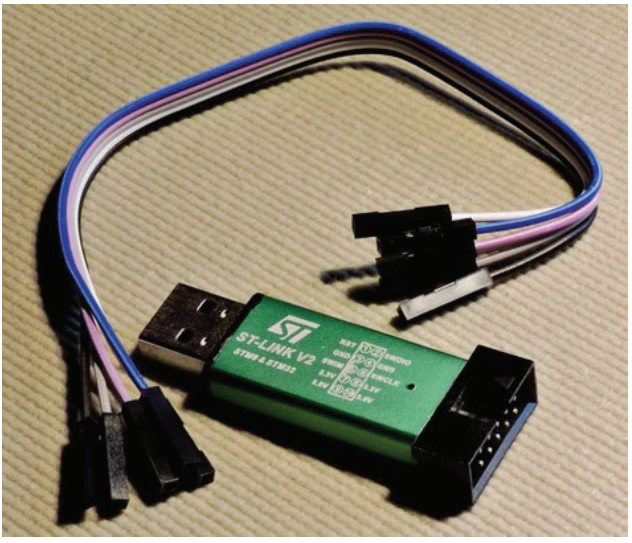


Рис. 1.2. Программатор ST-Link V2 с проводами

[4] В России используемую в этой книге плату Blue Pill с контроллером STM32F103C8T6 можно прибрести во многих интернет-магазинах, торгующих электроникой, а также через маркетплейсы (Ozon, WildBerries, Яндекс.Маркет).

[5] Отметим, что указанный автором программатор ST-Link V2 выпускается в различных вариантах оформления, среди которых есть весьма дорогие (с ЖК-дисплеем, многоконтактным кабелем JTAG-отладчика и пр.). Внешний вид самого простого и дешевого варианта, который и имеет в виду автор, аналогичен показанному на рис. 1.2.

Устройство STM32F103C8T6 можно запрограммировать несколькими способами, но в этой книге будет использоваться только USB-программатор ST-Link V2. Это упростит работу при разработке проекта и позволит осуществлять удаленную отладку.

Удлинительный USB-кабель[6] полезен при использовании с данным устройством. В комплекте его обычно не прилагают, потому следует подумать о том, чтобы его приобрести.

Макетная плата

Это почти само собой разумеется, но для проведения экспериментов необходима макетная плата. Она позволяет быстро и без применения пайки подключить компоненты, попробовать их, а затем выдернуть соединительные провода для следующего эксперимента.

Многие проекты в этой книге небольшие и требуют на плате места для одного устройства Blue Pill и, возможно, нескольких светодиодов или одного-двух других модулей. Однако в других экспериментах, например в главах 18 и 19, используются три устройства, обменивающиеся данными друг с другом через шину CAN. Я рекомендую вам приобрести комплект с макетными платами, на который поместятся четыре устройства (и останется дополнительное место для подключения других модулей и компонентов). Конечно, вы можете просто купить четыре небольших макетных платы, хотя это менее удобно.

На рис. 1.3 показан макетный комплект, который я использую в этой книге. Он не только достаточно большой, но также имеет подключенные шины питания вверху и внизу каждой платы, что упрощает составление схемы[7].

Провода DuPont (перемычки)

Возможно, вы не особо задумываетесь о разводке макетной платы, но обнаружите, что качество проводов-перемычек DuPont[8] может иметь огромное значение. Да, вы можете разрезать и зачистить обычные провода калибра AWG22 (или AWG24), но это неудобно и отнимает много времени. Гораздо

---

[6] То есть не соединительный USB-кабель (как, например, для подключения к плате Arduino Uno – типа AB), а именно *удлинительный*, конфигурации штырь (типа A) – гнездо (типа A). Без него программатор можно включать в USB-порт компьютера непосредственно, но это неудобно и опасно с точки зрения сохранности USB-разъемов.

[7] Для большинства экспериментов в этой книге достаточно одной-двух макетных плат половинного размера (длина 80–85 мм и примерно 400 контактов в 30+ столбцах) или одной полноразмерной, как у автора (длина 175–180 мм, ширина 55–68 мм, около 800 контактных гнезд в 60+ столбцах). Следует учесть, что макетные платы половинного размера могут «пристегиваться» друг к другу короткими сторонами, образуя платы в принципе любой желаемой длины.

[8] Наименование фирмы-изготовителя DuPont в отношении проводов-перемычек для макетных плат на отечественном рынке употребляется относительно редко (хотя тоже встречается, но компания «Дюпон» ассоциируется в основном совсем с другими продуктами). В США это примерно такое же ходовое наименование, как у нас все множительные аппараты называют «ксероксами» независимо от происхождения.

удобнее иметь наготове небольшую коробочку с перемычками. На рис. 1.4 показана небольшая случайная коллекция проводов DuPont типа «штырь–штырь».

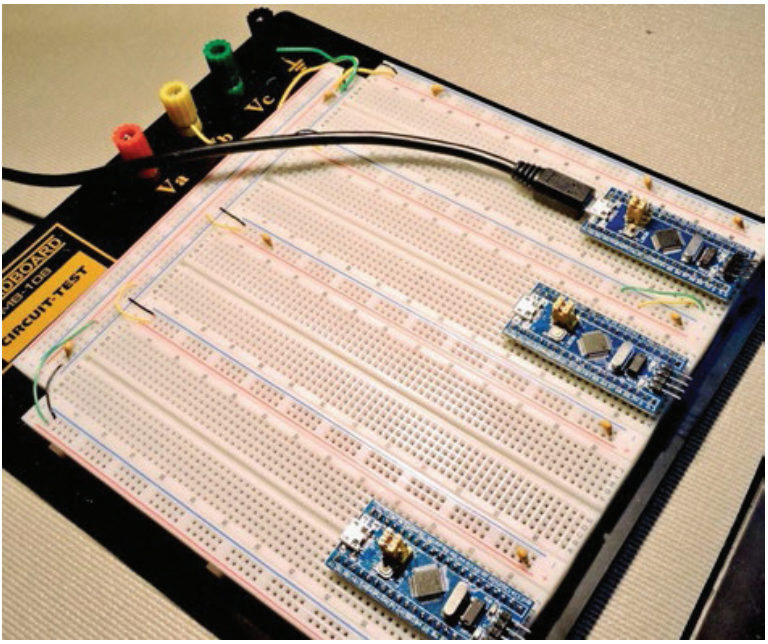


Рис. 1.3. Комплект макетных плат с подключенными шинами питания

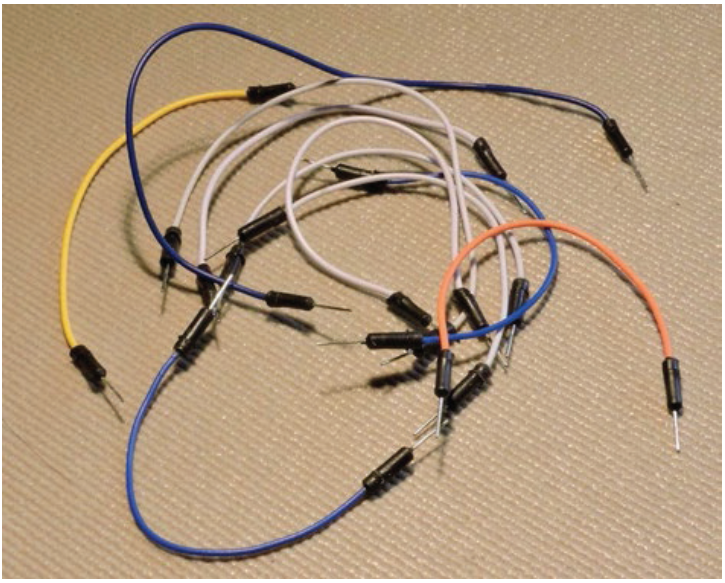


Рис. 1.4. Случайная коллекция проводов-перемычек DuPont

Провода DuPont «штырь–штырь» можно приобрести в различных вариантах. Я рекомендую приобрести несколько наборов, чтобы у вас были разные цвета и длины[9].

Развязывающие конденсаторы 0.1 мкФ

Вы можете обнаружить, что можете обойтись без развязывающих конденсаторов[10], но они рекомендуются (показаны на рис. 1.5 в виде желтых компонентов на шинах питания).

Если возможно, постарайтесь купить качественные конденсаторы, например на основе металлизированной полиэфирной пленки[11]. Номинальное напряжение может составлять всего 16 В. Их следует подключить к шинам питания на макетной плате между положительными и отрицательными шинами чтобы отфильтровать любые переходные напряжения и шумы.

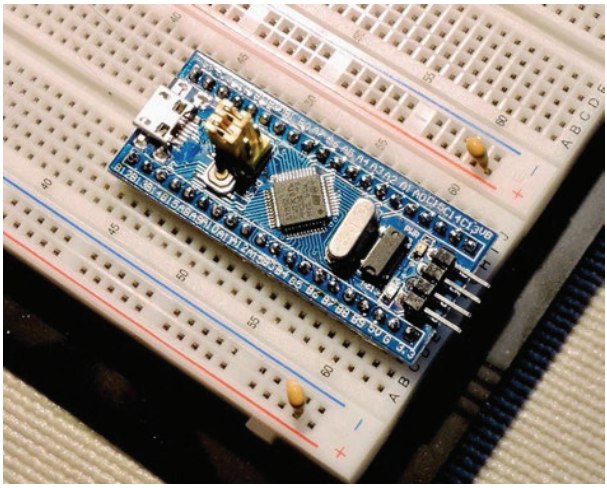


Рис. 1.5. Макетная плата с установленным модулем STM32F103C8T6 и развязывающими конденсаторами 0.1 мкФ

[9] Автор ориентируется на сборку схем с помощью макетной платы, потому упоминает только провода типа «штырь–штырь». Однако перемычки конфигураций «штырь–гнездо» или «гнездо–гнездо» позволяют быстро собирать макеты из модулей с установленными разъемами вообще без макетной платы (гнезда на таких перемычках полностью аналогичны отверстиям в макетной плате).

[10] Развязывающие конденсаторы включают между шинами питания, если в схеме присутствуют высокочастотные компоненты, особенно много потребляющие (с целью «развязки», т. е. изоляции их друг от друга по высокой частоте). В простых схемах с микроконтроллерными модулями их установка необязательна, так как они уже установлены на платах модулей.

[11] Однако на рис. 1.5 у автора показаны самые обычные многослойные керамические (К10-17 согласно отечественному обозначению) – в рядовых схемах их совершенно достаточно для целей «развязки». Металлопленочные конденсаторы с полиэфирным диэлектриком (типа, например, К73-17) используют обычно в высоковольтных схемах – в сети с напряжением 220 В и выше (а также в аналоговых схемах с требованием повышенной стабильности).

Адаптер последовательного порта USB-TTL

Это устройство необходимо для некоторых проектов, описанных в этой книге. На рис. 1.6 показан модуль адаптера, который я использовал. Адаптер используется для передачи данных на настольный компьютер или ноутбук. При отсутствии дисплея адаптер позволяет вам общаться с контроллером через виртуальный последовательный порт (поверх USB) с терминальной программой.

В интернет-магазинах доступно несколько разных типов адаперов, но будьте осторожны: вам необходимо устройство с аппаратными сигналами управления коммуникацией. В самых дешевых устройствах отсутствуют эти дополнительные сигналы, называемые RTS и CTS. Без них вы не сможете общаться на высоких скоростях (например, 115 200 бод) без потери данных.

Если вы используете Windows, также остерегайтесь покупать «аналоги» (точнее, подделки) под коммуникационный чип FTDI. Поступали сообщения о том, что программные драйверы FTDI блокируют поддельные устройства. Ваш выбор не обязательно должен включать FTDI, но, если устройство заявляет о совместимости с FTDI, проверьте поддержку драйвера на сайте продавца.

На рис. 1.6 вы можете заметить, что к концу кабеля прикреплена бирка. Эта бирка напоминает, какого цвета провод несет какую функциональность, чтобы подключить его правильно. Возможно, вы захотите сделать для себя что-то подобное.

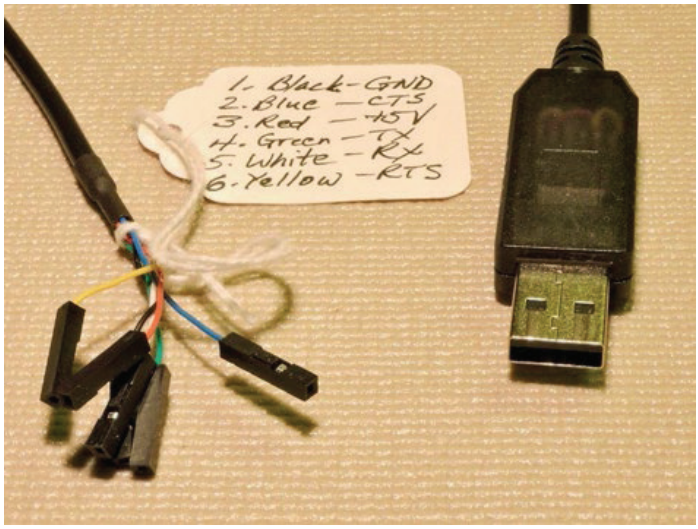


Рис. 1.6. Последовательный адаптер USB-TTL (5 В)

Обычно это устройства с напряжением 5 В и, следовательно, они совместимы с TTL-уровнями порта контроллера. Однако обратите внимание, что одной из особенностей устройств семейства STM32F103 является то, что многие выводы GPIO выдерживают напряжение 5 В, хотя микроконтроллер работает

от источника питания +3.3 В. Это позволяет использовать эти TTL-адаптеры без каких-либо переходных устройств. Подробнее об этом будет рассказано ниже. Можно приобрести аналогичные устройства, которые работают на уровне 3.3 В или могут переключаться между 5 и 3.3 В.

Источник питания

Большинство представленных проектов будут прекрасно работать от выходной мощности адаптера USB. Но если ваш проект потребляет ток, превышающий обычный[12], вам может понадобиться адаптер питания. На рис. 1.7 показан хороший источник, подключающийся прямо к шинам питания макетной платы.

Показанный на рисунке источник рекламировался как «Модуль питания для беспаечной макетной платы MB102, 3.3 В, 5 В». Если на вашей макетной плате отсутствуют шины питания, возможно, вам придется купить макетную плату другого типа.

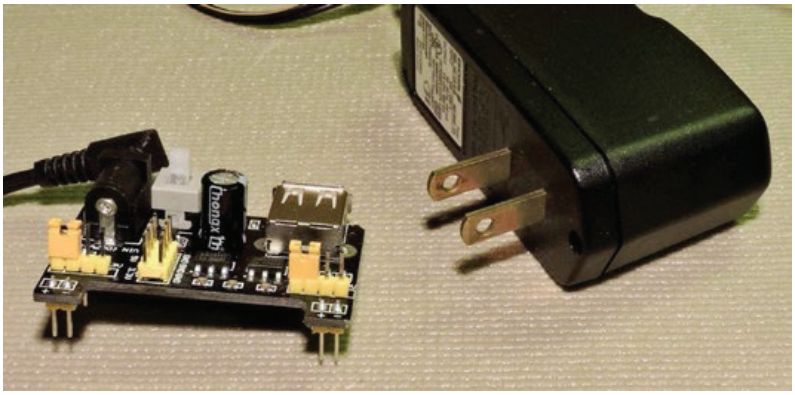


Рис. 1.7. Небольшой блок питания для макетной платы и адаптер на 7.5 В постоянного тока

MB102 удобен тем, что он может выдавать питание 3.3 или 5 В. Кроме того, на нем имеется кнопка включения/выключения питания.

Другим важным моментом является сетевой адаптер для подачи входного питания (он не входит в комплект MB102). Хотя MB102 принимает входное напряжение до 12 В, я обнаружил, что большинство настенных адаптеров постоянного тока на 9 В имеют напряжение холостого хода около 13 В и более. Я считаю, что это рискованно, потому что, если дешевый MB102 по какой-либо причине выйдет из строя, повышенное напряжение может просочиться и повредить блок контроллера.

Копаясь в своей коробке с разными адаптерами, я в конце концов нашел старое зарядное устройство для телефона Ericsson, рассчитанное на 7.5 В по-

[12] Напомним, что стандартный нагрузочный ток по шине питания USB равен 0.5 А. В некоторых современных стандартах (USB 3.0 и др.) допустимый ток может быть и выше, но рассчитывать на это не следует.

стоянного тока и ток 600 мА. Оно измерило напряжение без нагрузки 7.940 В. Это намного ближе к выходным напряжениям 5 и 3.3 В, которые регулирует MB102. Если вам придется приобрести адаптер питания, я рекомендую аналогичный блок[13].

Мелочи

Некоторые небольшие компоненты, возможно, у вас уже есть. В противном случае учтите, что вам понадобятся как минимум светодиоды и резисторы для использования в проекте. На рис. 1.8 показан случайный набор светодиодов и резисторная сборка SIP-9.

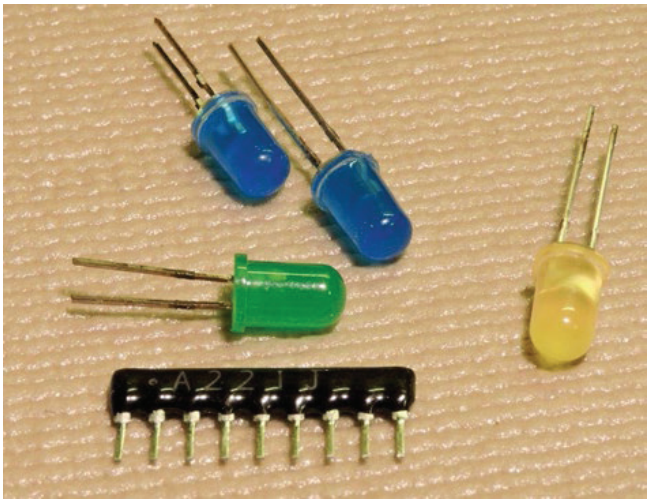


Рис. 1.8. Случайный набор из 5-мм светодиодов и резисторной сборки SIP-9

Обычно светодиод рассчитан на ток около 10 мА для нормальной яркости. Для светодиодов меньшего размера требуется всего от 2 до 5 мА. При напряжении питания около 3.3 В вам понадобится резистор сопротивлением около 220 Ом для ограничения тока (220 Ом ограничивают ток максимум пример-

[13] Чтобы избежать риска перенапряжения на входе питания, не следует использовать дешевые нестабилизированные сетевые адаптеры (как, очевидно, это сначала сделал автор). В продаже имеются стабилизированные адаптеры на 7.5 или 9 В, причем не гонитесь за дешевизной – в данном случае цена напрямую коррелирует с качеством. Среди таких адаптеров имеются и выдающие непосредственно 5 или 3.3 В (без промежуточных преобразователей типа указанного автором MB102) – их, конечно, тоже можно использовать, подключив к макетной плате через специальные гнезда с зажимами для проводов, которые также несложно приобрести отдельно. Однако предпочтительно использовать двухступенчатую схему, как у автора, – качество и надежность питания будут выше, к тому же подключать гораздо удобнее. Только сочетание, указанное автором, мало чем отличается от питания по USB, так что такая замена не имеет особого смысла. Мощность адаптера следует выбирать побольше (1–2 А), причем он должен выдавать ток, не меньший, чем плата источника.

но до 7 мА). Поэтому приобретите несколько резисторов сопротивлением 220 Ом (0.25 Вт будет достаточно)[14].

На рис. 1.8 показана одна деталь, которую вы, возможно, захотите приобрести, – это резисторная сборка SIP-9. Рисунок 1.9 иллюстрирует внутреннюю схему этой сборки. Если, например, вы хотите управлять восемью светодиодами, вам понадобится восемь токоограничивающих резисторов. Отдельные резисторы вполне годятся, но требуют дополнительных проводов и занимают место на макетной плате. Сборка SIP-9, напротив, имеет один вывод, общий для восьми резисторов. Остальные восемь выводов – это другие концы восьми внутренних резисторов. Используя этот тип корпуса, вы можете уменьшить количество деталей и необходимых проводов.

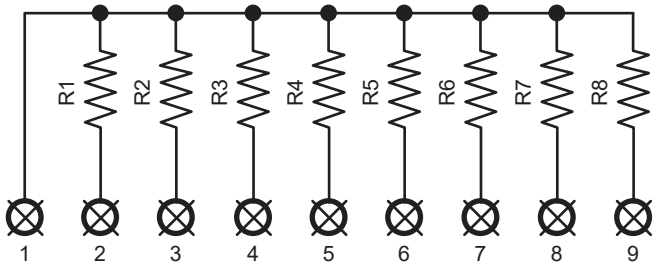


Рис. 1.9. Внутренняя схема резисторной сборки SIP-9

Итоги

В этой главе были представлены основные компоненты, которые рассматриваются в этой книге. Было также перечислено большинство дополнительных вещей, которые вам, возможно, придется приобрести. Следующая глава проведет вас через необходимые этапы установки программного обеспечения. Как только это будет закончено, мы сможем приступить к настоящей работе.

[14] Разумеется, для макета вам понадобятся резисторы с гибкими выводами (TH – монтируемые в отверстия), а не резисторы для поверхностного монтажа (SMD).

Глава 2 • Настройка программного обеспечения

Прежде чем вы сможете приступить к работе, связанной с проектами, вам необходимо установить кое-какое программное обеспечение, в котором задействовано множество составляющих. Несмотря на это, процесс должен протекать гладко, и его нужно будет выполнить только один раз.

Используемые соглашения о каталогах

На протяжении всей книги мне придется обращаться к различным подкаталогам прилагаемого программного обеспечения. Предполагается, что верхний уровень установленного вами программного обеспечения имеет название *~/stm32f103c8t6*. Поэтому, когда я ссылаюсь на путь *~/stm32f103c8t6/libopencm3/README.md*, я предполагаю, что он начинается с вашего домашнего каталога (~) (где бы он ни находился). Я часто буду использовать это соглашение для ясности, даже если ваш текущий каталог уже установлен правильно для файла, о котором идет речь.

Операционное ПО

Я предполагаю, что у вас есть среда POSIX (Linux/Unix), из которой можно запускать команды. Среды Linux или Raspberry Pi, использующие оболочку *bash*, пожалуй, наиболее естественны. Другие хорошие среды включают FreeBSD и macOS. Что касается FreeBSD, я также предполагаю, что вы используете оболочку *bash*.

В противном случае, если вы используете Windows, вам потребуется установить WSL (подсистему Windows для Linux). Это даст вам среду Linux для работы (см. раздел «Установки для пользователей Windows» далее в этой главе).

Пользователям Mac потребуется как минимум установить *git*. Вам также понадобится GNU *make*, особенно если вы используете FreeBSD. В системе BSD иногда GNU *make* устанавливается как *gmake*. Если вы используете Mac Homebrew (https://brew.sh), можете установить указанные утилиты следующим образом:

```
$ brew install make
$ brew install git
```

Если вы являетесь пользователем MacPorts (www.macports.org), вам нужно будет использовать эту платформу для установки *make* и *git*.

ПО для книги

Структура каталогов для сборки с помощью libopencm3 и FreeRTOS доступна на *github.com*. Выберите подходящее место для создания рабочего подкаталога. Эта книга предполагает домашний каталог:

```
$ cd ~
```

Клонирование с помощью учетной записи github

Если у вас есть учетная запись на github, просто используйте команду `git` для загрузки и создания подкаталога следующим образом (введите свой идентификатор пользователя и пароль github, если будет предложено):

```
$ git clone https://github.com/Apress/beginning-STM32-second-edition
```

Анонимная загрузка с github

Microsoft отключила анонимный доступ к *github.com*. Поэтому, если вы не хотите регистрировать идентификатор пользователя, используйте вместо этого следующую процедуру:

```
$ wget https://github.com/Apress/beginning-STM32-second-edition/archive/refs/heads/
master.zip
```

Другой способ: в раскрывающемся списке **Code** (Код) на веб-странице выберите **Download ZIP** (Загрузить ZIP) (в текущий каталог). В результате у вас должен остаться загруженный файл с именем *master.zip*.

В любом случае после скачивания *master.zip* его необходимо распаковать:

```
$ unzip master.zip
```

При разархивировании создается подкаталог с именем *beginning-STM32-second-edition-master*. Переименуйте его в более простое имя каталога *stm-32f103c8t6* или другое имя по вашему выбору:

```
$ mv beginning-STM32-second-edition-master stm32f103c8t6
```



На большинстве платформ Linux при необходимости можно установить unzip с помощью команды root:

```
# apt install unzip
```

libopencm3

Затем мы должны загрузить программное обеспечение libopencm3 в нужное место. У вас должна быть возможность клонировать его без учетной записи GitHub, поскольку они разрешили анонимную загрузку.

Перейдите в подкаталог, а затем введите команду `git clone` для libopencm3 следующим образом:

```
$ cd ~/stm32f103f8t6
$ git clone https://github.com/libopencm3/libopencm3.git
```

Это заполнит каталог *~/stm32f103c8t6/libopencm3* файлами и подкаталогами.

Ядро FreeRTOS

Следующая важная часть программного обеспечения – ядро FreeRTOS. Выполните следующее:

```
$ cd ~/stm32f103c8t6/rtos
$ git clone https://github.com/FreeRTOS/FreeRTOS-Kernel.git
```

Это создаст подкаталог *FreeRTOS-Kernel* в подкаталоге *rtos* и заполнит его исходными модулями ядра FreeRTOS.

Кросс-компилятор ARM

Если у вас еще не установлен кросс-компилятор ARM, его необходимо установить. Если вы используете Linux или Raspberry Pi, можете просто использовать команду `apt-get` для его установки. Несмотря на это, я рекомендую вместо этого загрузить и установить тулчейн (набор инструментов), как описано далее, поскольку некоторые инструменты кросс-компилятора плохо организованы и иногда неполны.

Выполните следующее.

1. В браузере посетите https://developer.arm.com.
2. Нажмите ссылку **Arm GNU Toolchain Downloads** (Загрузка тулчейна Arm GNU).
3. Прокрутите вниз и выберите в соответствии с вашей платформой **... hosted cross toolchains** (кросс-инструментальных тулчейнов, размещенных на), где на месте многоточия указано **x86_64 Linux**, **macOS (x86_64)** или **macOS (Apple Silicon)**. Обратите внимание, что «кросс-инструментальные тулчейны» должны быть «размещенные» (*hosted*).
4. Если, например, используется Intel Linux, мы затем загрузим *arm-gnu-toolchain-12.3.rel1-aarch64-arm-none-eabi.tar.xz*, в зависимости от того, что предлагает веб-сайт.
5. Создайте системный каталог */opt* (если у вас его еще нет):

   ```
   $ sudo -i
   # mkdir /opt
   ```

6. Перейдите в каталог */opt* и установите разрешения (от имени пользователя root):

```
# cd /opt
# chmod 755 /opt
```

7. Загрузите программное обеспечение:

```
# cd /opt
# wget -O a.xz 'https://wget/developer.arm.com/...'
```

На веб-сайте могут использоваться удивительно длинные URL-адреса для загрузки, поэтому я рекомендую вам использовать опцию `-O a.xz` для названия загруженного файла. Оставшуюся часть URL-адреса загрузки заключите в одинарные кавычки, чтобы оболочка не могла неправильно интерпретировать специальные символы.

8. На этой стадии вы распакуете свой компилятор:

```
# tar xJf a.xz
```

Используйте опцию `tar J`, если расширение имени файла *xz*. В противном случае используйте опцию `z`, если расширение *gz*. Если на вашем Mac не установлена команда GNU `tar`, вы можете установить ее с помощью MacPorts (www.macports.org) или Homebrew (https://brew.sh/).

9. После извлечения tar-файла он может создать длинное имя каталога, например *arm-gnu-toolchain-12.3.rel1-aarch64-arm-none-eabi*. Сейчас самое время сократить его до *gcc-arm*:

```
# mv arm-gnu-toolchain-12.3.rel1-aarch64-arm-none-eabi gcc-arm
```

Это изменит имя каталога */opt/arm-gnu-toolchain-12.3.rel1-x86_x64-arm-none-eabi* на более удобное */opt/gcc-arm*.

10. Теперь выйдите из пользователя root и вернитесь в сеанс разработчика. В этом сеансе добавьте каталог *bin* компилятора в ваш системный каталог *PATH*:

```
$ export PATH="/opt/gcc-arm/bin:$PATH"
```

11. На этом этапе вы сможете протестировать установленный кросс-компилятор:

```
$ arm-none-eabi-gcc --version
arm-none-eabi-gcc (Arm GNU Toolchain 12.3.Rel1 (Build arm-12.35)) 12.3.1 20230626
Copyright (C) 2022 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Если компилятор не запускается и вместо этого выдает такое сообщение:

```
$ arm-none-eabi-gcc --version
-bash: arm-none-eabi-gcc: command not found
```

значит, ваша переменная *PATH* либо настроена неправильно, не экспортирована, либо установленные инструменты используют другой префикс. При необходимости выполните следующие действия (вывод результата здесь немного сокращен):

```
$ ls -l /opt/gcc-arm/bin
total 75128
-rwxr-xr-x@ 1 root wheel 1016776 21 Jun 16:11 arm-none-eabi-addr2line
-rwxr-xr-x@ 2 root wheel 1055248 21 Jun 16:11 arm-none-eabi-ar
-rwxr-xr-x@ 2 root wheel 1749280 21 Jun 16:11 arm-none-eabi-as
-rwxr-xr-x@ 2 root wheel 1206868 21 Jun 19:08 arm-none-eabi-c++
-rwxr-xr-x@ 1 root wheel 1016324 21 Jun 16:11 arm-none-eabi-c++filt
-rwxr-xr-x@ 1 root wheel 1206788 21 Jun 19:08 arm-none-eabi-cpp
-rwxr-xr-x@ 1 root wheel 42648 21 Jun 16:11 arm-none-eabi-elfedit
-rwxr-xr-x@ 2 root wheel 1206868 21 Jun 19:08 arm-none-eabi-g++
-rwxr-xr-x@ 2 root wheel 1202596 21 Jun 19:08 arm-none-eabi-gcc
...
-rwxr-xr-x@ 2 root wheel 1035160 21 Jun 16:11 arm-none-eabi-nm
-rwxr-xr-x@ 2 root wheel 1241716 21 Jun 16:11 arm-none-eabi-objcopy
...
```

Если вы получили кросс-компилятор из источника, отличного от указанного, у вас могут отсутствовать имена префиксов. Если вы видите имя файла *gcc* вместо *arm-none-eabi-gcc*, нужно будет вызвать его просто как *gcc*. Но в этом случае будьте осторожны, потому что кросс-компилятор может перепутаться с компилятором вашей платформы. Префикс «arm-none-eabi-» предотвращает это. Когда вы собираетесь использовать кросс-платформенный *gcc*, убедитесь, что используется правильный компилятор, с помощью команды `type`:

```
$ type gcc
arm-none-eabi-gcc is hashed (/opt/gcc-arm/bin/gcc)
```

Если оболочка bash находит *gcc* в каталоге, отличном от того, в который вы установили, значит, *PATH* установлен неправильно.

Если вам необходимо изменить префикс всего тулчейна, то верхний уровень *~/stm32f103c8t6/Makefile.incl* следует отредактировать:

```
$ cd ~/stm32f103c8t6
$ nano Makefile.incl
```

Измените следующую строку соответствующим образом и сохраните ее:

```
PREFIX ?= arm-none-eabi
```

В обычной ситуации, когда используется кросс-платформенный префикс, вы также должны иметь возможность получить следующее подтверждение:

```
$ type arm-none-eabi-gcc
arm-none-eabi-gcc is hashed (/opt/gcc-arm/bin/arm-none-eabi-gcc)
```

Это подтверждает, что компилятор запускается из правильного каталога */opt/gcc-arm*.



Чтобы использовать тулчейн кросс-компилятора, переменную *PATH* необходимо будет модифицировать для каждого нового сеанса терминала. Для удобства вы можете создать сценарий, изменить файл *~/.bashrc* или создать для этого команду алиаса (псевдонима) оболочки.

Создание программного обеспечения

На этом этапе вы установили программное обеспечение книги, libopencm3, FreeRTOS и тулчейн кросс-компилятора ARM. Настроив переменную *PATH*, вы теперь сможете перейти в каталог *stm32f103c8t6* и ввести `make` (некоторым пользователям может потребоваться вместо этого использовать `gmake`):

```
$ cd ~/stm32f103c8t6
$ make
```

Сначала будет создан *~/stm32f103c8t6/libopencm3*, а затем все остальные подкаталоги.

Всегда существует вероятность того, что новая версия libopencm3 может создать проблемы со сборкой. Это трудно предвидеть, но вот некоторые возможности и решения.

1. Что-то в libopencm3 помечается кросс-компилятором как ошибка, хотя раньше это было допустимо. Можно сделать следующее:
   a) исправить или обойти проблему в исходниках libopencm3;
   b) попробовать более позднюю (или предыдущую) версию тулчейна кросс-компилятора. Новые тулчейны часто решают проблему;
   c) установить более старую версию libopencm3.
2. Что-то в программном обеспечении для этой книги сломалось. Проверьте репозиторий *git* на наличие обновлений. По мере того как проблемы становятся известны, исправления будут применяться и публиковаться там. Также проверьте файл *README.md* верхнего уровня.

Обычно вы можете игнорировать предупреждения компилятора, особенно в подкаталогах *libopencm3* или *FreeRTOS-Kernel*.

Инструмент ST-Link

Есть еще одна часть программного обеспечения, которую, возможно, потребуется установить. Если вы еще не установили его с помощью менеджера

пакетов вашей системы, вам необходимо установить его сейчас. Даже если оно у вас уже установлено, оно может устареть. Давайте проверим это:

```
$ st-flash
```

Найдите следующую строку на экране справки:

```
./st-flash [--debug] [--reset] [--serial <serial>] [--format <format>] [--flash=<fsize>]
{read|write} <path> <addr> <size>
```

Если вы не видите опцию `--flash=<fsize>`, возможно, вы захотите загрузить последнюю версию с github и собрать ее из исходного кода. Это необходимо только в том случае, если вы хотите использовать более 64 Кбайт флеш-памяти. Ни одна из демонстраций в этой книге не выходит за этот предел.

Пользователи сообщают, что многие устройства STM32F103C8T6 поддерживают 128 Кбайт флеш-памяти, хотя само устройство сообщает, что у него только 64 Кбайта. Следующая команда проверяет принадлежащее мне устройство:

```
$ st-info --probe
Found 1 stlink programmers
serial: 493f6f06483f53564554133f
openocd: "\x49\x3f\x6f\x06\x48\x3f\x53\x56\x45\x54\x13\x3f"
flash: 65536 (pagesize: 1024)
sram: 20480
chipid: 0x0410
descr: F1 Medium-density device
```

Сообщаемая информация указывает на то, что устройство поддерживает только 65 536 байт (64 Кбайт) флеш-памяти. Еще я знаю, что могу прошить до 128 Кбайт и пользоваться (у меня все имеющиеся поддерживают 128 Кбайт). Было высказано предположение, что и устройства F103C8, и устройства F103B8 используют один и тот же кремниевый кристалл. Я расскажу об использовании программатора ST Link V2 на своем устройстве в следующей главе.

Для пользователей, отличных от Windows: если у вас не установлены эти утилиты, сделайте это сейчас, используя `apt-get`, `brew`, `yum` или любой другой менеджер пакетов. Если пакет не установлен, вы можете скачать последнюю версию исходного `кода с github здесь`:

```
$ cd ~
$ git clone https://github.com/texane/stlink.git
$ cd ./stlink
$ make
$ cd build/Release
$ sudo make install
```

Возможно, вам сначала придется установить драйвер `libusb`. Если это необходимо, попробуйте следующее от имени пользователя root:

```
# apt install libusb-1.0-0-dev
```

Если у вас возникнут с этим вопросом проблемы, обратитесь к следующим онлайн-ресурсам:

- файл *README.md* по адресу https://github.com/texane/stlink,
- https://github.com/texane/stlink/blob/master/doc/compiling.md,
- некоторые дистрибутивы Linux могут потребовать от вас выполнения команды `sudo ldconfig` после установки.

Установки для пользователей Windows

С появлением WSL (подсистемы Windows для Linux) от Microsoft теперь можно запускать Linux из Windows 10 или 11. Это обеспечивает удобный способ использования инструментов с открытым исходным кодом, не уходя с рабочего стола.

Прежде чем начать, выполните обновление Windows, чтобы исправить любые проблемы в установленной версии Windows. Как только вы будете в курсе последних событий, посетите один из следующих ресурсов Microsoft, чтобы получить инструкции по установке WSL:

- https://learn.microsoft.com/en-us/windows/wsl/install-manual,
- https://learn.microsoft.com/en-us/windows/wsl/install.

Используйте первую ссылку (руководство по установке), если у вас более старая версия Windows 10, и посмотрите, сможете ли вы перейти на обновленную версию WSL2. Обязательно обновите WSL2, если это возможно. Новейшим пользователям Windows 10 или 11 следует вместо этого использовать вторую ссылку для установки (заканчивающуюся на «install»).

По умолчанию выбрана Ubuntu Linux, именно этот дистрибутив Linux рассматривается в этой книге. Другие дистрибутивы не рекомендуется использовать с этой книгой, поскольку в них может отсутствовать вспомогательное программное обеспечение, такое как инструменты для прошивки stm32 и сценарии, необходимые для связи с утилитой Windows *usbipd*.

Если в WSL уже установлен другой дистрибутив Linux, вы можете безопасно добавить Ubuntu Linux, не затрагивая текущие экземпляры WSL Linux.

Учетная запись и пароль Linux

В ходе установки WSL вам будет предложено ввести новое имя учетной записи и пароль для экземпляра Linux. Это не обязательно должно совпадать с вашей учетной записью в Windows и используется только для устанавливаемого дистрибутива Linux. Обратите внимание, что экземпляр Linux, установленный WSL, предназначен для каждого пользователя Windows отдельно и не может использоваться совместно с другими пользователями Windows.

Введите имя пользователя Linux в командной строке, показанной ниже:

```
Installing, this may take a few minutes...
Installation successful!
Please create a default UNIX user account. The username does not need to match your
Windows username.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username:
```

После указания имени пользователя вам будет предложено ввести пароль. Вы не увидите пароль при его вводе, поэтому делайте это осторожно. Если вы его испортите или забудете, вы сможете сбросить пароль позже. Для получения подробной информации перейдите по ссылке https://learn.microsoft.com/en-us/windows/wsl/setup/environment-set-up-your-linux-user-info.

Обновление WSL

Предоставляемое программное обеспечение поддержки WSL иногда обновляется Microsoft. Чтобы воспользоваться преимуществами обновлений, выполните следующие действия.

1. Нажмите клавиши <**Win**+**X**> и затем букву <**A**>.
2. Ответьте «Yes» (Да) на вопрос «Do you want this app to make changes to your device?» (Хотите ли вы, чтобы это приложение внесло изменения в ваше устройство?).
3. Введите команду `wsl --update`.
4. Введите `exit`, чтобы выйти из сеанса.

Запуск Ubuntu

Чтобы запустить Ubuntu Linux в WSL:

1) нажмите кнопку **Start** (Пуск).
2) щелкните по пункту меню **Ubuntu on Windows**.

Откроется окно с учетной записью, не являющейся администратором, без необходимости ввода пароля. Это учетная запись, под которой вы будете выполнять большую часть своей работы.

Обновление Ubuntu

Чтобы обновить Ubuntu Linux, вам понадобится `sudo`, чтобы перейти на «корневую» учетную запись. Здесь нужно будет указать пароль, который вы использовали при установке Linux:

```
indy@DESKTOP-8KO41TK:~$ sudo -i
[sudo] password for indy: <password>
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.90.1-microsoft-standard-WSL2 x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/advantage
...snip...
root@DESKTOP-8KO41TK:~#
```

Обратите внимание, как символ приглашения меняется с «$» на «#». Это означает, что теперь у вас есть root-доступ и вы можете выполнять административные функции. Выполните следующие команды (здесь я сократил подсказки до «#»):

```
# apt update
# apt upgrade
```

В зависимости от того, насколько актуальна ваша Ubuntu Linux, вы можете получить некоторые обновления. Если будет предложено, ответьте «Y» (Да).

Установка зависимостей

Если у вас свежеустановленный экземпляр Ubuntu Linux, вам, вероятно, потребуется установить некоторые зависимости пакетов для использования с этой книгой:

```
# apt install wget git make cmake unzip
```

Затем установите `stlink-tools` следующим образом (от имени пользователя root):

```
# apt install stlink-tools
```

При этом будут установлены такие команды, как `st-flash`, которые для WSL будут использовать утилиту *usbipd* (см. раздел «Установка usbipd в Windows» в этой главе).

На этом этапе вы можете следовать инструкциям, перечисленным ранее, начиная с раздела «ПО для книги». После того как вы скачали и установили программное обеспечение для книги, установили кросс-компилятор (*arm-gcc*) и скомпилировали исходный код, обязательно обратитесь к следующему разделу «Установка Windows usbipd».

Установка Windows usbipd

На момент написания WSL2 не может получить доступ к периферийным устройствам USB напрямую из Linux. Хотя можно прошить устройства STM32 с помощью инструмента Windows (вне сеанса Linux), вам может оказаться более удобным вместо этого использовать инструменты командной строки Ubuntu, такие как `st-flash`. Чтобы использовать их, установите программное обеспечение usbipd.

В терминале администратора Windows (клавиша <**Win**+**X**> и выбрать «Командная строка»)[15] выполните:

```
C:\Windows\system32> wget winget install usbipd
```

[15] В Windows вместо ввода команд с клавиатуры, привычного для линуксоида, проще командную строку и другие утилиты вызывать мышью через **Пуск** > **Программы**).

Затем нажмите **Yes** (Да) для установки. Возможно, вам придется перезагрузиться после завершения установки.

После повторного запуска сеанса терминала администратора Windows выполните следующие действия, чтобы проверить результат:

```
C:\Windows\system32> usbipd list
Connected:
BUSID VID:PID DEVICE STATE
3-3 045e:0040 Microsoft USB Wheel Mouse Optical Not shared
4-2 0518:0002 Office Keyboard Not shared
```

Содержание ответа, скорее всего, будет отличаться, но вы должны увидеть мышь и клавиатуру, если больше нет USB-устройств.

ПО Windows Ubuntu для usbipd

Чтобы программное обеспечение Linux могло использовать преимущества usbipd WSL, в Ubuntu требуется установленная библиотека *libusb-dev*. От имени пользователя root в Linux добавьте следующий пакет:

```
# apt install libusb-dev
```

Если вы используете usbip-win 2.0.0 или новее, вам также необходимо выполнить следующее:

```
# apt install linux-tools-virtual hwdata
# update-alternatives --install /usr/local/bin/usbip usbip \
   $(ls /usr/lib/linux-tools/*/usbip | tail -n1) 20
```

После подключения устройства программирования stm32 к USB-порту в сеансе терминала администратора Windows еще раз перечислите устройства:

```
PS C:\Windows\system32> usbipd list
Connected:
BUSID VID:PID   DEVICE                                   STATE
3-3   045e:0040 Microsoft USB Wheel Mouse Optical        Not shared
3-4   0bda:818b Realtek RTL8192EU Wireless LAN Networ... Not shared
4-2   0518:0002 Office Keyboard                          Not shared
4-3   0483:3748 STM32 STLink                           Shared
```



Устройство программирования USB ST-LINK описано в следующей главе.

Подключенное устройство должно появиться с названием «STM32 STLink» (см. идентификатор шины 4-3 в показанном примере сеанса). После этого в окне администратора Windows вы сможете выполнить действия, показанные ниже, заменив идентификатор шины «4-3» на идентификатор вашего собственного сеанса.

Опция `--distribution` не требуется, если имя вашего экземпляра WSL — «Ubuntu» (это значение по умолчанию). Не вводите квадратные скобки, если вам нужно назвать свой дистрибутив.

```
PS C:\Windows\system32> usbipd wsl attach -b 4-3 [--distribution Ubuntu-22.04]
```



Если вы недавно запускали apt-обновление, вам потребуется повторить запуск сценария update-alternatives.

В сеансе Ubuntu Linux USB-устройства перечисляются следующим образом:

```
# lsusb
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 002: ID 0483:3748 STMicroelectronics ST-LINK/V2
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
#
```

В этом списке Linux должно отобразить устройство под названием «STMicroelectronics ST-LINK/V2». Успех этой операции означает, что вы сможете выполнить следующую проверку (stm32 должен быть подключен к программатору):

```
# st-info --probe
Found 1 stlink programmers
version: V2J17S4
serial: 303030303030303030303031
flash: 131072 (pagesize: 1024)
sram: 20480
chipid: 0x0410
descr: F1xx Medium-density
#
```

Отображаемая информация будет различаться в зависимости от устройства, но программист должен иметь возможность запрашивать и получать сообщение о подключенном устройстве. Теперь WSL Ubuntu Linux может программировать устройство напрямую без необходимости использования специального инструмента для Windows.

Утилита STM32 ST-LINK

Если вы успешно установили программу usbipd для Windows и настроили его для Ubuntu Linux под WSL2, то описываемое здесь ПО является необязательным. В этом разделе описывается, как загрузить и установить утилиту STM32 *ST-LINK* для Windows.

Перейдите в веб-браузере по ссылке www.st.com/en/development-tools/stsw-link004.html.

Затем выполните следующие шаги.

1. Прокрутите вниз до раздела **Get Software** (Получить программное обеспечение).

2. Нажмите кнопку **Get latest** (Получить последнюю версию). Откроется страница с просьбой принять лицензию.
3. Откроется еще одна страница, требующая от вас зарегистрироваться и войти в систему или просто указать свое имя и адрес электронной почты.
4. Нажмите кнопку **Download** (Загрузить), чтобы на указанный адрес электронной почты была отправлена ссылка.
5. Нажмите ссылку **Download now** (Загрузить сейчас) в электронном письме.
6. Вы попадете на страницу под названием **STM32 ST-LINK utility** (Утилита STM32 ST-LINK), содержащую кнопку **Get Software** (Получить программное обеспечение). Нажмите на нее.
7. Затем вы опуститесь вниз с помощью другой кнопки **Download latest** (Загрузить последнюю версию). Поскольку вы зарегистрировались с использованием адреса электронной почты, это должно инициировать загрузку программного обеспечения.

При загрузке вам должен быть предоставлен файл с именем *en.stsw-link004.zip*. Откройте этот архив и нажмите на имеющийся там файл *setup.exe*. Установленное программное обеспечение включает и руководства к нему.

В качестве теста подключите программатор ST-LINK к USB-порту, а затем выберите в меню утилиты STM32 *ST-LINK* **Target** (Целевое устройство) > **Connect** (Подключиться). Утилита должна сразу найти устройство и отобразить его характеристики в правом верхнем углу окна (см. пример на рис. 2.1).

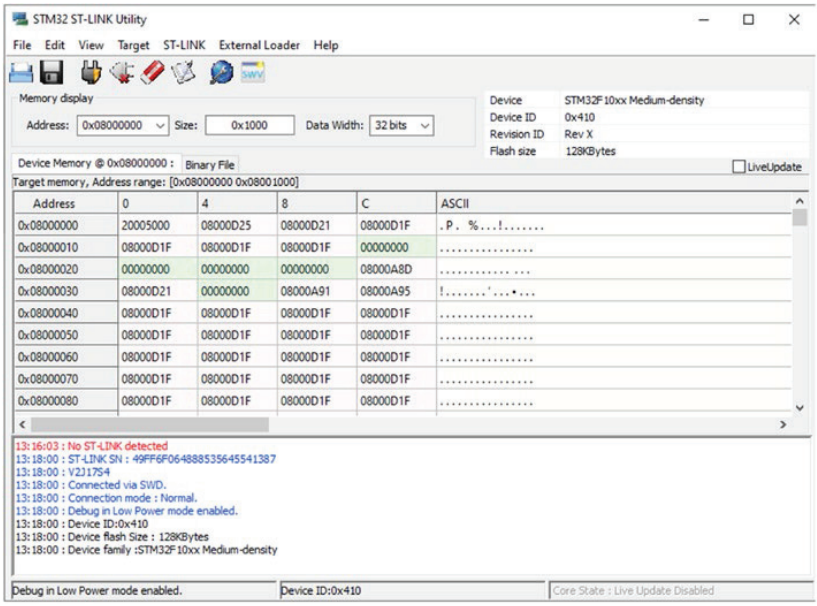


Рис. 2.1. Пример окна утилиты STM32 ST-LINK после подключения

Итоги

После установки программного обеспечения мы наконец можем обратить внимание на аппаратное обеспечение и что-то с ним сделать. В следующей главе мы рассмотрим варианты питания, а затем применим программатор ST-Link V2 для проверки устройства.

Глава 3 • Включаем питание и мигаем светодиодом

Купленная плата контроллера, скорее всего, запрограммирована на мигание при включении (возможно, вы это уже проверили). Это позволяет легко проверить работоспособность устройства. В этой главе еще необходимо обсудить несколько важных деталей, касающихся питания, сброса и светодиодов. Наконец, здесь будет рассмотрено использование программатора ST-Link V2 и проверка устройства.

Питание

Печатная плата STM32F103C8T6, также известная как плата Blue Pill, имеет ряд выводов, в том числе для подключения питания. Нет необходимости использовать все такие подключения одновременно. На самом деле лучше всего использовать только один набор выводов питания. Чтобы прояснить этот момент, давайте начнем с изучения вариантов электропитания. Рисунок 3.1 иллюстрирует выводы по краям печатной платы, включая питание.

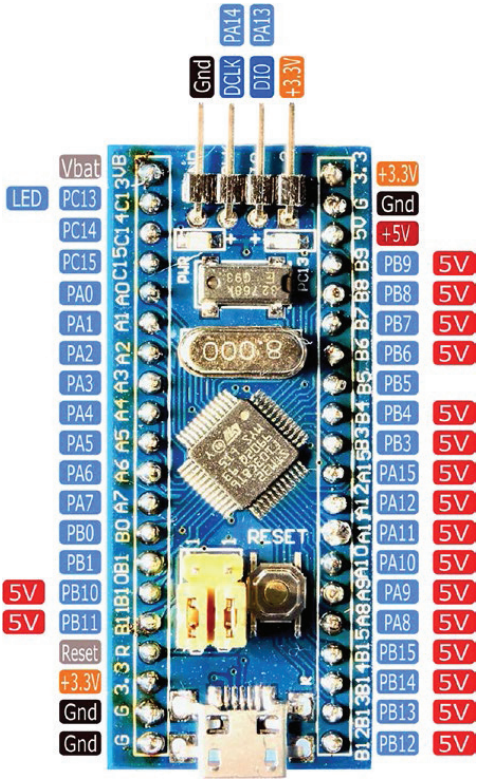


Рис. 3.1. Выводы питания и GPIO на плате STM32F103C8T6 Blue Pill. Питание может подаваться на выводы +5V, +3.3V или через кабель USB с соответствующим напряжением. Контакты, помеченные как «5V» (без знака «плюс»), представляют собой входы сигналов, устойчивые к напряжению 5 В. Контакты, содержащие в обозначении знак плюс, предназначены для поступления питания

Четыре вывода на верхнем торце платы используются для программирования устройства. Обратите внимание, что вывод для программирования, обозначенный DIO, также может быть выводом общего назначения (GPIO) PA13. Аналогично вывод DCLK может использоваться как GPIO PA14. Прочитав эту книгу, вы узнаете, как настраивать различные функции выводов STM32.

Обратите внимание, что входное напряжение питания у разъема программирования составляет +3.3 В. Электрически этот вывод такой же, как и любой другой, имеющий маркировку «+3.3V» на печатной плате (выделены светло-оранжевым цветом).

Стабилизатор +3.3 В

Микросхема STM32F103C8T6 рассчитана на напряжение питания от 2 до 3.3 В. На нижней стороне печатной платы Blue Pill имеется крошечный стабилизатор +3.3 В с надписью рядом «U1» (см. рис. 3.2). В моем устройстве использовался стабилизатор с SMD-кодом 4A2D, что означает принадлежность к серии XC6204. Стабилизатор на вашей плате может отличаться.

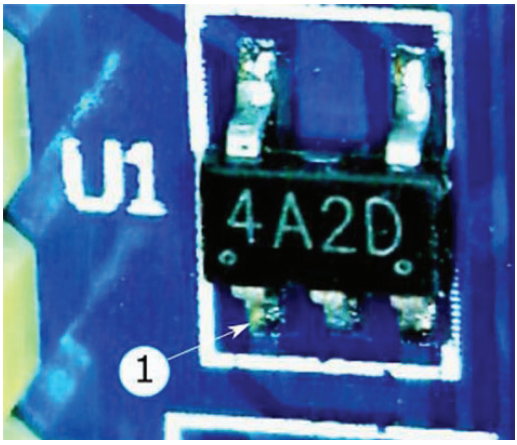


Рис. 3.2. Стабилизатор +3.3 В на нижней стороне печатной платы. Маркирован вывод 1 микросхемы 4A2D (серия XC6204)

На официальной схеме этой платы в качестве стабилизатор указан стабилизатор RT9193-33, поддерживающий ток 300 мА (см. https://stm32-base.org/assets/pdf/boards/original-schematic-STM32F103C8T6-Blue_Pill.pdf). Очевидно, моя печатная плата является клоном с более дешевой микросхемой стабилизатора. Стабилизатор серии XC6204 ограничен током 150 мА. Если вы не знаете особенностей вашего устройства, безопаснее всего предположить, что предел тока составляет 150 мА.

Энергопотребление микроконтроллера будет рассмотрено в следующей главе. Но для ориентировки было установлено, что программа мигания светодиодом в поставляемом устройстве использует ток около 30 мА (измеренный при входном питании +5 В). Эта величина включает в себя небольшой дополнительный ток, используемый самим регулятором.

В таблице данных STM32F103C8T6 указано максимальное потребление тока около 50 мА. Измерения, приведенные там, получены при работе с внешним кварцевым резонатором и всеми включенными периферийными устройствами при работе в режиме «run mode» на частоте 72 МГц. Вычитая 50 из максимального значения стабилизатора, принимаемого равным 150, вы получаете текущий запас около 100 мА от стабилизатора +3.3 В. Всегда приятно знать, каковы пределы!

USB-питание и вывод +5 В

При питании от USB-кабеля питание поступает через разъем micro-USB B. Этот источник выдает питание +5 В, регулируемое стабилизатором до уровня +3.3 В, необходимого микроконтроллеру. В правом верхнем углу рис. 3.1 есть вывод с надписью «+5 В» (со знаком плюс), который также можно использовать в качестве входа питания. С этого вывода напряжение идет на тот же вход стабилизатора, что и с разъема USB.

Из-за низких требований к току микроконтроллера вы также можете запитать устройство от адаптера последовательного порта USB-TTL. Многие USB-TTL-адаптеры имеют вывод +5 В, который может питать вашу плату. Чтобы быть уверенным, проверьте характеристики вашего адаптера.

Будьте осторожны, не подключайте USB-кабель и не подавайте одновременно +5 В[16]. Это может привести к повреждению вашего настольного компьютера/ноутбука через USB-кабель. Например, если напряжение вашего источника +5 В немного выше, вы будете подавать ток в USB-схему настольного компьютера[17].

Питание +3.3 В

Если у вас есть источник питания +3.3 В, можете оставить входы +5 В неподключенными. Подключите источник питания +3.3 В непосредственно ко входу +3.3 В (убедитесь, что USB-кабель отсоединен!). При подаче питания на вход +3.3 В вы подключаете питание к клемме VOUT стабилизатора (см. рис. 3.3). В этом случае на VIN регулятора не поступает напряжение 5 В. Вывод разрешения CE также подключен к VIN, но, когда VIN не подключен, вывод CE заземляется конденсатором. Низкий уровень CE заставляет регулятор отключить свои внутренние подсистемы. Иными словами,

[16] В этом отличие STM32 от Arduino, где внешнее питание можно подавать, не выключая USB (что, конечно, намного удобнее при отладке схемы, когда требуется «общение» с компьютером). К выводу +3.3V подключать внешнее питание при включенном USB также запрещается, см. далее. Заметьте, что при включенном USB можно использовать вывод +5V для питания внешних 5-вольтовых устройств, не устанавливая для них отдельного стабилизатора.

[17] А если наоборот, то ток пойдет в схему вашего внешнего источника. И неизвестно, кто кого победит в этом своеобразном «перетягивании каната», – факт, что один из источников в конце концов перегорит, и хорошо, если это будет дешевый адаптер, а не дорогущий ноутбук. Можно констатировать, что на платах STM32F103C8T6 с разводкой питания явный недосмотр, если учесть, что они рассчитаны в том числе и на любителей (см. далее многочисленные напоминания об использовании только одного источника).

стабилизатор отключается автоматически, когда на 5-вольтовом входе нет напряжения.

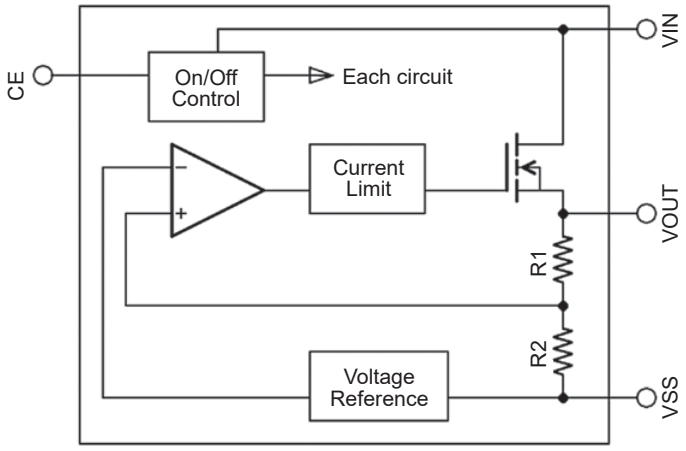


Рис. 3.3. Блок-схема стабилизатора напряжения 3.3 В

Однако в делитель напряжения на выходе стабилизатора поступает небольшой ток. Этот ток будет течь от +3.3 В к земле через внутренние сопротивления R1 и R2. Эти сопротивления достаточно велики, и ток незначителен. Но следует помнить об этом при измерении потребления от батареек сверхмалой мощности.

Не подавайте одновременно +5 и +3.3 В. Это может привести к повреждению стабилизатора или рабочего компьютера при подключенном USB-кабеле. Используйте только один источник питания!

Правило одного источника питания

Повторим общий совет – использовать только один источник питания. Следует еще раз подчеркнуть, что подключение к печатной плате Blue Pill более чем одного источника питания может привести к повреждению.

Это, как правило, очевидно для входов питания +3.3 и +5 В. Однако о чем можно легко забыть – так это о USB-кабеле. Учтите, что питание может поступать от адаптера последовательного порта USB-TTL, программатора ST-Link V2 или кабеля USB. Будьте осторожны при изменении схемы электропитания, особенно при переключении с программирования устройства на обычную конфигурацию питания.

В некоторых приложениях может потребоваться использование дополнительных источников питания, например при питании двигателей или реле. В этих случаях вы должны подавать необходимое питание на внешние цепи, но не на печатную плату контроллера. Общие соединения должны иметь только сигнальные линии и общий провод. Если это неясно, просто затвердите правило одного источника питания.

Общий провод

Обратный провод цепи питания (обычно отрицательный вывод) называется общим проводом[18]. На рис. 3.1 он отмечен черным цветом и обозначением «GND». Все общие выводы электрически соединены между собой внутри микросхемы, т. е. они взаимозаменяемы.

Сброс (RESET)

На печатной плате также имеется кнопка с надписью «RESET» и вывод с подписью «R» (на рис. 3.1 обозначен серым цветом). Этот вывод позволяет внешней цепи выполнить сброс МК, если это необходимо. На рис. 3.4 показана схема подключения кнопки Reset к контроллеру.

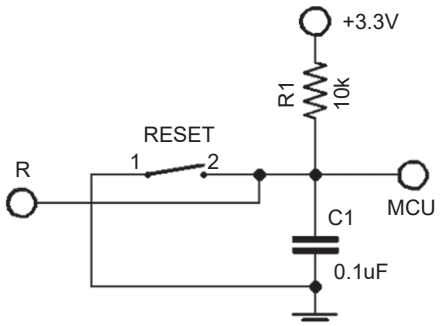


Рис. 3.4. Схема сброса STM32F103C8T6.
Вывод «R» находится на краю печатной платы

Быстрая проверка

Вероятно, вы уже протестировали свое устройство, но, если вы еще этого не сделали, сделайте это сейчас. Самый безопасный и простой способ – использовать USB-кабель с разъемом micro-USB. Подключите кабель к источнику питания USB, который не обязательно должен быть компьютером. После включения ваше устройство должно мигнуть. Если нет, то попробуйте нажать кнопку Reset. Также убедитесь, что перемычки boot-0 и boot-1 расположены, как показано на рис. 3.1 (обе перемычки должны быть расположены со стороны с надписью «0»).

[18] Автор называет общий (отрицательный) провод питания схемы и соответствующие выводы компонентов привычным для англоязычных текстов словом *ground*, т. е. «заземляющим». Так принято, но в общем случае называть «общий провод» «заземлением» безграмотно и может привести к фатальным последствиям для схемы в случае возникновения путаницы с настоящей электротехнической землей, с которой часто соединены корпуса стационарных компьютеров, измерительных приборов или сетевых источников питания. Сохранив общепринятое обозначение GND, мы в дальнейшем везде заменяем авторский термин *ground* на «общий» или «общий провод».

Есть два встроенных светодиода. Светодиод слева указывает на подачу питания (у меня он желтый, но у вас может быть иначе). Светодиод справа активируется программно через вывод GPIO PC13 (у меня он красный; опять же у вас может отличаться).



Некоторые пользователи сообщают, что разъем USB легко отламывается от печатной платы. Будьте осторожны, вставляя конец кабеля micro-USB.

Если у вас нет подходящего USB-кабеля, можете опробовать устройство, если подать +5 или +3.3 В к соответствующему разъему, как обсуждалось выше. Подойдет даже пара щелочных гальванических элементов, включенных последовательно на питание +3 В (напомним, что этот МК работает от 2 до 3.3 В).

На рис. 3.5 показано питание устройства от контакта +3.3 В разъема в верхней части печатной платы, к которому подключается программатор. Будьте осторожны при использовании зажимов типа «крокодил» и следите за тем, чтобы они не замыкали на другие контакты. Перемычки DuPont более безопасны.

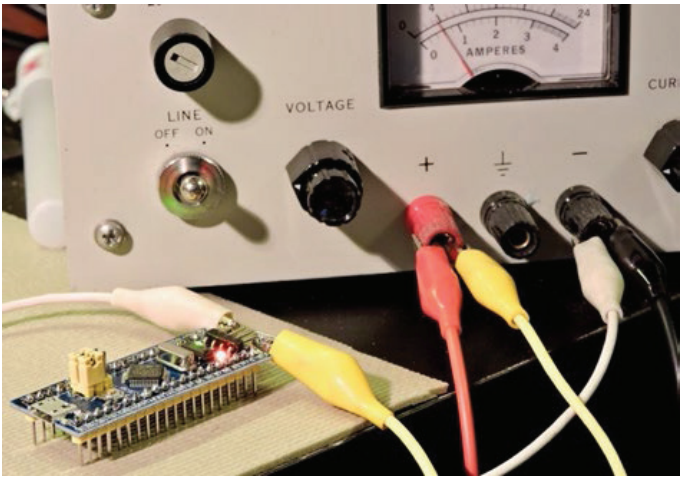


Рис. 3.5. STM32F108C8T6 мигает при питании от блока питания HP 6284A через вывод верхнего разъема (+3.3 В).

ST-Link V2

Следующий пункт, который следует отметить в списке действий к этой главе, – это подключить программатор и запустить утилиту *st-info*.

Когда вы получите программатор, вы, скорее всего, получите только четыре провода DuPont с гнездовыми концами. Это не очень удобно, но работает, если правильно выполнить подключение. Если вы часто переключаете устройства для программирования, вам понадобится сделать для этой цели специальный кабель. Схема подключения программатора показана на

рис. 3.6. В продаже доступны разные модели программатора, использующие разные выводы и типы проводов.

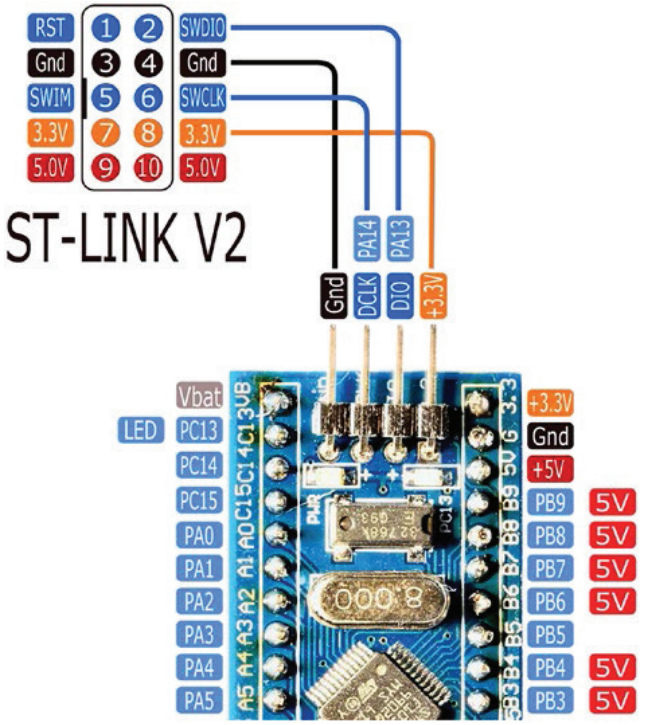


Рис. 3.6. Подключение программатора ST-Link V2 к устройству STM32F103C8T6. Проверьте соединения на имеющемся у вас устройстве: программаторы ST-Link могут отличаться

Подключив программатор согласно рис. 3.6, проверьте перемычки boot-0 и boot-1, расположенные рядом с кнопкой Reset. Они должны выглядеть так, как показано на рис. 3.1 (обе перемычки расположены со стороны с отметкой «0»).

Подключите программатор ST-Link V2 к порту USB или используйте удлинитель USB[19]. Как только вы это сделаете, индикатор питания должен немедленно загореться. Кроме того, светодиод PC13 должен мигать, если в вашем устройстве еще есть загруженная программа мигания. Рисунок 3.7 иллюстрирует настройку.

На компьютере запустите команду `st-info` следующим образом:

```
$ st-info --probe
Found 1 stlink programmers
```

[19] Не забывайте, что нельзя подавать питание от программатора на плату, подключенную к другому источнику питания!

```
  serial: 493f6f06483f53564554133f
openocd: "\x49\x3f\x6f\x06\x48\x3f\x53\x56\x45\x54\x13\x3f"
  flash: 131072 (pagesize: 1024)
  sram: 20480
  chipid: 0x0410
  descr: F1 Medium-density device
```

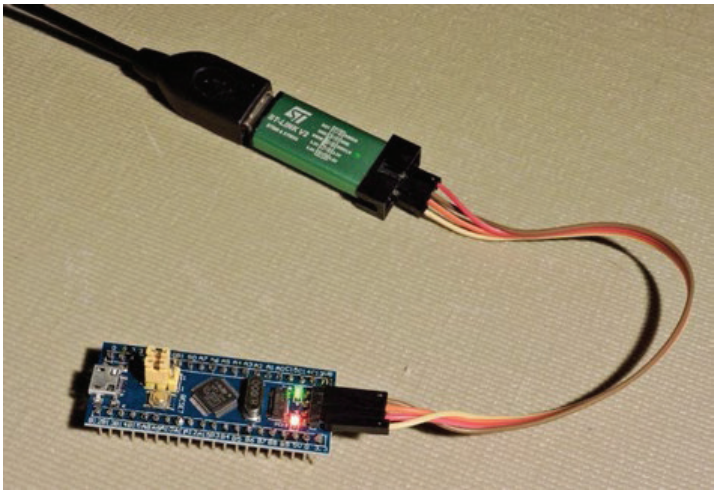


Рис. 3.7. Программатор ST-Link V2 с удлинительным USB-кабелем, подключенный к STM32F103C8T6 с помощью проводов DuPont

Команда `st-info` должна найти программатор ST-Link V2 и подключенный к нему STM32F103C8T6. Успешный результат должен быть похож на показанный у меня. Обратите внимание, что указывается серийный номер контроллера и объем SRAM (20 480 ~ 20 Кбайт). Указанный здесь объем флеш-памяти составляет 131 072 ~ 128 Кбайт, но также вы можете увидеть 64 Кбайта. В любом случае контроллер, вероятно, будет поддерживать 128 Кбайт.



Пользователям Windows, которые не могут настроить usbipd для работы с WSL, возможно, вместо этого потребуется использовать утилиту Windows, предоставленную *st-microelectronics.com* (см. раздел «Утилита Windows STM32 ST-LINK» в конце этой главы).

Утилита st-flash

Давайте теперь посмотрим, как вы можете использовать утилиту *st-flash* для чтения (и сохранения), записи (программирования) или стирания данных на устройстве STM32.

Чтение STM32

Сохранение содержимого памяти устройства в файл образа позволит вам восстановить исходную программу, если она понадобится вам позже. Сле-

дующий пример считывает данные из флеш-памяти вашего устройства, начиная с адреса 0x80000000, и сохраняет данные размером 0x1000 (4 Кбайта) в файл с именем *save.img* (если не указано иное, в этой книге для обозначения шестнадцатеричных чисел будет использоваться соглашение о префиксе 0x из языка C):

```
$ st-flash read ./saved.img 0x8000000 0x1000
st-flash 1.3.1-9-gc04df7f-dirty
2017-07-29T09:54:02 INFO src/common.c: Loading device parameters....
2017-07-29T09:54:02 INFO src/common.c: Device connected is: \
    F1 Medium-density device, id 0x20036410
2017-07-29T09:54:02 INFO src/common.c: SRAM size: 0x5000 bytes (20 KiB), \
    Flash: 0x20000 bytes (128 KiB) in pages of 1024 bytes
```

Чтобы проверить сохраненный файл образа содержимого памяти, используйте утилиту *hexedit* (возможно, вам придется использовать менеджер пакетов, чтобы установить ее на рабочий стол):

```
$ hexedit saved.img
```

Чтобы получить справку, находясь в утилите, нажмите F1. Вы можете использовать <**Ctrl+V**> для прокрутки страницы вниз. Используйте <**Ctrl+C**> для выхода в командную строку.

Просматривая файл образа, вы должны увидеть шестнадцатеричное содержимое примерно до смещения 0x4EC. С этого момента вы можете увидеть шестнадцатеричные байты 0xFF, представляющие собой незаписанную (стертую) флеш-память. Если вы не видите ничего, кроме нулей или байтов 0xFF, значит, что-то не так. Обязательно включите (в настройках утилиты) префикс 0x в аргументы адреса и размера команды.

Если вы не видите группу байтов 0xFF в конце сохраненного содержимого, возможно, вам нужно сохранить область памяти большего размера.

Запись flash

Запись во флеш-память, конечно, обратна чтению. Сохраненный образ памяти можно «прошить» с помощью подкоманды записи с помощью *st-flash*. Обратите внимание, что мы опускаем аргумент размера для этой команды. В этом примере мы записываем прочитанное обратно по тому же адресу:

```
$ st-flash write ./saved.img 0x8000000
st-flash 1.3.1-9-gc04df7f-dirty
2017-07-29T10:00:39 INFO src/common.c: Loading device parameters....
2017-07-29T10:00:39 INFO src/common.c: Device connected is: \
    F1 Medium-density device, id 0x20036410
2017-07-29T10:00:39 INFO src/common.c: SRAM size: 0x5000 bytes (20 KiB), \
    Flash: 0x20000 bytes (128 KiB) in pages of 1024 bytes
2017-07-29T10:00:39 INFO src/common.c: Ignoring 2868 bytes of 0xff at end of file
```

```
2017-07-29T10:00:39 INFO src/common.c: Attempting to write 1228 (0x4cc) \
    bytes to stm32 address: 134217728 (0x8000000)
Flash page at addr: 0x08000400 erased
2017-07-29T10:00:39 INFO src/common.c: Finished erasing 2 pages of 1024 (0x400) bytes
2017-07-29T10:00:39 INFO src/common.c: Starting Flash write for VL/F0/F3 core id
2017-07-29T10:00:39 INFO src/flash_loader.c: Successfully loaded flash loader in sram
  1/1 pages written
2017-07-29T10:00:39 INFO src/common.c: Starting verification of write complete
2017-07-29T10:00:39 INFO src/common.c: Flash written and verified! jolly good!
```

Эта операция восстановит сохраненный файл программы мигания во флеш-памяти вашего устройства. Светодиод может начать мигать сразу. В противном случае нажмите кнопку Reset (Сброс), чтобы принудительно перезагрузить устройство.

Стирание flash

Могут быть случаи, когда вы захотите принудительно полностью стереть данные с устройства. Возможно, вы собираетесь подарить свое устройство другу и хотите стереть свой последний эксперимент:

```
$ st-flash erase
st-flash 1.3.1-9-gc04df7f-dirty
2017-07-29T10:06:17 INFO src/common.c: Loading device parameters....
2017-07-29T10:06:17 INFO src/common.c: Device connected is: \
    F1 Medium-density device, id 0x20036410
2017-07-29T10:06:17 INFO src/common.c: SRAM size: 0x5000 bytes (20 KiB), \
    Flash: 0x20000 bytes (128 KiB) in pages of 1024 bytes
Mass erasing
```

После завершения этой операции ваше устройство должно быть полностью стерто. Светодиод должен перестать мигать. Ради интереса попробуйте восстановить программу, которую вы сохранили и тем самым сбросили настройки.

Утилита Windows STM32 ST-LINK

Если вам не удалось выполнить раздел «Установка Windows usbipd» из главы 2 или вы предпочитаете вместо этого использовать инструмент с графическим интерфейсом, то можно использовать утилиту STM32 ST-LINK.

Функция Info

В разделе «Утилита STM32 ST-LINK» главы 2 показано, как получить информацию об устройстве. Для этого соедините программатор STM32 с устройством, запустите утилиту и подключитесь к нему.

В следующих подразделах предполагается, что вы подключились к устройству через USB-порт.

Функция чтения Read

После запуска утилиты и подключения к устройству утилита с графическим интерфейсом отобразит содержимое флеш-памяти в прокручиваемом окне. Рисунок 2.1 иллюстрирует пример такого чтения.

Чтобы сохранить прочитанное содержимое в файл, выберите в меню **File** (Файл) > **Save as…** (Сохранить как…) (<**Ctrl+S**>). Откроется диалоговое окно, в котором вы можете выбрать существующий файл или ввести имя и расширение в поле, где написано **File name** (Имя файла). Формат файла выбирается по названию предоставленного расширения файла и должен быть одним из следующих: *.bin*, *.hex*, *.srec* или *.s19*. Чтобы сохранить файл образа в бинарной форме, используйте *.bin*. В этом примере для файла, сохраненного на рабочем столе, используется имя *saved.bin*.

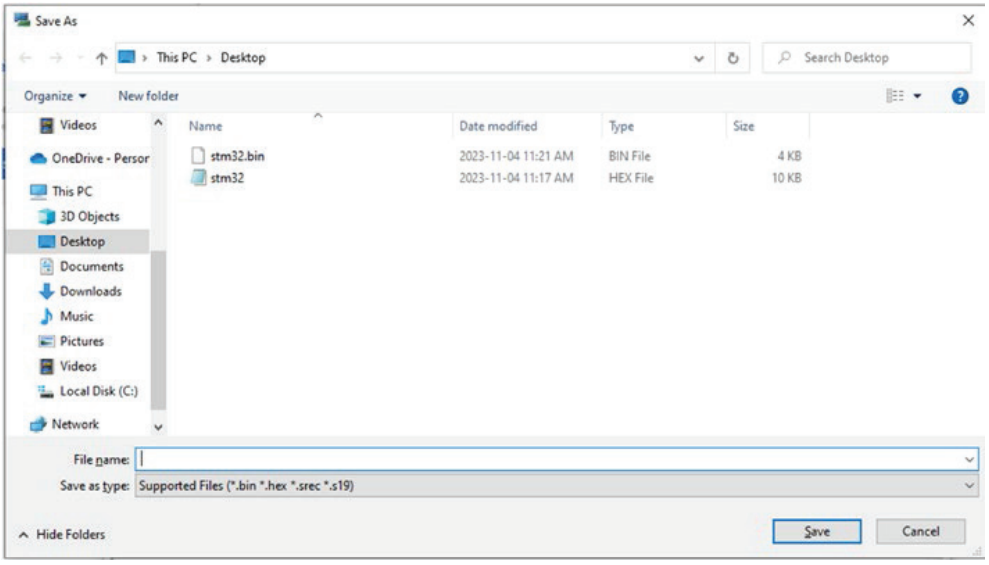


Рис. 3.8. Диалоговое окно **File** > **Save as…**

Вернитесь в окно терминала Ubuntu, чтобы мы могли использовать в нем инструмент шестнадцатеричного редактирования. Поскольку вы работаете в среде WSL Linux, необходимо найти файл рабочего стола на стороне Ubuntu Linux. Предполагая, что ваш рабочий стол находится на диске C, файлы Windows будут найдены в смонтированном каталоге */mnt/c*. В моем примере имя пользователя – 19053, поэтому для шестнадцатеричного редактирования сохраненного файла введите

```
$ hexedit /mnt/c/Users/19053/Desktop/stm32.bin
```

Замените здесь «19053» своим именем пользователя. Дополнительную информацию о *hexedit* см. в разделе «Чтение STM32» этой главы.

Функция записи Write

Программирование устройства STM32 можно выполнить, записав в него файл образа. Используя заданный файл *.img* или *.bin*, загрузите его в утилиту, выбрав в меню **File** (Файл) > **Open** (Открыть) и выберите файл. После загрузки содержимое памяти будет отмечено цветом, это указывает на то, что оно еще не записано.

Выполните этот шаг, выбрав в меню **Target** (Целевое устройство) > **Program & Verify** (Программировать и проверить) (<**Ctrl+P**>). Здесь мы будем использовать параметры по умолчанию. Нажмите **Start** (Пуск), чтобы начать. Если вы оставили отмеченным флажок **Reset after programming** (Сброс после программирования), а записанный образ представлял собой программу мигания, то устройство должно начать мигать после завершения программирования. На рис. 3.9 показаны дополнительные параметры, поддерживаемые утилитой.

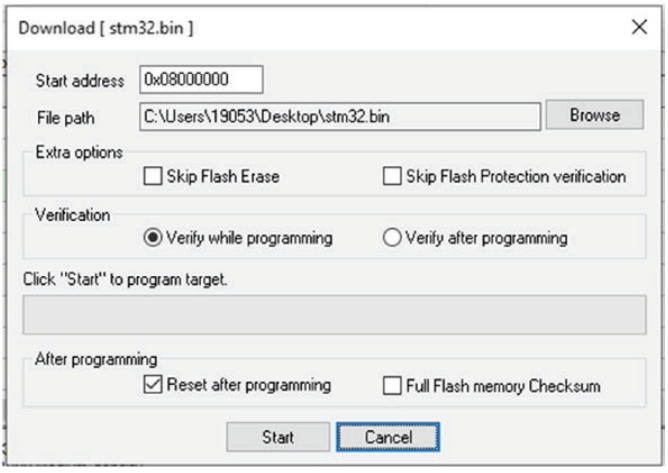


Рис. 3.9. Параметры диалогового окна **Target** > **Program & Verify**

При программировании файла образа, скомпилированного на стороне Ubuntu Linux (в WSL), вам необходимо найти файл в Ubuntu. Чтобы найти файл, вы можете использовать файловый менеджер Windows, как показано на рис. 3.10. Прокрутите левую сторону, пока не увидите пункт **Linux**, и нажмите на него. Это покажет один или несколько экземпляров Linux, которые вы включили. В моем примере у меня установлено два экземпляра Ubuntu Linux.

Углубившись в проводник (в этом примере я зашел в «Ubuntu»), вы можете найти и определить имя файла. Перейдя к папке, содержащей файл образа, вы сможете найти и определить путь к нему. Например, пользователь под именем Arie может найти образ *miniblink*, расположенный по адресу

\\wsl.localhost\Ubuntu\home\arie\stm32f103c8t6\miniblink\miniblink.bin.

Из этого мы узнаем, что файлы WSL Linux расположены по адресу

\\wsl.localhost\<Linux_Instance>\<Linux_Pathname>.

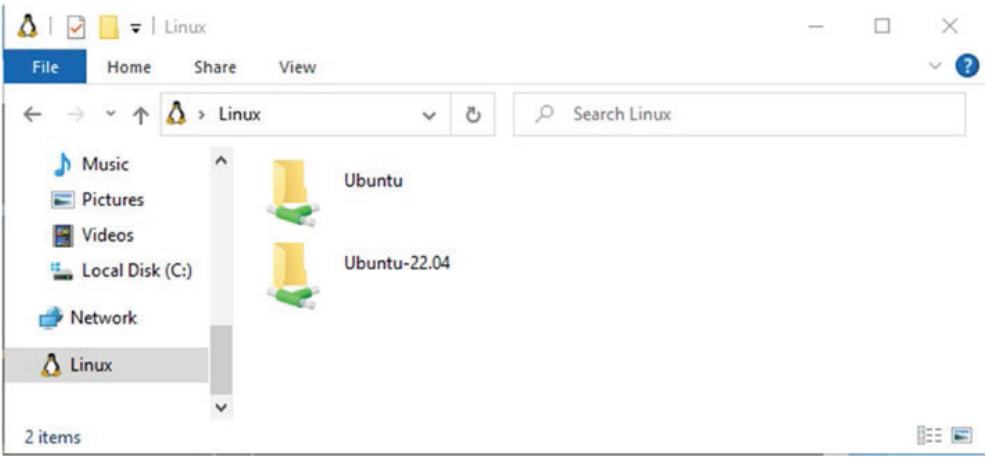


Рис. 3.10. Пример отображения Linux в файловом менеджере Windows

Пункт меню **File** (Файл) > **Open** (Открыть) утилиты STM32 *ST-LINK* позволит вам просмотреть файл в выбранном вами экземпляре Linux и найти файл так же, как в проводнике. Это удобнее, чем вводить полный путь к файлу образа.

Функция стирания Erase

Утилита также способна стирать флеш-память устройства. В меню выберите **Target** (Целевое устройство) > **Erase Chip** (Стереть чип) (<**Ctrl+E**>). Вам будет предложено подтвердить операцию. После подтверждения данные с устройства будут удалены. Ради интереса вы после этого можете записать образ, сохраненный в предыдущем разделе, обратно на устройство, чтобы восстановить программу.

Итоги

В этой главе в первую очередь представлена информация о вариантах электропитания. Это очень важно, поскольку сбои в этой области могут привести к необратимому повреждению. Далее вы подключаете устройство и убеждаетесь, что оно работает с прилагаемой программой мигания. Затем с помощью команды `st-link` (утилиты Windows *ST-LINK*) вы подтвердили, что программатор и устройство, которое нужно программировать, работают. Наконец, вы узнали, как использовать утилиту *st-flash* для чтения, записи и очистки флеш-памяти устройства.

Дополнительные источники

1. https://stm32-base.org/assets/pdf/boards/original-schematic-STM32F103C8T6-Blue_Pill.pdf (дата доступа 01.09.2024).

Глава 4 • GPIO

В этой главе мы собираемся использовать библиотеку libopencm3 для создания программы мигания Blink в исходном коде. Этот пример программы демонстрирует настройку и использование выводов общего назначения GPIO (General Purpose Input/Output). Представленная программа представляет собой слегка модифицированную версию примера программы libopencm3 под названием *miniblink*. Программа имеет иной период мигания, чтобы было очевидно, что выполняется именно ваш недавно прошитый код. После сборки и запуска этой программы мы обсудим интерфейс прикладного программирования API GPIO, предоставляемый libopencm3.

Сборка miniblink

Перейдите в подкаталог *miniblink*, как показано, и введите `make`:

```
$ cd ~/stm32f103c8t6/miniblink
$ make
gmake: Nothing to be done for 'all'.
```

Если вы видите предыдущее сообщение, возможно, вы уже произвели сборку всех проектов верхнего уровня (в этом нет ничего плохого). Однако если вы внесли изменения в файлы исходного кода, команда `make` должна автоматически обнаружить это и пересобрать затронутые компоненты. Здесь мы просто хотим принудительно пересобрать проект miniblink. Для этого введите `make clobber` в каталоге проекта, а затем `make`, как показано здесь:

```
$ make clobber
rm -f *.o *.d generated.* miniblink.o miniblink.d
rm -f *.elf *.bin *.hex *.srec *.list *.map
$ make
...
arm-none-eabi-size miniblink.elf
text data bss  dec hex filename
696     0   0  696 2b8 miniblink.elf
arm-none-eabi-objcopy -Obinary miniblink.elf miniblink.bin
```

Сделав это, вы увидите несколько длинных командных строк, выполняемых для компиляции и связывания вашего исполняемого файла с именем *miniblink.elf*. Однако для прошивки вашего устройства нам также понадобится файл образа. На последнем этапе сборки показано, как специфичная для ARM утилита *objcopy* используется для преобразования *miniblink.elf* в файл образа *miniblink.bin*.

Непосредственно перед последним шагом вы можете видеть, что команда `size`, специфичная для ARM, выгрузила размеры разделов данных и текста

вашей программы. Наша программа *miniblink* занимает всего 696 байт во флеш-памяти (см. позицию `text`) и не использует выделенную SRAM (позиция `data`). Несмотря на это, для стека вызовов функций по-прежнему используется SRAM.

Прошивка miniblink

Снова воспользовавшись фреймворком `make`, мы теперь можем прошить устройство новым образом программы. Подключите программатор ST-Link V2, проверьте соединения и выполните следующее:

```
$ make flash
/usr/local/bin/st-flash write miniblink.bin 0x8000000
st-flash 1.3.1-9-gc04df7f-dirty
2017-07-30T12:57:56 INFO src/common.c: Loading device parameters....
2017-07-30T12:57:56 INFO src/common.c: Device connected is: \
    F1 Medium-density device, id 0x20036410
2017-07-30T12:57:56 INFO src/common.c: SRAM size: 0x5000 bytes (20 KiB),\
    Flash: 0x20000 bytes (128 KiB) in pages of 1024 bytes
2017-07-30T12:57:56 INFO src/common.c: Attempting to write 696 (0x2b8) \
    bytes to stm32 address: 134217728 (0x8000000)
Flash page at addr: 0x08000000 erased
2017-07-30T12:57:56 INFO src/common.c: Finished erasing 1 pages of 1024 (0x400) bytes
...
2017-07-30T12:57:57 INFO src/common.c: Flash written and verified! jolly good!
```

Как только это будет сделано, ваше устройство должно автоматически перезагрузиться и запустить программу мигания *miniblink*. При использовании измененных постоянных времени вы должны увидеть, что оно теперь часто мигает с рабочим циклом в основном 70/30. Программа мигания вашего устройства, поставляемая в комплекте, вероятно, вместо этого использовала более медленный рабочий цикл 50/50. Если ваша схема мигания несколько отличается от описанной, не волнуйтесь. Важным моментом является то, что вы прошили и запустили другую программу.

Эта программа не использует тактовую частоту процессора, управляемую кристаллом. Она использует внутренний резисторно-конденсаторный (RC) генератор тактовой частоты. По этой причине одно устройство может мигать немного быстрее или медленнее, чем другое.



Если вместо этого вы используете утилиту Windows STM32 *ST-LINK*, следуйте инструкциям, приведенным в разделе «Функция записи Write» в главе 3 вместо `make flash`.

Исходный код miniblink.c

Давайте теперь рассмотрим исходный код программы *miniblink*, которую вы только что запустили. Если вы еще не находитесь в подкаталоге *miniblink*, перейдите туда сейчас:

```
$ cd ~/stm32f103c8t6/miniblink
```

В этом подкаталоге вы должны найти файл исходного кода программы *miniblink.c*. В листинге 4.1 показан текст программы.

Листинг 4.1. Файл *miniblink.c*

```
0019: #include <libopencm3/stm32/rcc.h>
0020: #include <libopencm3/stm32/gpio.h>
0021:
0022: static void
0023: gpio_setup(void) {
0024:
0025:   /* Enable GPIOC clock. */
0026:   rcc_periph_clock_enable(RCC_GPIOC);
0027:
0028:   /* Set GPIO8 (in GPIO port C) to 'output push-pull'. */
0029:   gpio_set_mode(GPIOC,GPIO_MODE_OUTPUT_2_MHZ,
0030:   GPIO_CNF_OUTPUT_PUSHPULL,GPIO13);
0031: }
0032:
0033: int
0034: main(void) {
0035:   int i;
0036:
0037:   gpio_setup();
0038:
0039:   for (;;) {
0040:     gpio_clear(GPIOC,GPIO13); /* LED on */
0041:     for (i = 0; i < 1500000; i++) /* Wait a bit. */
0042:       __asm__("nop");
0043:
0044:     gpio_set(GPIOC,GPIO13); /* LED off */
0045:     for (i = 0; i < 500000; i++) /* Wait a bit. */
0046:       __asm__("nop");
0047:   }
0048:
0049:   return 0;
0050: }
```



Номера строк, отображаемые слева, не являются частью исходного файла. Они используются только для удобства ориентации по тексту.

Структура программы достаточно проста. Она состоит из следующих основных разделов.

1. Основная функция программы `main`, объявленная в строках 33–50. Обратите внимание, что, в отличие от программ POSIX, функция `main` не имеет аргументов `argc` или `argv`.
2. В основной программе вызывается функция `gpio_setup()` для выполнения некоторой инициализации.

3. Строки 39–47 образуют бесконечный цикл, в котором светодиод включается и выключается. Обратите внимание, что оператор `return` в строке 49 никогда не выполняется и предоставляется только для того, чтобы компилятор не показывал ошибки.

Даже в этой простой программе есть что обсудить. Как мы увидим позднее, этот пример программы по умолчанию привязан к частоте процессора, поскольку она не определена. Мы обсудим этот момент в свое время.

Давайте сначала разберемся с простыми вещами. На рис. 4.1 показано, как мигающий светодиод подключен к микроконтроллеру. На этой схеме мы видим, что питание на светодиод поступает от источника +3.3 В через ограничительный резистор R1. Чтобы замкнуть цепь и обеспечить протекание тока, вывод GPIO PC13 должен подключать светодиод к общему проводу. Вот почему в комментарии к строке 40 говорится, что светодиод включается, хотя происходит вызов функции `gpio_clear()`. В строке 44 используется `gpio_set()` для выключения светодиода. Эта инвертированная логика используется просто из-за способа подключения светодиода[20].

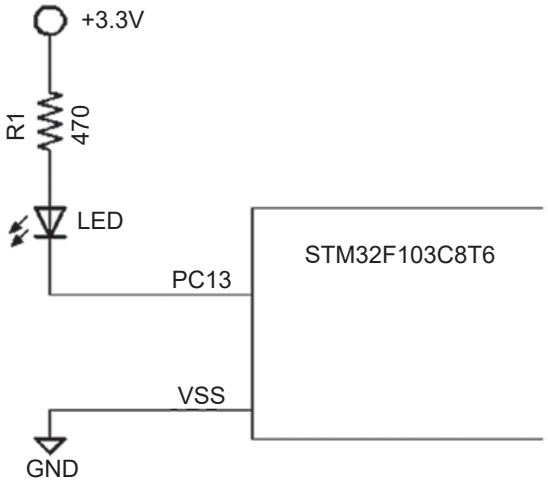


Рис. 4.1. Подключение светодиода к выводу PC13 на плате Blue Pill

Посмотрите еще раз на эти вызовы функций:

```
gpio_clear(GPIOC,GPIO13); /* LED on */
...
gpio_set(GPIOC,GPIO13); /* LED off */
```

[20] Автор имеет в виду, что функция с названием `gpio_clear` (очистить вывод) устанавливает уровень напряжения, равный нулю, что обычно означает выключение чего-то, подключенного к выводу, (а `gpio_set` – установка вывода – соответственно, включение), тогда как светодиод работает в противоположной логике.

Обратите внимание, что эти два вызова требуют двух аргументов, а именно:

1) название порта GPIO,
2) номер вывода данного порта.

Если вы привыкли к среде Arduino, вы ожидали увидеть что-то вроде следующего:

```
int ledPin = 13; // LED on digital pin 13
digitalWrite(ledPin, LOW);
...
digitalWrite(ledPin, HIGH);
```

В мире, отличном от Arduino, вы обычно работаете напрямую с портом и выводом. В библиотеке libopencm3 вы указываете, очищаете ли вы бит или устанавливаете его, в зависимости от имени функции (`gpio_clear()` или `gpio_set()`). Вы также можете выполнить переключение в противоположное состояние с помощью `gpio_toggle()`. Наконец, можно читать и записывать полный набор выводов по данному порту, используя `gpio_port_read()` и `gpio_port_write()` соответственно.

API-интерфейс GPIO

Это хороший момент для обсуждения функций libopencm3, предназначенных для использования выводов GPIO. Первое, что нужно сделать, – это включить соответствующие файлы заголовков, как показано ниже:

```
#include <libopencm3/stm32/rcc.h>
#include <libopencm3/stm32/gpio.h>
```

Файл *rcc.h* содержит определения, необходимые для включения тактов GPIO. Файл *gpio.h* необходим для остальных функций:

```
void gpio_set(uint32_t gpioport, uint16_t gpios);
void gpio_clear(uint32_t gpioport, uint16_t gpios);
uint16_t gpio_get(uint32_t gpioport, uint16_t gpios);
void gpio_toggle(uint32_t gpioport, uint16_t gpios);
uint16_t gpio_port_read(uint32_t gpioport);
void gpio_port_write(uint32_t gpioport, uint16_t data);
void gpio_port_config_lock(uint32_t gpioport, uint16_t gpios);
```

Во всех приведенных функциях аргумент `gpioport` может быть одним из макроопределений из табл. 4.1 (на других платформах STM32 могут быть дополнительные порты). Одновременно можно указать только один порт.

В функциях GPIO libopencm3 один или несколько битов порта могут быть установлены или очищены одновременно.

В табл. 4.2 перечислены поддерживаемые имена GPIO-выводов порта. Обратите также внимание на макроопределение с именем GPIO_ALL.

Порт	Описание
GPIOA	GPIO port A
GPIOB	GPIO port B
GPIOC	GPIO port C

Таблица 4.1. Наименования GPIO для STM32F103C8T6 в библиотеке libopencm3

Обозначение	Определение	Описание
GPIO0	(1 << 0)	бит 0
GPIO1	(1 << 1)	бит 1
GPIO2	(1 << 2)	бит 2
GPIO3	(1 << 3)	бит 3
GPIO4	(1 << 4)	бит 4
GPIO5	(1 << 5)	бит 5
GPIO6	(1 << 6)	бит 6
GPIO7	(1 << 7)	бит 7
GPIO8	(1 << 8)	бит 8
GPIO9	(1 << 9)	бит 9
GPIO10	(1 << 10)	бит 10
GPIO11	(1 << 11)	бит 11
GPIO12	(1 << 12)	бит 12
GPIO13	(1 << 13)	бит 13
GPIO14	(1 << 14)	бит 14
GPIO15	(1 << 15)	бит 15
GPIO_ALL	0xffff	все от 0 до 15

Таблица 4.2. Обозначения выводов GPIO порта в libopencm3

Примером для `GPIO_ALL` может быть следующее:

```
gpio_clear(PORTB,GPIO_ALL); // clear all PORTB pins
```

Особенностью серии STM32, которую поддерживает libopencm3, является возможность блокировки настроек ввода-вывода GPIO следующим образом:

```
void gpio_port_config_lock(uint32_t gpioport, uint16_t gpios);
```

После вызова `gpio_port_config_lock()` на порту `gpioport` для выбранных контактов GPIO конфигурация ввода-вывода замораживается до следующего сброса системы. Это может быть полезно в системах, критически важных для безопасности, и вы не хотите, чтобы ошибочная программа вносила в них изменения. Если выбранный GPIO установлен как вход или выход, он гарантированно останется таковым.

Конфигурация GPIO

Давайте теперь рассмотрим, как настраивать GPIO в функции `gpio_setup()`. В строке 26 листинга 4.1 содержится следующий интересный вызов:

```
rcc_periph_clock_enable(RCC_GPIOC);
```

Вы обнаружите, что серия STM32 очень гибко настраивается. В том числе устанавливаются базовые тактовые частоты, необходимые для различных портов GPIO и периферийных устройств. Показанная функция libopencm3 используется для включения системных тактовых импульсов для порта GPIOC. Если не включить этот источник, порт GPIOC не будет работать. Программные операции будут игнорироваться (видимый результат), в некоторых ситуациях система может зависнуть. Это одна из критических операций, про которые нужно не забывать.

Причина, по которой источник тактов отключен по умолчанию, заключается в экономии энергопотребления. Это важно для экономии заряда батареи.



Если периферийное устройство или GPIO не работают, убедитесь, что вы включили необходимый источник тактов.

Следующий вызов – функция `gpio_set_mode()` в строке 29:

```
gpio_set_mode(
  GPIOC, // Table 4.1
  GPIO_MODE_OUTPUT_2_MHZ, // Table 4.3
  GPIO_CNF_OUTPUT_PUSHPULL, // Table 4.4
  GPIO13 // Table 4.2
);
```

Эта функция требует четырех аргументов. Первый аргумент указывает порт GPIO (табл. 4.1). Четвертый аргумент указывает вывод(ы) GPIO (табл. 4.2). Значения третьего аргумента перечислены в табл. 4.3 и определяют общий режим порта GPIO.

Таблица 4.3. Определения режима GPIO

Название режима	Значение	Описание
GPIO_MODE_INPUT	0x00	Режим входа
GPIO_MODE_OUTPUT_2_MHZ	0x02	Режим выхода, 2 МГц
GPIO_MODE_OUTPUT_10_MHZ	0x01	Режим выхода, 10 МГц
GPIO_MODE_OUTPUT_50_MHZ	0x03	Режим выхода, 50 МГц

Макроопределение `GPIO_MODE_INPUT` определяет вывод GPIO в режиме входа, как ясно из названия. В списке есть также три макроопределения режима выхода.

Выбор каждого режима влияет на то, насколько быстро выходной контакт реагирует на изменение. В нашем примере программы был выбран вариант 2 МГц. Это было выбрано потому, что скорость изменения сигнала светодиода не будет заметна человеческим глазом. При выборе 2 МГц экономится электроэнергия и уменьшаются электромагнитные помехи.

Третий аргумент функции `gpio_set_mode()` дополнительно определяет, как следует настроить порт. В табл. 4.4 перечислены предоставленные названия макроопределений.

Таблица 4.4. Конфигурация специальных режимов выводов

Название режима	Значение	Описание
GPIO_CNF_INPUT_ANALOG	0x00	Режим аналогового входа
GPIO_CNF_INPUT_FLOAT	0x01	Цифровой вход, плавающий (по умолчанию)
GPIO_CNF_INPUT_PULL_UPDOWN	0x02	Цифровой вход, подтянутый вверх и вниз
GPIO_CNF_OUTPUT_PUSHPULL	0x00	Цифровой выход, двухтактный
GPIO_CNF_OUTPUT_OPENDRAIN	0x01	Цифровой выход, открытый сток
GPIO_CNF_OUTPUT_ALTFN_PUSHPULL	0x02	Альтернативный функциональный выход, двухтактный
GPIO_CNF_OUTPUT_ALTFN_OPENDRAIN	0x03	Альтернативный функциональный выход, открытый сток

Входные режимы портов

Имена определений, включающие слово «INPUT», применяются только тогда, когда второй аргумент подразумевает порт в режиме входа. Из табл. 4.4 мы видим, что входные режимы могут быть трех разновидностей:

- аналоговый,
- цифровой, плавающий вход,
- цифровой, подтянутый вверх и вниз.

Чтобы лучше понять, как работает вход GPIO в различных конфигурациях, изучите упрощенный рис. 4.2.

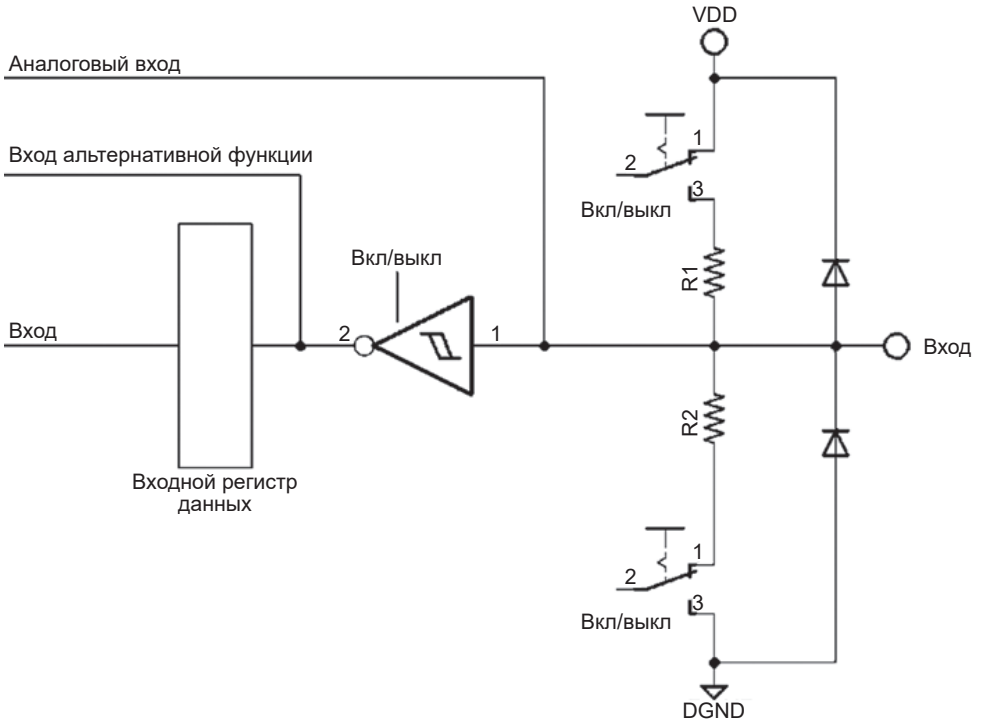


Рис. 4.2. Базовая структура GPIO в режиме входа

Логика контроллера находится слева по рисунку, внешний вход показан справа. К этому входу подключены два защитных диода, которые обычно срабатывают только в том случае, если поступающее напряжение становится отрицательным или превышает напряжение питания.

Когда порт настроен как аналоговый вход, переключатели, подключенные к резисторам R1 и R2, разомкнуты. Это делается для того, чтобы избежать повышения или понижения аналогового сигнала. При отключенных резисторах аналоговый вход направляется на линию с надписью «Аналоговый вход», т. е. напрямую на периферийное устройство аналого-цифрового преобразования (АЦП). Триггер Шмитта[21] перед регистром данных также отключен для экономии энергопотребления.

Когда входной порт настроен как цифровой вход, резисторы R1 или R2 работают, если только вы не выберете «плавающую» опцию `GPIO_CNF_INPUT_FLOAT`. Для обоих режимов цифрового входа включен триггер Шмитта с гистерезисом для обеспечения более чистого сигнала. Выход триггера Шмитта затем поступает на «Вход альтернативной функции» и в регистр входных данных GPIO. Подробнее об альтернативных функциях будет сказано позже, но простой ответ заключается в том, что вход может действовать как вход GPIO или как вход периферии.

Входы, допускающие напряжение 5 В, идентичны схеме, показанной на рис. 4.2, за исключением того, что защитный диод на стороне высокого напряжения позволяет напряжению подниматься с уровня 3.3 В как минимум до +5 В[22].



При настройке вывода как входа периферийной (альтернативной) функции обязательно используйте одно из альтернативных функциональных определений. В противном случае будут настроены только сигналы GPIO.

Выходные режимы портов

Когда порт GPIO настроен на выход, у вас есть четыре варианта на выбор:

- GPIO-двухтактный выход (push/pull),
- GPIO с открытым стоком,
- альтернативная функция: двухтактный выход (push/pull),
- альтернативная функция: открытый сток.

Для работы выхода, как GPIO, вы всегда выбираете неальтернативные функциональные режимы. Для задействования периферийного оборудования, такого, например, как USART, вместо этого вы выбираете один из альтернативных функциональных режимов. Распространенной ошибкой явля-

---

[21] Триггер Шмитта – пороговое устройство на входе логического элемента, обеспечивающее однозначное различение высокого и низкого уровня цифрового сигнала, вне зависимости от колебаний напряжения вблизи порога срабатывания элемента. Принцип действия триггера Шмитта основан на явлении гистерезиса: порог срабатывания при повышении напряжения выше, чем при его снижении.

[22] Пятивольтовые выводы STM32 выдерживают до V_{cc} + 4 В, т. е. максимум около 7 В (см. табл. 6 на стр. 37 руководства DS5319 по STM32F103x8 (раздел «Дополнительные источники», [2] к главе 14).

ется настройка использования GPIO, например `GPIO_CNF_OUTPUT_PUSHPULL` для выхода TX USART.

Правильный выбор в этом случае – `GPIO_CNF_OUTPUT_ALTFN_PUSHPULL` для периферийного устройства. Если вывод периферийных устройств не работает, спросите себя, выбрали ли вы одно из альтернативных значений функции.

На рис. 4.3 показана блок-схема GPIO при работе на выход. Для выходов, совместимых с 5-вольтовым напряжением (как и входов), единственное изменение в схеме заключается в том, что защитный диод верхнего плеча способен принимать напряжение до +5 В. Для портов, не допускающих напряжение 5 В, верхний защитный диод ограничивает напряжение до +3.3 В (фактически оно может подниматься до падения напряжения на одном диоде выше 3.3 В).

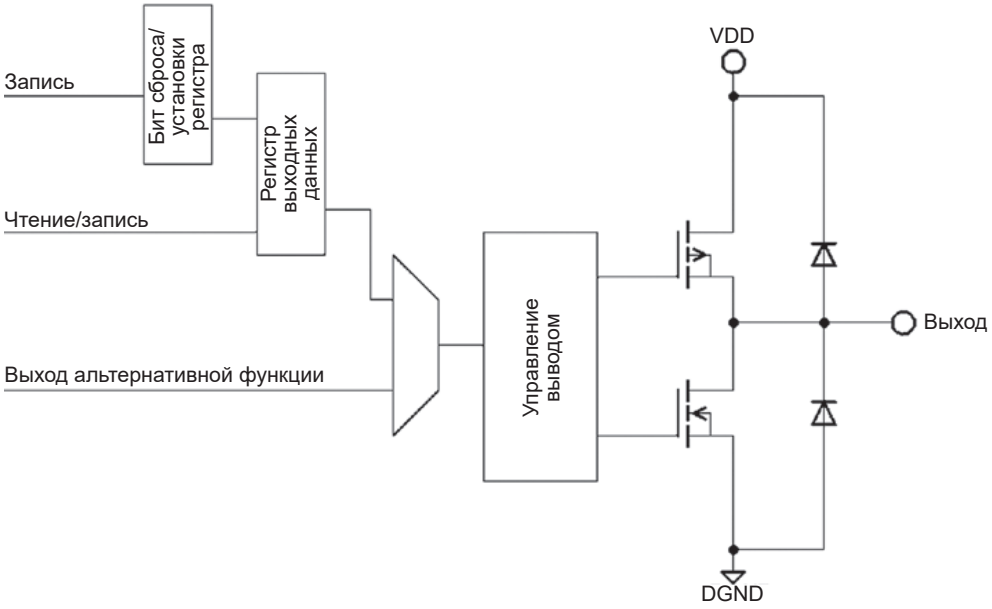


Рис. 4.3. Схема выходного драйвера GPIO

Схема управления выходом определяет, управляет ли она парой P-MOS- и N-MOS-транзисторов (в двухтактном режиме) или только N-MOS (в режиме с открытым стоком). В режиме открытого стока P-MOS-транзистор всегда выключен. Только когда вы записываете ноль на выход, N-MOS-транзистор включится и установит выходной вывод в низкий уровень. Запись единичного бита в порт с открытым стоком фактически отключает порт, поскольку оба транзистора переводятся в выключеное состояние.

Входные резисторы, показанные на рис. 4.2, отключены в режиме выхода. По этой причине на рис. 4.3 они не показаны.

Биты выходных данных выбираются либо из регистра выходных данных GPIO, либо из источника альтернативной функции. Состояния выходов GPIO поступают в регистр выходных данных, который может быть записан как

целое слово или как отдельные биты. Регистр позволяет изменять отдельные биты GPIO, как если бы они были одной атомарной операцией. Другими словами, прерывание не может произойти в середине операции над битами.

Поскольку выходные данные GPIO фиксируются в регистре выходных данных, можно прочитать текущие настройки вывода. Однако это не работает для альтернативных функций выводов.

Когда выход настроен для периферийного устройства, такого как USART, данные поступают от периферийного устройства через выходную линию альтернативной функции. Схема управления выводом данных на рис. 4.3 иллюстрирует факт необходимости настройки порта как GPIO или для альтернативных функций. Я говорю об этом лишний раз, чтобы вы не тратили время на отладку подобных проблем.

Двухтактный выход

Давайте проясним, что подразумевается под термином «двухтактный выход» (push/pull[23]). Просмотрите рис. 4.3 и обратите внимание, как разнополярные выходные транзисторы расположены друг над другом между VDD (питание) и DGND[24] (общий провод). В такой конфигурации в любой момент времени активен только один транзистор.

Когда на выводе GPIO устанавливается высокий уровень, верхний транзистор открывается, и выход эффективно подключается к источнику питания VDD. Выходное напряжение практически равно напряжению питания. Одновременно нижний транзистор принудительно закрывается, фактически удаляясь из схемы. Выход «подтягивается» к питанию и демонстрирует высокий уровень выходного сигнала.

Когда на выводе GPIO устанавливается низкий уровень, нижний транзистор открывается, а верхний транзистор принудительно отключается. Это эффективно соединяет выход с общим проводом, «подтягивая» выход к нулевому потенциалу. Такое чередование верхних и нижних транзисторов позволяет переключать выходной сигнал между высоким и низким уровнями сигнала. В этой конфигурации на выходе всегда имеется или высокий, или низкий уровень сигнала. Это отличает двухтактную схему от конфигурации с «открытым стоком», которую мы рассмотрим далее.

Выход с открытым стоком

В конфигурации GPIO с открытым стоком верхний транзистор принудительно отключен и всегда остается выключенным. Можно считать, что в схеме он отсутствует. GPIO с открытым стоком имеет высокий или низкий уровень, но участвует в этом только нижний транзистор.

Когда нижний транзистор открыт, он подтягивает выходной потенциал к нулевому значению, как и раньше. А вот когда он заперт, то никакой связи

---

[23] Буквально «тяни-толкай»; в отечественной литературе такое включение транзисторов часто называют пушпульным.

[24] Буква D в обозначении общего провода (DGND) означает digital (*цифровой*), т. е. это общий провод для цифровых схем (аналоговая «земля», соответственно, обозначается как AGND).

с выходом GPIO как будто вообще нет. Это обычно описывается как состояние высокого импеданса, вызывающее «плавание» выходного сигнала.

Конфигурация с открытым стоком полезна, когда несколько выходов подключены к общему подтягивающему резистору. Если ни один из выходов не активирован, резистор подтягивает сигнал до уровня питания. В противном случае, если какой-либо из выходов включен, уровень напряжения снижается до потенциала земли[25]. Это часто используется несколькими микросхемами, например, для сигнализации прерывания.

GPIO_MODE_OUTPUT_*_MHZ

Название конфигурации, такое как `GPIO_MODE_OUTPUT_2_MHZ`, может озадачить непосвященного. Для STM32F103C8T6 существуют следующие варианты:

- `GPIO_MODE_OUTPUT_2_MHZ`,
- `GPIO_MODE_OUTPUT_10_MHZ`,
- `GPIO_MODE_OUTPUT_50_MHZ`.

Этот аргумент функции `gpio_set_mode()` определяет, насколько быстро выходной GPIO меняет свой уровень (насколько велика скорость нарастания выходного сигнала). Частично ваш выбор будет определяться требованиями схемы, подключенной к данному выводу. Если схема имеет низкую скорость передачи данных (как, например, драйвер реле), выберите вариант с более низкой скоростью (`GPIO_MODE_OUTPUT_2_MHZ`). Это снижает рассеиваемую мощность и уменьшает радиочастотный шум (т. е. электромагнитные помехи, EMI). Если вам необходимо подключиться к каналу с высокой скоростью передачи данных, следует выбрать один из вариантов с более высокой скоростью. Но это приведет к увеличению энергопотребления и электромагнитных помех.

Выстраиваем уток в ряд

Происхождение поговорки «держать уток в ряд»[26] неясно, вариант, который мне нравится, относится к ярмарочному развлечению – стрельбе по ряду механических уток. Такое расположение позволяет стрелку собрать их все и выиграть приз.

Периферийные устройства на платформе STM32 обладают широкими возможностями настройки, что, однако, оставляет больше, чем обычно, возможностей для ошибок. Поэтому я ссылаюсь на идею «выстроить уток в ряд» как на сокращенный рецепт успеха. Если конфигурация периферийных устройств не работает должным образом, просмотрите список «уток в ряд».

Часто проблема заключается в пропуске или использовании неправильного имени настройки, который не вызывает предупреждение компилятора. Последовательность также часто важна – например, вам необходимо

[25] Подобное включение транзисторов еще называют «Проводное ИЛИ».

[26] Английское идиоматическое выражение «ducks in a row» (*утки в ряд*) означает, что все приготовления уже сделаны, порядок наведен, все организовано и можно приступать к следующему этапу действий.

включить тактовый сигнал перед настройкой устройства, которому нужен этот сигнал.

Входы GPIO

При настройке входных контактов GPIO используйте следующую процедуру. Это относится только к входам GPIO, а не к функциям периферии, таким как USART. Периферийные устройства требуют других соображений, особенно если используются альтернативные конфигурации выводов (они будут рассмотрены в этой книге позже).

1. Включите синхронизацию порта GPIO. Например, если вывод GPIO находится на порту C, включите тактовый сигнал с помощью вызова `rcc_periph_clock_enable(RCC_GPIOC)`. Вы должны включить каждый используемый порт индивидуально, используя имя `RCC_GPIOx`.
2. Установите режим работы на вход с помощью `gpio_set_mode()`, указав порт в аргументе 1 и макроопределение `GPIO_MODE_INPUT` в аргументе 2.
3. В вызове `gpio_set_mode()` выберите соответствующее название режима `GPIO_CNF_INPUT_ANALOG`, `GPIO_CNF_INPUT_FLOAT` или `GPIO_INPUT_PULL_UPDOWN` в зависимости от ситуации.
4. Наконец, укажите в последнем аргументе вызова `gpio_set_mode()` все необходимые номера контактов. Они объединяются вместе оператором «|», как, например, `GPIO12|GPIO15`.

Цифровой выход, двухтактный

Обычно цифровые выходы настроены на двухтактную конфигурацию (push/pool). Для этого распространенного случая:

1) включите синхронизацию порта GPIO. Например, если вывод GPIO находится на порту B, включите тактовый сигнал с помощью вызова `rcc_periph_clock_enable(RCC_GPIOB)`. Вы должны включить каждый используемый порт индивидуально, используя имя `RCC_GPIOx`;
2) установите режим выходного вывода с помощью `gpio_set_mode()`, указав порт в аргументе 1 и один из режимов `GPIO_MODE_OUTPUT_*_MHZ` в аргументе 2;
3) для некритических скоростей сигнала выберите наименьшее значение `GPIO_MODE_OUTPUT_2_MHZ` для экономии энергии и снижения электромагнитных помех;
4) укажите `GPIO_CNF_OUTPUT_PUSHPULL` в третьем аргументе вызова `gpio_set_mode()`. Не используйте альтернативные режимы `ALTFN` для использования GPIO (они предназначены только для использования периферийного оборудования);
5) наконец, укажите в последнем аргументе вызова `gpio_set_mode()` все необходимые номера контактов. Они объединяются вместе оператором «|», как, например, `GPIO12|GPIO15`.

Цифровой выход, открытый сток

При работе с шиной, где для установки логического нуля может использоваться более одного транзистора, может потребоваться выход с открытым

стоком. Примеры можно найти в связи по шинам I2C или CAN. Следующая процедура рекомендуется только для выходов GPIO с открытым стоком (не используйте эту процедуру для периферийных устройств!).

1. Включите синхронизацию порта GPIO. Например, если вывод GPIO находится на порту B, включите тактовые сигналы с помощью вызова `rcc_periph_clock_enable(RCC_GPIOB)`. Вы должны включить каждый используемый порт индивидуально, используя имя `RCC_GPIOx`.
2. Установите режим выходного контакта с помощью `gpio_set_mode()`, указав порт в аргументе 1 и один из режимов `GPIO_MODE_OUTPUT_*_MHZ` в аргументе 2. Для некритических скоростей сигнала выберите наименьшее значение `GPIO_MODE_OUTPUT_2_MHZ` для экономии энергии и снижения электромагнитных помех.
3. Укажите `GPIO_CNF_OUTPUT_OPENDRAIN` в аргументе 3 при вызове `gpio_set_mode()`. Не используйте альтернативные режимы `ALTFN` для использования GPIO (они предназначены только для использования периферийного оборудования).
4. Наконец, укажите в последнем аргументе вызова `gpio_set_mode()` все необходимые номера контактов. Они объединяются вместе оператором «|», как, например, `GPIO12|GPIO15`.

Характеристики GPIO

Это хороший момент, чтобы подвести итог возможностям выводов GPIO STM32. Многие из них допускают напряжение 5 В на входе, тогда как некоторые другие имеют ограничение тока на выходе. В соответствии с соглашением в документации STM32 порты часто обозначаются как PB5, например, для обозначения контакта GPIO5 порта GPIOB. Я буду использовать это соглашение на протяжении всей книги. В табл. 4.5 суммированы эти важные характеристики GPIO применительно к устройству Blue Pill.

Таблица 4.5. Возможности GPIO: если не указано иное, все контакты GPIO могут подавать или потреблять ток максимум 25 мА

Вывод	GPIO_PortA			GPIO_PortB			GPIO_PortC		
	3/5 В	Нач. сост.	Альт. функц.	3/5 В	Нач. сост.	Альт. функц.	3/5 В	Нач. сост.	Альт. функц.
GPIO0	3 В	PA0	Да	3 В	PB0	Да			
GPIO1	3 В	PA1	Да	3 В	PB1	Да			
GPIO2	3 В	PA2	Да	5 В	PB2/ BOOT1	Да			
GPIO3	3 В	PA3	Да	5 В	JTDO	Нет			
GPIO4	3 В	PA4	Да	5 В	JNTRST	Да			
GPIO5	3 В	PA5	Да	3 В	PB5	Да			
GPIO6	3 В	PA6	Да	5 В	PB6	Да			
GPIO7	3 В	PA7	Да	5 В	PB7	Да			
GPIO8	5 В	PA8	Нет	5 В	PB8	Да			

Таблица 4.5 (окончание)

Вывод	GPIO_PortA			GPIO_PortB			GPIO_PortC		
	3/5 В	Нач. сост.	Альт. функц.	3/5 В	Нач. сост.	Альт. функц.	3/5 В	Нач. сост.	Альт. функц.
GPIO9	5 В	PA9	Нет	5 В	PB9	Да			
GPIO10	5 В	PA10	Нет	5 В	PB10	Да			
GPIO11	5 В	PA11	Нет	5 В	PB11	Да			
GPIO12	5 В	PA12	Нет	5 В	PB12	Нет			
GPIO13	5 В	JTMS/ SWDIO	Да	5 В	PB13	Нет	3 В	3 mA 2 МГц	Да
GPIO14	5 В	JTCK/ SWCLK	Да	5 В	PB14	Нет	3 В	3 mA 2 МГц	Да
GPIO15	5 В	JTDI	Да	5 В	PB15	Нет	3 В	3 mA 2 МГц	Да

Столбец «Альт. функц.» в табл. 4.5 указывает, что на данном выводе имеются альтернативные функции. Выводы GPIO, отмеченные «5 В», могут безопасно выдерживать сигнал напряжением 5 В, тогда как отмеченные «3 В» могут принимать сигналы только до +3.3 В. Столбец с надписью «Нач. сост.» указывает состояние конфигурации вывода после сброса микроконтроллера.

Выводы GPIO PC13, PC14 и PC15 ограничены по току. Они могут потреблять втекающий ток максимум 3 мА (при низком уровне на выходе) и никогда не должны использоваться для подачи тока. Кроме того, в документации указано, что их никогда не следует настраивать для работы на частоте более 2 МГц, если они настроены как выходы.

Пороги входного напряжения

Учитывая, что STM32F103C8T6 может работать в широком диапазоне напряжений, входные пороговые напряжения GPIO не фиксированы, а вычисляются по формуле. В табл. 4.6 показано, чего можно ожидать от устройства Blue Pill, работающего при +3.3 В.

Таблица 4.6. Пороги входного напряжения при VDD = + 3.3 В[27]

Обозначение	Описание	Диапазон
V_{IL}	Стандартное входное напряжение низкого уровня	от 0 до 1.164 В
	Входы, устойчивые к 5 В	от 0 до 1.166 В
V_{IH}	Входное напряжение высокого уровня	от 1.833 до 3.3 В
	Входы, устойчивые к 5 В	от 1.546 до 5.0 В

Пороги выходного напряжения

Пороги выходного сигнала GPIO указаны в табл. 4.7 и основаны на данных для устройства Blue Pill, работающего при +3.3 В. Заметим, что диапазоны сужаются по мере увеличения тока.

[27] Данные порогов для любых напряжений питания – см. формулы в табл. 36 на стр. 61 официального руководства DS5319 по STM32F103: https://www.st.com/resource/en/datasheet/stm32f103c8.pdf.

Таблица 4.7. Уровни выходного напряжения GPIO при токе <= 20 мА

Обозначение	Описание	Диапазон
V_{OL}	Выходное напряжение низкого уровня	от 0.4 до 1.3 В
V_{OH}	Выходное напряжение высокого уровня	от 2 до 3.3 В

Программные задержки

Возвращаясь снова к программе, показанной в листинге 4.1, давайте рассмотрим следующий фрагмент этой программы:

```
0039: for (;;) {
0040:     gpio_clear(GPIOC,GPIO13); /* LED on */
0041:     for (i = 0; i < 1500000; i++) /* Wait a bit. */
0042:          __asm__("nop");
0043:
0044:     gpio_set(GPIOC,GPIO13); /* LED off */
0045:     for (i = 0; i < 500000; i++) /* Wait a bit. */
0046:          __asm__("nop");
0047: }
```

Первое, на что следует обратить внимание в этом фрагменте, – то, что количество циклов различается: 1 500 000 в строке 41 и 500 000 в строке 45. В результате светодиод горит 75 % времени и погашен в течение 25 %.

Оператор `__asm__("nop")` заставляет компилятор выдать команду ассемблера ARM `nop` в качестве тела обоих циклов. Почему это необходимо? Почему бы не закодировать пустой цикл, как показано ниже?

```
0041: for (i = 0; i < 1500000; i++) /* Wait a bit. */
0042:     ; /* empty loop */
```

Проблема с пустым циклом в том, что компилятор может его оптимизировать. Оптимизация компилятора постоянно совершенствуется, и конструкции такого типа могут рассматриваться как избыточные и удаляться из скомпилированного результата. Эта функция также чувствительна к параметрам оптимизации, используемым для компиляции. Этот трюк `__asm__` – один из способов заставить компилятор всегда создавать код цикла и выполнять пустую инструкцию `nop`[28].

Проблемы с программной задержкой

Программные задержки хороши тем, что их легко кодировать. Но с ними могут быть проблемы.

- Сколько итераций нужно для конкретной задержки по времени?
- Плохая переносимость исходного кода:

[28] Название инструкции `nop` расшифровывается как «no operation» (*нет операции*), т. е. процессор просто пропускает такт, ничего не делая.

  - задержка будет различаться для разных платформ;
  - задержка зависит от тактовой частоты процессора;
  - задержка будет варьироваться в зависимости от контекста выполнения.
- Тратит ресурсы ЦП, которые в многозадачной среде могут использоваться другими задачами.
- Задержки ненадежны при использовании вытесняющей многозадачности.

Первая проблема – сложность вычисления количества итераций, необходимых для достижения задержки. Число циклов зависит от нескольких факторов, а именно:

- тактовой частоты процессора;
- продолжительности цикла выполнения команды;
- одно- или многозадачной среды.

В программе *miniblink* не была установлена тактовая частота процессора. Следовательно, этот код зависит от используемой частоты по умолчанию. Экспериментальным путем можно получить длительность циклов, которая «кажется, работает». Но если вы запускаете те же циклы из SRAM вместо флеш-памяти, задержки будут короче. Это связано с тем, что для извлечения командных слов из SRAM не требуется времени на ожидание, тогда как получение инструкций из флеш-памяти может включать задержки ожидания, к тому же зависящие от выбранной тактовой частоты процессора.

В многозадачной среде, такой как FreeRTOS, запрограммированные задержки – совсем плохой выбор. Одна из причин заключается в том, что вы не знаете, сколько времени уходит на другие задачи.

Наконец, программные задержки не переносятся на другие платформы. Возможно, исходный код будет повторно использован на устройстве STM32F4, где эффективность выполнения другая. Потребуется ручное вмешательство в код, чтобы исправить недостаток синхронизации.

Все эти причины объясняют, почему FreeRTOS предоставляет API для синхронизации и задержки. Они будут рассмотрены позже, когда мы будем применять FreeRTOS в демонстрационных программах.

Итоги

В этой главе было затронуто много вопросов, хотя мы только начинаем. Мы использовали утилиту *st-flash* и запрограммировали устройство с помощью программы *miniblink*, которая отличается от поставляемой с вашим устройством.

Еще интереснее обсуждение API libopencm3 для GPIO и подробное рассмотрение программы *miniblink*. Мы подробно разобрали конфигурацию и работу вывода GPIO. Наконец, были обсуждены проблемы программных задержек.

Упражнения

1. Какой порт GPIO использует встроенный светодиод на плате Blue Pill? Укажите имя libopencm3 для этого порта.

2. Какой вывод GPIO использует встроенный светодиод на плате Blue Pill? Укажите имя libopencm3 для этого вывода.
3. Какой уровень на выводе необходим для включения встроенного светодиода на плате Blue Pill?
4. Какие два фактора влияют на выбранное количество циклов при программируемой задержке в однозадачных средах?
5. Почему программируемые задержки не используются в многозадачной среде?
6. Какие три фактора влияют на время цикла задержки?
7. Перечислите три режима входного порта GPIO.
8. Участвуют ли подтягивающие и заземляющие резисторы в работе аналогового входа?
9. Когда для входных портов включается триггер Шмитта?
10. Участвуют ли подтягивающие и заземляющие резисторы в выходных портах GPIO?
11. Какое название режима следует использовать при настройке выхода USART TX (передача) для работы вывода в двухтактном режиме?
12. При настройке вывода для использования светодиодов какой режим GPIO предпочтителен для снижения электромагнитных помех?

Глава 5 • FreeRTOS

В начале этой книги мы переходим к использованию многозадачной операционной системы FreeRTOS. Это дает ряд преимуществ главным образом потому, что программирование становится намного проще и в результате обеспечивает большую надежность.

Не все платформы способны поддерживать RTOS (Real-Time Operating System, операционную систему реального времени). Для каждой задачи в многозадачной системе требуется выделить некоторое пространство стека – области памяти, где хранятся переменные и адреса возврата вызовов функций. Например, на ATmega328 просто не хватит оперативной памяти, всего 2 Кбайта. С другой стороны, STM32F103C8T6 имеет 20 Кбайт SRAM, которые можно разделить между разумным количеством задач.

В этой главе представлена FreeRTOS, исходный код которой открыт и доступен бесплатно. Исходный код FreeRTOS распространяется по лицензии GPL License 2. Существует специальное положение, позволяющее вам распространять связанный с ней продукт без необходимости раскрытия собственного проприетарного исходного кода. Для получения подробной информации найдите текстовый файл с именем *LICENSE*.

Возможности FreeRTOS

Что делает RTOS желательной? Что это дает? Давайте рассмотрим некоторые основные категории сервисов FreeRTOS:

- многозадачность и планирование,
- очереди сообщений,
- семафоры и мьютексы,
- таймеры,
- группы событий.

Многозадачность

В среде Arduino все изначально выполнялось как одна задача с единым стеком для переменных и адресов возврата при вызове функций. Такой стиль программирования требует запуска цикла опроса каждого обслуживаемого события. На каждой итерации вам может потребоваться опросить датчик температуры, а затем вызвать другую процедуру как часть цикла опроса для трансляции этого результата[29].

[29] Справедливости ради следует отметить, что создание главного цикла опроса не единственный метод обработки событий в классических контроллерах, включая семейство AVR (к которому относится в том числе и ATmega328, применяемый в Arduino). Такой метод было фактически единственным для ранних примитивных моделей (вроде i8080), но с развитием микроэлектроники в контроллерах появилась развитая система прерываний, которая позволяет в ряде случаев обойтись вообще без обращения к главному циклу, оставив его пустым. Характерный

В FreeRTOS (RTOS) логические функции помещаются в отдельные задачи, которые выполняются независимо. Одна задача может отвечать за считывание и вычисление текущей температуры, другая – за передачу последней вычисленной температуры внешним устройствам. По сути, это становится парой программ, работающих одновременно.

Для очень простых приложений эти накладные расходы на планирование задач могут рассматриваться как излишние. Однако по мере увеличения сложности преимущества разделения проблемы на задачи становятся гораздо более очевидными.

FreeRTOS очень гибкая система. Он обеспечивает два типа планирования задач:

- вытесняющую многозадачность;
- кооперативную многозадачность.

При вытесняющей многозадачности задача выполняется до тех пор, пока не закончится отведенный временной интервал, или задача не будет заблокирована, или пока не будет явно передано управление. Планировщик задач определяет, какая задача будет запущена следующей, принимая во внимание приоритеты. Именно такой тип многозадачности будет использоваться в проектах этой книги.

Другая форма многозадачности – кооперативная. Разница в том, что текущая задача выполняется до тех пор, пока она не передаст управление. Временной интервал или тайм-аут отсутствуют. Если ни один вызов функции не блокируется (например, мьютекс, см. далее), то сопрограмма должна вызвать yield-функцию[30], чтобы передать управление другой задаче. Планировщик задач затем решает, какой задаче передать управление следующей. Такая форма планирования желательна для приложений, критичных к безопасности, которым требуется строгий контроль времени процессора.

Очереди сообщений

Как только вы перейдете на многозадачность, вы унаследуете проблемы со связью. Используя наш пример измерения температуры, зададимся вопросом, как задача измерения температуры безопасно передает значение задаче передачи данных о температуре? Если температура хранится в виде 4 байт, как передать это значение без перерыва? Вытесняющая многозадачность означает, что копирование 4 байт данных в другое место может быть прервано на полпути.

пример – подсистема отсчета времени в Arduino (от которой работают функции `millis()`, `delay()` и пр.; в том числе и `yield()`, см. следующее примечание), целиком функционирующая в фоновом режиме так, что начинающий «ардуинщик» даже не подозревает о том, что контроллер интенсивно работает с момента включения, незаметно выполняя довольно сложные операции. Такой подход иногда называют «бесплатной многозадачностью».

[30] Yield function (от *yield* – уступать) – функция, вызываемая регулярно автоматически по временны́м прерываниям. Она позволяет выполнить какую-то задачу, даже если основной цикл контроллера заблокирован, например, функцией задержки `delay()`.

Грубый способ решить эту проблему – запретить прерывания при копировании данных о температуре в место, используемое задачей трансляции. Но этот подход может оказаться недопустимым, если у вас часто возникают прерывания. Проблема усугубляется, когда копируемые объекты увеличиваются в размерах.

Функция очереди сообщений в FreeRTOS обеспечивает безопасный способ передачи полного сообщения. Очередь сообщений гарантирует, что будут получены только полные сообщения. Кроме того, он ограничивает длину очереди, чтобы задача отправки не могла использовать всю память. При использовании заранее определенной длины очереди задача добавления сообщений блокируется до тех пор, пока не освободится место. Когда задача блокируется, планировщик задач автоматически переключается на другую задачу, готовую к запуску, которая может удалить сообщения из той же очереди. Фиксированная длина дает очереди сообщений форму управления потоком.

Семафоры и мьютексы

В реализации очереди используется мьютексные операции[31]. Процесс добавления сообщения может потребовать выполнения нескольких инструкций. Тем не менее в системе с вытесняющей многозадачностью сообщение может быть добавлено наполовину, прежде чем оно будет прервано для выполнения другой задачи.

В FreeRTOS очередь предназначена для атомарного добавления сообщений. Для этого за кулисами используется разновидность мьютексной операции. Мьютекс – это устройство, работающее по принципу «все или ничего». Блокировка либо есть, либо ее нет.

Подобно мьютексам, существуют семафоры. Например, в некоторых ситуациях, когда вам может потребоваться ограничить определенное количество одновременных запросов, семафор может управлять этим процессом атомарным образом. Например, максимальное значение может быть равно трем. Тогда до трех запросов «получить» будут успешными. Дополнительные запросы «получить» будут блокироваться до тех пор, пока не будет выполнен один или несколько запросов «отдать» на возврат от ресурса.

Таймеры

Таймеры важны для многих приложений, включая различные программы мигания. Когда у вас есть несколько задач, потребляющих процессорное время, процедура задержки не только ненадежна, но и отнимает у других задач процессорное время, которое можно было бы использовать более продуктивно.

В любой системе RTOS обычно имеется системное прерывание, которое помогает управлять временем. Это системное прерывание не только отслеживает текущее количество «тиков», выполненных на данный момент, но также используется планировщиком для переключения задач.

[31] Mutex в дословном переводе означает «взаимное исключение».

В FreeRTOS вы можете отложить выполнение на определенное количество тактов. Это работает, отмечая текущее «время тика» и переходя к другой задаче, пока не наступит требуемое время тика. Таким образом, точность задержки ограничивается только настроенным тактовым интервалом. Это также позволяет другим задачам выполнять реальную работу до тех пор, пока не придет нужное время.

FreeRTOS также имеет возможность создания программных таймеров. По истечении времени таймера выполняется обратный вызов функции[32]. Этот подход экономит память, поскольку все таймеры будут использовать один и тот же стек.

Группы событий

Одна из часто возникающих проблем заключается в том, что задаче может потребоваться одновременное наблюдение за несколькими очередями. Например, задачу может потребоваться заблокировать до тех пор, пока не поступит сообщение из одной из двух разных очередей. FreeRTOS обеспечивает создание «наборов очередей» (queue sets). Это позволяет задаче не производить никаких действий до тех пор, пока какая-нибудь из очередей набора не пошлет сообщения.

А как насчет пользовательских событий? Группы событий могут быть созданы, чтобы событие представляла комбинация битов, соответствующих отдельным событиям. FreeRTOS позволяет задаче ждать, пока не произойдет эта комбинация событий. События могут запускаться из обычного кода задачи или из подпрограмм обслуживания прерываний ISR.

Программа blinky2

Перейдите в каталог демо *blinky2*:

```
$ cd ~/stm32f103c8t6/rtos/blinky2
```

В этом примере программы используется API FreeRTOS для реализации программы *Blink* в среде RTOS. В листинге 5.1 показана начальная часть файла исходного кода *main.c*. Обратите внимание на использование включаемых файлов:

- `FreeRTOS.h`,
- `task.h`,
- `libopencm3/stm32/rcc.h`,
- `libopencm3/stm32/gpio.h`.

Вы уже видели заголовочные файлы libopencm3. Заголовочный файл *task.h* определяет имена и функции, связанные с созданием задач. Наконец, есть

[32] Обратный вызов функции (функция обратного вызова, callback function) – отложенное выполнение функции, переданной другой функции в качестве параметра, после завершения последней.

FreeRTOS.h, который необходим каждому проекту для настройки и конфигурирования FreeRTOS. Мы проверим это после того, как закончим с *main.c*.

Программа *main.c* также определяет прототип функции с именем `vApplicationStackOverflowHook()` в строках 11–13 листинга 5.1. FreeRTOS не предоставляет для него прототип функции, поэтому мы должны предоставить его здесь, чтобы компилятор не жаловался на это[33].

Листинг 5.1. Начальная часть исходного кода *stm32/rtos/blinky2/main.c*

```
0001: /* Simple LED task demo, using timed delays:
0002: *
0003: * The LED on PC13 is toggled in task1.
0004: */
0005: #include "FreeRTOS.h"
0006: #include "task.h"
0007:
0008: #include <libopencm3/stm32/rcc.h>
0009: #include <libopencm3/stm32/gpio.h>
0010:
0011: extern void vApplicationStackOverflowHook(
0012:   xTaskHandle *pxTask,
0013:   signed portCHAR *pcTaskName);
```

В листинге 5.2 приведено определение необязательной функции `vApplicationStackOverflowHook()`. Эту функцию можно было бы исключить из программы, не создавая проблем. Оно представлено здесь, чтобы проиллюстрировать, как бы вы ее определили, если бы захотели.

Листинг 5.2. Файл *blinky2/main.c*, функция `vApplicationStackOverflowHook()`

```
0017: void
0018: vApplicationStackOverflowHook(
0019:   xTaskHandle *pxTask __attribute((unused)),
0020:   signed portCHAR *pcTaskName __attribute((unused))
0021: ) {
0022: for(;;); // Loop forever here..
0023: }
```

Если функция определена, FreeRTOS вызовет ее, когда обнаружит, что предел объема стека превышен. Это позволит разработчику приложения решить, что с этим делать. Например, вы можете захотеть мигать специальным красным светодиодом, указывающим на сбой программы[34].

В листинге 5.3 показана задача, выполняющая мигание светодиода. Он принимает аргумент `void *args`, который в этом примере не используется.

[33] Название функции означает, что это ловушка события переполнения стека приложения.

[34] Не путать с текущей задачей мигания `task1()`, о которой идет речь в следующих абзацах.

`__attribute((unused))` – это атрибут gcc, указывающий компилятору, что аргумент `args` не используется, и предотвращающий появление предупреждений об этом.

Листинг 5.3. Файл *blinky2/main.c*, функция `task1()`

```
0025: static void
0026: task1(void *args __attribute((unused))) {
0027:
0028:   for (;;) {
0029:       gpio_toggle(GPIOC,GPIO13);
0030:       vTaskDelay(pdMS_TO_TICKS(500));
0031:   }
0032: }
```

В остальном тело функции `task1()` очень простое. В цикле оно переключает состояние GPIO PC13. Далее выполняется задержка на 500 мс. Функция `vTaskDelay()` требует указания количества тиков таймера для задержки. Вместо этого часто удобнее указывать миллисекунды. Макроопределение `pdMS_TO_TICKS()` преобразует миллисекунды в тики в соответствии с вашей конфигурацией FreeRTOS.

Эта задача, конечно, предполагает, что все необходимые настройки FreeRTOS были выполнены заранее. Об этом заботится главная функция `main()`, показанная в листинге 5.4.

Листинг 5.3. Файл *blinky2/main.c*, функция `main()`

```
0034: int
0035: main(void) {
0036:
0037:  rcc_clock_setup_pll(&rcc_hse_configs[RCC_CLOCK_HSE8_72MHZ]);
0038:
0039:  rcc_periph_clock_enable(RCC_GPIOC);
0040:  gpio_set_mode(
0041:      GPIOC,
0042:      GPIO_MODE_OUTPUT_2_MHZ,
0043:      GPIO_CNF_OUTPUT_PUSHPULL,
0044:      GPIO13);
0045:
0046:  xTaskCreate(task1,"LED",100, NULL, configMAX_PRIORITIES-1, NULL);
0047:  vTaskStartScheduler();
0048:
0049:  for (;;);
0050:  return 0;
0051: }
```

Функция `main()` определена в строках 34 и 35 как возвращающая целое число `int`, хотя основная программа никогда не должна возвращать значе-

ние в этом контексте контроллера. Это просто убеждает компилятор в том, что код соответствует стандартам интерфейса портативной операционной системы POSIX. Оператор `return` в строке 50 никогда не выполняется.

Строка 37 иллюстрирует нечто новое – установление тактовой частоты процессора. Для устройства Blue Pill обычно нужно вызвать эту функцию для достижения наилучшей производительности. Она настраивает тактовый генератор так, чтобы высокоскоростной внешний генератор (HSE) использовал кварц с частотой 8 МГц, умноженный на 9 с помощью системы фазовой автоподстройки частоты (ФАПЧ), чтобы достичь тактовой частоты ЦП 72 МГц. Без этого вызова мы бы полагались на RC-синхронизацию (тактовый сигнал от резисторно-конденсаторного генератора).

Строка 39 включает тактовую частоту GPIO для порта C. Это первый шаг в последовательной настройке вывода GPIO PC13, который управляет встроенным светодиодом. Строки 40–44 определяют оставшиеся обязательные настройки, так что PC13 является выходным контактом на частоте 2 МГц в двухтактной конфигурации.

Строка 46 создает новую задачу, используя показанную ранее функцию с именем `task1()`. Мы даем задаче символическое имя «LED», которое может быть установлено по вашему выбору. Третий аргумент указывает, сколько слов требуется отвести для пространства стека. Обратите внимание, что речь здесь идет именно о «словах». Для платформы STM32 слово имеет размер 4 байта. Оценить пространство стека часто бывает непросто, и существуют способы его измерения (см. главу 21). А пока примем, что 400 байт (100 слов) достаточно.

Четвертый аргумент в строке 46 указывает на любые данные, которые вы хотите передать в свою задачу. Здесь нам не нужно передавать никаких данных, поэтому указываем `NULL`. Этот указатель передается в аргумент `args` в `task1()`. Пятый аргумент определяет приоритет задачи. В этом примере у нас, кроме основной, есть только одна задача. Мы просто придаем ей высокий приоритет. Последний аргумент позволяет вернуть дескриптор задачи, если мы предоставили указатель. Нам не нужен возвращаемый дескриптор, поэтому указывается `NULL`.

Одного создания задачи недостаточно для ее запуска. Вы можете создать несколько задач, прежде чем запускать планировщик задач. После вызова функции FreeRTOS `vTaskStartScheduler()` задачи начнутся с адреса функции, указанного в аргументе 1.

Будьте осторожны при выборе функций для вызова перед запуском планировщика задач. Некоторые из более сложных функций можно вызывать только после запуска планировщика. Есть и другие, которые можно вызвать только до запуска планировщика. При необходимости проверьте документацию FreeRTOS.

После запуска планировщика задач программа никогда не возвращается к строке 47 листинга 5.4, пока не будет остановлена. На этот случай принято помещать бесконечный цикл (строка 49) после вызова планировщика, чтобы предотвратить возврат из главной функции (строка 50).

Сборка и тестирование blinky2

Подключив программатор к устройству, выполните следующие действия:

```
$ make clobber
$ make
# make flash
```

Команда `make clobber` удаляет все встроенные или частично построенные компоненты, так что простой `make` снова полностью перекомпилирует проект. Команда `make flash` вызовет утилиту *st-flash* для записи новой программы на ваше устройство. При необходимости нажмите кнопку Reset на плате, но программа должна запуститься сама по себе.

Код показывает, что встроенный светодиод должен менять состояние каждые 500 мс. Если у вас есть осциллограф, можете убедиться, что это действительно происходит (проверьте сигнал на выводе PC13). Это не только подтверждает, что программа работает должным образом, но также свидетельствует, что наш файл *FreeRTOS.h* настроен правильно.

Выполнение

Пример программы довольно прост, но давайте подытожим происходящие действия на высоком уровне:

- одновременно выполняется функция `task1()`, включающая и выключающая встроенный светодиод. Эта задача синхронизируется с помощью таймера (функция `vTaskDelay()`);
- функция `main()` вызвала функцию `vTaskStartScheduler()`. Это дает управление планировщику FreeRTOS, который запускает и переключает различные задачи. Основной поток будет продолжать выполняться внутри FreeRTOS (внутри планировщика), пока задача не остановит планировщик.

Задача `task1()` имеет стек, выделенный из кучи для ее выполнения (мы отвели ему 100 слов). Если эта задача когда-либо будет удалена, этот объем будет возвращен в кучу. Основная задача в настоящее время выполняется в планировщике FreeRTOS с использованием предоставленного ей стека.

Хотя это может показаться элементарным, важно знать, куда распределяются ресурсы. Большим приложениям необходимо тщательно распределять память и ресурсы процессора, чтобы ни одна задача не оставалась неудовлетворенной. Здесь описывается только общая действующая структура контроля.

Использование вытесняющей многозадачности требует ответственности. Совместное использование данных задачами требует использования безопасных приемов распределения между потоками. Приведенный простой пример обходит проблему, поскольку задача только одна. Более сложные проекты потребуют взаимодействия между задачами.

Файл FreeRTOSConfig.h

Каждый из проектов, находящихся в подкаталогах *~/stm32f103c8t6/rtos*, имеет собственную копию файла *FreeRTOSConfig.h*. Это сделано специально, по-

скольку при этом индивидуально настраиваются ресурсы и функции RTOS, которые могут различаться в зависимости от проекта. Это также позволяет некоторым проектам исключать ненужные им функции FreeRTOS, в результате чего исполняемый файл становится меньше. В других случаях могут быть различия во времени, распределении памяти и других функциях, связанных с RTOS.

В листинге 5.1, строка 5, показано включение файла *FreeRTOS.h*. Этот файл, в свою очередь, приводит к включению вашего локального файла *FreeRTOSConfig.h*. Давайте теперь рассмотрим некоторые важные элементы конфигурации в файле *FreeRTOSConfig.h*, показанные в листинге 5.5.

Листинг 5.5. Некоторые элементы конфигурации в файле *FreeRTOSConfig.h*

```
0088: #define configUSE_PREEMPTION     1
0089: #define configUSE_IDLE_HOOK      0
0090: #define configUSE_TICK_HOOK      0
0091: #define configCPU_CLOCK_HZ (( unsigned long ) 72000000 )
0092: #define configSYSTICK_CLOCK_HZ   ( configCPU_CLOCK_HZ / 8 )
0093: #define configTICK_RATE_HZ (( TickType_t ) 250 )
0094: #define configMAX_PRIORITIES     ( 5 )
0095: #define configMINIMAL_STACK_SIZE (( unsigned short ) 128 )
0096: #define configTOTAL_HEAP_SIZE (( size_t ) ( 17 * 1024 ))
0097: #define configMAX_TASK_NAME_LEN  ( 16 )
0098: #define configUSE_TRACE_FACILITY 0
0099: #define configUSE_16_BIT_TICKS   0
0100: #define configIDLE_SHOULD_YIELD  1
0101: #define configUSE_MUTEXES        0
0102: #define configCHECK_FOR_STACK_OVERFLOW 1
```

Пожалуй, наиболее важный из этих элементов конфигурации – элемент `configUSE_PREEMPTION`. Ненулевое значение указывает на то, что нам нужно упреждающее планирование в FreeRTOS. Есть две функции-перехватчика, которые не использовались, поэтому `configUSE_IDLE_HOOK` и `configUSE_TICK_HOOK` установлены в ноль.

Следующие три строки настраивают FreeRTOS так, чтобы она могла рассчитать для нас правильное время:

```
0091: #define configCPU_CLOCK_HZ (( unsigned long ) 72000000 )
0092: #define configSYSTICK_CLOCK_HZ ( configCPU_CLOCK_HZ / 8 )
0093: #define configTICK_RATE_HZ (( TickType_t ) 250 )
```

Эти объявления указывают тактовую частоту процессора 72 МГц, счетчик системного таймера, который увеличивается каждые 8 тактов процессора, а также прерывание 250 раз в секунду (каждые 4 мс). Если вы установите неверные значения, FreeRTOS не сможет правильно определить тайминги и задержки.

Значение элемента `configMAX_PRIORITIES` определяет максимальное количество поддерживаемых приоритетов. Для каждого уровня приоритета тре-

буется ОЗУ в RTOS, поэтому количество уровней не следует устанавливать выше, чем необходимо.

Минимальный размер стека (MINIMAL_STACK_SIZE) определяет, сколько места требуется задачам FreeRTOS. Обычно это значение не следует изменять. Размер кучи (HEAP_SIZE) в байтах определяет, сколько оперативной памяти может быть выделено динамически. В этом примере 17 из 20 Кбайт SRAM доступны в виде кучи:

```
0095: #define configMINIMAL_STACK_SIZE (( unsigned short ) 128 )
0096: #define configTOTAL_HEAP_SIZE (( size_t ) ( 17 * 1024 ) )
```

Элемент configIDLE_SHOULD_YIELD должен быть включен, если вы хотите, чтобы задача простоя (IDLE) вызывала другую задачу, готовую к запуску. Наконец, для этого приложения был включен элемент configCHECK_FOR_STACK_OVERFLOW, чтобы мы могли продемонстрировать функцию vApplicationStackOverflowHook() из листинга 5.2. Если вам не нужна эта функция, отключите ее, установив ее на ноль.

Следующие элементы – примеры других настроек. Например, в нашем примере программы мы никогда не используем функцию vTaskDelete(), поэтому для INCLUDE_vTaskDelete установлено нулевое значение. Это снижает накладные расходы скомпилированного кода FreeRTOS. Однако нам нужна функция vTaskDelay(), поэтому элемент INCLUDE_vTaskDelay настроен в значение 1:

```
0111: #define INCLUDE_vTaskPrioritySet  0
0112: #define INCLUDE_uxTaskPriorityGet 0
0113: #define INCLUDE_vTaskDelete       0
0114: #define INCLUDE_vTaskCleanUpResources 0
0115: #define INCLUDE_vTaskSuspend      0
0116: #define INCLUDE_vTaskDelayUntil   0
0117: #define INCLUDE_vTaskDelay        1
```

Соглашение об именах FreeRTOS

Соглашение об именах FreeRTOS отличается от соглашения об именах, используемого библиотекой libopencm3. Группа FreeRTOS использует уникальное соглашение об именах для переменных, констант и функций. Как разработчик программного обеспечения, я не рекомендую включать информацию о типе в имена сущностей. Проблема в том, что типы могут меняться по мере развития проекта или его переноса на новую платформу. Когда это произойдет, вы столкнетесь с двумя ужасными вариантами:

1) оставить имена сущностей такими, какие они есть, и смириться с тем фактом, что информация о типе неверна;
2) отредактировать все ссылки на имена, чтобы они отражали новый тип.

Соглашение UNIX о включении буквы «p» для обозначения переменной-указателя обычно приемлемо, поскольку переменная редко меняется с ука-

зателя на экземпляр. Однако это также может произойти в C++, где можно использовать ссылочные переменные.

Несмотря на это странное соглашение об именах, давайте не будем тратить время на переписывание FreeRTOS или наложение на него слоев. В табл. 5.1 я просто назову соглашения, которые они использовали, чтобы вам было проще применять их API.

Таблица 5.1. Символы префикса типа FreeRTOS

Префикс	Описание
v	void
c	char
s	short
l	long
e	Перечисляемый тип (enum)
x	BaseType_t[35] и любой другой тип, не охваченный этим списком
u	Беззнаковый (дополнительно к вышеперечисленным)
p	Указатель (дополнительно к вышеперечисленным)

Исходя из этой таблицы, если переменная имеет тип беззнаковый `char`, она будет использовать префикс `uc`. Если переменная является указателем на беззнаковый `char`, она будет использовать `puc`.

Вы уже видели функцию `vTaskDelay()`, которая указывает на отсутствие возвращаемого значения (тип `void`). Функция FreeRTOS с именем `xQueueReceive()` возвращает тип `BaseType_t`, поэтому префикс имени функции – `x`.

Макроопределения FreeRTOS

FreeRTOS записывает имена макроопределений с префиксом, указывающим, где они определяются. Они перечислены в табл. 5.2.

Таблица 5.2. Префиксы макроопределений, используемые FreeRTOS

Префикс	Пример	Источник
`port`	`portMAX_DELAY`	`portable.h`
`task`	`taskENTER_CRITICAL()`	`task.h`
`pd`	`pdTRUE`	`projdefs.h`
`config`	`configUSE_PREEMPTION`	`FreeRTOSConfig.h`
`err`	`errQUEUE_FULL`	`projdefs.h`

Итоги

Программа *Blink*, какой бы простой она ни была, была представлена работающей под FreeRTOS как задача. И тем не менее код по-прежнему оставал-

[35] BaseType_t определяется как наиболее эффективный тип данных для архитектуры. Обычно это 32-битный тип в 32-битной архитектуре, 16-битный тип в 16-битной архитектуре и 8-битный тип в 8-битной архитектуре.

ся несложным и обеспечивал надежную синхронизацию, меняя состояние каждые 500 мс.

Мы также увидели, как настроить тактовую частоту МК, чтобы не приходилось использовать тактовую частоту RC-генератора, установленного по умолчанию. Точный отсчет времени важен для FreeRTOS. Была также проиллюстрирована дополнительная функция для перехвата события переполнения стека. Были рассмотрены конфигурация и соглашения FreeRTOS об именах, и вы увидели, насколько несложно создавать вытесняющие задачи.

Упражнения

1. Сколько задач выполняется в программе blinky2?
2. Сколько потоков управления работает в blinky2?
3. Что произойдет с частотой мигания blinky2, если значение `configCPU_CLOCK_HZ` установить равным 36000000?
4. Откуда берется стек задачи `task1`?
5. Когда именно начинается `task1()`?
6. Зачем нужна очередь сообщений?
7. Перейдите к проекту в *stm32/rtos/blinky*, соберите его и запустите. Затем ответьте на следующее (п. 8–12).
8. Несмотря на то что он использует цикл задержки выполнения, почему он работает с рабочим циклом почти 50 %?
9. Насколько сложно оценить, как долго горит светодиод на PC13? Почему?
10. С помощью осциллографа измерьте время включения и выключения PC13 (или посчитайте количество миганий в секунду и вычислите обратное значение). Сколько миллисекунд горит светодиод?
11. Если бы в этот проект была добавлена еще одна задача, которая потребляла большую часть ресурсов ЦП, как бы это повлияло на частоту мигания?
12. Добавьте в файл `main.c` задачу `task2`, которая ничего не делает, кроме выполнения в цикле команды `__asm__("nop")`. Создайте эту задачу в `main()` перед запуском планировщика. Как это повлияло на частоту мигания? Почему?

Глава 6 • USART

На печатной плате Blue Pill для сигнализации состояния программы имеется один светодиод, управляемый выводом GPIO. Излишне говорить, что было бы недостаточно, если бы это было все, что у вас было. Возможно, самым удобным периферийным устройством для связи на ранней стадии разработки является USART (Universal Synchronous/Asynchronous Receiver/Transmitter, универсальный синхронный/асинхронный приемник/передатчик), часто называемый просто последовательным (serial) портом.

В этой главе будет рассмотрено, как заставить USART в STM32 взаимодействовать с настольным компьютером через адаптер последовательного порта USB-TTL. Второй проект продемонстрирует тот же USART с использованием двух задач FreeRTOS и очереди сообщений.

Периферийное устройство USART/UART

В этой книге и в технической литературе в целом вы увидите, что термины «USART» и «UART» используются практически как синонимы. Разница между ними заключается в возможностях: USART – это синхронный/асинхронный приемник/передатчик, название UART исключает из обозначения слово «синхронный».

Периферийные устройства USART/UART отправляют данные по двум проводам последовательно. Один провод используется для отправки (TX), а другой – для приема (RX) данных. Подразумевается наличие общего провода между двумя конечными точками. Синхронная связь требует, чтобы одна сторона выступала в качестве ведущего и обеспечивала тактовый сигнал. Асинхронная связь не использует отдельный тактовый сигнал, но взамен требует, чтобы обе стороны точно согласовывали тактовую частоту устройства, обеспечивающую одинаковую скорость передачи данных. Асинхронная связь начинается со стартового бита и заканчивается стоповым битом для каждого символа.

Периферийные устройства USART, предоставляемые STM32F103, достаточно гибкие. Они действительно могут работать как USART или UART в зависимости от конфигурации. Для простоты в этой главе основное внимание будет уделено асинхронному режиму[36], поэтому название UART применимо к оставшейся части этой главы.

Асинхронные данные

На рис. 6.1 представлена осциллограмма передаваемого асинхронного байта 0x65 с пояснениями. Этот сигнал в TTL-уровнях[37] начинается (слева) с высо-

[36] Как и в большинстве практических случаев вообще, синхронный режим последовательного порта в реальных устройствах используется очень редко, только тогда, когда совершенно невозможно договориться о скорости передачи данных заранее.

[37] То есть измеренный на выводе TX микроконтроллера.

кого уровня (около 5 В) холостого хода линии. Начало символа отмечается младшим битом низкого уровня (около 0 В), известным как стартовый бит. Этот бит предупреждает получателя о том, что следом идут биты данных, начиная с младших битов (с прямым порядком байтов). В этом примере был показан символ с 8-битным значением. Конец символа отмечается стопбитом высокого уровня. Если стоповый бит не виден получателю, то отмечается ошибка.

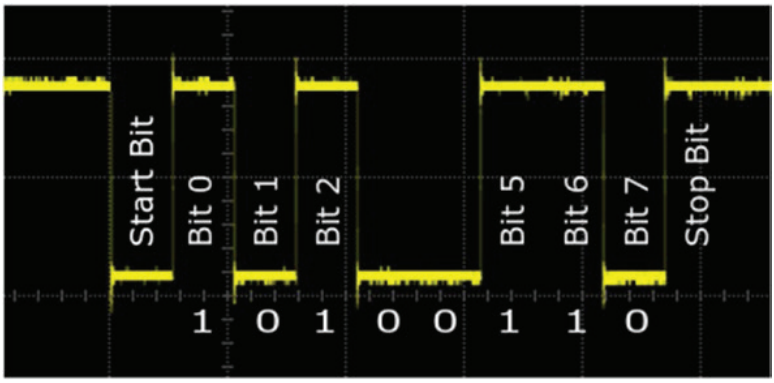


Рис. 6.1. Аннотированная осциллограмма сигнала UART для значения 0x65. Обратите внимание, что младшие биты отправляются первыми

Отправляемые значения могут иметь длину 8 или 9 бит[38]. Девятый бит является битом четности, если он включен. Стоповый бит (или биты) завершает передачу символа и может иметь длину 0.5, 1, 1.5 или 2 бита[39].

USB-адаптеры последовательного порта

Последовательный адаптер USB-TTL – чрезвычайно полезная вещь при работе с микроконтроллерами. При несложном подключении вы можете использовать терминальную программу на персональном компьютере для связи с STM32. Это устраняет необходимость в дорогостоящем ЖК-экране и клавиатуре.

Если вы еще не приобрели такой адаптер, вот несколько рекомендаций, на что обратить внимание:

- это должен быть переходник в уровни TTL (сигналы +5 или +3.3 В);

[38] В других микроконтроллерах значения символов могут быть также 5, 6 и 7-битными, что зависит от настроек; однако 8 и 9-битные все равно употребляются наиболее часто.

[39] Физически длина стопового бита не имеет особого значения, эти настройки устанавливают лишь минимальную длину для расчета длительности возможного таймаута на приемнике. Если присмотреться к осциллограмме на рис. 6.1, можно заметить, что стоповый бит, конец которого ничем не обозначен, неотличим от следующего за ним холостого хода линии (высокий уровень).

- USB-устройство должно поддерживаться вашей операционной системой[40];
- устройство поддерживает аппаратное управление потоком (наличие выводов RTS и CTS).

Наличие TTL-уровней важно. Обычные адаптеры USB/RS-232 работают при уровнях плюс и минус с амплитудой от 3 В и более. Их нельзя подключить напрямую к STM32.

Адаптеры последовательного порта USB-TTL подают сигнал от нуля до +5 В и могут использоваться с любым из 5-вольтовых входов. Компания STMicroelectronics предусмотрела, чтобы линии приема (RX) для UART 1 и UART 3 имели входы, устойчивые к напряжению 5 В. Отправка с 3.3-вольтового STM32 работает нормально, поскольку сигнал высокого уровня значительно превышает TTL-порог, необходимый для распознавания адаптером.

На рис. 6.2 показан адаптер, который используется автором. Напоминаем об обязательной поддержке аппаратного управления потоком в виде соединений для RTS и CTS. Без аппаратного управления потоком вы не сможете поддерживать более высокие скорости, например 115 200 бод, без потери данных. Смотрите также замечание о поддержке коммуникационного чипа FTDI на стр. 28.

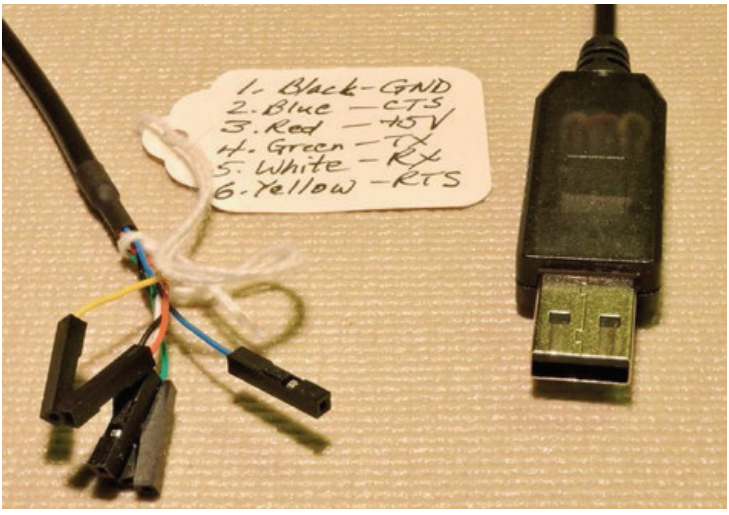


Рис. 6.2. Пример кабеля последовательного адаптера USB-TTL. Бирка была добавлена для обозначения, какого цвета провод несет какую функциональность

Подключение

Два проекта, представленные в этой главе, требуют подключения последовательного адаптера USB-TTL, чтобы иметь возможность просматривать выходные данные в вашем компьютере. На рис. 6.3 показаны необходимые соединения.

[40] Иными словами, необходимо наличие драйвера под операционную систему ПК.

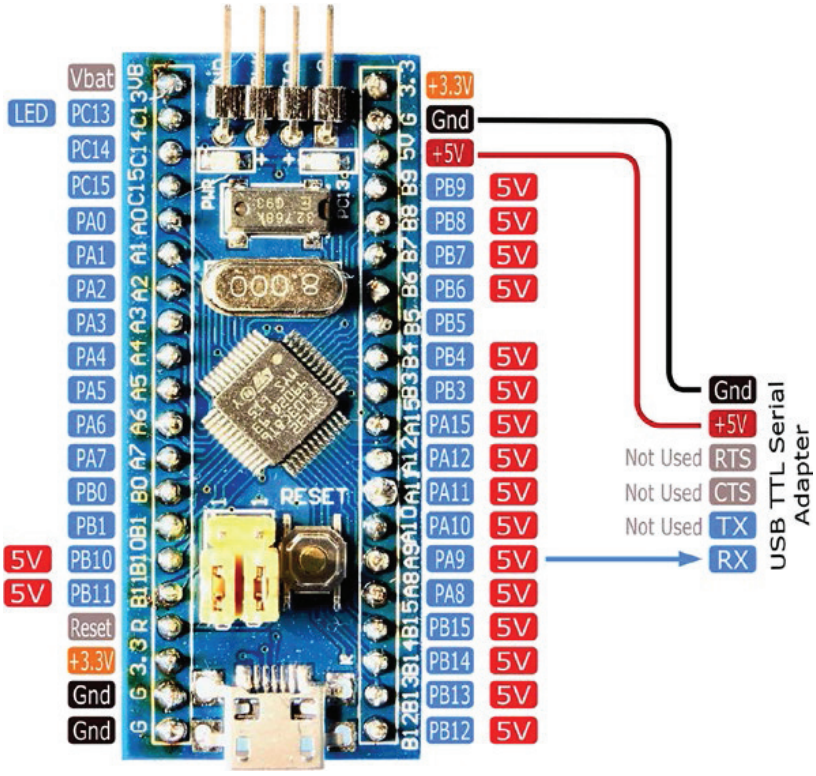


Рис. 6.3. Подключение адаптера USB-TTL для последовательной связи. В этом примере выводы управления потоком не используются

Скорость передачи данных, используемая в этой главе, составляет 38 400 бод, что является относительно низкой скоростью. Это позволяет нам избежать управления потоком для простоты нашей первой демонстрации. У вас должна быть возможность запитать микроконтроллер от линии +5 В последовательного адаптера, как показано на рисунке[41]. Подключите этот источник питания к входу +5 В вашей платы Blue Pill, чтобы встроенный регулятор подавал на МК напряжение 3.3 В.

Если это невозможно, то запитайте устройство отдельно. Обязательно обеспечьте общее заземление между источником питания, микроконтроллером и адаптером.

Наконец, обратите внимание, что для этих конкретных демонстраций требуется только одно соединение для передачи данных (RX). Это связано с тем, что эти демонстрационные программы только передают, а не получают данные с настольного компьютера.

[41] То есть фактически от USB-питания, поступающего с ПК с соответствующими ограничениями (как правило, ток не более 0.5 А). Отметим, что некоторые USB-TTL-адаптеры имеют встроенный стабилизатор питания 3.3 В, так что можно выбирать. Напоминаем, что при таком питании платы STM32, в отличие от плат Arduino, подавать стороннее питание запрещено.

Проект uart

Перейдите в следующий исходный каталог:

```
$ cd ~/stm32f103c8t6/rtos/uart
```

Перед подключением программатора убедитесь, что USB-адаптер отключен. Когда программатор готов, соберите программу и прошейте устройство следующим образом:

```
$ make clobber
$ make
$ make flash
```

После прошивки устройство начнет запускать программу и выведет текст. Чтобы увидеть его, вам нужно будет использовать программу эмулятора терминала. Отсоедините программатор ST-Link V2 и подключите USB-адаптер (соединения показаны на рис. 6.2). При подаче питания от адаптера вы должны увидеть горящими светодиод питания устройства STM32 и мигающий светодиод на PC13.

В этом тексте я буду использовать старую терминальную программу *minicom*, но есть еще одна хорошая программа – *putty*. При необходимости используйте менеджер пакетов вашей системы, чтобы установить любую из них.

Чтобы использовать адаптер последовательного порта, вам необходимо знать путь к устройству или COM-порт, специфичный для операционной системы. Для Mac или Linux вы можете обнаружить его, просто заглянув в каталог */dev*. На Mac устройство будет отображаться с префиксом */dev/cu*, когда оно подключено и активно (в противном случае найдите */dev/ttyusbserial**). Когда вы отключите его, это имя устройства исчезнет.

При использовании программы *minicom* вам необходимо сначала настроить коммуникационный порт, указав опцию -s:

```
$ minicom -s
+-----[configuration]------+
| Filenames and paths      |
| File transfer protocols  |
|>Serial port setup        |
| Modem and dialing        |
| Screen and keyboard      |
| Save setup as dfl        |
| Save setup as..          |
| Exit                     |
| Exit from Minicom        |
+--------------------------+
```

Прокрутите вниз до **Serial port setup** (Настройка последовательного порта) и нажмите <**Enter**>:

```
+-----------------------------------------------------------+
| A - Serial Device : /dev/cu.usbserial-A703CYQ5            |
| B - Lockfile Location : /usr/local/Cellar/minicom/2.7/var |
| C - Callin Program :                                      |
| D - Callout Program :                                     |
| E - Bps/Par/Bits : 2400 8O1                               |
| F - Hardware Flow Control : No                            |
| G - Software Flow Control : No                            |
|                                                           |
| Change which setting?                                     |
+-----------------------------------------------------------+
```

Введите «A», если указанный путь к устройству неверен. Убедитесь, что управление потоком отключено, при необходимости набрав «F» и/или «G». Наконец, введите «E», чтобы изменить настройки порта:

```
+-----------------+---------[Comm Parameters]----------+-------+
| A - Serial De|                          |       |
| B - Lockfile Loc| Current: 38400 8N1     |2.7/var|
| C - Callin Pro  | Speed Parity Data      |       |
| D - Callout Pro | A: <next> L: None S: 5 |       |
| E - Bps/Par/B   | B: <prev> M: Even T: 6 |       |
| F - Hardware Flo| C: 9600 N: Odd U: 7    |       |
| G - Software Flo| D: 38400 O: Mark V: 8  |       |
|                 | E: 115200 P: Space     |       |
| Change which    |                        |       |
+-----------------| Stopbits               |-------+
       | Screen a | W: 1              Q: 8-N-1 |
       | Save set | X: 2              R: 7-E-1 |
       | Save set |                            |
       | Exit     |                            |
       | Exit fro | Choice, or <Enter> to exit? |
       +----------+-----------------------------+
```

После отображения этой панели **Comm Parameters** (Параметры связи) вы можете ввести «Q», чтобы выбрать «8-N-1»[42], а затем ввести «D», чтобы установить скорость передачи данных 38 400 бод[43]. Затем дважды нажмите <**Enter**>, чтобы вернуться в главное меню.

В главном меню выберите **Save setup as...** (Сохранить настройки как...), чтобы сохранить настройки для следующего раза. Используйте, например, «chap6» в качестве имени и нажмите <**Enter**>. Если у вас возникли трудности с сохранением, возможно, это связано с тем, что в *minicom* из пакета установ-

[42] 8n1 означает 8 бит, нет проверки четности, 1 стоповый бит – самый распространенный режим последовательного порта.

[43] Боды (символы в секунду) – общепринятое обозначение единиц измерения скорости передачи в последовательном порту UART, для которого боды совпадают с битами в секунду. В общем случае во избежание неточностей следует указывать именно биты в секунду (бит/с, bps), так как для многих последовательных устройств (как, например, для модемов) боды и биты в секунду различаются.

лен каталог в месте, к которому у вас нет разрешений (большой вздох автора!). В этом случае вам следует использовать полный путь, начинающийся с косой черты, чтобы переопределить каталог.

```
+---------------------------------------+
|Give name to save this configuration?  |
|> chap6                                |
+---------------------------------------+
```

Если ваше устройство запустилось после прошивки, вы можете увидеть неполную первую строку, но остальная часть отклика должна быть примерно такой:

```
Welcome to minicom 2.7
OPTIONS:
Compiled on Sep 17 2016, 05:53:15.
Port /dev/cu.usbserial-A703CYQ5, 16:30:48
Press Meta-Z for help on special keys
UVWXYZ
0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ
0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ
0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ
0123456789:;<=>?@
```

Программа предназначена для медленной отправки повторяющихся строк с интервалом перед каждым следующим символом. Такая медленная скорость позволяет избежать необходимости управления потоком[44].

[44] Автор сильно преувеличивает медленность передачи при стандартных скоростях UART. Скорость 38 400 бит/с соответствует передаче 8-битного символа примерно за 0.25 мс, около 4000 символов в секунду. Если речь идет о передаче вручную, то даже рекордсмены по набору символов на клавиатуре выдают максимум 600 знаков в минуту, т. е. работают примерно в 400 раз медленнее. А примеры далее ориентированы на автоматическую посылку целых строк за один прием (см. пояснения к примеру на стр. 98). В этом случае передатчик все равно не может отправлять символы быстрее установленной скорости, потому приемнику просто не отправят следующий символ, пока не отправлен предыдущий. С синхронизацией приемника и передатчика при стандартных скоростях обмена обязаны справляться аппаратные схемы или подпрограммы управления портом, которые затормозят процесс передачи очередного символа и следующих за ним стоп-битов до окончания отправки предыдущего. Именно так и работает функция `usart_send_blocking()` в примере uart, см. стр. 99. В примере uart2 далее за время ожидания в передатчике даже успевает выполниться другая задача. Специальные меры для управления потоком данных (см. сноску 46 на стр. 104) требуются лишь при условии, что приемник перегружен параллельными задачами, и вы подозреваете, что он может не успевать принимать данные, идущие сплошным потоком, – вот тогда об этом нужно начинать задумываться. От скорости передачи это почти не зависит: надо еще учитывать, что при тактовых частотах контроллера во многие десятки мегагерц между скоростью обмена 115 200 бит/с (~10 Кбайт/с) и скоростью 9600 бит/с (~1 Кбайт/с) нет принципиальной разницы – в любом случае за время передачи успевают выполниться тысячи команд центрального процессора.

Если вам нужно перезапустить *minicom*, теперь вы можете использовать сохраненные настройки следующим образом:

```
$ minicom chap6
```

На Mac у *minicom* могут возникнуть проблемы с драйвером USB, если вы просто отключите адаптер, не выходя из программы. Чтобы выйти из *minicom*, используйте <**ESC+X**> (в некоторых системах вместо этого необходимо использовать <**Ctrl+A**> <**X**>).

```
+----------------------+
| Leave Minicom? |
| Yes         No |
+----------------------+
```

Yes (Да) должно быть выделено по умолчанию, что позволит вам просто нажать <**Enter**>. Если вы этого не видите, то вам нужно попробовать еще раз. Нажмите X сразу после нажатия <**ESC**> (или <**Ctrl+A**>), поскольку эта операция чувствительна ко времени. После того как *minicom* закрыл драйвер USB и закрылся, можно безопасно отключить адаптер. Если порт USB перестал откликаться, вы можете использовать другой порт или перезагрузиться.

Проект

Листинг 6.1 иллюстрирует основную программу *uart.c*. Единственное новое в основной функции – это вызов отдельной процедуры установки с именем *uart_setup()* в строке 94.

Листинг 6.1. Листинг основной программы *~/stm32f103c8t6/rtos/uart/uart.c*

```
0081: int
0082: main(void) {
0083:
0084:   rcc_clock_setup_pll(&rcc_hse_configs[RCC_CLOCK_HSE8_72MHZ]);
0085:
0086:   // PC13:
0087:   rcc_periph_clock_enable(RCC_GPIOC);
0088:   gpio_set_mode(
0089:       GPIOC,
0090:       GPIO_MODE_OUTPUT_2_MHZ,
0091:       GPIO_CNF_OUTPUT_PUSHPULL,
0092:       GPIO13);
0093:
0094:   uart_setup();
0095:
0096:  xTaskCreate(task1,"task1",100,NULL, configMAX_PRIORITIES-1, NULL);
0097:   vTaskStartScheduler();
0098:
```

```
0099:   for (;;);
0100:   return 0;
0101: }
```

Листинг 6.2 иллюстрирует код настройки для UART1. Обратите внимание, что в строках 31 и 32 включены две системы синхронизации. Выход TX UART1 по умолчанию выводится на PA9 (позже в книге мы рассмотрим ввод-вывод через альтернативные функции GPIO), поэтому подсистеме GPIOA необходимо включить тактовую частоту. Периферийному устройству USART1 также требуется тактовый сигнал, который включается в строке 32.

В строках 35–38 используется функция `gpio_set_mode()` для настройки выходного вывода GPIOA. Обратите внимание, что здесь выбрана опция `GPIO_MODE_OUTPUT_50_MHZ` с более высокой частотой, чем ранее, чтобы обеспечить более резкие изменения сигнала. Обратите особое внимание на то, что настройка `GPIO_CNF_OUTPUT_ALTFN_PUSHPULL` указывает, что вывод является ALTFN (а не GPIO) и что он должен использовать конфигурацию двухтактного выхода (push/pull). Аспект ALTFN здесь очень важен – распространенной ошибкой является выбор одной из конфигураций GPIO (извиняюсь за настойчивость).

В строке 38 указана настройка `GPIO_USART1_TX` библиотеки libopencm3, что на платформе STM32F103 соответствует выводу PA9. Использование вывода GPIO13 было бы также допустимым, хотя так код, как было указано, более переносим.

Строка 40 вызывает функцию `usart_set_baudrate()`, чтобы установить скорость передачи данных 38 400 бод. Эта функция вычисляет делитель тактовой частоты, необходимый для получения приблизительного значения скорости передачи данных. Произвольно выбранным скоростям передачи данных может не хватать точности, которой обладают стандартные скорости.

В строке 41 используется функция `usart_set_databits()` для настройки количества битов, которые будет содержать каждый символ. Здесь допустимыми вариантами являются 8 или 9. При включенной проверке четности это подразумевает 7 или 8 бит данных соответственно.

В строке 42 настраивается один стоповый бит, а в строке 43 периферийное устройство настроено только на передачу. Строка 44 указывает, что бит четности отправляться не будет, а строка 45 указывает, что аппаратное управление потоком данных использоваться не будет. Наконец, строка 46 разрешает работу периферийного устройства.

Листинг 6.2. Листинг `uart_setup()` в файле *stm32/rtos/uart/uart.c*

```
0028: static void
0029: uart_setup(void) {
0030:
0031:   rcc_periph_clock_enable(RCC_GPIOA);
0032:   rcc_periph_clock_enable(RCC_USART1);
0033:
0034:   // UART TX on PA9 (GPIO_USART1_TX)
0035:   gpio_set_mode(GPIOA,
```

```
0036:       GPIO_MODE_OUTPUT_50_MHZ,
0037:       GPIO_CNF_OUTPUT_ALTFN_PUSHPULL,
0038:       GPIO_USART1_TX);
0039:
0040:   usart_set_baudrate(USART1,38400);
0041:   usart_set_databits(USART1,8);
0042:   usart_set_stopbits(USART1,USART_STOPBITS_1);
0043:   usart_set_mode(USART1,USART_MODE_TX);
0044:   usart_set_parity(USART1,USART_PARITY_NONE);
0045:   usart_set_flow_control(USART1,USART_FLOWCONTROL_NONE);
0046:   usart_enable(USART1);
0047: }
```

Как видите, есть несколько деталей UART, которые требуют настройки, и все они должны соответствовать тем, что вы используете в принимающей программе настольного компьютера.

Функция `task1()` нашего приложения отправляет данные другой подпрограмме `uart_putc()`, которая представлена далее. Функция `task1()` показана в листинге 6.3. Задача предназначена для вывода строк текста следующего вида:

```
0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

В рамках цикла переключается встроенный светодиод PC13 (строка 65). Это дает нам уверенность в том, что программа работает, если возникнут проблемы с передачей результатов вывода UART на настольный компьютер. Перед каждым символом задача ожидает 200 мс, чтобы замедлить отправку (строка 66). Это избавляет нас от необходимости иметь дело с управлением потоком данных[45]. Строки 67–74 передают символ (строка 67 увеличивает счетчик `c`) путем вызова `uart_putc()`.

Листинг 6.3. Функция `task1( )` прикладной программы

```
0060: static void
0061: task1(void *args __attribute__((unused))) {
0062:   int c = '0' - 1;
0063:
0064:   for (;;) {
0065:      gpio_toggle(GPIOC,GPIO13);
0066:      vTaskDelay(pdMS_TO_TICKS(200));
0067:      if ( ++c >= 'Z' ) {
0068:          uart_putc(c);
0069:          uart_putc('\r');
```

[45] Задержка передачи здесь может понадобиться только для того, чтобы приемная программа не «молотила» выводимые строки без перерыва, не давая их прочесть. Рекомендуется попробовать удалить задержку (к тому же явно избыточную по длительности) – программа должна работать без нее (см. по этому поводу сноску 44 на стр. 95).

```
0070:           uart_putc('\n');
0071:           c = '0' - 1;
0072:       } else {
0073:           uart_putc(c);
0074:       }
0075:   }
0076: }
```

В листинге 6.4 показана функция `uart_putc()`, которая просто вызывает процедуру `usart_send_blocking()` из libopencm3. Как следует из названия функции, управление не возвращается до тех пор, пока USART не будет готов принять дополнительные данные. В следующем примере будет применен более удобный подход.

Листинг 6.4. Функция `uart_putc()` в файле *uart.c*

```
0052: static inline void
0053: uart_putc(char ch) {
0054:   usart_send_blocking(USART1,ch);
0055: }
```

По сути, этот пример сводится к следующим основным моментам:

1) как настроить и включить UART для передачи,
2) как применить билиотеку libopencm3 для отправки данных на UART1.

Несмотря на то что основной поток выполняет планирование задач, а приложение выполняется в функции `task1()`, дизайн программы не слишком хорош. Пример работает так, как представлено, но давайте его еще немного разовьем и исправим недостатки дизайна.

Проект uart2

Перейдите в следующий каталог проекта:

```
$ cd ~/stm32f103c8t6/rtos/uart2
```

Исходный модуль `uart.c` в этом проекте имеет некоторые улучшения. Во-первых, есть новый включаемый файл с именем `queue.h` (строка 15), предоставляемый FreeRTOS. Это позволяет объявить дескриптор очереди сообщений `uart_txq` в строке 21 (листинг 6.5).

Листинг 6.5. Включаемые файлы, используемые *stm32/rtos/uart2/uart.c*

```
0013: #include <FreeRTOS.h>
0014: #include <task.h>
0015: #include <queue.h>
0016:
0017: #include <libopencm3/stm32/rcc.h>
```

```
0018: #include <libopencm3/stm32/gpio.h>
0019: #include <libopencm3/stm32/usart.h>
0020:
0021: static QueueHandle_t uart_txq; // TX queue for UART
```

Процедура установки остается той же, за исключением создания очереди сообщений в строке 47 листинга 6.6. Вызов создает очередь сообщений, которая будет содержать максимум 256 сообщений, каждое из которых имеет длину 1 байт. Затем переменная `uart_txq` получает действительный дескриптор очереди.

Листинг 6.6. Функция `uart_setup()`

```
0026: static void
0027: uart_setup(void) {
0028:
0029:   rcc_periph_clock_enable(RCC_GPIOA);
0030:   rcc_periph_clock_enable(RCC_USART1);
0031:
0032:   // UART TX on PA9 (GPIO_USART1_TX)
0033:   gpio_set_mode(GPIOA,
0034:       GPIO_MODE_OUTPUT_50_MHZ,
0035:       GPIO_CNF_OUTPUT_ALTFN_PUSHPULL,
0036:       GPIO_USART1_TX);
0037:
0038:   usart_set_baudrate(USART1,38400);
0039:   usart_set_databits(USART1,8);
0040:   usart_set_stopbits(USART1,USART_STOPBITS_1);
0041:   usart_set_mode(USART1,USART_MODE_TX);
0042:   usart_set_parity(USART1,USART_PARITY_NONE);
0043:   usart_set_flow_control(USART1,USART_FLOWCONTROL_NONE);
0044:   usart_enable(USART1);
0045:
0046:   // Create a queue for data to transmit from UART
0047:   uart_txq = xQueueCreate(256,sizeof(char));
0048: }
```

Процедура записи символов теперь запускается из функции `uart_task()`, которая спланирована как задача в листинге 6.7.

Листинг 6.7. Функция задачи `uart_task()`

```
0053: static void
0054: uart_task(void *args __attribute__((unused))) {
0055:   char ch;
0056:
0057:   for (;;) {
0058:       // Receive char to be TX
0059:       if ( xQueueReceive(uart_txq,&ch,500) == pdPASS ) {
0060:          while ( !usart_get_flag(USART1,USART_SR_TXE) )
```

```
0061:              taskYIELD(); // Yield until ready
0062:           usart_send(USART1,ch);
0063:       }
0064:       // Toggle LED to show signs of life
0065:       gpio_toggle(GPIOC,GPIO13);
0066:   }
0067: }
```

Функция `uart_task()` работает в цикле, начиная со строки 57. Функция FreeRTOS `xQueueReceive()` вызывается для получения сообщения. Последний аргумент 500 указывает, что этот вызов должен закончиться по истечении 500 тактов. По истечении времени встроенный светодиод на PC13 может переключаться, указывая на то, что программа все еще работает. Функция `xQueueReceive()` возвращает `pdFAIL` по истечении времени ожидания.

Когда `xQueueReceive()` возвращает `pdPASS`, это означает, что задача получила сообщение. В нашей демонстрации сообщение принимается в виде одного символа в переменную `ch`. Как только мы получили символ из очереди, нам нужно отправить его в UART.

Обратите внимание на цикл `while` в строках 60 и 61. Он вызывает функцию FreeRTOS `TaskYIELD()`до тех пор, пока UART не сможет принять другой символ (в строке 62). Библиотека libopencm3 предоставляет функцию `usart_get_flag()`, позволяющую проверять различные флаги регистров состояния. Таким образом, проверяется бит TXE регистра статуса (передача пуста). Пока этот регистр указывает «не пуст», мы указываем планировщику выполнить другую задачу, вызывая `taskYIELD()`.

Если мы не передаем управление, функция `usart_send_blocking()` просто крутится, ожидая готовности UART. Если UART не будет готов вовремя, этот процесс будет расходовать процессорное время до тех пор, пока не истечет заданный временной интервал этой задачи. Такое зацикливание попрежнему будет нормально работать для приложения, но будет тратить время процессора, которое можно было бы с большей пользой использовать в другом месте. Поскольку в строке 62 флаг TXE указывает, что UART готов, вместо этого мы можем использовать функцию `usart_send()`.

Функции `uart_task()` и `demo_task()` выполняются одновременно. Листинг 6.8 иллюстрирует новую функцию `demo_task()`, которая ставит в очередь пары строк для отправки. Листинг 6.10 иллюстрирует изменения в основной программе, внесенные для реализации этих двух задач.

Листинг 6.8. Функция `demo_task()`, создающая повторяющуюся пару строк

```
0081: /*******************************************************
0082: * Demo Task:
0083: * Simply queues up two line messages to be TX, one second
0084: * apart.
0085: *******************************************************/
0086: static void
0087: demo_task(void *args __attribute__((unused))) {
0088:
```

```
0089:   for (;;) {
0090:       uart_puts("Now this is a message..\n\r");
0091:       uart_puts(" sent via FreeRTOS queues.\n\n\r");
0092:       vTaskDelay(pdMS_TO_TICKS(1000));
0093:   }
0094: }
```

Функция `demo_task()` вызывает новую функцию, показанную в листинге 6.9.

Функция `uart_puts()` просто вызывает функцию `uart_putc()` для помещения каждого символа строки в UART. Важно отметить, что вызов `xQueueSend()` в строке 77 заблокируется, если очередь заполнится. Третий аргумент указывает задержку `portMAX_DELAY`, чтобы порт заблокировался, пока очередь не освободится. Поскольку это вызов от FreeRTOS, функция знает, что нужно передать управление другой задаче, когда очередь заполнена.

Листинг 6.9. Функция `uart_puts()` использует `uart_putc()` для передачи строки символов в UART.

```
0072: static void
0073: uart_puts(const char *s) {
0074:
0075:   for ( ; *s; ++s ) {
0076:       // blocks when queue is full
0077:       xQueueSend(uart_txq,s,portMAX_DELAY);
0078:   }
0079: }
```

Основная программа в листинге 6.10 просто дважды вызывает `xTaskCreate()`, чтобы установить две задачи. Одна выполняет функцию `uart_task()` (строка 114), а другая выполняет `demo_task()` (строка 115). Порядок создания неважен, поскольку планировщик FreeRTOS не запускается ранее строки 117.

Листинг 6.10. Основная программа для *stm32/rtos/uart2/uart.c*

```
0099: int
0100: main(void) {
0101:
0102:   rcc_clock_setup_pll(&rcc_hse_configs[RCC_CLOCK_HSE8_72MHZ]);
0103:
0104:   // GPIO PC13:
0105:   rcc_periph_clock_enable(RCC_GPIOC);
0106:   gpio_set_mode(
0107:        GPIOC,
0108:        GPIO_MODE_OUTPUT_2_MHZ,
0109:        GPIO_CNF_OUTPUT_PUSHPULL,
0110:        GPIO13);
0111:
0112:   uart_setup();
0113:
```

```
0114:   xTaskCreate(uart_task,"UART",100,NULL,configMAX_PRIORITIES-1,NULL);
0115:   xTaskCreate(demo_task,"DEMO",100,NULL,configMAX_PRIORITIES-2,NULL);
0116:
0117:   vTaskStartScheduler();
0118:   for (;;);
0119:   return 0;
0120: }
```

Общий обзор работы программы.

1. Задача `demo_task()` вызывает функцию `uart_puts()` для отправки строк текста в UART с интервалом в одну секунду.
2. Функция `uart_puts()` вызывает `uart_putc()` для постановки символов в очередь сообщений, на которую ссылается дескриптор `uart_txq`. Если очередь заполнена, управление `demo_task()` прекращается.
3. Задача `uart_task()` удаляет символы из очереди, на которую ссылается дескриптор `uart_txq`.
4. Каждый полученный символ доставляется в UART для отправки при условии, что порт готов. Когда UART занят, управление задачей уступает другой задаче.
5. Хотя это был простой пример, мы видим элегантность FreeRTOS в действии. Одна задача производит, а другая потребляет. Контуры управления для обоих тривиальны. Разбивая приложение на задачи, мы разбиваем проблему на управляемые компоненты. Мы видим, что межзадачная связь может быть безопасно реализована через очередь сообщений FreeRTOS.

USART API

Для справки давайте перечислим функции API USART, используемые в этой главе, и разберем их аргументы. Кроме этих, добавим расширенные функции API, позволяющие хранить ссылку в одном месте.

Для перечисленных функций некоторым аргументам требуются специальные макроопределения, которые предоставляются библиотекой libopencm3. Они перечислены в табл. 6.1–6.7. В заголовке таблицы указано имя аргумента, на который ссылаются значения. Например, в табл. 6.2 перечислены заданные имена макроопределений для аргумента четности.

В табл. 6.1 перечислены все устройства USART, доступные для STM32F103C8T6. Для каждой функции указаны выводы по умолчанию. Например, USART2 по умолчанию получает данные на PA3, если не была применена конфигурация с альтернативной функцией.

*Таблица 6.1. USART, доступные для STM32F103C8T6 (аргумент **usart**)*

USART №	Имя	5V	TX	RX	CTS	RTS
1	USART1	Yes	PA9	PA10	PA11	PA12
2	USART2	No	PA2	PA3	PA0	PA1
3	USART3	Yes	PB10	PB11	PB14	PB12

Таблица 6.2. Значения проверки четности USART (аргумент `parity`)

Имя	Описание
`USART_PARITY_NONE`	Нет проверки
`USART_PARITY_EVEN`	Четная четность
`USART_PARITY_ODD`	Нечетная четность
`USART_PARITY_MASK`	Маска

Таблица 6.3. Режимы работы USART (аргумент `mode`)

Имя	Описание
`USART_MODE_RX`	Только прием
`USART_MODE_TX`	Только передача
`USART_MODE_TX_RX`	Передача и прием
`USART_MODE_MASK`	Маска

Таблица 6.4. Стоповые биты USART (аргумент `stopbits`)

Имя	Описание
`USART_STOPBITS_0_5`	0.5 стоп-бита
`USART_STOPBITS_1`	1 стоп-бит
`USART_STOPBITS_1_5`	1.5 стоп-бита
`USART_STOPBITS_2`	2 стоп-бита

Таблица 6.5. Выводы управления потоком USART[46]

Имя	Описание
`USART_FLOWCONTROL_NONE`	Нет аппаратного управления потоком
`USART_FLOWCONTROL_RTS`	Аппаратное управление потоком RTS
`USART_FLOWCONTROL_CTS`	Аппаратное управление потоком CTS
`USART_FLOWCONTROL_RTS_CTS`	Аппаратное управление потоком RTS и CTS
`USART_FLOWCONTROL_MASK`	Маска

Таблица 6.6. Количество бит данных USART (аргумент `bits`)

Имя	Биты данных (без проверки четности)	Биты данных (с проверкой четности)
`8`	8	7
`9`	9	8

[46] Названия выводов RTS и CTS управления потоком данных расшифровываются как Clear To Send («очищено для передачи») и Ready To Send («готов к передаче»). Последнюю иногда расшифровывают как Request To Send («запрос передачи»). Эти выводы используются для предотвращения потерь данных, связанных с неготовностью одной из сторон принять данные. Обычно при соединении двух устройств CTS (вход) одного устройства соединяется с RTS (выход) другого. Когда входной буфер переполнен, RTS равен 1. Когда входной буфер снова получает место для данных, RTS становится равным 0. CTS проверяется перед отправкой данных. Если CTS равен 1, значит, у удаленного устройства RTS равен 1, т. е. оно не готово принять данные.

Таблица 6.7. Биты флагов статуса USART (аргумент `flag`)

Имя	Описание
USART_SR_CTS	Флаг «очищено для передачи»
USART_SR_LBD	Флаг обнаружения разрыва линии
USART_SR_TXE	Буфер передаваемых данных пуст
USART_SR_TC	Передача завершена
USART_SR_RXNE	Регистр данных чтения не пуст
USART_SR_IDLE	Обнаружена незанятая линия
USART_SR_ORE	Ошибка переполнения
USART_SR_NE	Флаг ошибки шума
USART_SR_FE	Ошибка фрейма
USART_SR_PE	Ошибка четности

Включаемые файлы

```
#include <libopencm3/stm32/rcc.h>
#include <libopencm3/stm32/usart.h>
```

Тактовые сигналы

```
rcc_periph_clock_enable(RCC_GPIOx);
rcc_periph_clock_enable(RCC_USARTn);
```

Конфигурация

```
void usart_set_mode(uint32_t usart, uint32_t mode);
void usart_set_baudrate(uint32_t usart, uint32_t baud);
void usart_set_databits(uint32_t usart, uint32_t bits);
void usart_set_stopbits(uint32_t usart, uint32_t stopbits);
void usart_set_parity(uint32_t usart, uint32_t parity);
void usart_set_flow_control(uint32_t usart, uint32_t flowcontrol);
void usart_enable(uint32_t usart);
void usart_disable(uint32_t usart);
```

DMA[47]

```
void usart_enable_rx_dma(uint32_t usart);
void usart_disable_rx_dma(uint32_t usart);
void usart_enable_tx_dma(uint32_t usart);
void usart_disable_tx_dma(uint32_t usart);
```

Прерывания

```
void usart_enable_rx_interrupt(uint32_t usart);
void usart_disable_rx_interrupt(uint32_t usart);
void usart_enable_tx_interrupt(uint32_t usart);
```

[47] Direct Memory Acsess, прямой доступ к памяти.

```
void usart_disable_tx_interrupt(uint32_t usart);
void usart_enable_error_interrupt(uint32_t usart);
void usart_disable_error_interrupt(uint32_t usart);
```

Статус ввод/вывод

```
bool usart_get_flag(uint32_t usart, uint32_t flag)
void usart_send(uint32_t usart, uint16_t data)
uint16_t usart_recv(uint32_t usart)
```

Последовательность настроек

Ниже приводится краткое описание настроек (за исключением прерываний и DMA), которые необходимо выстроить в определенной последовательности, чтобы периферийное устройство UART работало правильно.

1. Включите соответствующие тактовые сигналы GPIO для всех задействованных выводов: `rcc_periph_clock_enable(RCC_GPIOx)`.
2. Включите тактовый сигнал для выбранного периферийного устройства UART: `rcc_periph_clock_enable(RCC_USARTn)`.
3. Настройте режим выводов с помощью `gpio_set_mode()`:
   а) для выходов выберите `GPIO_CNF_OUTPUT_ALTFN_PUSHPULL` в качестве третьего аргумента (обратите внимание на `ALTFN`);
   б) для входов выберите `GPIO_CNF_INPUT_PULL_UPDOWN` или `GPIO_CNF_INPUT_FLOAT`.
4. `usart_set_baudrate()`.
5. `usart_set_databits()`.
6. `usart_set_stopbits()`.
7. `usart_set_mode()`.
8. `usart_set_parity()`.
9. `usart_set_flow_control()`.
10. `usart_enable()`.

FreeRTOS

В этой главе мы использовали несколько функций API FreeRTOS, часть из которых мы видели раньше. Для вашего удобства они будут суммированы здесь.

Задачи

Ниже приведены функции FreeRTOS, которые мы использовали для создания задач, запуска планировщика и задержки выполнения соответственно:

```
BaseType_t xTaskCreate(
    TaskFunction_t pvTaskCode, // function ptr
    const char * const pcName, // string name
    unsigned short usStackDepth, // stack size in words
```

```
    void *pvParameters, // Pointer to argument
    uBaseType_t uxPriority, // Task priority
    TaskHandle_t *pxCreatedTask // NULL or pointer to task handle
); // Returns: pdPass or errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY
void vTaskStartScheduler(void); // Start the task scheduler
void vTaskDelay(TickType_t xTicksToDelay);
void taskYIELD();
```

Значение указателя `pvTaskCode` – это просто указатель на функцию следующего вида:

```
void my_task(void *args)
```

Значение, передаваемое в `args`, берется из `pvParameters` в вызове `xTaskCreate()`. Если это не требуется, значение может быть указано со значением `NULL`. Глубина стека `usStackDepth` указывается в словах (по 4 байта каждое).

Каждая задача имеет связанный с ней приоритет, который предоставляется аргументом `uxPriority`. Если вы выполняете все задачи с одинаковым приоритетом, укажите значение `configMAX_PRIORITIES-1` или просто используйте значение 1. Если вам не нужны разные приоритеты, установите для них одно и то же значение (причины см. в главе 21). Имейте в виду, что при необходимости вы можете создавать дополнительные задачи после вызова планировщика `vTaskStartScheduler()`.

Обычно используемое макроопределение для `vTaskDelay()` выглядит следующим образом:

```
pdMS_TO_TICKS(ms) // Macro: convert ms to ticks
```

Это преобразует время в миллисекундах в количество тактов для удобства программирования.

Очереди

Функции API очереди, используемые в этой главе, включают следующее:

```
QueueHandle_t xQueueCreate(
    UBaseType_t uxQueueLength, // Max # of items
    UBaseType_t uxItemSize     // Item size (bytes)
);                             // Returns: handle else NULL
BaseType_t xQueueSend(
    QueueHandle_t xQueue,      // Queue handle
    const void *pvItemToQueue, // pointer to item
    TickType_t xTicksToWait    // 0, ticks or portMAX_DELAY
);                             // Returns: pdPASS or errQUEUE_FULL
BaseType_t xQueueReceive(
    QueueHandle_t xQueue,      // Queue handle
    void *pvBuffer,            // Pointer to receiving item buffer
    TickType_t xTicksToWait    // 0, ticks or portMAX_DELAY
);                             // Returns: pdPASS or errQUEUE_EMPTY
```

Функция `xQueueCreate()` выделяет область памяти для созданной очереди и возвращает ее дескриптор. Аргумент `uxQueueLength` указывает максимальное количество элементов, которые могут храниться в очереди. Значение `uxItemSize` определяет размер каждого элемента.

Функция `xQueueSend()` добавляет элемент в очередь. Элемент, на который указывает `pvItemToQueue`, копируется в хранилище очереди. И наоборот, `xQueueReceive()` берет элемент из очереди и копирует его в хранилище вызывающего объекта по адресу `pvBuffer`. Размер этого буфера должен быть не меньше размера, заданного в `uxItemSize`, иначе это приведет к повреждению памяти. Если нет элемента для получения, вызов блокируется в соответствии с `xTicksToWait`.

Итоги

В этой главе продемонстрирован общий рецепт настройки и активации USART в асинхронном режиме. Это позволяет плате Blue Pill отправлять данные на настольный компьютер для отладки или создания отчетов.

При этом была предоставлена демонстрация задач FreeRTOS и очередей сообщений. Такой подход разделил отправляющую и принимающую стороны приложения на отдельные задачи. Это упростило программирование, поскольку каждому нужно было заниматься только своей собственной работой. Очередь сообщений обеспечивала канал для взаимодействия между задачами приложения.

Упражнения

1. Каково состояние уровня TTL сигнала покоя USART?
2. Данные USART имеют последовательность с прямым или обратным порядком битов?
3. Какие тактовые частоты должны быть включены для использования UART?
4. Что означает аббревиатура 8n1?
5. Что произойдет, если вы предоставите UART-данные для отправки, когда приемное устройство еще не освободилось?
6. Можно ли создавать задачи до, после или до и после вызова `vTaskStartScheduler()`?
7. Каков минимальный размер буфера, определяемый функцией `xQueueReceive()`?
8. Как указать, что функция `xQueueSend()` должна немедленно вернуть управление, если очередь заполнена?
9. Как указать, что `xQueueReceive()` должен блокироваться, если очередь пуста?
10. Что произойдет с задачей, если `xQueueReceive()` обнаружит, что очередь пуста, и придется ждать ее наполнения?

Глава 7 • Последовательный порт USB

Одной из приятных особенностей микроконтроллера STM32 является наличие периферийного устройства USB. Благодаря USB можно напрямую взаимодействовать с настольной платформой в различных режимах. Одним из таких гибких режимов является эмуляция последовательного канала USB между микроконтроллером и настольным компьютером.

В этой главе будет рассмотрено совместное использование libopencm3 и FreeRTOS для обеспечения удобных средств связи. Вы будете использовать класс работы USB CDC (communication device class, класс устройства связи). Это обеспечивает очень удобное средство взаимодействия с платой Blue Pill.

Проблема с USB на плате Blue Pill

Во-первых, давайте проясним ситуацию по поводу проблемы Blue Pill USB. О какой именно проблеме вы, возможно, читали на интернет-форумах?

Оказывается, печатная плата изготовлена с резистором 10 кОм (R_{10}), подтягивающим к питанию +3.3 В, что неправильно. Для полноскоростного USB сопротивление должно составлять 1.5 кОм. Вы можете проверить это, измерив сопротивление с помощью цифрового мультиметра между выводом PA12 на печатной плате и контактом +3.3 В. Скорее всего, вы увидите значение 10 кОм[48].

Однако этот дефект не всегда мешает его работе. Например, у меня не было проблем с подключением USB от STM32 к MacBook Pro. Но ваш случай может отличаться. Прямой способ исправить это – заменить резистор R_{10} на печатной плате, но это сложно, поскольку резистор невероятно мал.



Многие люди сообщают на интернет-форумах, что их USB-разъем Blue Pill сломался или стал неработоспособным. Соблюдайте особую осторожность при вставке кабеля.

Устранить проблему проще всего, подключив параллельно ему еще один резистор. Если подключить резистор сопротивлением 1.8 кОм параллельно резистору сопротивлением 10 кОм, общее сопротивление составит около 1.5 кОм. На рис. 7.1 показано, как автор припаял резистор к одному из своих блоков. Резистор 0.125 Вт просто аккуратно припаивается между контактом PA12 и контактом +3.3 В. Это некрасиво, но работает!

Чтобы увидеть, как сопротивление влияет на разницу, посмотрите на экран осциллографа (рис. 7.2 и 7.3). Вот как выглядела линия D+ со значением сопротивления по умолчанию 10 кОм.

---

[48] Обязательно проверьте значение, если соберетесь выполнять доработку. Согласно официальной схеме на сайте https://stm32-base.org сопротивление резистора R_{10} должно составлять 4.7 кОм. Вполне возможно, USB будет работать нормально с таким значением.

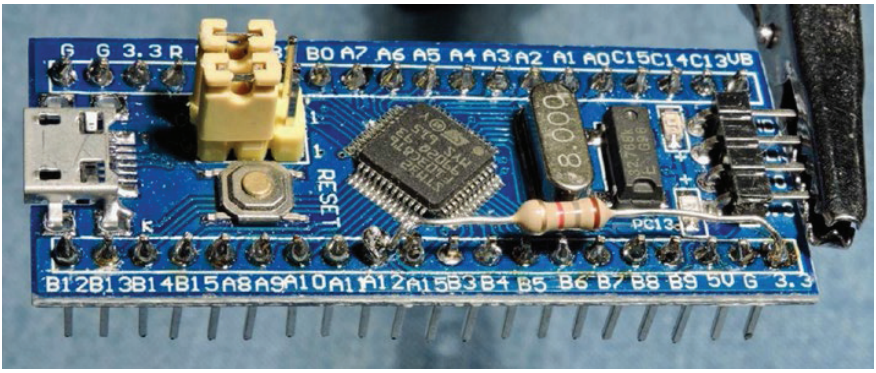


Рис. 7.1. Исправление подтяжки линии USB добавлением резистора сопротивлением 1.8 кОм

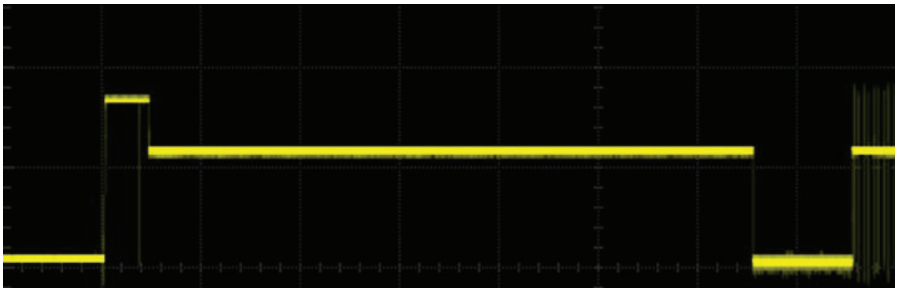


Рис. 7.2. Осциллограмма линии D+ с использованием нагрузочного сопротивления 10 кОм

На рисунке вы можете видеть скачок в начале, за которым следует спад, возможно, до уровня 70 %. Справа, где начинаются высокочастотные сигналы, вы можете видеть, что между импульсами уровень сигнала остается примерно на уровне 70 %. Подключите это устройство к другому USB-порту или USB-концентратору, и ухудшение качества может быть еще сильнее.

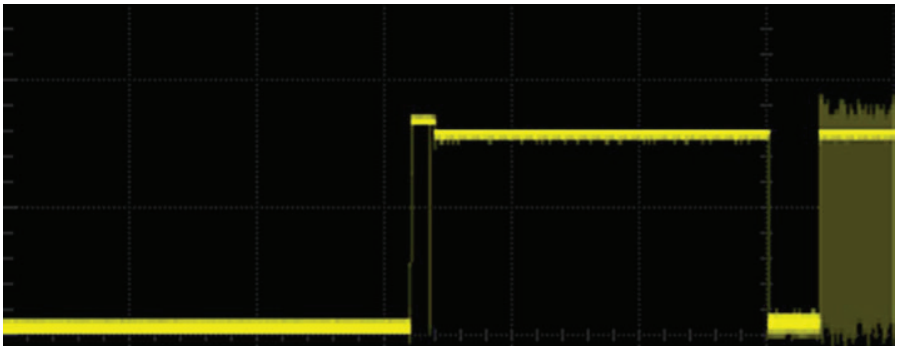


Рис. 7.3. Осциллограмма линии D+ с подтягивающим сопротивлением 1.5 кОм

Сравните это с рис. 7.3, на котором показана осциллограмма после установки подтягивающего сопротивления 1.5 кОм.

Вы можете видеть, что сигнал остается намного выше, возможно, на уровне 90 % от питания. Это помогает обеспечить улучшенные пороги сигнала и увеличить помехоустойчивость.

Введение в USB

USB – популярное средство связи персонального компьютера с различными периферийными устройствами, такими как принтеры, сканеры, клавиатуры и мышь. Частично его успех обусловлен стандартизацией и низкой стоимостью. В стандарт также входят USB-концентраторы, позволяющие экономично расширить сеть для размещения дополнительных устройств.

При USB-связи весь трафик направляется хостом. Каждое устройство регулярно опрашивается в зависимости от его конфигурации и требований. Клавиатуре нечасто приходится отправлять данные, например, в то время как звуковому записывающему устройству необходимо отправлять объемные данные в режиме реального времени. Эти различия предусмотрены стандартом USB и являются частью конфигурации устройства.

Каналы и конечные точки

USB использует концепцию конечных точек с соединительными каналами для передачи данных. Канал передает информацию, а конечные точки отправляют или получают. Каждое USB-устройство имеет по крайней мере одну конечную точку, известную как конечная точка 0. Это конечная точка по умолчанию является и контрольной точкой, которая позволяет хосту и устройству настраивать операции и параметры, специфичные для устройства. Это происходит во время обнаружения устройств.

На рис. 7.4 представлено высокоуровневое представление конечных точек 0, 1 и 2, которые мы будем использовать в примере программы. Технически конечная точка 0 – это всего лишь один канал. Здесь он изображен в виде двух каналов, поскольку конечная точка управления разрешает обратный ответ хосту. Все остальные конечные точки передают данные только в одном направлении. Обратите внимание, что «In» (Вход) и «Out» (Выход) на рис. 7.4 помечены с точки зрения хост-контроллера.

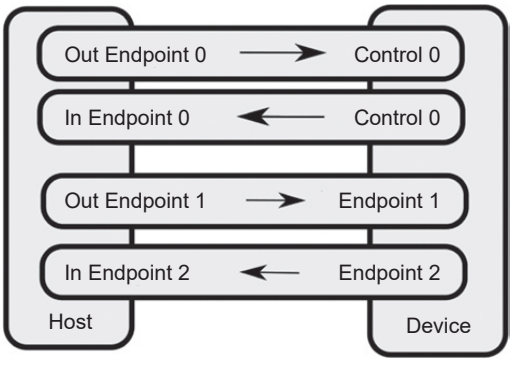


Рис. 7.4. USB-каналы и конечные точки

Устройство может иметь дополнительные конечные точки, но в нашем примере USB CDC нужны только две в дополнение к требуемой конечной точке управления 0:

- конечная точка 1 – это конечная точка приема устройства (выход хоста), специфицируется как 0x01;
- конечная точка 2 – это конечная точка отправки устройства (вход хоста), специфицируется как 0x82.

Как видно из нумерации, бит 7 номера конечной точки указывает, является ли она входом или выходом (по отношению к хост-контроллеру). Значение 0x82 указывает в шестнадцатеричном формате, что конечная точка 2 (с единичным битом 7) отправляет данные (с точки зрения устройства). В отличие от сокета TCP/IP, USB-каналы передают данные только в одном направлении.

Как вы, возможно, поняли, один из потенциально запутанных аспектов программирования USB заключается в том, что вход и выход указываются в коде с точки зрения хост-контроллера. Например, конечная точка 0x82 является принимающей (входной) конечной точкой с точки зрения хоста. Это может сбивать с толку при написании программ для устройства. Помните об этом при настройке дескрипторов USB.

Это было обязательное краткое введение в USB. На эту тему написаны целые книги, и заинтересованному читателю рекомендуется поискать их. Наше внимание будет ограничено успешным использованием периферийного USB-устройства в интересах нашего STM32. Давайте начнем!

Периферийное устройство последовательного порта USB

Когда МК прошит и подключен к компьютеру, вам необходимо получить к нему доступ из вашей операционной системы как к последовательному устройству. Это действие зависит от операционной системы, что немного усложняет ситуацию. Исходный код контроллера находится в следующем каталоге:

```
$ cd ~/stm32f103c8t6/rtos/usbcdcdemo
$ make clobber
$ make
$ make flash
```

Эти шаги позволят собрать и записать код в ваш контроллер. В следующих разделах описываются детали USB-канала со стороны настольного компьютера.

Последовательное USB-устройство в Linux

В Linux, когда STM32 прошит и подключен к USB-порту, вы можете использовать команду `lsusb` для просмотра подключенных устройств:

```
$ lsusb
Bus 002 Device 003: ID 0483:5740 STMicroelectronics STM32F407
```

В этом примере у меня было только одно устройство. Не беспокойтесь по поводу обозначения STM32F407. Это всего лишь описание устройства с идентификатором 0483:5740, зарегистрированного STMicroelectronics. Но как

узнать, какой путь к устройству использовать? После подключения кабеля попробуйте следующее:

```
$ dmesg | grep 'USB ACM device'
[ 709.468447] cdc_acm 2-7:1.0: ttyACM0: USB ACM device
```

Очевидно, это не очень удобно для пользователя, но вы обнаружите, что имя устройства – */dev/ttyACM0*. Следующий листинг подтверждает это:

```
$ ls -l /dev/ttyACM0
crw-rw---- 1 root dialout 166, 0 Jan 25 23:38 /dev/ttyACM0
```

Следующая проблема – наличие разрешений на использование устройства. Обратите внимание, что группа устройства – это `dialout`. Добавьте себя в группу `dialout` (замените `fred` своим собственным идентификатором пользователя):

```
$ sudo usermod -a -G dialout fred
```

Выйдите из системы и войдите снова, чтобы убедиться, что у вас правильная группа:

```
$ id
uid=1000(fred) gid=1000(fred) groups=1000(fred),20(dialout), 24(cdrom),...
```

Членство в группе `dialout` избавляет вас от необходимости использовать root-доступ для доступа к последовательному устройству.

Последовательное USB-устройство в macOS

Возможно, самый простой способ найти USB-устройство в macOS – просто перечислить откликающиеся устройства:

```
$ ls -l /dev/cu.*
crw-rw-rw- 1 root wheel 35, 1 6 Jan 15:14 /dev/
cu.Bluetooth-Incoming-Port
crw-rw-rw- 1 root wheel 35, 3 6 Jan 15:14 /dev/cu.FredsiPhone-Wireless
crw-rw-rw- 1 root wheel 35, 45 26 Jan 00:01 /dev/cu.usbmodemFD12411
```

В демоверсии USB новое устройство будет выглядеть примерно так: */dev/cu.usbmodemFD12411*. Номер устройства может отличаться, поэтому в названиях ищите *cu.usbmodem*. Обратите внимание, что все разрешения предоставлены.

Последовательное USB-устройство в Windows

Последовательные устройства под Windows отображаются как COM-устройства в диспетчере устройств после подключения кабеля и установки драйвера. На рис. 7.5 приведен пример скриншота.

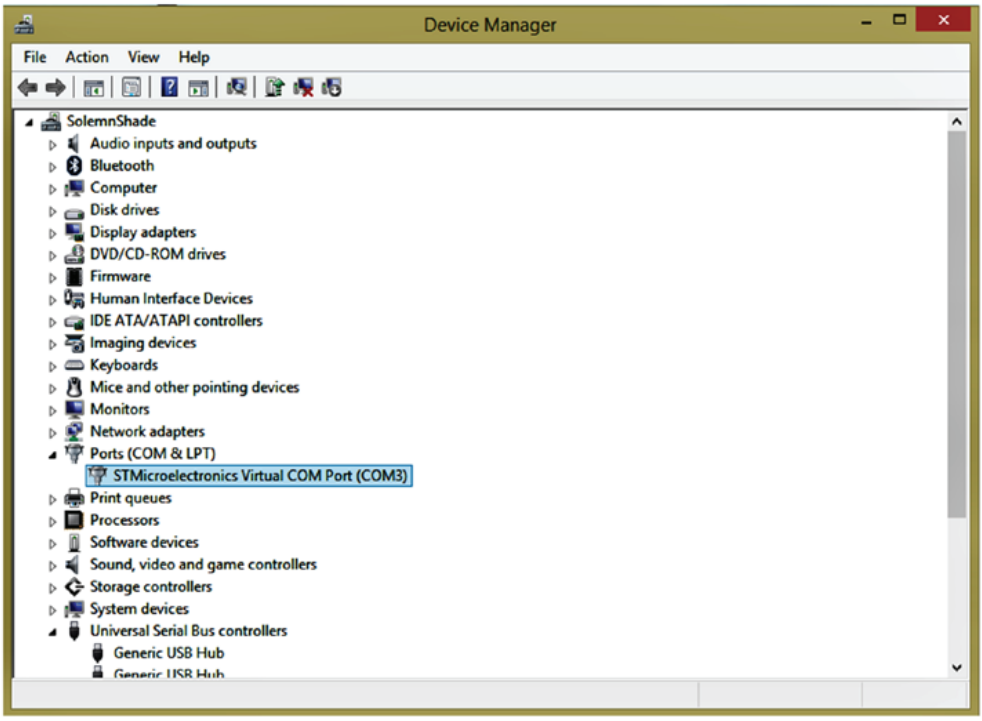


Рис. 7.5. Пример окна диспетчера устройств Windows

В этом примере USB-устройство подключено как порт COM3 Windows. Если вы используете *Cygwin* под Windows, путь к устройству – */dev/ttyS2* (вычтите 1 из номера COM-порта).

USB-выводы GPIO

Серия STMF103 поддерживает USB только на контактах GPIO PA11 (USB_DM) и PA12 (USB_DP). Альтернативных конфигураций для USB нет. Кроме того, нет необходимости настраивать PA11 и PA12, поскольку они автоматически конфигурируются при включении периферийного устройства USB (см. справочное руководство «RM0008. Reference manual» (ссылка в разделе «Дополнительные источники», [1] в конце этой главы). Это, возможно, единственное периферийное устройство, которое ведет себя таким образом, и эта небольшая деталь об альтернативных конфигурациях скрыта в указанном справочном руководстве. Однако вам все равно необходимо включить тактовую частоту для GPIOA и периферийного устройства USB.

Исходный демокод

Прежде чем запускать прилагаемое демонстрационное программное обеспечение, давайте рассмотрим некоторые связанные с USB части кода, находящегося в каталоге (еще раз):

```
$ cd ~/stm32f103c8t6/rtos/usbcdcdemo
```

Код, который будет обсуждаться, находится в исходном модуле *usbcdc.c*. В листинге 7.1 показан код инициализации периферийного устройства USB с использованием драйвера libopencm3 и FreeRTOS для очередей данных.

Листинг 7.1. Функция `usb_start()` для инициализации USB

```
0386: void
0387: usb_start(void) {
0388:   usbd_device *udev = 0;
0389:
0390:   usb_txq = xQueueCreate(128,sizeof(char));
0391:   usb_rxq = xQueueCreate(128,sizeof(char));
0392:
0393:   rcc_periph_clock_enable(RCC_GPIOA);
0394:   rcc_periph_clock_enable(RCC_USB);
0395:
0396:   // PA11=USB_DM, PA12=USB_DP
0397:   udev = usbd_init(&st_usbfs_v1_usb_driver,&dev,&config,
0398:      usb_strings,3,
0399:      usbd_control_buffer,sizeof(usbd_control_buffer));
0400:
0401:   usbd_register_set_config_callback(udev,cdcacm_set_config);
0402:
0403:  xTaskCreate(usb_task,"USB",200,udev,configMAX_PRIORITIES-1,NULL);
0404: }
```

Строки 390 и 391 создают очереди FreeRTOS, которые будут использоваться для связи с потоком USB туда и обратно соответственно.

Поскольку включение периферийного устройства USB автоматически берет на себя управление GPIO PA11 и PA12, все, что нам нужно сделать, – это включить тактовую частоту GPIO и USB в строках 393 и 394. После этого подпрограмма libopencm3 `usbd_init()` выполняет все остальное в строках 397–399.

После инициализации периферийного устройства обратный вызов `cdcacm_set_config()` регистрируется в строке 401. Наконец, в строке 403 создается задача FreeRTOS для обслуживания событий USB.

Функция cdcacm_set_config()

Когда хост-контроллер обращается к периферийному устройству USB, он вызывает обратный вызов, показанный в листинге 7.2, для настройки/перенастройки устройства USB CDC.

Листинг 7.2. Обратный вызов `cdcadm_set_config()`

```
0030: // True when USB configured:
0031: static volatile bool initialized = false;
...
```

```
0252: static void
0253: cdcacm_set_config(
0254:   usbd_device *usbd_dev,
0255:   uint16_t wValue __attribute__((unused))
0256: ) {
0257:
0258:   usbd_ep_setup(usbd_dev,
0259:       0x01,
0260:       USB_ENDPOINT_ATTR_BULK,
0261:       64,
0262:       cdcacm_data_rx_cb);
0263:   usbd_ep_setup(usbd_dev,
0264:       0x82,
0265:       USB_ENDPOINT_ATTR_BULK,
0266:       64,
0267:       NULL);
0268:   usbd_register_control_callback(
0269:       usbd_dev,
0270:       USB_REQ_TYPE_CLASS | USB_REQ_TYPE_INTERFACE,
0271:       USB_REQ_TYPE_TYPE | USB_REQ_TYPE_RECIPIENT,
0272:       cdcacm_control_request);
0273:
0274:   initialized = true;
0275: }
```

Из строк 258–262 видно, что обратный вызов `cdcacm_data_rx_cb()` зарегистрирован, чтобы он мог получать данные. С точки зрения хоста это порт OUT, обозначенный как конечная точка 0x01 (конечная точка OUT 1).

Затем строки 263–267 регистрируют другую конечную точку, которая с точки зрения хост-контроллера рассматривается как порт IN. Следовательно, конечная точка IN 2 указана со старшим битом в константе 0x82.

Наконец, запросы управления будут вызывать обратный вызов `cdcacm_control_request()`, как зарегистрировано в строках 268–272.

Наконец, в строке 274 инициализированной логической переменной присваивается значение `true`, чтобы другие задачи могли знать состояние готовности инфраструктуры USB.

Функция cdc_control_request()

Инфраструктура USB использует обратный вызов `cdcacm_control_request()` для обработки специализированных сообщений (листинг 7.3). Этот драйвер реагирует на два типа сообщений `req->bRequest`, первый из которых предназначен для устранения недостатков Linux (строки 203–209).

Листинг 7.3. Функция обратного вызова `cdcacm_control_request()`

```
0190: static int
0191: cdcacm_control_request(
0192:   usbd_device *usbd_dev __attribute__((unused)),
0193:   struct usb_setup_data *req,
```

```
0194:   uint8_t **buf __attribute__((unused)),
0195:   uint16_t *len,
0196:   void (**complete)(
0197:     usbd_device *usbd_dev,
0198:     struct usb_setup_data *req
0199:   ) __attribute__((unused))
0200:   ) {
0201:
0202:   switch (req->bRequest) {
0203:   case USB_CDC_REQ_SET_CONTROL_LINE_STATE:
0204:     /*
0205:     * The Linux cdc_acm driver requires this to be implemented
0206:     * even though it's optional in the CDC spec, and we don't
0207:     * advertise it in the ACM functional descriptor.
0208:     */
0209:     return 1;
0210:   case USB_CDC_REQ_SET_LINE_CODING:
0211:       if ( *len < sizeof(struct usb_cdc_line_coding) ) {
0212:          return 0;
0213:       }
0214:       return 1;
0215:   }
0216:   return 0;
0217: }
```

Строки 210–214 проверяют длину структуры и возвращают ошибку, если длина выходит за пределы строки (строка 212). В противном случае возврат единицы указывает на статус «обработано» (строка 214).

Функция cdcacm_data_rx_cb()

Этот обратный вызов вызывается инфраструктурой USB, когда данные отправляются по шине в микроконтроллер STM32. Первое, что выполняется в строке 228, – это определение, сколько осталось буферного пространства, назначенного переменной `rx_avail`. Если свободного места недостаточно, обратный вызов просто возвращается в строке 233. Хост снова отправит те же данные позже.

Если у нас есть место для каких-то данных, мы решаем, сколько именно в строке 236. Вызов `usbd_ep_read_packet()` в строке 239 затем получает некоторые или все полученные данные. Строки 241–244 отправляют его в приемную очередь для задачи приема, см. листинг 7.4.

Листинг 7.4. Обратный вызов USB-приема

```
0222: static void
0223: cdcacm_data_rx_cb(
0224:   usbd_device *usbd_dev,
0225:   uint8_t ep __attribute__((unused))
0226: ) {
0227:   // How much queue capacity left?
```

```
0228:   unsigned rx_avail = uxQueueSpacesAvailable(usb_rxq);
0229:   char buf[64]; // rx buffer
0230:   int len, x;
0231:
0232:   if ( rx_avail <= 0 )
0233:      return; // No space to rx
0234:
0235:   // Bytes to read
0236:   len = sizeof buf < rx_avail ? sizeof buf : rx_avail;
0237:
0238:   // Read what we can, leave the rest:
0239:   len = usbd_ep_read_packet(usbd_dev,0x01,buf,len);
0240:
0241:   for ( x=0; x<len; ++x ) {
0242:       // Send data to the rx queue
0243:       xQueueSend(usb_rxq,&buf[x],0);
0244:   }
0245: }
```

Задача USB task

Задача, которую мы создали для обработки USB, представляет собой бесконечный цикл, начинающийся со строки 284. Цикл должен вызывать подпрограмму драйвера `usbd_poll()` из библиотеки libopencm3 достаточно часто, чтобы хост поддерживал соединение USB. Это делается в начале цикла в строке 285 листинга 7.5.

Листинг 7.5. Функция `usb_task()`

```
0278: static void
0279: usb_task(void *arg) {
0280:   usbd_device *udev = (usbd_device *)arg;
0281:   char txbuf[32];
0282:   unsigned txlen = 0;
0283:
0284:   for (;;) {
0285:       usbd_poll(udev); /* Allow driver to do its thing */
0286:       if ( initialized ) {
0287:          while ( txlen < sizeof txbuf
0288:           && xQueueReceive(usb_txq,&txbuf[txlen],0)== pdPASS)
0289:              ++txlen; /* Read data to be sent */
0290:          if ( txlen > 0 ) {
0291:             if ( usbd_ep_write_packet(udev,0x82,
                          txbuf,txlen) != 0 )
0292:                txlen = 0; /* Reset if sent ok */
0293:          } else {
0294:              taskYIELD(); /* Then give up CPU */
0295:          }
0296:       }
0297:   }
0298: }
```

Булева переменная (volatile bool)[49] `initialized` проверяется в строке 286. Пока `initialized` не примет значение `true`, следует избегать других вызовов USB, таких как `usbd_ep_write_packet()`.

После инициализации драйвера в строках 287–289 производится проверка очереди передачи. Из очереди для отправки берется как можно больше символов, находящихся в очереди. Отправка данных USB происходит в строках 290–292. Если символов для передачи нет, FreeRTOS вызывает функцию `taskYIELD()`, чтобы предоставить процессорное время другой задаче.

Отсюда видно, что цель этой задачи – просто отправить все байты данных, находящиеся в очереди, на USB-хост. Получение данных происходит в другом месте.

USB-прием

Когда приложению требуется прочитать последовательные данные, оно вызывает `usb_getc()` или процедуры-оболочки, такие как `usb_getline()`. Листинг 7.6 иллюстрирует код `usb_getc()`.

В строке 367 вы можете видеть, что она вызывает функцию `xQueueReceive()` для извлечения байта полученных данных из очереди. Если данных нет, вызов будет заблокирован параметром, заданным как `portMAX_DELAY`. Как только будет выполнен обратный вызов `cdcacm_data_rx_cb()` и поставит данные в очередь, этот код получит данные и разблокируется.

Хотя этого никогда не должно происходить, возврат –1 в строке 369 выполняется, если очередь была уничтожена или по другой причине стала нефункциональной. Обычно один символ возвращается в строке 370.

Листинг 7.6. Листинг функции `usb_getc()`

```
0362: int
0363: usb_getc(void) {
0364:   char ch;
0365:   uint32_t rc;
0366:
0367:   rc = xQueueReceive(usb_rxq,&ch,portMAX_DELAY);
0368:   if ( rc != pdPASS )
0369:      return -1;
0370:   return ch;
0371: }
```

USB-отправка

Чтобы отправить байт данных на USB, он помещается в переменную FreeRTOS `usb_txq` с помощью функции `usb_putc()`, как показано в листинге 7.7. Однако перед этим в строке 307 выполняется проверка, чтобы убедиться, что драйвер USB готов. Если он еще недоступен, в строке 308 вызывается `TaskYIELD()` для освобождения процессора.

[49] Тип переменной volatile означает, что значение переменной может меняться извне и что компилятор не должен оптимизировать эту переменную. Переменная `initialized` формируется в строке 409 (листинг 7.7 далее).

Как только становится известно, что драйвер USB готов, байт ставится в очередь в строке 312, где он блокируется, если очередь заполнена. Как только байты будут удалены из этой очереди, появившийся байт помещается в очередь, и вызов функции заканчивается.

Листинг 7.7. Отправка данных через USB с использованием функции `usb_putc()`

```
0303: void
0304: usb_putc(char ch) {
0305:   static const char cr = '\r';
0306:
0307:   while ( !usb_ready() )
0308:       taskYIELD();
0309:
0310:   if ( ch == '\n' )
0311:       xQueueSend(usb_txq,&cr,portMAX_DELAY);
0312:       xQueueSend(usb_txq,&ch,portMAX_DELAY);
0313:   }
...
0407:   bool
0408:   usb_ready(void) {
0409:     return initialized;
0410: }
```

Чтобы сделать посылаемые строки более читабельными, функция `usb_putc()` проверяет, отправляете ли вы символ новой строки `\n` (также известный как перевод строки). Если да, то в строке 311 сначала отправляется символ возврата каретки (`\r`). В Unix/Linux этот тип обработки известен как режим «cooked mode»[50]. Принимающая сторона в эмуляторе терминала переместит курсор в начало строки, прежде чем перейти к следующей строке.

Демопрограмма последовательного порта USB

Чтобы продемонстрировать ввод-вывод через последовательный порт USB, здесь приведен измененный текст с открытым исходным кодом по мотивам игры, написанный Джеффом Трантером. Его исходный код, найденный в модуле *adventure.c*, был модифицирован для использования только что рассмотренных USB-подпрограмм.

```
$ make clobber
$ make
$ make flash
```

[50] То есть «приготовленный». Режим «cooked mode» означает распознавание специальных управляющих символов (из первых 32 символов таблицы ASCII). В данном случае обозначаются концы строк символами возврат каретки (\r, CR, 0x0D) + перевод строки (\n, LF, 0x0A), что означает создание новой строки. В отличие от «cooked mode», в режиме «raw mode» («сыром») распознавание специальных символов как команд не производится.

После прошивки контроллера аккуратно вставьте USB-кабель в плату STM32 и подключите другой конец кабеля к ноутбуку или ПК. Предполагая, что вы знаете имя устройства (указанное ранее в этой главе), настройте minicom или другую терминальную программу (при необходимости просмотрите инструкции minicom в главе 6). Я рекомендую вам сохранить эти настройки под именем профиля, например «usb2», поскольку они отличаются от настроек USB, используемых далее в этой книге.

Когда все готово, запустите эмулятор терминала следующим образом:

```
$ minicom usb2
Welcome to minicom 2.7
OPTIONS:
Compiled on Sep 17 2016, 05:53:15.
Port /dev/cu.usbmodemFD12411, 16:56:26
Press Meta-Z for help on special keys
...
     Abandoned Farmhouse Adventure
          By Jeff Tranter
Your three-year-old grandson has gone
missing and was last seen headed in the
direction of the abandoned family farm.
It's a dangerous place to play. You
have to find him before he gets hurt,
and it will be getting dark soon...
?
```

Не волнуйтесь, если вы пропустили вводный текст в показанном сеансе (вы, очевидно, можете прочитать его здесь или выключить *minicom* и начать все с начала). Такое может случиться, если вам пришлось возиться с настройкой *minicom*.

С первого экрана вы можете прочитать о приключении. Информацию можно получить, набрав «help»:

```
? help
Valid commands:
go east/west/north/south/up/down
look
use <object>
examine <object>
take <object>
drop <object>
inventory
help
You can abbreviate commands and
directions to the first letter.
Type just the first letter of
a direction to move.
?
```

Игра состоит из использования некоего глагола, а иногда и объекта. Следующий сеанс дает вам образец[51]:

```
? look
You are in the driveway near your car.
You see:
key
have to find him before he gets hurt,
and it will be getting dark soon...
?
```

Итоги

USB – это большая тема, поскольку порт приходится адаптировать для самых разных целей. Последовательный поток символов, показанный в этой главе, является только одним из многих применений USB. Кроме того, в исходном модуле *usbcdc.c* были объявлены, но не описаны структуры управления. Заинтересованному читателю предлагается изучить их и поэкспериментировать с исходным кодом. О USB написано несколько книг, и этот проект дает вам основу, с которой можно начать.

Вы также узнали, как можно создать удобный USB-интерфейс между STM32 и вашим ноутбуком или ПК. Для настройки USB не требовались скорости передачи данных, биты данных, стоповые биты, четность или управление потоком. При условии, что на USB-хосте присутствует необходимая поддержка драйверов, это так же просто, как подключить кабель.

Хотя основное внимание уделялось USB как средству последовательной связи, в демонстрации также были освещены некоторые возможности FreeRTOS, такие как задачи и очереди сообщений. Раздельное выполнение задач и безопасная связь между задачами значительно упрощают разработку приложений.

Наконец, небольшой дефект USB Blue Pill на самом деле исправить не так уж и сложно. Учитывая мощность микроконтроллера STM32, доступного по цене значительно более слабых платформ вроде Arduino, ни у кого нет причин упускать удовольствие!

Дополнительные источники

1. STMicroelectronics. «RM0008. Reference manual». www.st.com/resource/en/reference_manual/cd00171190.pdf (дата доступа 09.09.2024), табл. 29, стр. 167.

[51] Перевод текста сеанса игры:
? смотрим
Вы находитесь на подъездной дорожке рядом со своей машиной.
Вы видите:
ключ
надо найти его, прежде чем он потеряется,
и скоро стемнеет...
?

Упражнения

1. Какая подготовка GPIO необходима перед включением периферийного USB-устройства?
2. Какие альтернативные конфигурации GPIO доступны для USB?
3. Какую процедуру libopencm3 необходимо регулярно вызывать для обработки событий USB?

Глава 8 • Флеш-память с интерфейсом SPI

Каким бы богатым на ресурсы ни был микроконтроллер STM32, иногда пользователю требуется дополнительное постоянное хранилище данных. Небольшие приложения могут оставлять свободными остатки программной флеш-памяти, которые можно использовать, но, если вы собираете большие объемы данных, вы, вероятно, обратите внимание на решение с дополнительной флеш-памятью.

В этой главе описывается связь с чипами Winbond W25Q32 или W25Q64 с использованием периферийного устройства SPI в ведущем режиме. Чип W25Q32 обеспечивает 4 Мбайта стираемой флеш-памяти, а W25Q64 – 8 Мбайт. Эти чипы можно приобрести по цене несколько долларов за штуку, что делает их привлекательными для многих приложений.

Представление W25QXX

Чипы W25Q32/64 обеспечивают достаточный объем памяти, при этом для связи требуется всего несколько проводов. Они работают при питании от 2.7 до 3.6 В, потребляют 50 мкА тока в режиме ожидания и примерно 15 мА при чтении данных. Для записи и стирания требуется немного больший ток – 25 мА. Кроме того, чипы W25QXX можно отключать под управлением программного обеспечения для экономии энергии, когда это необходимо.

Поскольку эти чипы используют для связи шину SPI, давайте кратко рассмотрим, как работает SPI.

Шина последовательного периферийного интерфейса SPI

Последовательный периферийный интерфейс (Serial Peripheral Interface, SPI) – это синхронный последовательный интерфейс, который обеспечивает связь на коротких расстояниях с использованием трех проводов и сигнала выбора микросхемы chip select ($\overline{CS}$). Одно из устройств на конце шины работает как ведущее (master), а остальные устройства являются ведомыми (slave). Протокол SPI был разработан компанией Motorola в конце 1980-х годов и с тех пор стал стандартом де-факто (https://en.wikipedia.org/wiki/Serial_Peripheral_Interface_Bus). Рисунок 8.1 иллюстрирует связь одного ведущего устройства с одним ведомым устройством. Так будет настроен демонстрационный проект этой главы. К шине можно было подключить дополнительные ведомые устройства, но каждое из них будет иметь свой собственный сигнал $\overline{CS}$.

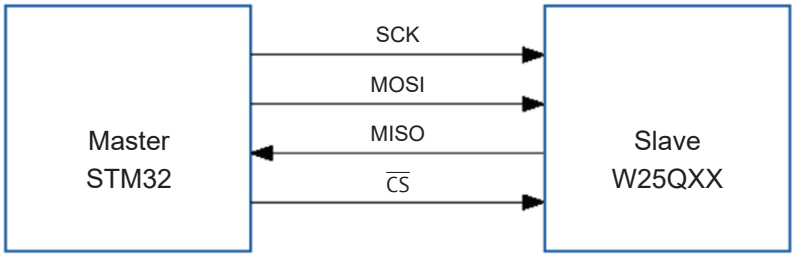


Рис. 8.1. Пример SPI от одного ведущего к одному ведомому

Линия системного тактового сигнала (SCK) обеспечивает тактовые импульсы, которые синхронизируют передаваемые и принимаемые биты данных. Сигнал MOSI (Master Out Slave In) является выходной линией данных ведущего и входной ведомого устройства, а MISO (Master In Slave Out) – это выходная линия ведомого и входная ведущего устройства. Четвертый сигнал – это выбор чипа $\overline{CS}$, который используется для активации выбранного ведомого устройства. Он отображается с надчеркиванием $\overline{CS}$ (или предшествующей косой чертой /CS), чтобы указать, что активное состояние – низкого уровня. Иногда этот сигнал называют попросту выбором ведомого устройства slave select $\overline{SS}$.

Одной из уникальных особенностей шины SPI является ее метод связи. Когда ведущий отправляет биты данных по линии MOSI, ведомый одновременно возвращает биты данных мастеру по линии MISO. Рисунок 8.2 иллюстрирует, как пара обменивающихся устройств ведет себя аналогично двум сдвиговым регистрам, соединенным в кольцо.

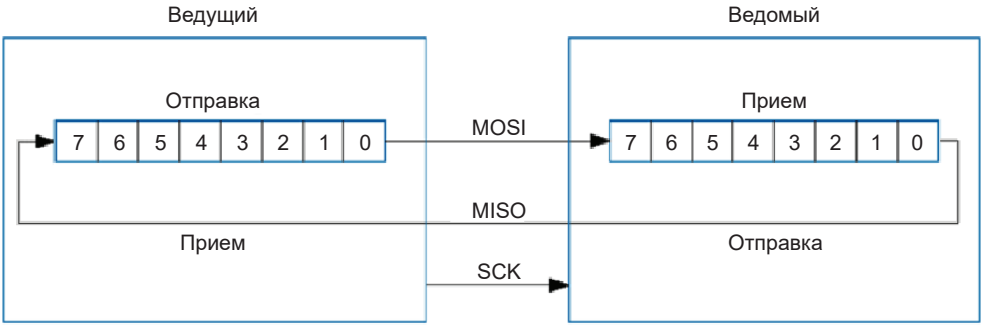


Рис. 8.2. Ведомый и ведущий SPI как пара сдвиговых регистров

Мастер SPI генерирует тактовые импульсы (SCK), по которым данные сдвигаются из регистра отправки в регистр приема. После получения полной длины слова принимающее и отправляющее устройство обмениваются содержимым регистров и могут одновременно читать полученные данные.

Конструкция шины SPI приводит к некоторому необычному программированию. Ведомое устройство может не знать, какие данные отправлять, пока

не получит командное слово от ведущего. Следовательно, когда ведущий отправляет командное слово ведомому, первое слово данных, полученное от ведомого, отбрасывается, поскольку оно бессмысленно. Как только ведомое устройство получило командное слово, оно знает, как ответить. Но чтобы отправить ответ, ведомому устройству необходимо, чтобы мастер отправил фиктивное слово, чтобы ответ переместился в регистр ведущего устройства. Из-за этой особенности программирование SPI требует отбрасывания некоторых полученных данных и отправки фиктивных слов. Размер слова для SPI-контроллера STM32 может иметь длину 8 или 16 бит.

Сигнал chip select

Вы можете спросить: «Зачем нам нужна линия выбора микросхемы, если в этой демонстрации задействован только один ведомый?» Проблема в том, что по шине может передаваться шум. Чтобы защититься от этого, ведомому устройству необходимо знать, когда начинается и заканчивается передача. Например, если на линии SCK возникает шум, ведомое устройство может принять неверную команду: например, прочесть ее как функцию «стирания чипа», что будет иметь катастрофические последствия. По этой причине $\overline{\text{CS}}$ переходит в низкий уровень до того, как ведущий отправляет первый бит данных. Это сообщает ведомому устройству, что на подходе первый бит. Когда будет отправлено последнее слово данных, $\overline{\text{CS}}$ возвращается в высокое состояние, сигнализируя об окончании передачи. Флеш-память Winbond будет настаивать на этом перед выполнением операции записи или стирания; в противном случае команда игнорируется.

Соединения и напряжение

При подключении UART необходимо соединить TX с RX, а RX с TX и т. д., в зависимости от назначения устройства и того, как производитель обозначил соединения. Это может сбить с толку. С шиной SPI ситуация очень проста: линия SCK всегда подключается к SCK, MOSI всегда к MOSI, MISO всегда к MISO, а $\overline{\text{CS}}$ – к $\overline{\text{CS}}$.

Напряжение шины SPI может варьироваться, обычно оно составляет 5 или 3.3 В. Устройства Winbond W25QXX могут работать на уровне 3.3 В, что упрощает взаимодействие с STM32.

Схема подключения SPI-флеш-памяти

На рис. 8.1 показана полная схема нашего проекта флеш-памяти SPI. Чип Winbond включает в себя некоторые другие выводы, которые мы не обсуждали, а именно:

- $\overline{\text{WP}}$ (Write Protect, подключен к +3.3 В для разрешения записи),
- $\overline{\text{HOLD}}$ – вход удержания (подключен к +3.3 В, когда не используется).

Обе эти функции не используются в этой главе и должны быть подключены к источнику питания +3.3 В. Сигнал $\overline{\text{WP}}$ – это опция безопасности, которая

может оказаться полезной в некоторых приложениях. Когда $\overline{WP}$ подключен к низкому уровню, запись или стирание невозможны.

Управление выводом $\overline{NSS}$

Особенностью периферийного устройства STM32 SPI, которая, судя по сообщениям на форумах, раздражала многих людей, является требование «подтяжки» вывода $\overline{CS}$ в режиме ведущего устройства SPI. Если вы забудете установить подтягивающий резистор R1, показанный на рис. 8.3, вы обнаружите, что схема не работает. Ответом на эту проблему на форумах было предложено вместо этого использовать программное управление выводом (в двухтактном режиме GPIO).

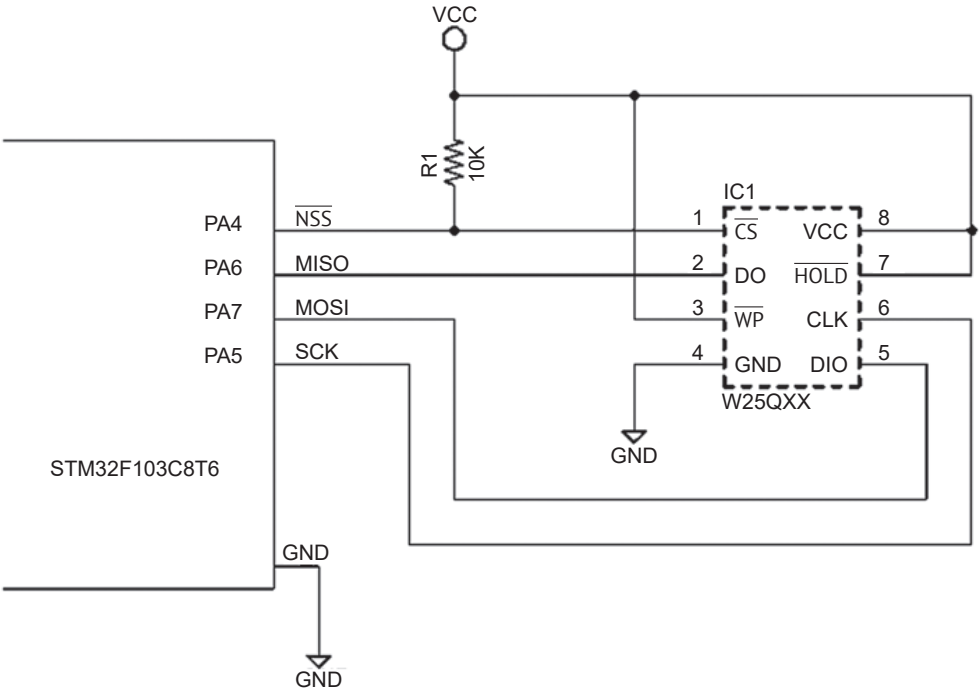


Рис. 8.3. К STM32 подключен W25Q32 или W25Q64.
Если у вас плата с другой разводкой, номера контактов платы сопоставляйте по функциям (например, CLK памяти подключен к SCK на плате контроллера)

Эта проблема основана на предположении, что аппаратно вывод $\overline{NSS}$ ($\overline{CS}$) управляется в двухтактном режиме (push/pull). В конце концов, именно так это обычно и настраивается во время настройки SPI (см. строку 652 в листинге 8.2 ниже). Но это явно не так, и документация STM32 слаба в этом отношении. Единственный намек на поведение вывода содержится в справочном руководстве RM0008, раздел «25.3 SPI functional description», пункт «Slave select (NSS) management»: «Сигнал NSS устанавливается на низкий

уровень, когда ведущее устройство начинает связь, и сохраняется на низком уровне до тех пор, пока SPI не будет отключен». При этом нигде не упоминается, что сигнал $\overline{\text{NSS}}$ имеет высокий уровень. Кроме того, есть примечания по применению AN2576, в которых описывается связь STM32F10xxx SPI и флеш-памяти M25P64. На рис. 5 в этом документе показан подтягивающий резистор сопротивлением 10 кОм, который нигде не упоминается.

Эта характеристика периферии SPI не совсем удивительна, когда читаешь о ее особенностях. Одной из рекламируемых функций является поддержка режима нескольких ведущих. В этом режиме работы вывод $\overline{\text{NSS}}$ должен функционировать как выход с открытым стоком. В идеальном мире периферийное устройство должно было бы обеспечивать двухтактный выход в режиме с одним главным устройством и использовать открытый сток для режима с несколькими ведущими устройствами. Но здесь дело обстоит иначе. Чтение спецификаций и работа с оборудованием часто приводят к интересным головоломкам.

Если вы только начинаете заниматься цифровой электроникой, то простой ответ в этой схеме заключается в том, что вам нужен резистор сопротивлением 10 кОм. Возможно, некоторые читатели пробормочут: «К чему вся эта суета? Почему бы просто не управлять $\overline{\text{NSS}}$ с помощью команд GPIO?» Это слово «просто» создает столько проблем!

С чисто логической точки зрения и без учета небольшой потери эффективности управление $\overline{\text{NSS}}$ по GPIO вполне допустимо, хотя и неудобно для кода. Но основная причина необходимости аппаратного управления уровнями $\overline{\text{NSS}}$ заключается в уменьшении вероятности искажения сообщений SPI шумом. Промежуток времени между запуском транзакции SPI и активацией линии $\overline{\text{NSS}}$ при аппаратном управлении короче, чем при программной настройке GPIO. Аналогично аппаратное управление SPI может быстрее деактивировать линию $\overline{\text{NSS}}$ в конце транзакции раньше, чем это может сделать программное действие GPIO. Времена не сильно отличаются, но это снижает вероятность повреждения сообщений.

Чтобы не «размазывать белую кашу по чистому столу» дальше, заметим, что еще одна причина использования аппаратного управления линией $\overline{\text{NSS}}$ заключается в том, что это избавляет нас от необходимости самостоятельно проводить какие-то действия. Это может показаться высказыванием «капитана Очевидности», но это также означает, что мы не можем забыть отключить вывод $\overline{\text{NSS}}$. Если бы мы забыли, последняя операция записи или стирания была бы проигнорирована флеш-чипом. Самые худшие и коварные ошибки – это те, которые остаются незамеченными до тех пор, пока не становится слишком поздно выяснить причину.

Обратите внимание, что V_{cc} здесь соответствует источнику питания +3.3 В. Сигналы $\overline{\text{WP}}$ и $\overline{\text{HOLD}}$ имеют активный низкий уровень и должны быть подключены к выводу Vcc, чтобы нормально реализовывать их функциональность.

На рис. 8.4 показаны две основных разновидности корпуса, в которых поставляется W25Q32.

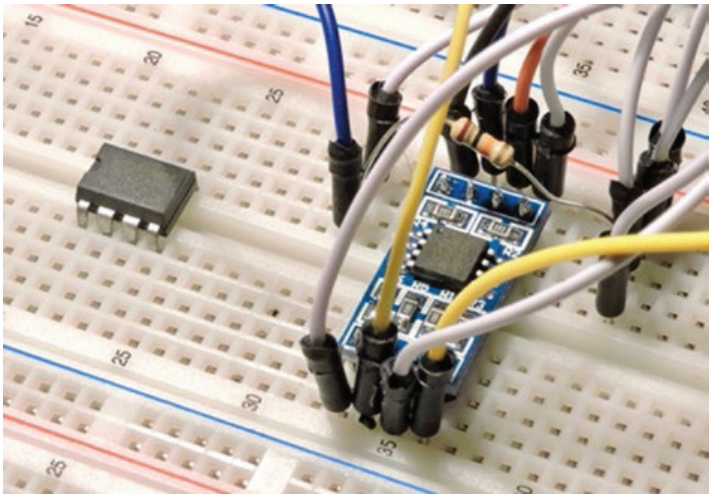


Рис. 8.4. W25Q32 в корпусе DIP (слева) и W25Q32 в корпусе SOIC на печатной плате (справа)

Конфигурация STM32 SPI

Для связи с внешним флеш-чипом нам необходимо настроить и подготовить периферийное устройство STM32 SPI. Демонстрационный исходный код этой главы находится в следующем каталоге:

```
$ cd ~/stm32f103c8t6/rtos/winbond
```

В табл. 8.1 приведены контакты GPIO, которые будут использоваться для подключения периферийного устройства SPI1 к флеш-чипу Winbond. Они также показаны на схеме на рис. 8.3.

Таблица 8.1. Используемые выводы STM32 для SPI1

Вывод GPIO	Функция SPI	Описание
PA4	$\overline{CS}$	Выбор чипа (активный низкий уровень)
PA5	SCK	Системные такты
PA6	MISO	Вход ведущего, выход ведомого
PA7	MOSI	Выход ведущего, вход ведомого

В листинге 8.1 представлен общий исходный код инициализации основной программы. Строка 679 включает тактовую частоту для GPIOA, поскольку наше периферийное устройство SPI использует выводы этого порта (табл. 8.1). Остальная настройка SPI выполняется в строке 685 в функции `spi_setup()`, которую мы рассмотрим отдельно.

Листинг 8.1. Инициализация основной программы `main()`

```
0674: int
0675: main(void) {
0676:
0677:   rcc_clock_setup_pll(&rcc_hse_configs[RCC_CLOCK_HSE8_72MHZ]);
0678:
0679:   rcc_periph_clock_enable(RCC_GPIOA);
0680:   rcc_periph_clock_enable(RCC_GPIOC);
0681:
0682:   // LED on PC13
0683:   gpio_set_mode(GPIOC,GPIO_MODE_OUTPUT_2_MHZ,
GPIO_        CNF_OUTPUT_PUSHPULL,GPIO13);
0684:
0685:   spi_setup();
0686:   gpio_set(GPIOC,GPIO13); // PC13 = on
0687:
0688:   usb_start(1);
0689:   std_set_device(mcu_usb); // Use USB for std I/O
0690:   gpio_clear(GPIOC,GPIO13); // PC13 = off
0691:
0692:   xTaskCreate(monitor_task,"monitor",
             500,NULL,configMAX_PRIORITIES-1,NULL);
0693:   vTaskStartScheduler();
0694:   for (;;);
0695:   return 0;
0696: }
```

Чтобы мы могли сосредоточиться на SPI в этой главе, мы используем библиотеку для обеспечения USB-связи с настольным компьютером. Статическая библиотека находится здесь:

- *~/stm32f103c8t6/rtos/libwwg/libwwg.a*.

Исходный код библиотеки находится в следующих двух каталогах:

- *~/stm32f103c8t6/rtos/libwwg/include*;
- *~/stm32f103c8t6/rtos/libwwg/src*.

Строки 688 и 689 выполняют инициализацию USB, позволяя программе взаимодействовать с терминальной программой. Строка 689 просто перенаправляет все вызовы `std_printf()` на `usb_printf()` и т. д. Если позже вы решите использовать UART для связи, можно настроить это перенаправление на него.

В листинге 8.2 показана процедура `spi_setup()`, которая учитывает особенности SPI. Для инициализации периферийного устройства SPI1 используются следующие шаги.

1. Для SPI1 включается источник тактов (строка 648).
2. Выводы GPIOA настроены (строки 649–654):
   а) на альтернативный функциональный выход (двухтактный);
   б) частоту 50 МГц (для быстрого нарастания/спада);
   в) для выводов PA4, PA5 и PA7.
3. GPIO PA6 настроен на вход без подтягивающего резистора (строки 655–660).

4. Периферийное устройство SPI1 сбрасывается (строка 661).
5. SPI1 настроено на использование следующего:
   а) делитель тактовой частоты FPCLK = 256 (строка 664);
   б) полярность SCK равна 0 (низкий уровень) в режиме ожидания (строка 655);
   в) фаза тактирования возникает при первом переходе (строка 666);
   г)длина слова составляет 8 бит (строка 667);
   д) при сдвиге первым идет старший бит MSB (строка 668).
6. SPI1 использует аппаратное управление $\overline{CS}$ (т. е. не использует программное управление ведомыми устройствами, строка 670).
7. Периферийное устройство SPI может подтвердить $\overline{CS}$ (строка 671).

Листинг 8.2. Настройка периферийного устройства SPI

```
0645: static void
0646: spi_setup(void) {
0647:
0648:   rcc_periph_clock_enable(RCC_SPI1);
0649:   gpio_set_mode(
0650:       GPIOA,
0651:         GPIO_MODE_OUTPUT_50_MHZ,
0652:         GPIO_CNF_OUTPUT_ALTFN_PUSHPULL,
0653:         GPIO4|GPIO5|GPIO7 // NSS=PA4,SCK=PA5,MOSI=PA7
0654:   );
0655:   gpio_set_mode(
0656:       GPIOA,
0657:         GPIO_MODE_INPUT,
0658:         GPIO_CNF_INPUT_FLOAT,
0659:         GPIO6 // MISO=PA6
0660:   );
0661:   rcc_periph_reset_pulse(RST_SPI1);
0662:   spi_init_master(
0663:         SPI1,
0664:         SPI_CR1_BAUDRATE_FPCLK_DIV_256,
0665:         SPI_CR1_CPOL_CLK_TO_0_WHEN_IDLE,
0666:         SPI_CR1_CPHA_CLK_TRANSITION_1,
0667:         SPI_CR1_DFF_8BIT,
0668:         SPI_CR1_MSBFIRST
0669:   );
0670:   spi_disable_software_slave_management(SPI1);
0671:   spi_enable_ss_output(SPI1);
0672: }
```

Тактовая частота SPI

Одна из самых неприятных особенностей платформы STM32 – сложность системы синхронизации. Вопрос, на который мы хотим получить ответ: какую частоту обеспечивает режим SPI_CR1_BAUDRATE_FPCLK_DIV_256? Частично ответ заключается в определении f_{PCLK}. Для STM32F103 SPI1 использует тактовую частоту шины APB2, максимальная частота которой составляет 72 МГц. SPI2

использует тактовую частоту шины APB1, максимальная частота которой составляет 36 МГц.

Основная программа использовала следующую функцию libopencm3 для установки некоторых основных тактовых частот:

```
0677: rcc_clock_setup_pll(&rcc_hse_configs[RCC_CLOCK_HSE8_72MHZ]
```

Используя эти настройки, мы можем подытожить величину f_{PCLK} для SPI, как показано в табл. 8.2.

Таблица 8.2. Частоты для SPI1 и SPI2 при условии ***rcc_clock_setup_in_hse_8mhz_out_72mhz()***

Такты	Шина	Периферия	f_{PCLK}
PCLK1	APB1	SPI2	36 МГц
PCLK2	APB2	SPI1	72 МГц

Обладая этой информацией, мы можем обобщить выбор делителей тактовой частоты SPIx, как показано в табл. 8.3. С помощью цифрового запоминающего осциллографа я проверил, что при запуске демонстрационной программы период сигнала SCK составляет около 3.56 мкс. Как и ожидалось, это соответствует частоте около 281 кГц.

Таблица 8.3. Частоты делителя тактовой частоты SPI

Делитель	Обозначение	Частота SPI1	Частота SPI2
2	SPI_CR1_BAUDRATE_FPCLK_DIV_2	36 МГц	18 МГц
4	SPI_CR1_BAUDRATE_FPCLK_DIV_4	18 МГц	9 МГц
8	SPI_CR1_BAUDRATE_FPCLK_DIV_8	9 МГц	4.5 МГц
16	SPI_CR1_BAUDRATE_FPCLK_DIV_16	4.5 МГц	2.25 МГц
32	SPI_CR1_BAUDRATE_FPCLK_DIV_32	2.25 МГц	1.125 МГц
64	SPI_CR1_BAUDRATE_FPCLK_DIV_64	1.125 МГц	562.5 кГц
128	SPI_CR1_BAUDRATE_FPCLK_DIV_128	562.5 кГц	281.25 кГц
256	SPI_CR1_BAUDRATE_FPCLK_DIV_256	281.25 кГц	140.625 кГц

Для этой демопрограммы я выбрал низкую частоту, чтобы гарантировать хорошие результаты на макетной плате. Иногда при использовании макетов с длинными проводами шум может нарушить связь SPI. Имея в своем распоряжении исходный код, вы можете попробовать увеличить скорость передачи данных после первоначального успеха. Микросхема Winbond должна считывать данные при частотах до 50 МГц, но SPI1 на платформе STM32 ограничен 36 МГц.

Режимы синхронизации SPI

Процедура `spi_setup()` в листинге 8.2 использовала следующий параметр конфигурации:

```
0665: SPI_CR1_CPOL_CLK_TO_0_WHEN_IDLE
```

Что это значит для программиста?

SPI может работать в одном из четырех режимов, что может привести к путанице. Флеш-чипы Winbond, используемые в этой главе, могут работать в режиме 0 или 3. На рис. 8.5 показаны взаимосвязи между различными конфигурациями в библиотеке libopencm3. Следует иметь в виду, что режим работы SPI определяют полярность и фаза тактового сигнала вместе.

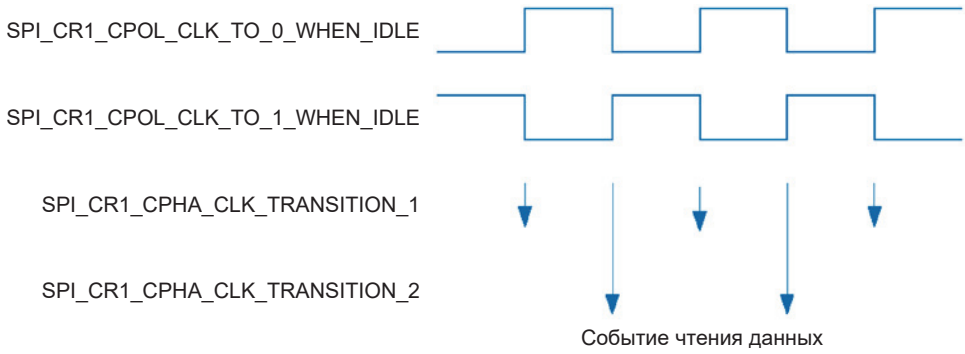


Рис. 8.5. Полярность тактовых импульсов и конфигурации фаз

Когда используется `SPI_CR1_CPOL_CLK_TO_0_WHEN_IDLE` с `SPI_CR1_CPHA_CLK_TRANSITION_1`, приемник производит выборку входных данных при нарастающем переходе тактовой частоты (короткие стрелки на рис. 8.5). Если используется та же полярность тактового сигнала и вместо него настроен `SPI_CR1_CPHA_CLK_TRANSITION_2`, данные отбираются по заднему фронту тактового сигнала (длинные стрелки).

Ситуация меняется на обратную, когда используется полярность `SPI_CR1_CPOL_CLK_TO_1_WHEN_IDLE` (вторая строка сверху на рис. 8.5). Задний фронт (короткие стрелки) тактового сигнала используется, когда настроен `SPI_CR1_CPHA_CLK_TRANSITION_1`; в противном случае используется нарастающий фронт (длинные стрелки).

В табл. 8.4 приведены режимы SPI с использованием названий режимов libopencm3. Зная, что микросхемы Winbond W25QXX будут работать в режиме 0 или 3, можно прийти к выводу, что они работают только по нарастающему фронту сигнала SCK.

Таблица 8.4. Сводка режимов SPI по номерам

Режим SPI	Полярность такта	Фаза
0	`SPI_CR1_CPOL_CLK_TO_0_WHEN_IDLE`	`SPI_CR1_CPHA_CLK_TRANSITION_1`
1	`SPI_CR1_CPOL_CLK_TO_0_WHEN_IDLE`	`SPI_CR1_CPHA_CLK_TRANSITION_2`
2	`SPI_CR1_CPOL_CLK_TO_1_WHEN_IDLE`	`SPI_CR1_CPHA_CLK_TRANSITION_1`
3	`SPI_CR1_CPOL_CLK_TO_1_WHEN_IDLE`	`SPI_CR1_CPHA_CLK_TRANSITION_2`

Следует отметить еще один момент, касающийся полярности тактовой частоты SPI, по крайней мере, в отношении названий режимов в libopencm3.

Имя SPI_CR1_CPOL_CLK_TO_0_WHEN_IDLE описывает полярность тактового сигнала в простое во время выбора чипа. На рис. 8.6 представлена осциллограмма активности SCK при низком уровне $\overline{CS}$. До активации $\overline{CS}$ тактовый сигнал находился на высоком уровне. Но во время активности SPI ($\overline{CS}$ активен) тактовый сигнал действительно простаивал на низком уровне. Это важно иметь в виду при изучении сигналов SPI.

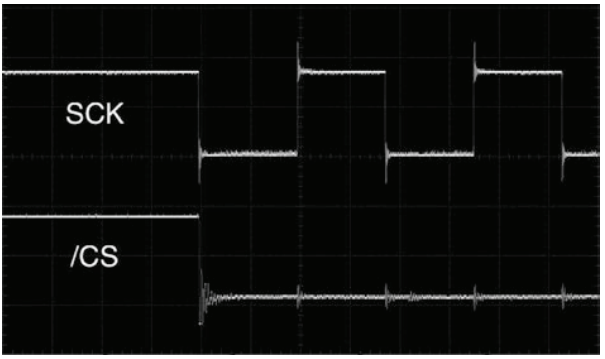


Рис. 8.6. Осциллограмма SCK и $\overline{CS}$ при низком уровне ожидания SCK

Порядок битов и длина слова

Теперь мы можем рассмотреть заключительные аспекты конфигурации SPI1 (из листинга 8.2):

```
0662: spi_init_master(
0663:     SPI1,
0664:     SPI_CR1_BAUDRATE_FPCLK_DIV_256,
0665:     SPI_CR1_CPOL_CLK_TO_0_WHEN_IDLE,
0666:     SPI_CR1_CPHA_CLK_TRANSITION_1,
0667:     SPI_CR1_DFF_8BIT,
0668:     SPI_CR1_MSBFIRST
0669: );
```

Название режима SPI_CR1_DFF_8BIT (строка 667) указывает, что длина слова равна байту (8 бит). Название режима SPI_CR1_MSBFIRST (строка 668) указывает, что мы будем передавать самый старший бит первым (с обратным порядком байтов). Другие варианты libopencm3 – SPI_CR1_DFF_16BIT и SPI_CR1_LSBFIRST.

Процедура настройки SPI заканчивается еще двумя вызовами:

```
0670: spi_disable_software_slave_management(SPI1);
0671: spi_enable_ss_output(SPI1);
```

Строка 670 просто указывает, что мы будем использовать аппаратное управление выводом $\overline{NSS}$ ($\overline{CS}$). Этот вызов, возможно, не является строго необходимым после сброса, но он служит для документирования нашего намерения. Строка 671 указывает, что периферийное устройство SPI должно

подтвердить управление выводом $\overline{\text{NSS}}$ ($\overline{\text{CS}}$). После выполнения этих шагов устройство SPI1 готово к использованию.

Ввод-вывод через SPI

После настройки устройства SPI мы теперь можем инициировать команды на шине SPI и обмениваться данными с Winbond W25Q32 или его более вместительным родственником W25Q64. В этом примере прерывания не используются, поскольку наш код является ведущим и сам устанавливает время транзакций. Ведомый следует нашим командам. Следовательно, если наша программа по какой-либо причине задержится, ведомый будет ждать. Если существует риск чрезвычайно длительных задержек, вы можете использовать прерывания, чтобы избежать влияния шума на шине.

SPI может выполнять операции только отправки, только чтения или двунаправленной отправки и получения[52]. В нашем приложении чип Winbond будет отправлять данные обратно, поэтому мы всегда будем использовать функцию `spi_xfer()`, чтобы можно было как отправлять, так и получать байты. Эти процедуры будут проиллюстрированы в следующих разделах.

Чтение регистра SR1

В листинге 8.3 показан код из модуля *main.c*, который выполняет чтение через SPI первого регистра статуса устройства Winbond (SR1).

Листинг 8.3. Процедура чтения регистра статуса `w25_read_sr1()`

```
0059: static uint8_t
0060: w25_read_sr1(uint32_t spi) {
0061:   uint8_t sr1;
0062:
0063:   spi_enable(spi);
0064:   spi_xfer(spi,W25_CMD_READ_SR1);
0065:   sr1 = spi_xfer(spi,DUMMY);
0066:   spi_disable(spi);
0067:   return sr1;
0068: }
```

Процесс начинается с включения SPI1 в строке 63 (SPI1 передается как аргумент `spi`). Это заставляет аппаратное обеспечение устанавливать сигнал $\overline{\text{CS}}$ и включать тактовый сигнал (SCK). Затем вызывается функция `spi_xfer()` библиотеки libopencm3 для отправки байта, который задан следующим образом:

```
0036: #define W25_CMD_READ_SR1 0x05
```

[52] В реальности это означает всего лишь наличие/отсутствие программного доступа к операциям записи или чтения регистра данных. Аппаратно, как вы видели (см. рис. 8.2), процесс передачи всегда происходит путем обмена информацией между сторонами.

В рамках транзакции функция `spi_xfer()` возвращает байт с флеш-устройства, который в данном случае отбрасывается (строка 64). Полученное значение отбрасывается, поскольку флеш-устройство не знает, что нам отправлять, пока не получит нашу команду. Подобные вещи обычны в транзакциях SPI.

В строке 65 теперь отправляется фиктивное значение (ноль), чтобы контроллер мог получить ответ флеш-устройства теперь, когда оно знает, о чем мы запрашиваем. Это возвращаемое значение сохраняется в переменной `sr1`.

Наконец, устройство SPI1 отключается (строка 66), чтобы завершить транзакцию SPI. В этот момент аппаратное обеспечение отменяет подтверждение $\overline{\text{CS}}$, освобождая шину SPI. Флеш-устройство перейдет в режим ожидания. В конце процедуры прочитанное значение переменной `sr1` возвращается вызывающей стороне.

После завершения настройки устройства SPI использование периферийного устройства становится приятным и простым.

Ожидание готовности

Команда чтения статуса – единственная команда, которую флеш-устройство Winbond принимает в любое время. Все остальные запросы требуют, чтобы устройство было «не занято». Если к устройству обращаются во время, когда оно занято, запрос игнорируется. Это функция флеш-устройства Winbond, не связанная с SPI.

Поскольку флеш-устройство может быть занято записью страницы или стиранием секторов, удобно использовать функцию запроса состояния готовности. В листинге 8.4 показан используемый код.

Листинг 8.4. Функция ожидания готовности Winbond

```
0081: static void
0082: w25_wait(uint32_t spi) {
0083:
0084:   while ( w25_read_sr1(spi) & W25_SR1_BUSY )
0085:                 taskYIELD();
0086: }
```

Приведенная функция `w25_wait()` запрашивает регистр состояния SR1, а затем проверяет бит `W25_SR1_BUSY`. Если этот бит установлен, подпрограмма вызывает функцию `TaskYIELD()` в строке 85, чтобы позволить другим задачам использовать процессор. Когда этот бит становится нулевым, процедура просто завершается.

Чтение ID производителя

Команда чтения идентификатора (ID) производителя интересна взаимодействием игнорируемых полученных данных и фиктивных записей. Листинг 8.5 иллюстрирует этот процесс.

Листинг 8.5. Функция чтения ID производителя

```
0107: static uint16_t
0108: w25_manuf_device(uint32_t spi) {
0109:   uint16_t info;
0110:
0111:   w25_wait(spi);
0112:   spi_enable(spi);
0113:   spi_xfer(spi,W25_CMD_MANUF_DEVICE); // Byte 1
0114:   spi_xfer(spi,DUMMY); // Dummy1 (2)
0115:   spi_xfer(spi,DUMMY); // Dummy2 (3)
0116:   spi_xfer(spi,0x00); // Byte 4
0117:   info = spi_xfer(spi,DUMMY) << 8; // Byte 5
0118:   info |= spi_xfer(spi,DUMMY); // Byte 6
0119:   spi_disable(spi);
0120: return info;
0121: }
```

Обратите внимание, как строка 111 вызывает функцию `w25_wait()` для блокировки до тех пор, пока флеш-устройство не будет готово к новой команде. Затем на шину SPI в строке 113 записывается команда чтения информации об устройстве производителя. Возвращенный байт отбрасывается.

В строках 114 и 115 отправляются фиктивные байты (использовалось нулевое значение). Это необходимо для того, чтобы направить флеш-устройству достаточно тактовых импульсов для подготовки данных ответа. Обратите внимание, что полученные байты также игнорируются (флеш-устройство еще не было готово). В строке 116 записано значение 0x00 (в соответствии со спецификациями флеш-устройства), чтобы начать прием данных в следующем байте.

Наконец, в строках 117 и 118 отправляются еще два фиктивных байта, чтобы заставить ведомое устройство передать свои данные. Полученные значения сохраняем в 16-битной переменной `info`, которая позже возвращается программе в строке 120.

Это может показаться сумасшедшей транзакцией, но SPI-транзакции часто именно такие.

Запись флеш-памяти[53]

Микросхемы Winbond очень тщательно защищают вашу флеш-память. Они предлагают защиту областей поддерживаемой памяти с помощью программного обеспечения и, кроме того, защищают все устройство целиком установкой сигнала $\overline{\text{WP}}$ микросхемы. Последнее имеет смысл для материнских плат настольных ПК, где вы не хотите потерять код или настройки BIOS.

Безопасность флеш-памяти чипа имеет еще одно следствие. После включения питания, после операции записи или стирания данных микросхема возвращается в режим «запись запрещена». Чтобы выполнить запись данных, ей

[53] Рекомендуется перед изучением этого раздела ознакомиться со следующим («Стирание флеш-памяти»).

должна предшествовать операция «разрешения записи». Это делается путем установки опции «Write Enable Latch» (Защелка включения записи) в регистре статуса SR1. В листинге 8.6 показан код, используемый для этой цели.

Листинг 8.6. Включение разрешения записи

```
0095: static void
0096: w25_write_en(uint32_t spi,bool en) {
0097:
0098:   w25_wait(spi);
0099:
0100:   spi_enable(spi);
0101:   spi_xfer(spi,en ? W25_CMD_WRITE_EN : W25_CMD_WRITE_DI);
0102:   spi_disable(spi);
0103:
0104: w25_wait(spi);
0105: }
```

Строка 98 ожидает готовности устройства (иначе любые дальнейшие команды будут игнорироваться). В зависимости от того, была ли вызвана процедура для включения или отключения записи, соответствующая команда отправляется в строке 101. Поскольку установка разрешения может потребовать некоторого времени, мы ждем готовности устройства в строке 104 перед возвратом из функции.

Вероятно, следует снова прочитать регистр SR1, чтобы увидеть, удалось это или нет. Если на чипе был установлен активный сигнал $\overline{\text{WP}}$, эта операция завершится неудачно. В нашем проекте мы запрограммировали $\overline{\text{WP}}$ как неактивный, поэтому проверка была проигнорирована.

Когда запись во флеш-память включена, мы можем записать один или несколько байтов флеш-памяти. В листинге 8.7 представлена функция `w25_write_data()` для программирования записи данных.

Листинг 8.7. Функция записи данных

```
0211: static unsigned // New address is returned
0212: w25_write_data(uint32_t spi,uint32_t addr,void *data,uint32_t bytes) {
0213:   uint8_t *udata = (uint8_t*)data;
0214:
0215:   w25_write_en(spi,true);
0216:   w25_wait(spi);
0217:
0218:   if ( w25_is_wprotect(spi) ) {
0219:       std_printf("Write disabled.\n");
0220:       return 0xFFFFFFFF; // Indicate error
0221:   }
0222:
0223:   while ( bytes > 0 ) {
0224:       spi_enable(spi);
0225:       spi_xfer(spi,W25_CMD_WRITE_DATA);
```

```
0226:       spi_xfer(spi,addr >> 16);
0227:       spi_xfer(spi,(addr >> 8) & 0xFF);
0228:       spi_xfer(spi,addr & 0xFF);
0229:       while ( bytes > 0 ) {
0230:           spi_xfer(spi,*udata++);
0231:           --bytes;
0232:           if ( (++addr & 0xFF) == 0x00 )
0233:               break;
0234:       }
0235:       spi_disable(spi);
0236:
0237:       if ( bytes > 0 )
0238:           w25_write_en(spi,true); // More to write
0239:   }
0240:   return addr;
0241: }
```

Строка 215 включает опцию «Write Enable Latch» и проверяет, установлена ли она в строке 218 (путем чтения SR1). Если опция не установлена, терминальная программа получает сообщение «Write disabled» (Запись отключена) в строке 219 перед возвратом кода ошибки.

Цикл в строках 223–239 выдает команду записи данных (`W25_CMD_WRITE_DATA`) и три байта адреса флеш-памяти. Затем байты передаются в цикле в строках 229–234. Новый адрес флеш-памяти возвращается в младших 24 битах возвращаемого значения.

Внешний цикл (строка 223) предназначен для записи в страницы размером 256 байт. Чип Winbond перенесет адрес на ту же самую страницу, если вы попытаетесь пересечь границы страниц, поэтому этот код записывает каждую страницу как отдельную команду SPI.

Стирание флеш-памяти

Легко забыть, что мы имеем дело с флеш-памятью. Чтобы значения были успешно записаны, сначала необходимо стереть затронутую память. В стертом состоянии байт во флеш-памяти имеет значение 0xFF (все биты установлены в 1). Как только байт записан как 0x00, его нельзя вернуть обратно в 0xFF без операции стирания (ни один бит не может быть снова установлен в 1).

Несмотря на это, при записи флешки можно схитрить. Допустим, у вас есть 1 байт, который представляет восемь ячеек хранения данных, где единичный бит представляет собой доступную ячейку, а нулевой бит – используемую (запрограммированную) ячейку. Если текущее значение байта равно 0x7F, т. е. старший бит (бит 7) очищен, вы можете сделать доступной следующую ячейку, обнулив следующий бит без выполнения стирания. Запись значения 0x3F (или даже 0xBF) очистит следующий бит, в результате чего будет получено обратное значение 0x3F. (Обратите внимание, что запись 0xBF также работает, поскольку седьмой бит игнорируется, если он уже равен нулю.) Программирование единичных битов не требует операций, но программирование нулевых битов обращает единичные биты в нули.

Однако, какой бы умной ни была схема, реальность требует в конечном итоге использования операции стирания. Трудность здесь состоит в том, что стирание должно выполняться крупными блоками. Микросхемы W25QXX позволяют стирать следующим образом:

- один сектор (4 Кбайта);
- блок 32 Кбайта;
- блок 64 Кбайта;
- весь объем памяти.

В листинге 8.8 показан код стирания всей флеш-памяти, используемый в демонстрационных целях. Статус защиты от записи проверяется в строках 170–173. Если защита микросхемы установлена, сообщение отказа выдается в строке 171 перед возвратом ошибки. Строки 175–177 выполняют стирание чипа, а остальная часть функции проверяет, прошла ли операция успешно. Если отключение защиты записи не сработало, функция вернет соответствующее сообщение, которое означает, что команда не выполнена или была проигнорирована. Если это произойдет, скорее всего, возникла проблема с программным обеспечением или сообщение на шине SPI каким-то образом повреждено.

Листинг 8.8. Функция стирания памяти

```
0167: static bool
0168: w25_chip_erase(uint32_t spi) {
0169:
0170:   if ( w25_is_wprotect(spi) ) {
0171:       std_printf("Not Erased! Chip is not write enabled.\n");
0172:       return false;
0173:   }
0174:
0175:   spi_enable(spi);
0176:   spi_xfer(spi,W25_CMD_CHIP_ERASE);
0177:   spi_disable(spi);
0178:
0179:   std_printf("Erasing chip..\n");
0180:
0181:   if ( !w25_is_wprotect(spi) ) {
0182:      std_printf("Not Erased! Chip erase failed.\n");
0183:      return false;
0184:   }
0185:
0186:   std_printf("Chip erased!\n");
0187:   return true;
0188: }
```

Остальные функции стирания практически такие же, за исключением того факта, что они идентифицируют номер блока, который стирают. Листинг 8.9 иллюстрирует используемую процедуру.

Листинг 8.9. Процедура стирания блока `w25_erase_block()`

```
0233: static void
0234: w25_erase_block(uint32_t spi,uint32_t addr,uint8_t cmd) {
0235:   const char *what;
0236:
0237:   if ( w25_is_wprotect(spi) ) {
0238:       std_printf("Write protected. Erase not performed.\n");
0239:       return;
0240:   }
0241:
0242:   switch ( cmd ) {
0243:   case W25_CMD_ERA_SECTOR:
0244:       what = "sector";
0245:       addr &= ~(4*1024-1);
0246:       break;
0247:   case W25_CMD_ERA_32K:
0248:       what = "32K block";
0249:       addr &= ~(32*1024-1);
0250:       break;
0251:   case W25_CMD_ERA_64K:
0252:       what = "64K block";
0253:       addr &= ~(64*1024-1);
0254:       break;
0255:   default:
0256:       return; // Should not happen
0257:   }
0258:
0259:   spi_enable(spi);
0260:   spi_xfer(spi,cmd);
0261:   spi_xfer(spi,addr >> 16);
0262:   spi_xfer(spi,(addr >> 8) & 0xFF);
0263:   spi_xfer(spi,addr & 0xFF);
0264:   spi_disable(spi);
0265:
0266:   std_printf("%s erased, starting at %06X\n",
0267:      what,(unsigned)addr);
0268: }
```

Аргумент, передаваемый как `addr`, считается номером блока, который необходимо стереть. Аргумент `cmd` затем указывает, какой тип стирания следует выполнить. Затем строка 242 определяет, какой тип стирания выполняется, и выполняет операцию маски над адресом в соответствии с используемым размером блока.

Команда стирания происходит в строках 260–263, куда передаются команда и 3 байта номера блока.

Чтение флеш-памяти

После того как флеш-память записана, нам нужна возможность прочитать ее обратно. В листинге 8.10 показана функция для выполнения этой задачи.

Листинг 8.10. Функция чтения SPI Flash

```
0191: static uint32_t // New address is returned
0192: w25_read_data(uint32_t spi,uint32_t addr,void *data,uint32_t bytes) {
0193:   uint8_t *udata = (uint8_t*)data;
0194:
0195:   w25_wait(spi);
0196:
0197:   spi_enable(spi);
0198:   spi_xfer(spi,W25_CMD_FAST_READ);
0199:   spi_xfer(spi,addr >> 16);
0200:   spi_xfer(spi,(addr >> 8) & 0xFF);
0201:   spi_xfer(spi,addr & 0xFF);
0202:   spi_xfer(spi,DUMMY);
0203:
0204:   for ( ; bytes-- > 0; ++addr )
0205:       *udata++ = spi_xfer(spi,DUMMY);
0206:
0207:   spi_disable(spi);
0208:   return addr;
0209: }
```

Аргумент `addr` указывает относительный адрес флеш-памяти для чтения, а аргументы `data` и `bytes` указывают, куда поместить считанные данные. В строке 198 выдается команда чтения SPI, а затем 3 байта адресной информации передаются ведомому устройству. Процедура чтения требует записи фиктивного байта после адреса (строка 202). После этого цикл в строках 204 и 205 считывает обратно байты, переданные флеш-устройством.

Команда Winbond «Fast Read» (быстрое чтение) здесь использовалась не ради скорости. У обычной команды чтения есть проблема: она оборачивается в пределах текущей 256-байтовой страницы. Чтобы разрешить чтение за пределы страницы, вместо этого используется команда быстрого чтения `W25_CMD_FAST_READ`.

Демопрограмма

Хватит технических разговоров! Время для демонстрации. Если вы еще этого не сделали, соберите программу сейчас (в каталоге *~/stm32f103c8t6/rtos/winbond*):

```
$ make clobber
$ make
...
arm-none-eabi-size main.elf
  text  data  bss  dec   hex  filename
  17376 1472 18104 36952 9058 main.elf
```

Подключите программатор и выполните следующие действия:

```
$ make flash
/usr/local/bin/st-flash write main.bin 0x8000000
```

```
st-flash 1.3.1-9-gc04df7f-dirty
2017-11-04T10:03:37 INFO src/usb.c: -- exit_dfu_mode
2017-11-04T10:03:37 INFO src/common.c: Loading device parameters....
...
2017-11-04T10:03:39 INFO src/common.c: Starting verification of write
  complete
2017-11-04T10:03:39 INFO src/common.c: Flash written and verified!
  jolly good!
```

После прошивки платы STM32 сначала отключите программатор (важно!), а затем подключите USB-кабель между STM32 и компьютером. В качестве терминальной программы я использую *minicom*, но вы можете использовать другую, если хотите. Чтобы подключиться через USB, вам необходимо узнать имя устройства, которое вы будете использовать. Если вам нужна помощь в этом, прочтите главу 7.



Если у вас *minicom* установлен так, что для каталога сохранения по умолчанию требуются права root, можете использовать команду `sudo minicom -s`.

Как настроить minicom для использования порта, см. главу 6, раздел «Проект uart».

Запуск демоверсии

После завершения настройки *minicom* вы сможете подключить USB-кабель STM32 и запустить *minicom* (в указанной последовательности). Используйте имя сохраненных настроек в качестве аргумента в командной строке *minicom* (здесь я использовал «usb»):

```
$ minicom usb
```

При подключении через USB к STM32 *minicom* должен отобразить следующее:

```
Welcome to minicom 2.7
OPTIONS:
Compiled on Sep 17 2016, 05:53:15.
Port /dev/cu.usbmodemWGDEM1, 10:17:40
Press Meta-Z for help on special keys
```

Теперь мы можем общаться с STM32. Нажмите <**Enter**>, чтобы запустить демонстрационный код для отображения следующего меню:

```
Winbond Flash Menu:
0 ... Power down
1 ... Power on
a ... Set address
```

```
d ... Dump page
e ... Erase (Sector/Block/64K/Chip)
i ... Manufacture/Device info
h ... Ready to load Intel hex
j ... JEDEC ID info
r ... Read byte
p ... Program byte(s)
s ... Flash status
u ... Read unique ID
w ... Write Enable
x ... Write protect
Address: 000000
:
```

Нажатие любой неизвестной командной буквы или нажатие клавиши <**Enter**> приведет к повторному отображению этого меню. Первое, что нужно сделать, – это посмотреть, можем ли мы прочитать статус флеш-устройства. Нажмите <**s**> (или <**S**>), чтобы прочитать статус:

```
: S
SR1 = 00 (write protected)
SR2 = 00
```

Демопрограмма сообщает, что регистр SR1 W25Q25 равен шестнадцатеричному 00, а регистр SR2 также равен 00. Если вместо этого вы видите FF, возможно, вы неправильно обмениваетесь данными через SPI.

Чтобы включить разрешение записи, нажмите <**W**>:

```
: W
SR1 = 02 (write enabled)
SR2 = 00
```

Отсюда вы можете видеть, что регистр SR1 флеш-устройства теперь читается как шестнадцатеричное 02, что указывает на то, что запись теперь включена. При включенной записи вы можете выполнить стирание чипа (нажмите <**E**>):

```
: E
Erase what?
s ... Erase 4K sector
b ... Erase 32K block
z ... Erase 64K block
c ... Erase entire chip
anything else to cancel
: c
Erasing chip..
Chip erased!
```

Не паникуйте, если стирание займет несколько секунд. Это не программа зависла, просто чип Winbond усердно работает.

Теперь давайте установим адрес и проверим, действительно ли чип стерт. Введите <**A**> и введите ноль, а затем <**Enter**>:

```
: A
Address: 0
Address: 000000
```

Теперь давайте выгрузим целиком страницу (256 байт), набрав <**D**>:

```
: D
000000 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
000010 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
000020 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
000030 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
000040 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
000050 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
000060 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
000070 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
000080 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
000090 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
0000A0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
0000B0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
0000C0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
0000D0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
0000E0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
0000F0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF................
Address: 000100
```

Кажется, это подтверждает стирание по крайней мере страницы 0. Обратите внимание, что адрес увеличился на страницу, поэтому продолжающиеся нажатия <**D**> позволят отображать последующие страницы.

Теперь давайте запишем несколько байтов. Следуйте показанному сеансу:

```
: W
SR1 = 02 (write enabled)
SR2 = 00
: A
Address: 0
Address: 000000
: P
$000000 AA BB CC DD EE
$000005 5 bytes written.
: D
000000 AA BB CC DD EE FF FF FF FF FF FF FF FF FF FF FF ................
000010 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
000020 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
```

В сеансе мы сбрасываем адрес в ноль после разрешения записи, а затем набираем <**P**>, чтобы запрограммировать несколько байтов. Между каждым из байтов данных AA, BB и т. д. нажмите <**Enter**>. Еще одно нажатие <**Enter**> приведет к выходу из программного режима.

Чтобы убедиться, что данные были записаны, страница 0 была выгружена для просмотра. Данные ASCII также можно запрограммировать, введя одинарную кавычку, за которой следует один символ, который вы хотите ввести. Следующий сеанс иллюстрирует это (страница 0 предполагается стертой):

```
: P
$000000 'H 'e 'l 'l 'o ' 'W 'o 'r 'l 'd '!
$00000C 12 bytes written.
: D
000000 48 65 6C 6C 6F 20 57 6F 72 6C 64 21 FF FF FF FF Hello World!....
000010 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
000020 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
```

Это немного утомительно, но текст был введен успешно.

Теперь давайте проверим природу программирования флеш-памяти. Установите адрес стертого места и запрограммируйте его как 0x7F. Здесь мы будем использовать шестнадцатеричный адрес 10:

```
: P
$000010 7F
$000011 1 bytes written.
: D
000000 48 65 6C 6C 6F 20 57 6F 72 6C 64 21 FF FF FF FF Hello World!....
000010 7F FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
```

Убедитесь, что во второй строке слева отображается 7F. Теперь запрограммируйте шестнадцатеричное местоположение 10 с шестнадцатеричным значением BF:

```
: A
Address: 10
Address: 000010
: P
$000010 BF
$000011 1 bytes written.
: D
000000 48 65 6C 6C 6F 20 57 6F 72 6C 64 21 FF FF FF FF Hello World!....
000010 3F FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ?...............
```

Обратите внимание, что старший бит (бит 7) ячейки 000010 остается неизменным. Но поскольку бит 6 (в 0xBF) был нулем, был запрограммирован новый нулевой бит (бит 6), что привело к получению значения 0x3F.

ID производителя

ID флеш-чипа можно проверить с помощью команд меню «I» и «J»:

```
: I
Manufacturer $EF Device $15 (W25X32)
: J
Manufacturer $EF Type $40 Capacity $16 (W25X32)
```

Режим Power Down

Микросхему Winbond можно отключить для экономии электроэнергии. Используйте пункты меню «0» и «1» для выключения и включения соответственно:

```
: 0
: 1
```

При включении в режиме чтения ток, потребляемый устройством, у меня составлял около 19.4 мА. В выключенном состоянии (режим Power Down) ток снижался до 0.7 мкА.

Итоги

Это была длинная глава, так что дайте себе передышку. В приведенной демонстрационной программе есть несколько неизученных опций, таких как уникальный идентификатор и загрузка Intel hex. Обязательно проверьте функцию уникального идентификатора. Функция загрузки Intel hex будет использоваться в следующей главе для программирования оверлеев, так что следите за обновлениями. Есть также ряд функций W25QXX, которые не были изучены, например многочисленные функции защиты. Чтобы получить полный спектр возможностей этой микросхемы, прочтите технические описания производителя. Простой поиск в Google по запросу «W25Q64 datasheet PDF» покажет то, что вам нужно.

Завершение изучения этой главы означает, что теперь вы обладаете знаниями о протоколе SPI и о том, как его применять в STM32 под FreeRTOS с использованием библиотеки libopencm3. Может показаться, что было слишком много сведений, и это действительно так. В следующей главе наше внимание будет обращено на одно практическое применение внешней флеш-памяти: оверлеи.

Дополнительные источники

1. «Serial Peripheral Interface», «Википедия». https://en.wikipedia.org/wiki/Serial_Peripheral_Interface_Bus (дата доступа 01.09.2024). То же по-русски: https://ru.wikipedia.org/wiki/Serial_Peripheral_Interface (дата доступа 01.09.2024).

Упражнения

1. Сколько линий данных используется SPI в двунаправленных каналах? Каковы названия сигналов в этих линиях?
2. Откуда берутся тактовые сигналы?
3. Какие уровни напряжения используются для сигналов SPI?
4. Почему для вывода STM32 NSS необходимо использовать подтягивающий резистор?
5. Почему в некоторых транзакциях SPI необходимо отправлять фиктивное значение?

Глава 9 • Оверлеи

Сегодня мало что слышно о наложениях кода – оверлеях. Учитывая сегодняшнюю, казалось бы, неограниченную виртуальную память на настольных компьютерах и серверах, приложения часто не проверяют риск нехватки памяти. Тем не менее на заре компьютерной эры при использовании основной памяти мэйнфреймов и только что появившегося IBM PC нехватка памяти была частой проблемой. Оверлеи помогли добиться большего с меньшими затратами.

Оверлеи продолжают играть важную роль в микроконтроллерах и сегодня из-за ограниченного объема памяти этих устройств. Проектирование может начинаться с выбора микроконтроллера, но позже обнаружится, что данное программное обеспечение не подходит. Если это происходит на поздней стадии цикла разработки продукта, необходимо найти решение для использования существующего микроконтроллера или отказаться от каких-то функций программного обеспечения.

Дизайнер может знать, что некоторые разделы кода не нужны часто. Например, полнофункциональный интерпретатор BASIC может заменять сегмент кода программы, когда это необходимо. В остальное время этот код останется неиспользованным и не обязательно будет резидентным.

В интернете не так уж много информации о том, как использовать оверлеи в GCC (см. список литературы в конце главы, {1]). О сценариях загрузки ведется много дискуссий, но остальные подробности обычно оставляются в качестве упражнения для читателя. Эта глава посвящена полной демонстрации рабочего примера. В этой демонстрации оверлеи из флеш-чипа с SPI-интерфейсом будут заменять код в области оверлеев SRAM, где он будет выполняться. Учитывая, что эти флеш-чипы предлагают 4 или 8 Мбайт памяти для размещения кода, только ваше воображение является пределом, когда речь идет о крупных приложениях на STM32.

Проблема компоновки

При разработке приложений ваши знания подсказывают написать код C, который транслируется компилятором в один или несколько объектных файлов (**.o*). Если приложение достаточно невелико, вы компонуете его в один окончательный файл **.elf*, который предназначен для размещения в доступной флеш-памяти микроконтроллера. Утилите *st-flash* для STM32 необходим файл образа памяти, поэтому на следующем этапе сборки сначала файл *.elf* преобразуется в двоичный образ:

```
$ arm-none-eabi-objcopy -Obinary main.elf main.bin
```

Затем файл образа *main.bin* загружается во флеш-память по адресу 0x8000000:

```
$ st-flash write main.bin 0x8000000
```

Это процесс типовой компоновки, но как создавать оверлеи? Давайте начнем с демопроекта Winbond. Перейдите в следующий подкаталог:

```
cd ~/stm32f103c8t6/rtos/winbond
```

Затем выполните следующее:

```
$ make clobber
$ make
```

При этом проект перекомпилируется, и в конце всего этого стадия компоновки будет выглядеть примерно так (строки разбиты для удобства чтения):

```
arm-none-eabi-gcc --static -nostartfiles -Tstm32f103c8t6.ld \
-mthumb -mcpu=cortex-m3 -msoft-float -mfix-cortex-m3-ldrd \
-Wl,-Map=main.map -Wl,--gc-sections main.o rtos/heap_4.o \
rtos/list.o rtos/port.o rtos/queue.o rtos/tasks.o \
rtos/opencm3.o -specs=nosys.specs -Wl,--start-group \
-lc -lgcc -lnosys -Wl,--end-group \
-L/Users/ve3wwg/stm32f103c8t6//rtos/libwwg -lwwg \
-L/Users/ve3wwg/stm32f103c8t6/libopencm3/lib \
-lopencm3_stm32f1 -o main.elf
```

Весь процесс управляется сценарием компоновки, указанным в опции `-Tstm32f103c8t6.ld`. Например, если в команде сборки Linux не указан параметр -T, он будет использоваться по умолчанию. Но давайте рассмотрим файл, предоставленный в проекте Winbond.

Раздел MEMORY

Сценарий компоновщика содержит в начале раздел описания памяти MEMORY, который выглядит следующим образом:

```
MEMORY
{
    rom (rx) : ORIGIN = 0x08000000, LENGTH = 64K
    ram (rwx) : ORIGIN = 0x20000000, LENGTH = 20K
}
```

Эта часть сценария загрузки объявляет две области памяти. Это области, в которые мы собираемся загрузить код (ROM) или выделить место для выполняемого кода (RAM). Если вы создаете большие приложения, я бы посоветовал вам в сценарии компоновщика изменить размер этого диска на 128 Кбайт и использовать команду `st-link`, чтобы прошить его с помощью опции `--flash=128k` (команда `make bigflash` укажет эту опцию из предоставлен-

ного файла *make*). Как отмечалось ранее, STMF103C8T6, похоже, поддерживает 128 Кбайт, несмотря на утверждение, что существует только 64 Кбайта.

После расширения до 128 Кбайт раздел MEMORY должен выглядеть следующим образом:

```
MEMORY
{
    rom (rx) : ORIGIN = 0x08000000, LENGTH = 128K
    ram (rwx) : ORIGIN = 0x20000000, LENGTH = 20K
}
```

Необязательные атрибуты в скобках, например `rwx`, описывают предполагаемое использование области памяти (чтение r, запись w и выполнение x). В документации GCC говорится, что они поддерживаются для обратной совместимости с компоновщиком AT&T, но в остальном проверяются только на достоверность.

Параметр `ORIGIN = 0x20000000` указывает, где физически находится блок оперативной памяти. Параметр `LENGTH` – это размер в байтах. Давайте сравним это представление с базовой картой физической памяти микроконтроллера, показанной на рис. 9.1.

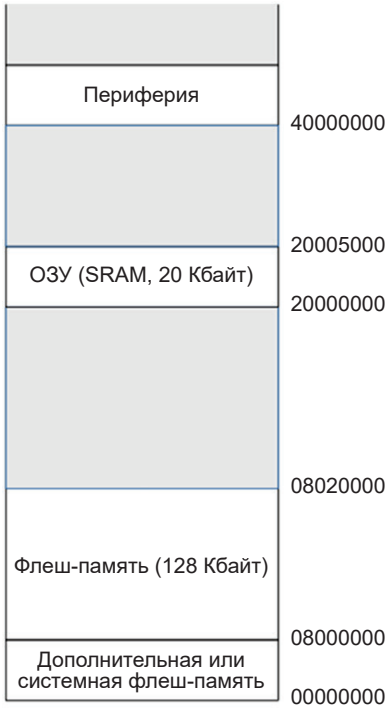


Рис. 9.1. Базовая карта памяти STM32F103C8T6 (адреса в шестнадцатеричном формате)

Память, которая находится в области, начинающейся с 0x00000000, зависит от переключателей BOOT0 и BOOT1. Обычно используется настройка BOOT0 = 0, в результате чего системная флеш-память появляется как в нулевой позиции, так и в 0x08000000. Это позволяет коду запуска контроллера программировать флеш-память.

Выше по адресу 0x20000000 мы находим статическое ОЗУ (SRAM). Размер этой области памяти для STM32F103C8T6 составляет 20 Кбайт. Теперь давайте посмотрим на оставшуюся часть сценария загрузки, чтобы понять, как он работает.

ENTRY и EXTERN

Основным драйвером процесса загрузки будет область SECTIONS файла, которую мы рассмотрим далее, но сначала обсудим две записи. Это записи по ключевым словам `ENTRY` и `EXTERN`:

```
ENTRY(reset_handler)
EXTERN (vector_table)
```

Эти записи не находятся в областях MEMORY или SECTIONS сценария загрузки, а размещаются отдельно. Ключевое слово `ENTRY` указывает процедуру, которой передается управление при запуске. Ключевое слово `EXTERN` идентифицирует область данных, которая будет определять начальный вектор прерывания. В используемой среде они будут предоставлены в модуле с именем *vector.o* из статической библиотеки libopencm3 следующим образом:

```
~/stm32f103c8t6/libopencm3/lib/libopencm3_stm32f1.a
```

Если вам нужно каким-либо образом изменить запуск или вам просто интересно, просмотреть текст модуля *vector.o* можно следующим образом:

```
~/stm32f103c8t6/libopencm3/lib/cm3/vector.c
```

Поскольку ключевое слово `ENTRY` ссылается на обозначение `reset_handler`, при загрузке загружается модуль *vector.c* (если вы не указали свой собственный). Это избавляет вас от необходимости определять всю необходимую инициализацию. Когда обработчик сброса, на который указывает `ENTRY`, выполнит всю настройку, он вызовет вашу программу `main()`.

Раздел SECTIONS

Вот здесь в сценарии загрузки все становится интереснее. В общих чертах вы обнаружите, что этот раздел выглядит следующим образом:

```
SECTIONS
{
  .text : {
```

```
      *(.vectors) /* Vector table */
      *(.text*) /* Program code */
      . = ALIGN(4);
      *(.rodata*) /* Read-only data */
      . = ALIGN(4);
  } >rom
  ...etc...
}
```

Здесь сокращена часть содержимого раздела, чтобы вы могли сосредоточиться на главном. Из показанного фрагмента видно, что в скрипте загрузки встречаются комментарии, оформленные, как в языке C, например `/* comment */`. Пусть вас не смущает оставшаяся часть со странным синтаксисом. Давайте разберемся.

Конкретный раздел начинается с имени, которое в этом примере `.text`. Имена разделов могут состоять практически из любых символов, однако случайные символы или пробелы должны быть заключены в прямые кавычки («). В противном случае ожидается, что символ будет окружен пробелами.

Объявленный раздел называется `.text`, за ним следует двоеточие (:), а затем открывающая и в конце закрывающая фигурные скобки. После закрывающей скобки мы видим надпись `>rom`. Это указывает, что все, описанное между фигурными скобками, будет помещено в область памяти с именем `rom` (т. е. во флеш-память).

То, что находится между фигурными скобками, описывает входные данные и вычисления значений имен. Давайте сначала посмотрим на один пример входа:

```
*(.vectors)
```

Это означает, что любой входной файл (*), содержащий раздел объектного файла с именем `.vectors`, должен быть включен в определяемый выходной раздел (в данном случае `.text`).

Имейте в виду, что есть два типа разделов:

1) секции, включающие объекты (например, `.vectors` в примере),
2) выходные секции (например, `.text` в примере).

Звездочка в начале подразумевает любой файл; вместо нее можно указывать конкретные имена файлов. Например, являются возможными следующие два примера:

```
*.o(.vectors) /* любой с расширением .o имеющий секцию .vectors */
special.o(.vectors) /* файл special.o имеющий секцию .vectors */
```

Если мы сократим пример до входных данных, получим следующее:

```
.text : {
*(.vectors) /* Vector table */
```

```
*(.text*) /* Program code */
*(.rodata*) /* Read-only data */
} >rom
```

Если исходить из приведенного примера, то мы загружаем из любого входного файла, из его объектной секции `.vectors`, `.text` или `.rodata`. Они будут загружены в область памяти с именем `rom`. Вы все еще со мной?

А что насчет того, другого заклинания вуду? Символ точки (.) используется в качестве текущего местоположения в разделе. Эта практика, несомненно, исходит из использования ассемблером точки для "location assignment counter" (*счетчика назначения позиции*). В сценарии компоновки символ точки служит той же цели:

```
.text : {
    *(.vectors) /* Vector table */
    *(.text*) /* Program code */
    . = ALIGN(4);
    *(.rodata*) /* Read-only data */
    . = ALIGN(4);
} >rom
```

Значение точки в середине (после строки `.text`) – указать местоположение в конце раздела `.text` (в этой точке компоновки), но округленное до границы 4-байтового слова (с использованием специальной функции `ALIGN(4)`). В этом случае текущая позиция сдвигается до позиции следующей границы 4-байтового слова. То же самое вызывается второй раз после загрузки раздела ввода `.rodata` (read-only data, *данные только для чтения*), так что, если в секцию `.text` будет загружено что-нибудь еще, оно также будет выровнено по нужной границе.

Обратите внимание, что выражения, подобные тем, которые содержат точку, заканчиваются точкой с запятой (;). Символические обозначения также могут быть заданы таким же образом. Например, в том же скрипте между объявлениями разделов вы найдете следующее:

```
} >rom
. = ALIGN(4);
_etext = .;
```

Поскольку эти последние две строки являются выражениями, появляющимися за пределами определения раздела (который загружается в `rom`), точка здесь будет относиться к последней указанной позиции (внутри `rom`). Отсюда мы видим, что позиция точки выравнивается по границе следующего слова, а затем присваивается обозначению `_etext`. При необходимости в этих выражениях допускается арифметика. Тогда обозначение `_etext` в вашей программе будет иметь значение адреса первого байта после конца вашей области «только для чтения» во флеш-памяти (`rom`).

PROVIDE

Ключевое слово PROVIDE, используемое в сценарии компоновщика, дает вам возможность определить символическое обозначение, если оно необходимо (на него ссылаются). Если на обозначение не ссылаются, его определение исключается из компоновки во избежание конфликтов имен. В вашем сценарии загрузки можно найти следующее:

```
PROVIDE(_stack = ORIGIN(ram) + LENGTH(ram));
```

В этом выражении утверждается, что необходимо предоставить имя `_stack`, если на него где-то ссылаются. Тогда его значение рассчитывается как начальный адрес области памяти плюс длина области памяти. Другими словами, это начальный адрес стека (в конце доступной области SRAM), растущего затем в сторону уменьшения адресов.

Перемещение данных

При управлении компоновщиком возникает одна проблема: ему необходимо поместить некоторые данные во флеш-память, но при этом помещать ссылки на эти данные, как если бы они существовали в SRAM. Если посмотреть на это с другой стороны, мы будем использовать некоторые данные в SRAM, но они не будут храниться в SRAM до тех пор, пока какой-нибудь код запуска не скопирует их туда. Вот пример вашего скрипта загрузки:

```
.data : {
    _data = .;
    *(.data*) /* Read-write initialized data */
    . = ALIGN(4);
    _edata = .;
} >ram AT >rom
```

Здесь мы определяем два имени:

1) `_data` (начало данных),
2) `_edata` (конец данных).

Особый интерес представляет последняя строка:

```
} >ram AT >rom
```

Как вы, наверное, догадались, это означает, что имена должны определяться так, как если бы они находились в оперативной памяти (SRAM), но вместо этого они будут записываться во флеш-раздел (ROM). Код инициализации в модуле *vector.o*, обсуждавшийся ранее, скопирует эти данные из флеш-памяти в конечную ячейку SRAM перед вызовом `main()`.

Это работает для перемещения любых символических ссылок. Например, если у вас есть статическая константа `int`, которая не была объявлена как `const`, то она будет предназначена для SRAM. Адрес этой константы будет установ-

лен компоновщиком где-то в SRAM (адреса от 0x20000000 до 0x20004FFF). Однако само значение `int` будет загружено во флеш-память (где-то между 0x08000000 и 0x0801FFFF). При инициализации запуска необходимо скопировать это значение в SRAM, чтобы сделать его действительным.

Имейте это в виду, когда пойдет речь об оверлеях.

Определение оверлея

Теперь, когда вы вооружены и опасны, давайте начнем с использования компоновщика для определения оверлеев. Перейдите в каталог проекта *overlay1*:

```
$ cd ~/stm32f103c8t6/rtos/overlay1
```

В этом подкаталоге проекта находится модифицированная версия сценария компоновщика *stm32f103c8t6.ld*, которую мы и рассмотрим.

Первое интересное – то, что мы объявили четыре области памяти вместо обычных двух:

```
MEMORY
{
    rom (rx) : ORIGIN = 0x08000000, LENGTH = 128K
    ram (rwx) : ORIGIN = 0x20000000, LENGTH = 18K
    /* Overlay area in RAM */
    ovl (rwx) : ORIGIN = 0x20004800, LENGTH = 2K
    /* Give external flash its own storage */
    xflash (r) : ORIGIN = 0x00000000, LENGTH = 4M
}
```

Область оперативной памяти была сокращена на 2 Кбайта, чтобы оставить место для новой оверлейной области SRAM с именем `ovl`. Память, адресованная от 0x20004800 до 0x20004FFF, зарезервирована для выполнения нашего оверлейного кода.

Другая новая область, называемая `xflash`, определена таким образом, чтобы компоновщик мог генерировать код, который будет находиться во внешней флеш-памяти. Подробнее об этом будет рассказано позже.

Оставшуюся часть сценария компоновщика можно найти в файле далее в следующем виде:

```
OVERLAY : NOCROSSREFS {
    .fee {
        .overlay1_start = .;
        *(.ov_fee) /* fee() */
        *(.ov_fee_data) /* static data for fee() */
    }
    .fie { *(.ov_fie) } /* fie() */
    .foo { *(.ov_foo) } /* foo() */
    .fum { *(.ov_fum) } /* fum() */
```

```
} >ovl AT >xflash
PROVIDE (overlay1 = .overlay1_start);
```

Давайте теперь разберем это по отдельности. Ключевое слово OVERLAY указывает компоновщику загружать все разделы в оверлейный раздел с перекрытием физической памяти. В показанном примере разделы .fee, .fie, .foo и .fum начинаются в одном и том же месте. Учитывая, что >ovl помещает этот код в область оверлейной памяти, все они будут иметь начальный адрес 0x20004800. Конечно, понятно, что не все из них могут находиться в этом пространстве одновременно.

Символ .overlay1_start фиксирует начальный адрес оверлейной области и в конечном итоге передается в оператор PROVIDE, так что обозначение overlay1 будет содержать начальный адрес оверлея 0x20004800. Это имя можно использовать в программе на C.

Ключевое слово NOCROSSREFS предоставляет еще одну важную функцию компоновщика. Было бы невозможно, чтобы один оверлей вызывал или ссылался на другой в том же регионе. В одном регионе одновременно может находиться только один оверлейный код. Вызов функции fie() из функции fee() был бы катастрофой. Ключевое слово NOCROSSREFS указывает компоновщику рассматривать такой сценарий как ошибку.

Наконец, обратите внимание на следующую строку:

```
} >ovl AT >xflash
```

Она предписывает компоновщику переместить код, как если бы он выполнялся по адресу оверлея (ovl) (в SRAM), но вместо этого поместить этот код оверлея в область памяти xflash. Область памяти xflash позже потребует некоторой специальной обработки, но нам нужно, чтобы компоновщик сначала проделал эту хитрость.

Важная концепция здесь заключается в том, что все, что попадает в xflash, предназначено для флеш-устройства Winbond, начиная с нулевого адреса этой флеш-памяти. Это было установлено ключевым словом ORIGIN следующим образом:

```
xflash (r) : ORIGIN = 0x00000000, LENGTH = 4M
```

Код оверлея

Раздел, объявленный как .fee, состоит из двух входных разделов, которые происходят из разделов .ov_fee и .ov_fee_data. Это пример объявления кода и данных в одном оверлее, представленный в листинге 9.1.

Листинг 9.1. Объявление оверлейной функции fee()

```
0027: int fee(int arg) __attribute__((noinline,section(".ov_fee")));
...
0115: int
```

```
0116: fee(int arg) {
0117:   static const char format[] // Placed in overlay
0118:       __attribute__((section(".ov_fee_data")))
0119:       = "***********\n"
0120:         "fee(0x%04X)\n"
0121:         "***********\n";
0122:
0123:   std_printf(format,arg);
0124:   return arg + 0x0001;
0125: }
```

Чтобы сообщить компилятору, что функция `fee()` должна перейти в секцию `.ov_fee` (в объектном файле), мы должны использовать ключевое слово GCC `__attribute__` (строка 27). Этот атрибут можно указать только в прототипе функции.

Ключевое слово `noinline` не позволяет GCC встраивать код для метода `fee()`. Это особенно важно для нашей демонстрации, поскольку функция достаточно мала, чтобы ее можно было встроить в точку вызова GCC.

Второй аргумент `section(".ov_fee")` называет секцию, в которую должен быть записан код нашей функции `fee()` в объектном файле *main.o*. Данные только для чтения, объявленные в строках 117–121, указаны для помещения в раздел `.ov_fee_data`. Компилятор настаивает на том, чтобы этот раздел данных отличался от раздела кода функции.

Остальные функции проще, но используют ту же идею. В листинге 9.2 показана оверлейная функция `fie()`.

Листинг 9.2. Код оверлейной функции `fie()`

```
0028: int fie(int arg) __attribute__((noinline,section(".ov_fie")));
...
0131: int
0132: fie(int arg) {
0133:
0134:   std_printf("fie(0x%04X)\n",arg);
0135:   return arg + 0x0010;
0136: }
```

Опять же, имя оверлея указано в прототипе функции (строка 28). Объявление функции в строках 131–136 обычное. Обратите внимание: в отличие от `fee()`, строковая константа, используемая в строке 134, станет частью неоверлейного кода в области ROM.

Заглушки оверлея

Прежде чем код оверлея может быть выполнен, код из флеш-памяти SPI должен быть скопирован в оверлейную область в SRAM. По этой причине каждая из оверлейных функций использует функцию-заглушку (stub) и диспетчер оверлеев, как показано в листинге 9.3.

Листинг 9.3. Функция-заглушка для `fee()`

```
0164: static int
0165: fee_stub(int arg) {
0166:   int (*feep)(int arg) = module_lookup(&__load_start_fee);
0167:
0168:   return feep(arg);
0169: }
```

Наша функция `fee()` принимает аргумент `int` и возвращает значение `int`. Следовательно, функция-заглушка должна делать то же самое. Однако, прежде чем мы сможем вызвать оверлейную функцию, вызывается функция `module_lookup()`, чтобы проверить, находится ли она уже в области `ovl` (оверлея), и, если нет, скопировать ее туда сейчас. Наконец, нам нужно знать адрес функции, что вернет функция `module_lookup()`, чтобы мы могли ее вызвать.

Диспетчер оверлеев

Обычно требуется какой-либо диспетчер оверлеев, особенно когда используется несколько областей. Наша демонстрационная программа настраивает таблицу оверлеев, используя структуру `s_overlay`:

```
0036: typedef struct {
0037:   short regionx; // Overlay region index
0038:   void *vma; // Overlay's mapped address
0039:   char *start; // Load start address
0040:   char *stop; // Load stop address
0041:   unsigned long size; // Size in bytes
0042:   void *func; // Function pointer
0043: } s_overlay;
```

В этом демонстрационном примере используется только одна оверлейная область; следовательно, индекс regionx всегда имеет нулевое значение. Однако, если вы поддерживаете, например, три оверлейные области, этот индекс может иметь значение 0, 1 или 2. Он используется для отслеживания того, какой оверлей в данный момент находится в области, поэтому его код не нужно копировать.

Член под названием `vma` – это отображаемый адрес оверлея (его местоположение в SRAM при выполнении). Члены `start` и `stop` – это адреса внешней флеш-памяти (из области `xflash`), которые нам нужно загрузить. Размер `size` будет содержать рассчитанный размер оверлея в байтах, а последний член `func` будет содержать указатель функции в SRAM.

Вы сейчас все еще размышляете над значениями `start` и `stop`? Добавьте себе очков, если да. Вопрос в том, как демонстрационная программа находит расположение кода во внешней флеш-памяти для загрузки?

VMA и адреса загрузки

VMA (virtual memory address, адрес виртуальной памяти) и адрес загрузки для оверлеев различаются. Мы организовали запись кода и данных оверлея

в область памяти xflash. Эти адреса загрузки будут начинаться с нуля, поскольку именно с него начинаются адреса внешней флеш-памяти. VMA для этого кода будут рассчитаны для области оверлеев в SRAM.

Это следует подчеркнуть: мы не можем использовать VMA для нашей таблицы оверлеев, так как они отображаются в одну и ту же область SRAM. Некоторые указатели функций могут даже совпадать. Однако адреса загрузки (из внешней флеш-памяти) будут уникальными. Это позволяет нам использовать их в качестве символических обозначений в нашей таблице оверлеев.

В демонстрационной программе для удобства программирования используется макроопределение:

```
0048: #define OVERLAY(region,ov,sym) \
{ region, &ov, &__load_start_ ## sym, &__load_stop_ ## sym, 0, sym }
```

Поскольку мы используем только одну оверлейную область, параметр region всегда будет равен нулю. Но если вы решите добавить еще один, вы можете указать индекс в качестве первого параметра.

Параметр ov относится к начальному адресу оверлея. Параметр sym позволяет нам указать оверлейную функцию. Подробнее об этом поговорим после того, как проиллюстрируем таблицу демонстрационной программы:

```
0056: // Overlay table:
0057: static s_overlay overlays[N_OVLY] = {
0058:   OVERLAY(0,overlay1,fee),
0059:   OVERLAY(0,overlay1,fie),
0060:   OVERLAY(0,overlay1,foo),
0061:   OVERLAY(0,overlay1,fum)
0062: };
```

В содержимом демонстрационной таблицы обозначение overlay1 упоминается как символьное имя, описывающее начальный адрес оверлея в SRAM. Сценарий загрузки определяет начало этой области как адрес 0x20004800 (для 2 Кбайт). Напомним, что имя было определено в скрипте загрузки следующим образом:

```
PROVIDE (overlay1 = .overlay1_start);
```

Присмотревшись к записи таблицы

```
0058: OVERLAY(0,overlay1,fee),
```

мы видим, что аргумент 3 предоставляется как fee. Итого макроопределение разворачивается в следующую строку:

```
{ 0, &overlay1, &__load_start_fee, &__load_stop_fee, 0, fee }
```

Откуда берутся имена `__load_start_fee` и `__load_stop_fee`? Они автоматически генерируются компоновщиком при обработке раздела `.fee`. Эти две строки можно найти в файле *main.map*, созданном компоновщиком:

```
0x0000000000 PROVIDE (__load_start_fee, LOADADDR (.fee))
0x0000000045 PROVIDE (__load_stop_fee, (LOADADDR (.fee) + SIZEOF (.fee)))
```

Из этого мы узнаем, что секция `.fee` загружается по нулевому адресу в области памяти `xflash` (внешняя флеш-память) и имеет длину 0x45 (69) байт.

Использование в коде имен, созданных компоновщиком

Одна вещь, которая сбивает с толку новых пользователей при использовании имен, созданных компоновщиком, таких как, например, `__load_start_fee`, заключается в том, что они пытаются использовать значения по этим адресам, а не сами адреса. Давайте проясним это на примере кода:

```
extern long __load_start_fee;
```

Как правильно использовать имя `__load_start_fee`? Оно является:

1) `__load_start_fee` (значением) или
2) `&__load_start_fee` (адресом)?

Я уже ответил выше. Решение 2 – правильный ответ, но почему?

Первое решение подразумевало бы, что компоновщик поместил 4 байта памяти по адресу `__load_start_fee`, содержащих значение этого имени. Но компоновщик определяет значение имени как адрес, поскольку память не выделяется.

Возвращаясь к таблице, которую использует диспетчер оверлеев, мы видим, что члены структуры первой записи заполняются следующим образом:

```
0036: typedef struct {
0037:   short regionx; // 0 (overlay index)
0038:   void *vma; // &overlay1
0039:   char *start; // &__load_start_fee
0040:   char *stop; // &__load_stop_fee
0041:   unsigned long size; // 0 (initially)
0042:   void *func; // A pointer inside SRAM
0043: } s_overlay
```

Затем эта запись определяет адрес оверлейной области SRAM в члене структуры `vma`, используя адрес `&overlay1`, предоставленный компоновщиком. Аналогично участники запуска и остановки также используют адреса, предоставленные компоновщиком. Элемент размера будет рассчитываться один раз во время выполнения. Наконец, член `func` получает значение `fee`. Что-что с этим происходит?

Поскольку компилятор знает, что `fee` является именем точки входа в функцию `fee()`, простая ссылка на него служит адресом. Этот трюк компоновщика может сбить с толку.

Функция диспетчера оверлеев

Давайте, наконец, представим функцию `overlay` (листинг 9.4). Значением, передаваемым в качестве модуля аргумента, является адрес загрузки оверлея, например `&__load_start_fee`. Это адрес, по которому компоновщик поместил код оверлея, поступающего из внешней флеш-памяти.

Листинг 9.4. Функция диспетчера оверлеев

```
0071: static void *
0072: module_lookup(void *module) {
0073:   unsigned regionx; // Overlay region index
0074:   s_overlay *ovl = 0; // Table struct ptr
0075:
0076:   std_printf("module_lookup(%p):\n",module);
0077:
0078:   for ( unsigned ux=0; ux<N_OVLY; ++ux ) {
0079:       if ( overlays[ux].start == module ) {
0080:           regionx = overlays[ux].regionx;
0081:           ovl = &overlays[ux];
0082:           break;
0083:       }
0084:   }
0085:
0086:   if ( !ovl )
0087:       return 0; // Not found
0088:
0089:   if ( !cur_overlay[regionx] || cur_overlay[regionx] != ovl ) {
0090:        if ( ovl->size == 0 )
0091:            ovl->size = (char *)ovl->stop - (char *)ovl->start;
0092:        cur_overlay[regionx] = ovl;
0093:
0094:   std_printf("Reading %u from SPI at 0x%04X into 0x%04X\n",
0095:       (unsigned)ovl->size,
0096:       (unsigned)ovl->start,
0097:       (unsigned)ovl->vma);
0098:
0099:   w25_read_data(SPI1,(unsigned)ovl->start,ovl->vma,ovl->size);
0100:
0101:   std_printf("Returned...\n");
0102:   std_printf("Read %u bytes: %02X %02X %02X...\n",
0103:      (unsigned)ovl->size,
0104:      ((uint8_t*)ovl->vma)[0],
0105:      ((uint8_t*)ovl->vma)[1],
0106:      ((uint8_t*)ovl->vma)[2]);
0107:   }
0108:   return ovl->func;
0109: }
```

Строки 78–84 выполняют линейный поиск по таблице в поисках совпадения по адресу модуля (совпадение происходит в строке 79). Если совпадение найдено, индекс записи сохраняется в `regionx` (строка 80). Затем адрес

записи таблицы оверлеев фиксируется в строке 81 в `ovl` перед выходом из цикла.

Если цикл был завершен без совпадения, в строке 87 возвращается 0 (ноль). Это фатально, если используется в качестве вызова функции, и указывает на ошибку в приложении.

В строке 89 проверяется, допустим ли оверлейный код и загружен он уже или нет. Если оверлей необходимо прочитать, выполняются строки 90–107, чтобы подготовить его к использованию. Если размер оверлея еще неизвестен, он рассчитывается и сохраняется в таблице в строках 90–91. В строке 92 отслеживается, какой оверлей загружен в данный момент. Строка 99 выполняет чтение по SPI с флеш-устройства по адресу флеш-памяти устройства `ovl->start` в оверлейную SRAM-память по адресу `ovl->vma` для количества `ovl->size` байт.

При загрузке кода оверлея указатель функции возвращается в строке 108.

Функция-заглушка

Чтобы упростить использование оверлеев, в качестве заменителя обычно используется функция-заглушка, чтобы ее можно было вызывать как обычную функцию. В листинге 9.5 показана функция-заглушка для оверлейной функции `fee()`.

Листинг 9.5. Функция-заглушка `fee()`

```
0164: static int
0165: fee_stub(int arg) {
0166:   int (*feep)(int arg) = module_lookup(&__load_start_fee);
0167:
0168:   return feep(arg);
0169: }
```

Функция-заглушка просто вызывает диспетчер оверлеев с правильным именем (в данном случае `&__load_start_fee`). Как только указатель функции будет записан в `feep`, можно безопасно вызывать функцию, поскольку диспетчер оверлеев может загружать код при необходимости. Указатель функции `feep` позволяет вызывать функцию с правильными аргументами и возвращать нужное значение оверлея.

Демопрограмма

Демонстрационная программа *main.c* (листинг 9.6) выполняет некоторую инициализацию для SPI и USB. Затем запускается задача `task1` для выполнения ввода-вывода через USB-терминал.

Листинг 9.6. Инициализация

```
0247: int
0248: main(void) {
```

```
0249:
0250:   rcc_clock_setup_pll(&rcc_hse_configs[RCC_CLOCK_HSE8_72MHZ]);
0251:   rcc_periph_clock_enable(RCC_GPIOC);
0252:   gpio_set_mode(GPIOC,GPIO_MODE_OUTPUT_2_MHZ,
            GPIO_CNF_OUTPUT_PUSHPULL,GPIO13);
0253:
0254:   usb_start(1);
0255:   std_set_device(mcu_usb); // Use USB for std I/O
0256:
0257:   w25_spi_setup(SPI1,true,true,true,
            SPI_CR1_BAUDRATE_FPCLK_DIV_256);
0258:
0259:   xTaskCreate(task1,"task1",100,NULL,configMAX_PRIORITIES-1,NULL);
0260:   vTaskStartScheduler();
0261:   for (;;);
0262:   return 0;
0263: }
```

Чтобы перестроить этот проект с нуля, выполните:

```
$ make clobber
$ make
```

Но пока не прошивайте STM32.

Извлечение оверлеев

Прежде чем вы сможете использовать оверлеи, необходимо загрузить их код на флеш-устройство W25Q32. Помните, мы поместили код оверлея в область памяти компоновщика `xflash`? Теперь нам нужно получить это из вывода компоновщика и загрузить в SPI-устройство.

Возможно, вы заметили, что команда `make` выполнила в этом проекте несколько дополнительных шагов:

```
arm-none-eabi-gcc --static -nostartfiles -Tstm32f103c8t6.ld ... -o main.elf
for v in fee fie foo fum ; do \
arm-none-eabi-objcopy -O ihex -j.$v main.elf $v.ov ; \
cat $v.ov | sed '/^:04000005/d;/^:00000001/d' >>all.hex ; \
done
arm-none-eabi-objcopy -Obinary -R.fee -R.fie -R.foo -R.fum main.elf main.bin
```

После обычного шага линковки (`arm-none-eabi-gcc`) вы увидите некоторые дополнительные команды оболочки, выполняемые как часть цикла `for`. Для каждой из секций наложения (`fee`, `fie`, `foo` и `fum`) выдается пара команд следующего вида:

```
arm-none-eabi-objcopy -O ihex -j.$v main.elf $v.ov
cat $v.ov | sed '/^:04000005/d;/^:00000001/d' >>all.hex
```

Первая команда извлекает указанный раздел из выходных данных в формате Intel hex (`-O ihex`). Если переменная `v` есть имя «fee», раздел .fee (`-j.fee`) извлекается в файл с именем *fee.ov*. Следующая команда `sed` просто удаляет из hex-файла записи типа 05 и 01[54], которые нам не нужны, и объединяет их все в файл *all.hex*.

Последний шаг требует, чтобы мы удалили разделы оверлеев из *main.elf*, чтобы окончательный образ загрузки не включал оверлеи. Если бы мы оставили их в загрузочном образе, то `st-flash` попыталась бы загрузить их в STM32 и потерпела бы неудачу.

```
arm-none-eabi-objcopy -Obinary -R.fee -R.fie -R.foo -R.fum main.elf main.bin
```

Эта команда записывает файл образа *main.bin* (опция `-Obinary`) и удаляет разделы `.fee`, `.fie`, `.foo` и `.fum` с помощью опции `-R`. *Main.bin* – это файл образа, который команда `st-flash` будет использовать для загрузки.



Чтобы облегчить доступ из *minicom*, вы можете скопировать файл *all.hex* в свой домашний каталог или в */tmp*.

Чтобы загрузить код оверлея на флеш-память Winbond, используйте для этого проект winbond из соответствующего каталога:

```
cd ~/stm32f103c8t6/rtos/winbond
```

Пересоберите этот проект и прошейте его на свой STM32:

```
$ make clobber
$ make
$ make flash
```

Однако перед запуском *minicom* убедитесь, что в вашей системе установлена следующая команда:

```
$ type ascii-xfr
ascii-xfr is /usr/local/bin/ascii-xfr
```

Обычно она устанавливается вместе с *minicom* и может оказаться в другом каталоге вашей системы. Если команда не найдена, вам нужно это исправить (возможно, переустановить minicom).

Затем отключите программатор и подключите USB-кабель. Запускаем *minicom*:

```
$ minicom usb
```

[54] О типах записи в файлах формата Intel hex см. https://radiohlam.ru/hexformat.

При работающем minicom проверьте настройки загрузки. Нажмите <**Esc+O**> (или используйте <**Ctrl+A**> <**O**>, если необходимо), чтобы открыть меню, затем выберите **File Transfer Protocols** (Протоколы передачи файлов). Если меню не появилось, попробуйте еще раз. Между нажатием клавиши <**Esc**> и буквы O (o) не может быть большой задержки.

Найдите имя протокола «ascii», которое обычно находится в конце списка. Введите букву для записи (в моей системе буква I) и нажмите <**Enter**>, чтобы войти в область ввода **Program** (Программа). Измените эту запись, чтобы она выглядела следующим образом:

```
/usr/local/bin/ascii-xfr -n -e -s -l75
```

Самая важная опция – это `-l75` (буква «l» в нижнем регистре), которая вызывает задержку в 75 мс после отправки каждой текстовой строки. Без разумной задержки загрузка не удастся. Вероятно, вам также следует установить другие параметры, как показано в этом примере.

Остальные флаги опций, как известно, работают следующим образом:

```
Name U/D FullScr IO-Red. Multi
Y    U    N      Y       N
```

Нажмите <**Enter**> для перемещения по списку настроек ввода. Нажмите <**Enter**> еще раз, чтобы вернуться в главное меню, затем выберите **Save setup as USB** (Сохранить настройки как USB). Теперь вы сможете использовать сеанс *minicom* для загрузки файла *all.hex*.

Выйдя из меню или в изначальный *minicom*, нажмите <**Enter**>, чтобы программа отобразила меню:

```
Winbond Flash Menu:
  0 ... Power down
  1 ... Power on
  a ... Set address
  d ... Dump page
  e ... Erase (Sector/Block/64K/Chip)
  i ... Manufacture/Device info
  h ... Ready to load Intel hex
  j ... JEDEC ID info
  r ... Read byte
  p ... Program byte(s)
  s ... Flash status
  u ... Read unique ID
  w ... Write Enable
  x ... Write protect
Address: 000000
:
```

Убедитесь, что ваша флеш-память SPI отвечает, и при необходимости очистите ее:

```
: W
SR1 = 02 (write enabled)
SR2 = 00
: E
Erase what?
  s ... Erase 4K sector
  b ... Erase 32K block
  z ... Erase 64K block
  c ... Erase entire chip
anything else to cancel
: s
sector erased, starting at 000000
Sector erased.
:
```

Здесь наш адрес по-прежнему равен нулю, но если нет, установите его равным нулю сейчас:

```
: A
Address: 0
Address: 000000
:
```

Разрешите запись снова (стирание отключает разрешение), а затем подготовьтесь к загрузке hex-файла:

```
: W
SR1 = 02 (write enabled)
SR2 = 00
: H
Ready for Intel Hex upload:
00000000 _
```

Теперь нажмите <**Esc+S**> (или <**Ctrl+A**> <**S**>), чтобы открыть меню **Upload** (Загрузить), и выберите «ascii»:

```
+-[Upload]--+
| zmodem |
| ymodem |
| xmodem |
| kermit |
| ascii |<-- выбор
+-----------+
```

Появится другое меню, в котором вы сможете выбрать файл для загрузки. Я рекомендую просто нажать <**Enter**> и ввести имя файла (*all.hex*). Я копирую свой файл в домашний каталог, так что мне нужно только ввести «all.hex»:

```
+--------------------------------------+
|No file selected - enter filename: |
|> all.hex |
+--------------------------------------+
```

При нажатии клавиши <**Enter**> появляется окно загрузки, в котором код Intel hex из файла *all.hex* отправляется на плату STM32.

Чтобы проверить, что он туда попал, можно получить дамп страницы следующим образом:

```
: D
000000 10 B5 04 46 01 46 02 48 00 F0 06 F8 60 1C 10 BD ...F.F.H....`...
000010 20 48 00 20 00 00 00 00 5F F8 00 F0 FD 18 00 08 H. ...._.......
000020 2A 2A 2A 2A 2A 2A 2A 2A 2A 2A 2A 0A 66 65 65 28 ***********.fee(
000030 30 78 25 30 34 58 29 0A 2A 2A 2A 2A 2A 2A 2A 2A 0x%04X).********
000040 2A 2A 2A 0A 00 10 B5 04 46 01 46 03 48 00 F0 06 ***.....F.F.H...
000050 F8 04 F1 10 00 10 BD 00 BF 30 31 00 08 5F F8 00 .........01.._..
000060 F0 FD 18 00 08 10 B5 04 46 01 46 03 48 00 F0 06 ........F.F.H...
000070 F8 04 F5 00 70 10 BD 00 BF 3D 31 00 08 5F F8 00 ....p....=1.._..
000080 F0 FD 18 00 08 10 B5 04 46 01 46 03 48 00 F0 06 ........F.F.H...
000090 F8 04 F5 40 50 10 BD 00 BF 4A 31 00 08 5F F8 00 ...@P....J1.._..
0000A0 F0 FD 18 00 08 FF FF FF FF FF FF FF FF FF FF FF ................
0000B0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
0000C0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
0000D0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
0000E0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
0000F0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF ................
```

Вы должны увидеть текст программы `fee()` в ASCII-части дампа справа. Теперь вы закончили загрузку флеш-памяти!



Всегда выходите из *minicom* (<**Esc-X**>) перед отсоединением USB-кабеля. В противном случае драйвер USB может зависнуть или отключиться.

Демопрограмма (продолжение)

Теперь выйдите из *minicom* и отсоедините USB-кабель, затем вернитесь в каталог проекта *overlay1*:

```
$ cd ~/stm32f103c8t6/rtos/overlay1
```

Прошейте в STM32 код примера оверлея (*main.bin*):

```
$ make flash
```

По завершении отключите программатор и подключите USB-кабель. Запустите *minicom*:

```
SPI SR1 = 00
Enter R when ready:
_
```

На этом этапе демонстрационная программа ожидает вашего разрешения, чтобы попытаться выполнить оверлеи. Нажмите <**R**>, чтобы попробовать:

```
OVERLAY TABLE:
[0] { regionx=0, vma=0x20004800, start=0x0, stop=0x45, \
size=0, func=0x20004801 }
[1] { regionx=0, vma=0x20004800, start=0x45, stop=0x65, \
size=0, func=0x20004801 }
[2] { regionx=0, vma=0x20004800, start=0x65, stop=0x85, \
size=0, func=0x20004801 }
[3] { regionx=0, vma=0x20004800, start=0x85, stop=0xa5, \
size=0, func=0x20004801 }
fang(0x0001)
module_lookup(0x0):
Reading 69 from SPI at 0x0000 into 0x20004800
Returned...
Read 69 bytes: 10 B5 04...
**********
fee(0x0001)
**********
module_lookup(0x45):
Reading 32 from SPI at 0x0045 into 0x20004800
Returned...
Read 32 bytes: 10 B5 04...
fie(0x0002)
module_lookup(0x65):
Reading 32 from SPI at 0x0065 into 0x20004800
Returned...
Read 32 bytes: 10 B5 04...
foo(0x0012)
module_lookup(0x85):
Reading 32 from SPI at 0x0085 into 0x20004800
Returned...
Read 32 bytes: 10 B5 04...
fum(0x0212)
calls(0xA) returned 0x3212
It worked!!
SPI SR1 = 00
Enter R when ready:
```

Если демопрограмма доходит до слов «It worked!!» (Сработало!!) и снова предложит вам ввести букву «R», то ваши оверлеи работают. Обратите внимание, что изначально в дампе таблицы оверлеев размеры равны нулю. Но если вы наберете <**R**> еще раз, вы увидите, что размер в байтах заполнен:

```
OVERLAY TABLE:
[0] { regionx=0, vma=0x20004800, start=0x0, stop=0x45, size=69, func=0x20004801 }
[1] { regionx=0, vma=0x20004800, start=0x45, stop=0x65, size=32, func=0x20004801 }
[2] { regionx=0, vma=0x20004800, start=0x65, stop=0x85, size=32, func=0x20004801 }
[3] { regionx=0, vma=0x20004800, start=0x85, stop=0xa5, size=32, func=0x20004801 }
```

Размер оверлея `.fee` самый большой, поскольку мы включили в код некоторые строковые текстовые данные.

В выводе сеанса следующее может сбить с толку:

```
module_lookup(0x0):
Reading 69 from SPI at 0x0000 into 0x20004800
```

Первый используемый адрес `&__load_start_fee` – это адрес внешней флеш-памяти 0x0 (не путать с нулевым указателем!). Но это просто первый байт, доступный в этой флеш-памяти. Вторая строка указывает, что из флеш-памяти по адресу 0x0000 было загружено 69 байт. Мы также видим сообщенный адрес оверлея 0x20004800, в который код был загружен для выполнения.

```
fie(0x0002)
module_lookup(0x65):
Reading 32 from SPI at 0x0065 into 0x20004800
```

Отсюда мы видим, что функция `fie()` вызывается со значением аргумента 2. Она расположена по адресу 0x65 во внешней флеш-памяти и загружается в ту же оверлейную область по адресу 0x20004800.

Ловушка при изменении кода

Программисты всегда ищут короткие пути, поэтому я хочу предупредить вас об одной ловушке, в которую легко попасть. Во время разработки этого проекта я предположил, что мне не нужно повторно загружать файл оверлея *all.hex* во внешнюю флеш-память, поскольку эти процедуры не изменились. Однако расположение вызываемой ими процедуры `std_printf()` вне оверлейного кода меняется в обычном режиме.

Подпрограммы, которые вызывают ваши оверлеи, могут перемещаться по мере изменения и перекомпиляции кода. Когда это произойдет, функции оверлея аварийно завершат работу при вызове с устаревшими адресами функций. Всегда обновляйте код оверлея, даже если изменяется код, им не являющийся.

Итоги

Это была техническая глава, и она поэтому была длинной. Однако преимущество для вас состоит в том, что вы держите в руках полный рецепт реализации собственных оверлеев. Вы больше не ограничены пределом флеш-памяти STM32f103C8T6 в 128 Кбайт. Расправьте крылья и летите!

Дополнительные источники

1. «Overlay (programming)», «Википедия». https://en.wikipedia.org/wiki/Overlay_(programming) (дата доступа 01.09.2024); то же по-русски: https://ru.wikipedia.org/wiki/Overlay_(программирование) (дата доступа 01.09.2024).
2. Steve Chamberlain, «Command Language», главы «Using LD», «GNU Linker», 1d ed. www.math.utah.edu/docs/info/ld_3.html#SEC13 (дата доступа 01.09.2024).

Упражнения

1. Почему в структуре, тип которой определен как `s_overlay`, члены определяются как указатели на имена, а не как длинные целые числа?
2. Почему в сценарий компоновщика была добавлена область памяти `xflash`?
3. Какова цель функции оверлея-заглушки?
4. Какова цель `noinline` в декларации GNU `__attribute__((noinline,section(".ov_fee")))`? Зачем это нужно?
5. Где находится объявление `__attribute((section("..."))`?

Глава 10 • Часы реального времени (RTC)

В приложениях часто важно отслеживание времени. По этой причине платформа STM32 предоставляет встроенные часы реального времени (RTC). Документация для RTC при изучении кажется почти тривиальной, но неосторожных ждут свои трудности.

В этой главе будет рассмотрена настройка процедуры обработки прерываний (ISR) для прерываний, происходящих раз в секунду, а также дополнительно функция тревоги (alarm). После освоения этой информации у вас не останется причин жаловаться на нехватку данных о времени.

Демонстрационные проекты

Демопрограммы для этой главы взяты из следующих двух каталогов:

```
$ cd ~/stm32f103c8t6/rtos/rtc
$ cd ~/stm32f103c8t6/rtos/rtc2
```

Первоначально основное внимание будет уделено первому проекту, в котором реализован один обработчик прерывания (ISR). Во втором примере, который будет рассмотрен ближе к концу главы, будет использоваться другое прерывание ISR.

RTC с одним прерыванием

Платформа STM32F1 обеспечивает до двух прерываний для RTC. Имена точек входа при использовании библиотеки libopencm3 следующие[55]:

```
#include <libopencm3/cm3/nvic.h>
void rtc_isr(void);
void rtc_alarm_isr(void);
```

В нашей первой демонстрации будет использоваться только процедура `rtc_isr()`, поскольку ее проще всего настроить и использовать. Обслуживаемые прерыванием события следующие:

1) односекундное событие (тик таймера один раз в секунду),
2) событие тревоги (alarm),
3) событие переполнения таймера.

[55] Название указанного далее модуля *nvic.h* и соответствующие обращения в коде этого раздела относятся к NVIC, что означает Nested Vectored Interrupt Controller, контроллер встроенных векторных прерываний.

Для RTC можно определить один сигнал тревоги, при котором прерывание будет генерироваться по наступлении заданного времени тревожного сигнала когда-то в будущем. Его можно использовать в качестве грубого сторожевого таймера (watchdog).

Несмотря на то что размер счетчика таймера RTC составляет 32 бита, по прошествии достаточного времени счетчик в конечном итоге переполнится. Иногда на этом этапе необходимо выполнить специальный учет времени. Ожидаемый сигнал тревоги также может нуждаться в настройке.

В нашей первой демонстрации все эти необязательные события будут передаваться через процедуру `rtc_isr()`. Это самый простой подход.

Конфигурация RTC

В следующих разделах будет описана настройка часов RTC с использованием функций библиотеки libopencm3.

Синхронизация RTC

RTC в STM32F103C8T6 может использовать один из трех возможных источников синхронизации, а именно:

1) тактовый генератор LSE (кварцевый генератор 32.768 кГц), который продолжает работать даже при отключении напряжения питания при условии, что сохраняется напряжение батареи VBAT. Предоставляемая частота RTCCLK составляет 32.768 кГц. Выбор установки – `RCC_LSE`;
2) тактовый генератор LSI (RC-генератор ~40 кГц), работает только при подаче основного питания. Предоставляемая частота RTCCLK составляет примерно 40 кГц. Выбор установки – `RCC_LSI`;
3) тактовый генератор HSE (кварцевый генератор 8 МГц), только при подаче основного питания. Частота RTCCLK рассчитывается по формуле:

$$\frac{8\ \text{МГц}}{128} = 62.5\ \text{кГц}.$$

Выбор установки – `RCC_HSE`.

Источником синхронизации, используемым в проекте этой главы, является тактовый генератор HSE, который выбирается следующим вызовом функции libopencm3:

```
rtc_awake_from_off(RCC_HSE);
```

Настройка предделителя

Поскольку мы выбрали тактовую частоту RCC_HSE в качестве источника для периферийного устройства RTC и используем 8 МГц / 128 = 62 500 Гц в качестве частоты RTCCLK, теперь мы можем определить тактовую частоту по следующей формуле:

$$f = \frac{62\ 500}{divisor}\ \text{Гц}.$$

В этой главе мы будем использовать делитель *divisor* = 62 500, чтобы частота составляла 1 Гц. Это обеспечивает время между тиками таймера в одну секунду. Однако вы можете использовать значение делителя, например 6250, для создания тиков с интервалом в десятую долю секунды, если это необходимо. Используя libopencm3, мы устанавливаем делитель следующим образом:

```
rtc_set_prescale_val(62500);
```

Начальное значение счетчика

Обычно счетчик RTC запускается с нуля, но не существует закона, обязывающего это делать. В демонстрационной программе мы собираемся инициализировать его числом примерно за 16 с до переполнения, чтобы мы могли продемонстрировать прерывание переполнения таймера.

Счетчик можно инициализировать следующим вызовом:

```
rtc_set_counter_val(0xFFFFFFF0);
```

Счетчик RTC имеет размер 32 бита, поэтому он может считать до 0xFFFFFFFF, после чего генерируется событие переполнения. Приведенный выше код инициализирует счетчик до числа 16 отсчетов перед переполнением.

Флаги RTC

Регистр управления RTC (`RTC_CRL`) содержит три интересующих нас флага (с использованием имен библиотеки libopencm3):

1) `RTC_SEC` (тики);
2) `RTC_ALR` (тревога);
3) `RTC_OW` (переполнение).

Эти флаги можно проверить и очистить соответственно, используя следующие вызовы libopencm3:

```
rtc_check_flag(flag);
rtc_clear_flag(flag);
```

Прерывание и настройка

В листинге 10.1 показаны шаги, необходимые для инициализации односекундных тиков RTC и получения прерываний.

Листинг 10.1. Настройка периферийного устройства RTC для прерываний

```
0166: static void
0167: rtc_setup(void) {
0168:
0169:   rcc_enable_rtc_clock();
0170:   rtc_interrupt_disable(RTC_SEC);
0171:   rtc_interrupt_disable(RTC_ALR);
0172:   rtc_interrupt_disable(RTC_OW);
```

```
0173:
0174:   // RCC_HSE, RCC_LSE, RCC_LSI
0175:   rtc_awake_from_off(RCC_HSE);
0176:   rtc_set_prescale_val(62500);
0177:   rtc_set_counter_val(0xFFFFFFF0);
0178:
0179:   nvic_enable_irq(NVIC_RTC_IRQ);
0180:
0181:   cm_disable_interrupts();
0182:   rtc_clear_flag(RTC_SEC);
0183:   rtc_clear_flag(RTC_ALR);
0184:   rtc_clear_flag(RTC_OW);
0185:   rtc_interrupt_enable(RTC_SEC);
0186:   rtc_interrupt_enable(RTC_ALR);
0187:   rtc_interrupt_enable(RTC_OW);
0188:   cm_enable_interrupts();
0189: }
```

Поскольку периферийное устройство RTC после сброса отключено, в строке 169 оно включается, чтобы его можно было инициализировать. Строки 170–172 отключают источники прерываний, чтобы предотвратить их появление во время настройки периферийного устройства.

В строке 175 выбирается источник тактовых сигналов RTC, а в строке 176 настраивается тактовая частота равной один раз в секунду. Строка 177 инициализирует счетчик RTC за 16 с до переполнения, чтобы продемонстрировать событие переполнения без длительного ожидания.

Строка 179 включает контроллер прерываний для обработчика прерываний `rtc_isr()`. Все прерывания временно подавляются в строке 181, чтобы обеспечить возможность окончательной настройки прерывания без генерации прерываний. Строки 182–184 очищают флаги RTC. Строки 185–187 позволяют генерировать прерывания, когда эти флаги установлены. И наконец, прерывания обычно снова разрешаются в строке 188.

На этом этапе периферийное устройство RTC готово генерировать прерывания.

Процедура обработки прерываний

В листинге 10.2 показан код, используемый для обработки прерываний RTC. Пусть вас не беспокоит длина программы, поскольку на самом деле там мало что происходит.

Листинг 10.2. Процедура обработки прерываний RTC

```
0057: void
0058: rtc_isr(void) {
0059:   UBaseType_t intstatus;
0060:   BaseType_t woken = pdFALSE;
0061:
0062:   ++rtc_isr_count;
```

```
0063:   if ( rtc_check_flag(RTC_OW) ) {
0064:       // Timer overflowed:
0065:       ++rtc_overflow_count;
0066:       rtc_clear_flag(RTC_OW);
0067:       if ( !alarm ) // If no alarm pending, clear ALRF
0068:           rtc_clear_flag(RTC_ALR);
0069:   }
0070:
0071:   if ( rtc_check_flag(RTC_SEC) ) {
0072:       // RTC tick interrupt:
0073:       rtc_clear_flag(RTC_SEC);
0074:
0075:       // Increment time:
0076:       intstatus = taskENTER_CRITICAL_FROM_ISR();
0077:       if ( ++seconds >= 60 ) {
0078:           ++minutes;
0079:           seconds -= 60;
0080:       }
0081:       if ( minutes >= 60 ) {
0082:           ++hours;
0083:           minutes -= 60;
0084:       }
0085:       if ( hours >= 24 ) {
0086:           ++days;
0087:           hours -= 24;
0088:       }
0089:       taskEXIT_CRITICAL_FROM_ISR(intstatus);
0090:
0091:       // Wake task2 if we can:
0092:       vTaskNotifyGiveFromISR(h_task2,&woken);
0093:       portYIELD_FROM_ISR(woken);
0094:       return;
0095:   }
0096:
0097:   if ( rtc_check_flag(RTC_ALR) ) {
0098:       // Alarm interrupt:
0099:       ++rtc_alarm_count;
0100:       rtc_clear_flag(RTC_ALR);
0101:
0102:       // Wake task3 if we can:
0103:       vTaskNotifyGiveFromISR(h_task3,&woken);
0104:       portYIELD_FROM_ISR(woken);
0105:       return;
0106:   }
0107: }
```

Счетчик прерываний с именем `rtc_isr_count` увеличивается в строке 62. Он используется только для вывода факта вызова ISR в нашем демонстрационном коде.

В строках 63–69 проверяется, не переполнился ли счетчик таймера, и если да, то очищается флаг переполнения (`RTC_OW`). Если ожидающих сигналов

тревоги нет, флаг тревоги также очищается. Похоже, что флаг тревоги всегда устанавливается, когда происходит переполнение таймера, независимо от того, был ли ожидающий сигнал тревоги или нет. Насколько я могу судить, это одна из недокументированных функций.

Обычно процедура `rtc_isr()` запускается по тику таймера. Строки 71–95 обслуживают односекундный тик. После очистки флага прерывания (`RTC_SEC`) в строке 76 начинается критическая секция. Это способ FreeRTOS отключить прерывания ISR до тех пор, пока программа не закончит критическую секцию (строка 89 ее завершает). Здесь очень важно, чтобы время плавно увеличивалось от секунд до минут, часов и дней, не прерываясь посередине. В противном случае может произойти прерывание с более высоким приоритетом и обнаружить, что текущее время – 12:05:60 или 13:60:45.

Строка 92 – это проверка уведомлений для второй задачи (будет рассмотрена в ближайшее время). Уведомление (если значение `woken` истинно) появляется в строке 93. Идея состоит в том, что выполнение задачи 2 будет заблокировано до тех пор, пока не поступит это уведомление. Оставайтесь с нами, чтобы узнать больше об этом.

Наконец, строки 97–106 обрабатывают возникновение прерывания RTC при установленном флаге тревоги. Помимо увеличения счетчика `rtc_alarm_count` для отображения в демопрограмме, процедура очищает флаг тревоги `RTC_ALR` в строке 100. Строки 103 и 104 предназначены для уведомления задачи 3, которая также будет рассмотрена далее.

Обслуживание прерываний

Внимательный читатель, вероятно, заметил, что процедура ISR не всегда обрабатывает все три источника прерываний за один вызов. Что произойдет, если `RTC_SEC`, `RTC_OW` и `RTC_ALR` при вызове процедуры `rtc_isr()` установлены все, но очищается, например, только `RTC_SEC`?

Любой из этих флагов может вызвать возникновение прерывания. Поскольку мы включили все три источника прерываний, процедура `rtc_isr()` будет продолжать вызываться до тех пор, пока не будут очищены все флаги.

Уведомление о задаче

FreeRTOS поддерживает эффективный механизм, позволяющий задаче блокировать собственное выполнение до тех пор, пока другая задача или прерывание не уведомит ее о необходимости выполнения. В листинге 10.3 показан код, используемый для задачи 2.

Листинг 10.3. Задача 2. Использование уведомления о задаче

```
0144: static void
0145: task2(void *args __attribute__((unused))) {
0146:
0147:   for (;;) {
0148:       // Block execution until notified
0149:       ulTaskNotifyTake(pdTRUE,portMAX_DELAY);
0150:
```

```
0151:       // Toggle LED
0152:       gpio_toggle(GPIOC,GPIO13);
0153:
0154:       mutex_lock();
0155:       std_printf("Time: %3u days %02u:%02u:%02u isr_count: %u, " alarms: %u,
overflows: %u\n",
0156:         days,hours,minutes,seconds,
0157:         rtc_isr_count,rtc_alarm_count,
              rtc_overflow_count);
0158:       mutex_unlock();
0159:   }
0160: }
```

Как и многие другие задачи, она начинается с бесконечного цикла в строке 147. Однако первое, что выполняется в этом цикле, – это вызов `ulTaskNotifyTake()` с тайм-аутом «ждать вечно» (аргумент 2 `portMAX_DELAY`). Задача 2 будет остановлена на этом этапе до тех пор, пока о ней не поступит уведомление. Единственное место, где уведомление возникает, – в строках 92 и 93 обработчика прерывания `rtc_isr()`. Как только происходит прерывание, вызов функции ожидания в строке 149 возвращает управление, и выполнение продолжается. Это позволяет функции `printf` в строках 155–157 сообщать время.

При запуске демонстрации вы увидите, что это уведомление появляется раз в секунду при вызове процедуры `rtc_isr()`. Это очень удобная синхронизация между ISR и не-ISR. Если вы заметили вызовы `mutex_lock()`/`mutex_unlock()`, просто держите их в голове до следующего раздела.

Задача 3 использует аналогичный механизм для сигналов тревоги, показанный в листинге 10.4.

Листинг 10.4. Задача 3 и уведомление о тревоге

```
0126: static void
0127: task3(void *args __attribute__((unused))) {
0128:
0129:   for (;;) {
0130:       // Block execution until notified
0131:       ulTaskNotifyTake(pdTRUE,portMAX_DELAY);
0132:
0133:       mutex_lock();
0134:       std_printf("*** ALARM *** at %3u days %02u:%02u:%02u\n",
0135:              days,hours,minutes,seconds);
0136:       mutex_unlock();
0137:   }
0138: }
```

Строка 131 блокирует выполнение задачи 3 до тех пор, пока не поступит уведомление от процедуры `rtc_isr()` (строки 103 и 104 листинга 10.2). Таким образом, задача 3 остается заблокированной до тех пор, пока ISR не обнаружит сигнал тревоги.

Мьютексы

Мьютексы (от mutual exclusion, *взаимное исключение*) необходимы для блокировки конкуренции нескольких задач за общий ресурс. В этой демонстрации нам нужно иметь возможность вывести полную строку текста, прежде чем позволить другой задаче сделать то же самое. Использование подпрограмм `mutex_lock()` и `mutex_unlock()` предотвращает вывод конкурирующих задач в середину нашей собственной строки текста. Эти процедуры используют API FreeRTOS и показаны в листинге 10.5.

Листинг 10.5. Функции мьютекса

```
0039: static void
0040: mutex_lock(void) {
0041:   xSemaphoreTake(h_mutex,portMAX_DELAY);
0042: }
0048: static void
0049: mutex_unlock(void) {
0050:   xSemaphoreGive(h_mutex);
0051: }
```

Дескриптор мьютекса создается в основной программе следующим вызовом:

```
0259: h_mutex = xSemaphoreCreateMutex();
```

Если вы новичок в использовании мьютексов, скажем, что при блокировке (захвате) мьютекса происходит следующее.

1. Если мьютекс уже заблокирован, вызывающая задача блокируется (ждет), пока он не освободится.
2. Когда мьютекс освободится, будет предпринята попытка его заблокировать. Если другая конкурирующая задача первой захватила блокировку, вернитесь к шагу 1, чтобы заблокировать, пока мьютекс снова не станет свободным.
3. В противном случае блокировка теперь принадлежит вызывающему объекту, пока мьютекс не будет разблокирован.

Как только задача, использующая мьютекс, будет выполнена, она может освободить мьютекс. Разблокировка мьютекса может позволить продолжить выполнение другой заблокированной задачи. В противном случае, если мьютекс не ожидает никакой другой задачи, он просто разблокируется.

Демопрограмма

Основная демонстрационная программа, представленная в этой главе, может быть запущена с использованием преобразователя уровней TTL-UART на базе адаптера USB-TTL или напрямую через USB-кабель. Выбор UART, пожалуй, лучший, поскольку соединение USB иногда приводит к задержкам,

маскирующим синхронизацию операторов печати. Тем не менее оба работают одинаково хорошо, когда дела идут, и USB-кабель обычно более удобен.

Следующий оператор определяет, какой подход вы хотите использовать (в файле с именем *main.c*):

```
0020: #define USE_USB 0 // Set to 1 for USB
```

По умолчанию используется UART. Если вы установите значение 1, вместо него будет использоваться USB-устройство. Настройка UART или USB происходит в функции `main()` файла *main.c* (листинг 10.6).

Листинг 10.6. Основная программа для инициализации UART или USB

```
0251: int
0252: main(void) {
0253:
0254:   rcc_clock_setup_pll(&rcc_hse_configs[RCC_CLOCK_HSE8_72MHZ]);
0255:
0256:   rcc_periph_clock_enable(RCC_GPIOC);
0257:   gpio_set_mode(GPIOC,GPIO_MODE_OUTPUT_50_MHZ,
            GPIO_CNF_OUTPUT_PUSHPULL,GPIO13);
0258:
0259:   h_mutex = xSemaphoreCreateMutex();
0260:   xTaskCreate(task1,"task1",350,NULL,1,NULL);
0261:   xTaskCreate(task2,"task2",400,NULL,3,&h_task2);
0262:   xTaskCreate(task3,"task3",400,NULL,3,&h_task3);
0263:
0264:   gpio_clear(GPIOC,GPIO13);
0265:
0266: #if USE_USB
0267:   usb_start(1,1);
0268:   std_set_device(mcu_usb); // Use USB for std I/O
0269: #else
0270:   rcc_periph_clock_enable(RCC_GPIOA); // TX=A9,RX=A10,
            CTS=A11,RTS=A12
0271:   rcc_periph_clock_enable(RCC_USART1);
0272:
0273:   gpio_set_mode(GPIOA,GPIO_MODE_OUTPUT_50_MHZ,
0274:       GPIO_CNF_OUTPUT_ALTFN_PUSHPULL,GPIO9|GPIO11);
0275:   gpio_set_mode(GPIOA,GPIO_MODE_INPUT,
0276:       GPIO_CNF_INPUT_FLOAT,GPIO10|GPIO12);
0277:   open_uart(1,115200,"8N1","rw",1,1); // RTS/CTS flow control
0278:  // open_uart(1,9600,"8N1","rw",0,0); // UART1 9600 with no f.control
0279:   std_set_device(mcu_uart1); // Use UART1 for std I/O
0280: #endif
0281:
0282:   vTaskStartScheduler();
0283:   for (;;);
0284:
0285:   return 0;
0286: }
```

Управление устройством инициализируется в строках 267 и 268 для USB и в строках 270–279 для UART. Для UART в строке 277 по умолчанию используется скорость передачи данных 115 200 бод. Это хорошо работает, если вы используете аппаратное управление потоком данных. Если по какой-то причине ваш адаптер USB-TTL не поддерживает аппаратное управление потоком данных, закомментируйте строку 277 и раскомментируйте строку 278. При более низкой скорости передачи данных (9600 бод) вы сможете безопасно работать без управления потоком[56].

Строка 268 или 279 определяет, будут ли `std_printf()` и др. направляться на устройство USB или на устройство UART.

Как мы отмечали ранее, мы использовали мьютекс, чтобы гарантировать, что текстовые строки будут выведены на консоль полностью (через USB или UART). Этот мьютекс FreeRTOS создается в строке 259.

Задача 1 (строка 260) будет нашей основной «консолью», позволяющей нам вводить символы, влияющие на работу демоверсии. Задача 2 (строка 261) будет переключать встроенный светодиод (вывод PC13) для каждого тика RTC, а также выводить текущее время с момента запуска (см. листинг 10.3). Наконец, задача 3 выведет уведомление о тревоге, когда тревога сработает (см. листинг 10.4).

Листинг 10.7 иллюстрирует задачу консоли (задача 1).

Листинг 10.7. «Консольная задача», задача 1

```
0220: static void
0221: task1(void *args __attribute__((unused))) {
0222:   char ch;
0223:
0224:   wait_terminal();
0225:   std_printf("Started!\n\n");
0226:
0227:   rtc_setup(); // Start RTC interrupts
0228:   taskYIELD();
0229:
0230:   for (;;) {
0231:     mutex_lock();
0232:     std_printf("\nPress 'A' to set 10 second alarm,\n"
0233:         "else any key to read time.\n\n");
0234:     mutex_unlock();
0235:
0236:     ch = std_getc();
0237:
0238:     if ( ch == 'a' || ch == 'A' ) {
0239:         mutex_lock();
0240:         std_printf("\nAlarm configured for 10 seconds"
                  " from now.\n");
0241:         mutex_unlock();
0242:         set_alarm(10u);
```

[56] См. по этому поводу сноску 44 на стр. 95.

```
0243:      }
0244:   }
0245: }
```

Задача 1 предназначена для синхронизации с пользователем, который подключается по USB или UART-каналу, используя *minicom*. Строка 224 вызывает функцию `wait_terminal()`, которая предлагает пользователю нажать любую клавишу, после чего эта функция завершает работу. Затем часы RTC инициализируются в строке 227, и выполняется первоначальный вызов `TaskYIELD()`. Это помогает получить все данные до входа в цикл задачи 1.

Основной цикл задачи 1 просто выдает сообщение с предложением нажатия символа «A» для установки десятисекундного сигнала тревоги. Любая другая клавиша просто приведет к повторению цикла консоли. Поскольку таймер RTC управляется прерываниями, прерывание `rtc_isr()` вызывается каждую секунду, а также при возникновении переполнения или аварийного сигнала. Это, в свою очередь, уведомляет задачу 2 или задачу 3, как обсуждалось ранее.

Соединения UART1

При использовании UART-адаптера (USB-TTL) соединения UART выполняются в соответствии с табл. 10.1. В этом случае мы можем запитать STM32 от устройства USB-TTL. Если вы питаете STM32 от другого источника, опустите соединение +5 В.

Таблица 10.1. Подключение адаптера UART (USB-TTL) к STM32F103C8T6

GPIO	UART1	USB-TTL	Описание
A9 (вых)	TX	RX	STM32 посылает адаптеру UART
A10 (вх)	RX	TX	STM32 принимает от адаптера UART
A11 (вых)	CTS	RTS	STM32 Clear To Send[57]
A12 (вх)	RTS	CTS	STM32 Request To Send
+5V		+5V	Питание STM32 от адаптера USB-TTL (необходимо отключить при питании от другого источника)
GND		GND	Общий провод



Был выбран UART1, поскольку он использует GPIO, устойчивый к напряжению 5 В.

С точки зрения соединений и работы эмулятора терминала USB-кабель является гораздо более простым вариантом.

Запуск демопрограммы

В зависимости от того, как вы настроили запуск программы *main.c*, вы будете использовать USB-кабель или UART-адаптер. USB – самый простой вариант: просто подключите и работайте. Если вы используете UART, вам необходимо настроить терминальную программу (*minicom*) так, чтобы она соответствова-

[57] К этой и следующей строкам таблицы см. примечание в сноске к табл. 6.5 на стр. 104.

ла параметрам связи: заданная скорость передачи, 8 бит данных, без контроля четности и один стоповый бит.

Перекомпилируйте и прошейте проект следующим образом:

```
$ make clobber
$ make
$ make flash
```

После запуска терминальной программы (я использую minicom) вы должны увидеть приглашение нажать любую клавишу:

```
Welcome to minicom 2.7
OPTIONS:
Compiled on Sep 17 2016, 05:53:15.
Port /dev/cu.usbserial-A703CYQ5, 19:13:38
Press Meta-Z for help on special keys
Press any key to start...
Press any key to start...
Press any key to start...
```

Нажмите любую клавишу, чтобы начать работу (я использовал клавишу <**Enter**>):

```
Press any key to start...
Started!
Press 'A' to set 10 second alarm,
else any key to read time.
Time: 0 days 00:00:01 isr_count: 1, alarms: 0, overflows: 0
Time: 0 days 00:00:02 isr_count: 2, alarms: 0, overflows: 0
Time: 0 days 00:00:03 isr_count: 3, alarms: 0, overflows: 0
...
Time: 0 days 00:00:20 isr_count: 20, alarms: 0, overflows: 0
Time: 0 days 00:00:21 isr_count: 21, alarms: 0, overflows: 0
Time: 0 days 00:00:22 isr_count: 22, alarms: 0, overflows: 0
Time: 0 days 00:00:23 isr_count: 23, alarms: 0, overflows: 0
Time: 0 days 00:00:24 isr_count: 24, alarms: 0, overflows: 1
Time: 0 days 00:00:25 isr_count: 25, alarms: 0, overflows: 1
Time: 0 days 00:00:26 isr_count: 26, alarms: 0, overflows: 1
Time: 0 days 00:00:27 isr_count: 27, alarms: 0, overflows: 1
```

После запуска включается функция `rtc_isr()`, и свидетельство о ежесекундных прерываниях реализуется в выводимых сообщениях. Значение счетчика `isr_count` указывает, как часто прерывание вызывало ISR-процедуру `rtc_isr()`. Обратите внимание, что счетчик переполнений увеличивается, когда `isr_count` равен 24. Это указывает на то, что счетчик RTC переполнился и теперь перезапускается с нуля.

Прошедшее время в днях, часах, минутах и секундах отображается в каждом сообщении. Эти значения были рассчитаны в критическом разделе процедуры `rtc_isr()`.

Чтобы создать сигнал тревоги, нажмите <**A**>. При этом запустится ожидание сигнала тревоги, срок действия которого истекает через десять секунд. В следующем сеансе сигнал тревоги начинается примерно в 00:07:40, а тревожное сообщение появляется в 00:07:50, как и ожидалось:

```
Time: 0 days 00:07:38 isr_count: 458, alarms: 0, overflows: 1
Time: 0 days 00:07:39 isr_count: 459, alarms: 0, overflows: 1
Alarm configured for 10 seconds from now.
Press 'A' to set 10 second alarm,
else any key to read time.
Time: 0 days 00:07:40 isr_count: 460, alarms: 0, overflows: 1
Time: 0 days 00:07:41 isr_count: 461, alarms: 0, overflows: 1
...
Time: 0 days 00:07:48 isr_count: 468, alarms: 0, overflows: 1
Time: 0 days 00:07:49 isr_count: 469, alarms: 0, overflows: 1
*** ALARM *** at 0 days 00:07:50
Time: 0 days 00:07:50 isr_count: 471, alarms: 1, overflows: 1
Time: 0 days 00:07:51 isr_count: 472, alarms: 1, overflows: 1
```

В листинге 10.8 показан фрагмент кода, запускающий десятисекундный сигнал тревоги. Он использует процедуру `set_alarm()` из libopencm3 в строке 242. Вызов функции устанавливает регистр для запуска сигнала тревоги через десять секунд от текущего момента.

Листинг 10.8. Код срабатывания сигнала тревоги

```
0236:    ch = std_getc();
0237:
0238:    if ( ch == 'a' || ch == 'A' ) {
0239:        mutex_lock();
0240:        std_printf("\nAlarm configured for 10 seconds"
                        " from now.\n");
0241:        mutex_unlock();
0242:        set_alarm(10u);
0243:    }
```

Прерывание rtc_alarm_isr()

Если вы изучали информацию из описания STM32F103C8T8, вы, вероятно, знаете, что существует вторая возможная точка входа в обработчик прерывания RTC. В описании об этом говорится довольно загадочно, если только вы не знаете, что они подразумевают под ссылками на «глобальное прерывание RTC» и «EXTI Line 17». Подпрограмма `rtc_isr()` – это «глобальное прерывание RTC», а «EXTI Line 17» означает что-то другое.

Контроллер EXTI представляет собой контроллер внешних прерываний. Назначение этого контроллера – позволить входным линиям GPIO вызывать прерывание при возрастании или падении сигнала. Поэтому я думаю, что вас

простят, если вы спросите: «Что такого внешнего в RTC?» Демонстрационный код, реализующий прерывание `rtc_alarm_isr()`, находится в следующем каталоге:

```
$ cd ~/stm32f103c8t6/rtos/rtc2
```

EXTI-контроллер

Как упоминалось ранее, контроллер EXTI[58] позволяет входным линиям GPIO вызывать прерывание, если их входной уровень возрастает или падает, в зависимости от установок (или при любом изменении уровня). Все выводы GPIO 0 отображаются на прерывание EXTI0. Для STM32F103C8T6 это означает порты GPIO PA0, PB0 и PC0. Другие контроллеры STM32 с дополнительными портами GPIO также могут иметь PD0, PE0 и т. д. Аналогично прерывание EXTI15 вызывается портами GPIO PA15, PB15, PC15 и т. д.

Это определяет EXTI0–EXTI15 как источники прерываний. Но, помимо них, существует еще до четырех источников[59]:

- EXTI16 – выход PVD (программируемый детектор напряжения);
- EXTI17 – событие тревоги RTC;
- EXTI18 – событие пробуждения USB;
- EXTI19 – событие пробуждения Ethernet (нет на F103C8T6).

Это внутренние события, которые также могут создавать прерывания. Непосредственный интерес здесь представляет событие тревоги RTC (EXTI17).

Настройка EXTI17

Чтобы получать прерывания `rtc_alarm_isr()`, мы должны настроить событие EXTII17 для вызова прерывания. Для этого требуются следующие шаги, согласно библиотеке libopencm3:

1) `#include <libopencm3/stm32/exti.h>`
2) `exti_set_trigger(EXTI17,EXTI_TRIGGER_RISING);`
3) `exti_enable_request(EXTI17);`
4) `nvic_enable_irq(NVIC_RTC_ALARM_IRQ);`

Шаг первый – требуется дополнительный включаемый файл libopencm3. Второй шаг указывает, какое событие мы хотим использовать в качестве спускового крючка, и настраивает нарастающий фронт (`RISING`) тревожного события. Шаг третий разрешает прерывание EXTI17 на периферийном устройстве. Наконец, четвертый шаг позволяет контроллеру прерываний обработать событие `RTC_ALARM_IRQ`.

Поскольку обработка сигналов тревоги здесь отделена от обработчика `rtc_isr()`, новый метод `rtc_alarm_isr()` в листинге 10.9 выглядит довольно простым.

---

[58] Как вы уже догадались, EXTI есть сокращение от «external interrupt» (*внешнее прерывание*).

[59] Эти дополнительные источники имеют фиксированные внутренние соединения, номер вывода для них указывать не требуется.

Листинг 10.9. Процедура `rtc_alarm_isr()`

```
0098: void
0099:   rtc_alarm_isr(void) {
0100:   BaseType_t woken = pdFALSE;
0101:
0102:   ++rtc_alarm_count;
0103:   exti_reset_request(EXTI17);
0104:   rtc_clear_flag(RTC_ALR);
0105:
0106:   vTaskNotifyGiveFromISR(h_task3,&woken);
0107:   portYIELD_FROM_ISR(woken);
0108: }
```

Обработчик практически идентичен обработке тревожных событий, представленной ранее, но есть один дополнительный шаг, который необходимо соблюдать, а именно:

```
0103: exti_reset_request(EXTI17);
```

Этот вызов libopencm3 необходим для сброса прерывания EXTI17 в дополнение к обычному сбросу флага `RTC_ALR` в строке 104.

Эта дополнительная настройка для `EXTI17` не особенно обременительна, но может быть сложно разобраться в описании. Теперь, когда вы увидели секрет, это не составит труда.

Запустите демоверсию RTC2 так же, как и RTC. Единственная разница между ними заключается в обработке прерываний.

Итоги

В этой главе были рассмотрены настройка и использование часов реального времени. Из этой презентации становится ясно, что RTC не является сложным периферийным устройством внутри STM32. Однако не следует недооценивать полезность наличия надежного и точного времени[60].

Несмотря на простоту, есть области, которые требуют тщательного рассмотрения, например правильная обработка переполнения таймера. Это становится еще более важным, если используются единицы с более высо-

[60] Следует заметить, что так называемые RTC в STM32 настоящими часами реального времени не являются (как и аналогичные устройства в других семействах контроллеров). Это, по сути, обычный суммирующий счетчик-таймер с возможностью вызова прерываний по некоторым событиям. Считать минуты-часы и дни, не говоря уж о вычислении дней недели и календарных дат (что отдельно не самая простая задача), этот счетчик не способен. Единственное, что его отличает от обычных таймеров в микроконтроллерах, – возможность автономной работы от отдельной батарейки, а также широкий выбор источников тактового сигнала, включая отдельный «часовой» кварц 32 768 Гц. Если вам требуется реальный календарь и отсчет в точных миллисекундах, следует использовать внешние модули RTC на специально разработанных «часовых» микросхемах.

ким разрешением, такие как 1/100 с[61]. Когда используется функция тревоги, событие переполнения счетчика RTC также может потребовать специальной обработки.

Знание того, как настроить прерывание EXTI17, также позволит вам аналогично настроить сигнал GPIO. Процедура та же, за исключением того, что вы указываете EXTIn для конкретного GPIOn.

Упражнения

1. Каковы три возможных источника прерываний от RTC?
2. Какова цель вызовов `TaskENTER_CRITICAL_FROM_ISR` и `TaskEXIT_CRITICAL_FROM_ISR`?
3. Сколько битов имеет счетчик RTC?
4. Какой источник синхронизации продолжает работать, когда контроллер STM32 отключается?
5. Какой источник синхронизации является наиболее точным?

[61] По поводу переполнения надо учитывать, при что тактовом сигнале в одну секунду емкости 32-разрядного счетчика хватит примерно на 136 лет; но при длительности тактов в районе миллисекунд этот срок уменьшается менее чем до пары месяцев.

Глава 11 • Интерфейс I2C

Шина I2C является удобной аппаратной системой главным образом потому, что для связи ей требуется всего два провода. Шина также известна под другими названиями, например IIC (inter-integrated circuit, межинтегральная схема) или TWI (two-wire interface, двухпроводной интерфейс). Компания Phillips Semiconductors разработала шину I2C, которую Intel позже расширила протоколом SMBus. Эти протоколы в основном взаимозаменяемы, но в этой главе я сосредоточусь на I2C.

Учитывая полезность шины I2C, неудивительно, что платформа STM32 включает в себя аппаратное периферийное устройство для нее. В этой главе рассматривается, как использовать периферийное устройство совместно с расширяющей GPIO микросхемой PCF8574.

Шина I2C

Одной из отличительных особенностей I2C как шины последовательной связи является то, что для нее требуется всего два провода. Источник питания и заземление в этот подсчет не включены. Задействованы две линии связи:

- системные такты (обычно обозначаются как SCL),
- системные данные (обычно обозначаются как SDA).

Каждая из этих линий в покое находится под высоким уровнем напряжения (обычно 5 или 3.3 В). Любое устройство на шине может генерировать сигнал данных, понижая напряжение на линии до нуля вольт. Это хорошо работает для выхода с открытым коллектором биполярного транзистора или открытым стоком полевого (FET) транзистора. Когда транзистор активен, он действует как контакты переключателя, замыкающие линию шины на землю. Когда шина простаивает, подтягивающий резистор повышает напряжение линии. По этой причине каждая из линий I2C должна иметь как минимум один общий подтягивающий резистор.

Ведущий и ведомый

Поскольку каждое устройство на шине способно отключать каждую из линий, должен существовать какой-то протокол для организации работы. В противном случае на шине будет происходить несколько передач одновременно, и никто из получателей не сможет понять искаженные сообщения. По этой причине протокол I2C часто использует одно ведущее (master) и множество ведомых (slave) устройств. В более сложных системах возможно наличие более одного ведущего контроллера, что выходит за рамки данной главы.

Ведущее устройство всегда начинает сеанс обмена, подавая тактовый сигнал. Исключением из того, что линия SCL управляется ведущим устройством, является то, что ведомые устройства могут иногда растягивать такты, что-

бы выиграть дополнительное время (когда это поддерживается ведущим). Растяжение тактового сигнала происходит, когда ведомое устройство продолжает удерживать линию SCL на низком уровне после того, как ведущее устройство освободило его.

Ведомые устройства отвечают только при адресации к ним. Каждое ведомое устройство имеет уникальный 7-битный адрес (есть возможность расширить его до 10 бит), поэтому оно знает, когда ему было отправлено сообщение шины. Это одна из особенностей, которая отличает I2C от шины SPI. Устройство I2C выбирается по адресу, а устройства SPI выбираются по линии выбора микросхемы.

Старт и стоп

I2C простаивает, когда линии SDA и SCL находятся на высоком уровне. В этом случае ни одно устройство – ведущее или ведомое – не снижает уровень напряжения на линиях шины.

На начало сеанса (старта транзакции) I2C указывают следующие события:

1) линия SCL остается высокой,
2) линия SDA опущена.

Шаг 2 обычно происходит в течение одного такта, хотя это не обязательно так. Когда шина простаивает, достаточно увидеть, что линия SDA переходит в низкий уровень, а SCL остается высоким. На рис. 11.1 показаны состояния шины I2C старта, стопа и повторного старта.

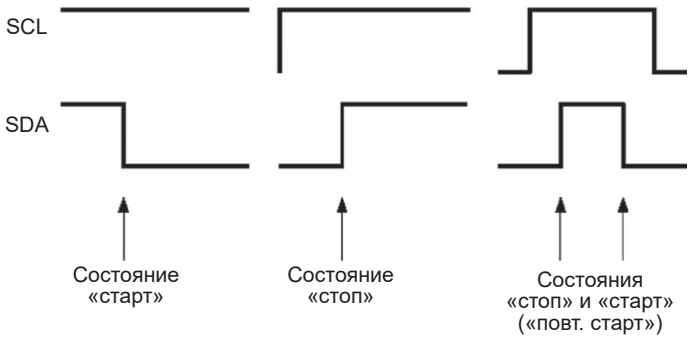


Рис. 11.1. Сигналы старта, стопа и повторного старта I2C

Повторный старт – это оптимальный способ подачи сигналов стопа и старта с целью удержания шины при продолжении длительной транзакции I2C по тому же адресу. Позже мы обсудим это подробнее.

Биты данных

Биты данных передаются в момент перехода от высокого к низкому уровню тактового сигнала (SCL). Рисунок 11.2 иллюстрирует этот принцип.

Выборка состояний линии шины SDA происходит там, где показаны стрелки. Высокое или низкое состояние линии SDA считывается в момент, когда линия SCL переходит на низкий уровень (ведущим устройством).

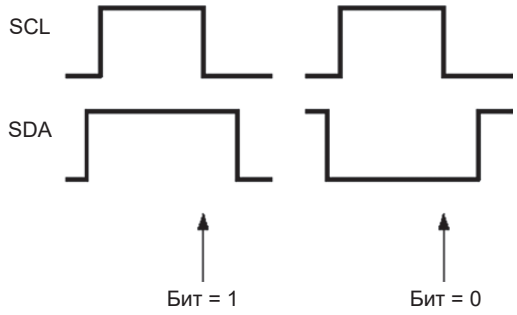


Рис. 11.2. Битовые сигналы данных I2C

I2C-адрес

Прежде чем мы рассмотрим всю транзакцию шины, давайте опишем адресный байт, который используется для идентификации ведомого устройства, к которому происходит обращение (рис. 11.3). В дополнение к адресу бит чтения/записи R/W указывает на намерение выполнить чтение с ведомого устройства или запись на него.

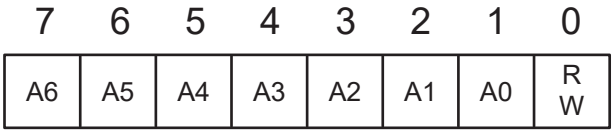


Рис. 11.3. 7-битный формат адреса I2C

7 бит адреса в байте адреса сдвинуты на 1 бит влево, а бит чтения/записи находится крайним правым (самый младший). Бит R/W определяется следующим образом:

- бит R/W = 1 указывает, что последует операция чтения с ведомого устройства;
- бит R/W = 0 указывает, что последует операция записи на ведомое устройство.

Адрес и бит чтения/записи перед ним всегда следуют за стартовым или повторным стартовым сигналом на шине I2C. Сигнал «старт»[62] требует, чтобы контроллер I2C проверил, что шина не используется другим ведущим устройством; это делается с помощью процедуры арбитража шины (если поддерживается несколько ведущих устройств). Но как только доступ к шине

[62] Отметим, что автор этой книги (вслед за руководством по STM32) называет сигналы (состояния шины) «старт» и «стоп» стартовым и стоповым битами – видимо, по аналогии с протоколом UART. Это неверное название: в протоколе UART это действительно биты (высокий или низкий уровни линии данных), а в I2C это именно состояния или сигналы, так как соответствуют некоему событию перепада уровней на одной из линий при нахождении другой в определенном состоянии (т. е. касаются двух линий одновременно, что никак не соответствует понятию бита).

определен, шина принадлежит ведущему контроллеру до тех пор, пока она не будет освобождена стоп-сигналом.

Повторный старт позволяет продолжить текущую транзакцию без дальнейшего арбитража шины. Поскольку за ним должны следовать адрес и бит чтения/записи, это позволяет обслуживать несколько подчиненных устройств за одну транзакцию. Альтернативно одно и то же подчиненное устройство может быть адресовано, но доступно в другом режиме чтения/записи.

Адреса ведомых устройств при отправке сдвигаются на 1 бит влево. Это приводит к умножению адреса на два. Указанный в коде адрес 0x42 на самом деле равен 0x21. Обратите внимание на это в документации.

I2C-транзакции

На рис. 11.4 показаны процедуры записи и чтения (без повторного старта).

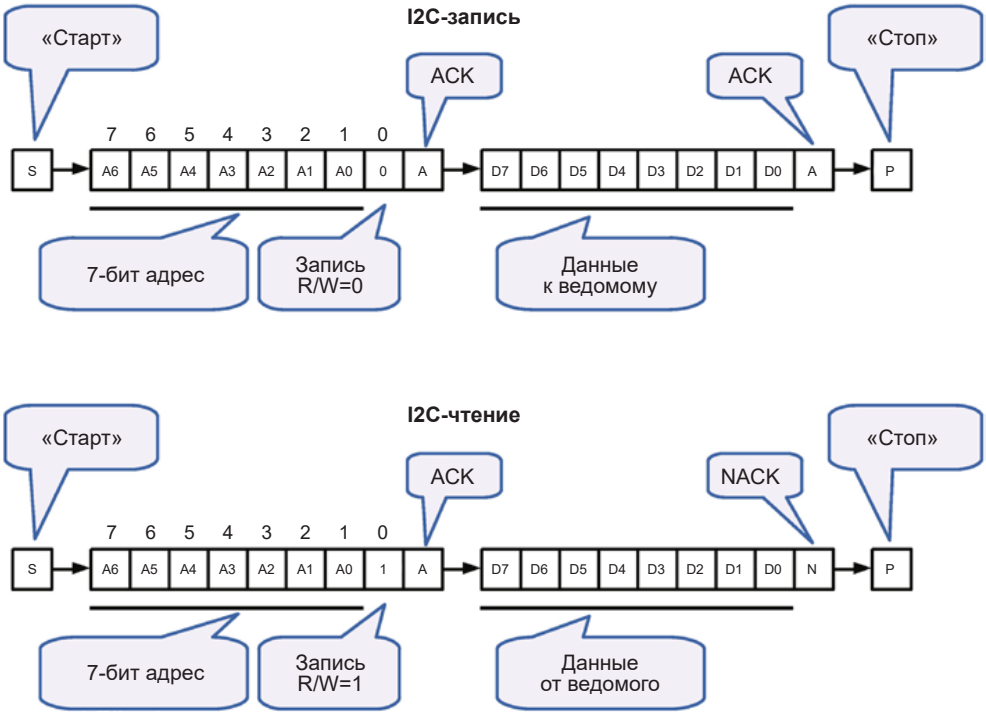


Рис. 11.4. Сеанс записи и сеанс чтения I2C

Верхняя часть рис. 11.4 иллюстрирует простую процедуру записи. Основной ход событий таков:

1) контроллер I2C получает управление шиной и выдает стартовый сигнал;
2) ведущий (контроллер) формирует 7 бит адреса, за которыми следует бит = 0, указывая, что это будет сеанс записи;

3) ведомое устройство подтверждает запрос и переводит линию данных в низкий уровень, формируя бит ACK (подтверждение). Если ни одно ведомое устройство не отвечает, на линии данных останется высокий уровень, что равносильно получению контроллером отрицательного подтверждения NACK;
4) поскольку это процедура записи, записывается байт данных;
5) ведомое устройство подтверждает получение байта данных, переводя линию данных в низкий уровень для формирования сигнала ACK;
6) ведущий больше не отправляет данные, поэтому формирует стоповый сигнал и освобождает шину I2C.

Запрос на чтение аналогичен:

1) контроллер I2C получает управление шиной и выдает состояние «старт»;
2) ведущий (контроллер) записывает 7 бит адреса, за которыми следует бит = 1, указывая, что это будет сеанс чтения;
3) ведомое устройство подтверждает запрос и переводит линию данных на низкий уровень, формируя сигнал ACK. Если ни одно ведомое устройство не отвечает, на линии данных останется высокий уровень, что приведет к получению контроллером NACK;
4) поскольку это процедура чтения, ведущее устройство продолжает посылать тактовые импульсы, чтобы позволить ведомому устройству синхронизировать биты данных в своем ответе с ведущим;
5) с каждым тактовым импульсом ведомое устройство посылает восемь бит данных в ведущий контроллер;
6) во время, когда ведущим должен быть послан сигнал ACK, контроллер обычно отправляет NACK, так как больше не нужно читать байты;
7) контроллер отправляет стоповый сигнал, который всегда завершает сеанс связи с ведомым устройством (независимо от последнего отправленного ACK/NACK).

PCF8574: расширитель GPIO

Для тестирования шины I2C в этой главе мы будем использовать микросхему расширения GPIO PCF8574 (рис. 11.5). Это отличный чип для создания дополнительных линий GPIO при условии, что вам не нужна высокая скорость (демопрограмма работает по шине I2C на частоте 100 кГц).

Питание +3.3 В подается на контакт 16, а общий провод – на контакт 8. Контакты от A0 до A2 используются для выбора адреса микросхемы в качестве ведомого (табл. 11.1). Выводы от P0 до P7 – это линии битов данных GPIO, которые могут быть входными или выходными. Контакт 14 подключается к линии синхронизации (SCL), а контакт 15 – к линии данных (SDA). Вывод 13 ($\overline{\text{INT}}$) можно использовать в качестве сигнала уведомления о некоторых операциях (см. далее).

Адресные линии от A0 до A2 программируются как нули при соединении с общим проводом и как единицы при подключении к Vcc (+3.3 В в нашем случае). Если у вас микросхема PCF8575A, то адрес нужно брать из правой колонки. Более ранняя микросхема PCF8574 использует шестнадцатеричные адреса в левом столбце таблицы.

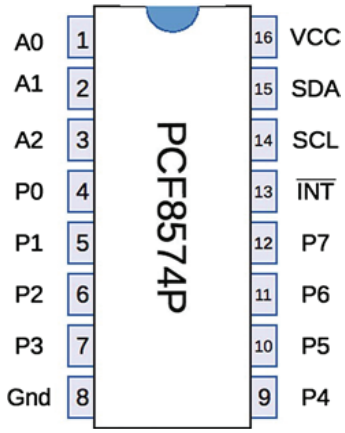


Рис. 11.5. Разводка выводов PCF8574P[63]

Таблица 11.1. Конфигурация адреса PCF8574

A0	A1	A2	Адрес PCF8574	Адрес PCF8574A
0	0	0	0x20	0x38
0	0	1	0x21	0x39
0	1	0	0x22	0x3A
0	1	1	0x23	0x3B
1	0	0	0x24	0x3C
1	0	1	0x25	0x3D
1	1	0	0x26	0x3E
1	1	1	0x27	0x3F

Подключение через I2C

На рис. 11.6 показаны три устройства PCF8574P, подключенные к STM32 через шину I2C.

Схема выглядит немного перегруженной, но это не страшно. Обратите внимание, что шина I2C состоит только из пары линий SCL и SDA, исходящих из STM32. Эти две линии подтягиваются резисторами R1 и R2 соответственно. Каждое ведомое устройство также подключено к этим линиям шины, что позволяет любому из них реагировать, когда оно распознает свой адрес.

Обратите внимание, что микросхемы IC1, IC2 и IC3 имеют разное подключение адресных контактов A0, A1 и A2. Это задает каждому устройству свой собственный адрес (см. табл. 11.1). Эти адреса должны быть уникальными.

[63] Суффикс «P» в данном случае означает тип корпуса PDIP; полное наименование варианта PCF8574A в таком корпусе будет PCF8574AP.

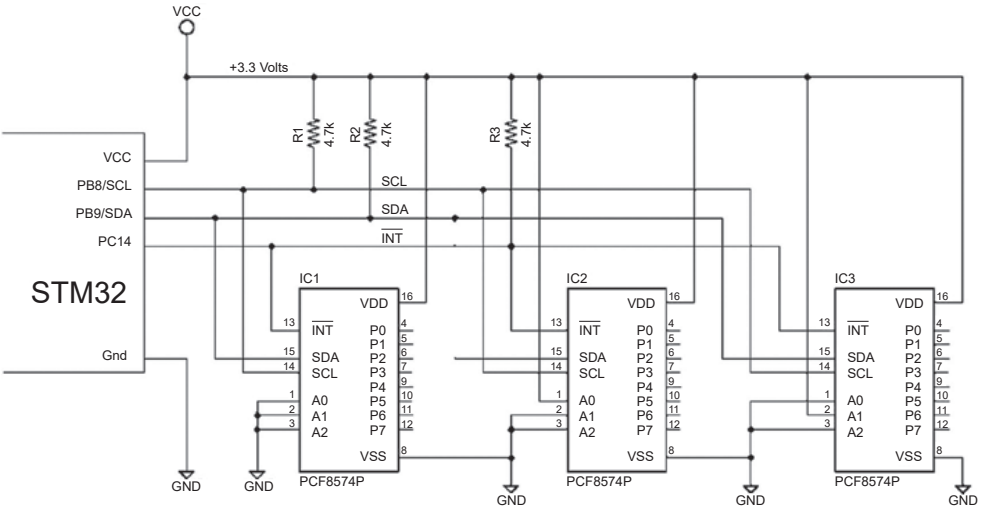


Рис. 11.6. К STM32 подключено три ведомых устройства PCF8574P с использованием шины I2C.

Линия $\overline{\text{INT}}$ микросхемы PCF8574

Остальные соединения на рис. 11.6 – это питание и линия $\overline{\text{INT}}$. Эта линия – дополнительный компонент интерфейса, который не имеет ничего общего с самой шиной I2C. Вы можете даже не подключать выводы $\overline{\text{INT}}$ устройств PCF8574P вовсе, особенно если устройства никогда не используются для ввода через дополнительные GPIO.

Линия $\overline{\text{INT}}$ сигнализирует об изменениях входных GPIO и обычно подключается к линии прерывания микропроцессора. Это избавляет MCU от постоянного опроса устройств I2C, например чтобы узнать, была ли нажата кнопка. Если какая-либо входная линия меняется с высокого уровня на низкий или с низкого на высокий, транзистор с открытым стоком в PCF8574 включается и переводит $\overline{\text{INT}}$ в низкий уровень. Этот уровень остается низким до тех пор, пока устройство не «обслужит» свое прерывание. Простое чтение или запись на периферийное устройство – это все, что необходимо для обслуживания прерывания.

Линия $\overline{\text{INT}}$ не определяет, какое именно ведомое устройство зарегистрировало изменение. Контроллер все равно должен опрашивать участвующие подчиненные устройства, чтобы узнать, где произошло изменение.

Есть небольшое ограничение, о котором важно помнить. Если изменение уровня GPIO происходит слишком быстро, прерывание не будет сгенерировано. Также возможно, что событие изменения GPIO произойдет во время цикла ACK/NACK, когда прерывание очищается. Возникающее тогда прерывание также может быть потеряно. В описании PCF8574 от компании NXP Semiconductors указано, что требуется 4 мкс с момента обнаружения изменения GPIO до активации линии $\overline{\text{INT}}$. Оставшееся время будет со-

стоять из реакции микроконтроллера на прерывание и его программной обработки.

Конфигурация PCF8574

В описании NXP Semiconductors порты ввода-вывода PCF8574 описаны как квазидвунаправленные. Это означает, что порты GPIO (от P0 до P7) могут использоваться в качестве выходов или считываться как входы напрямую, без какой-либо настройки через регистр устройства.

Чтобы отправить выходное значение, вы просто записываете его на устройство PCF8574 по шине I2C. С другой стороны, GPIO в качестве входов требуют небольшой хитрости: если вам нужны входы GPIO, вы сначала записываете 1 бит в порт GPIO. Чтобы увидеть, как это работает, просмотрите рис. 11.7.

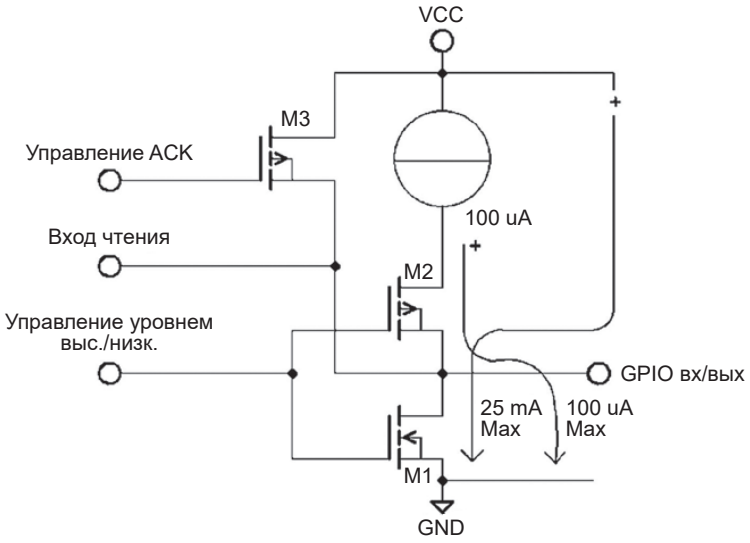


Рис. 11.7. Упрощенная схема GPIO микросхемы PCF8574

Начните с верхней части схемы, где находится Vcc (+3.3 В). Внутри микросхемы находится источник постоянного тока на 100 мкА, включенный последовательно с транзисторами M2 и M1. Следовательно, когда в порт GPIO записывается высокий уровень (бит = 1), транзистор M2 включается, а M1 выключается (управляется внутренним сигналом «управление уровнем»). Транзистор M3 в это время выключен.

Если вы замкнули вывод GPIO на землю, источник постоянного тока ограничит ток до 100 мкА, когда он протекает через M2 и выходит из вывода GPIO на землю (крайняя правая стрелка на рис. 11.7). При таком замыкании внутренние компоненты PCF8574 способны распознавать низкий уровень на внутреннем выводе «Вход чтения», подключенном к GPIO, который теперь

считывается как нулевой бит. Записывая бит = 1 в GPIO, вы позволяете внешней цепи снизить уровень напряжения или оставить его высоким. Ток всегда ограничен до 100 мкА, поэтому вреда никому не будет.

Если бы вместо этого в вывод GPIO был записан бит = 0, транзистор M1 всегда был бы включен, замыкая выход GPIO на общий провод. В результате с линии «Вход чтения» всегда будет читаться бит, равный нулю. Используя простое правило записи единичного бита в GPIO, вы можете определить, когда он замыкается на землю в качестве входа.

Подключение нагрузки к GPIO микросхемы PCF8574

Квазидвунаправленная конструкция GPIO имеет свои последствия. Вы уже видели, что ток закороченного выхода GPIO ограничен 100 мкА. Это означает, что GPIO не может выступать в качестве источника тока для светодиода. Типичному светодиоду требуется ток около 10 мА, что в 100 раз больше, чем способен обеспечить этот GPIO!

Однако транзистор M1 способен выдерживать максимум 25 мА, если вы используете вывод GPIO для подачи питания на светодиод. На рис. 11.8 показано, как управлять светодиодом.

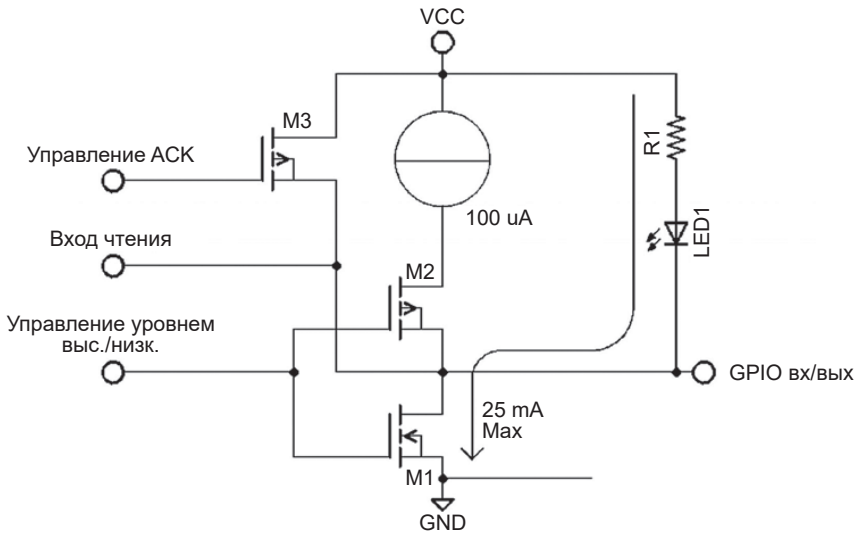


Рис. 11.8. Управление нагрузкой с помощью PCF8574

Светодиод и резистор R1 питаются от напряжения Vcc, которое не ограничено по току. Таким образом, M1 действует как переключатель, пропускающий ток на землю, зажигая светодиод. Логический смысл этого заключается в том, что вам нужно записать бит = 0, чтобы включить светодиод.



Хотя ток высокого уровня PCF8574 ограничен величиной 100 мкА, этого достаточно для управления другими входными сигналами CMOS-микросхем[64].

Формирование фронтов

Когда выход GPIO установлен в единичное состояние, для поднятия его до Vcc доступен ток только 100 мкА. Это представляет собой некоторую проблему в виде большого времени нарастания при переключении от низкого потенциала.

Разработчики PCF8574 добавили схему с транзистором M3 (рис. 11.8), который обычно закрыт. Однако при записи в устройство каждый вывод GPIO, устанавливаемый в единичное состояние, получает добавку от M3 во время цикла I2C ACK/NACK. Это помогает обеспечить быстрый переход от низкого к высокому уровню выходного сигнала. После завершения цикла ACK/NACK M3 снова выключается, оставляя ограничитель тока 100 мкА для поддержания высокого уровня выходного сигнала.

Демосхема

Рисунок 11.9 иллюстрирует окончательную схему демонстрационной программы. Примечательные изменения в сравнении с общим примером (рис. 11.6) заключаются в том, что используется только один чип PCF8574 с двумя светодиодами и одной кнопкой.

При сборке демонстрационной схемы (рис. 11.9) не забудьте включить подтягивающий резистор R3, чтобы потенциал покоя вывода контроллера PC14 был высоким. Также обратите внимание, что оба светодиода питаются от шины Vcc, поэтому выводы P0 и P1 PCF8574 должны обеспечить втекающий ток для включения светодиодов. Вывод P7 PCF8574 будет считывать кнопку, следовательно, в покое имеет высокий уровень (напомним, что вывод в PCF8574 получает высокий уровень от источника постоянного тока 100 мкА). При нажатии кнопки вывод P7 замыкается на общий провод, в результате чего считывается нулевой бит.

В проекте RTC2 прерывание EXTI использовалось для создания отдельного прерывания тревоги. Требуется еще несколько шагов для получения прерывания на PC14 по сигналу $\overline{\text{INT}}$. Давайте посмотрим на используемое программное обеспечение. Программное обеспечение проекта для этой главы находится в следующем каталоге:

```
$ cd ~/stm32f103c8t6/rtos/i2c-pcf8574
```

[64] Микросхемы по технологии CMOS (complementary metal oxide semiconductor, комплементарный металл-оксидный полупроводник, КМДП) отличаются близким к нулю потреблением тока по входу (в статическом установившемся режиме). Практически все современные цифровые микросхемы (включая и микроконтроллеры, и микросхемы низкой степени интеграции, как PCF8574) выполняются с внешними выводами, совместимыми с CMOS.

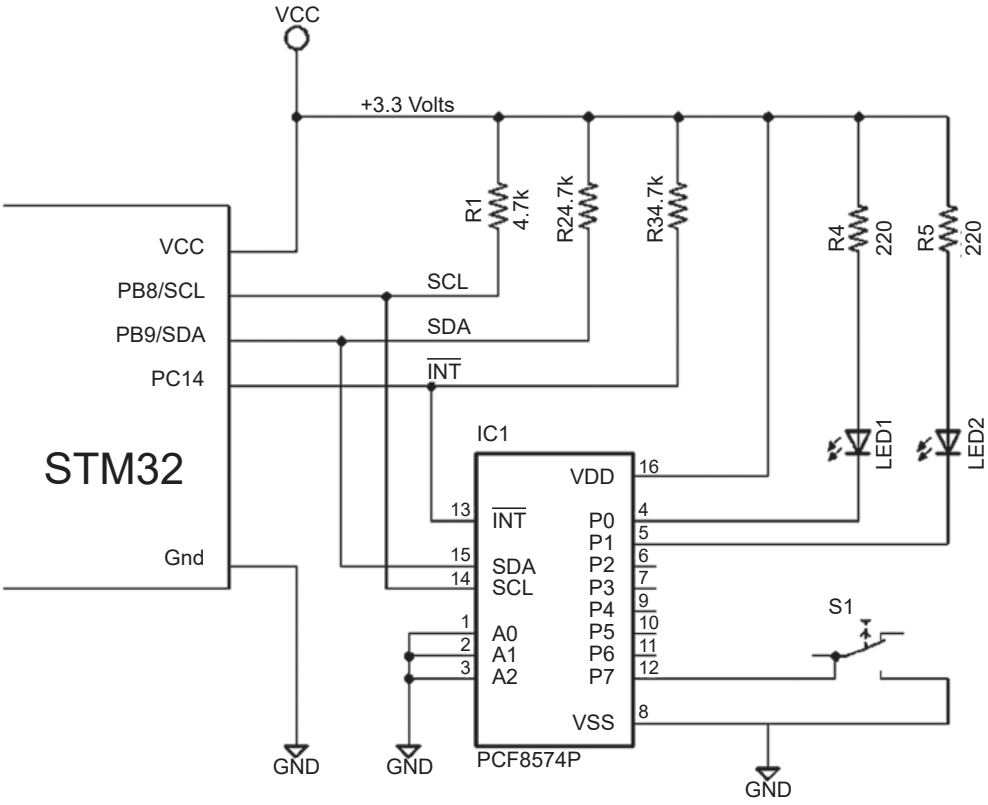


Рис. 11.9. Демонстрационная схема I2C со светодиодами и кнопкой

Сначала мы рассмотрим настройку I2C и прерываний EXTI в процедуре `main()` файла *main.c* (листинг 11.1).

Листинг 11.1. Первоначальная настройка периферийных устройств I2C и прерываний EXTI

```
0197: int
0198: main(void) {
0199:
0200: rcc_clock_setup_pll(&rcc_hse_configs[RCC_CLOCK_HSE8_72MHZ]);
0201: rcc_periph_clock_enable(RCC_GPIOB); // I2C
0202: rcc_periph_clock_enable(RCC_GPIOC); // LED
0203: rcc_periph_clock_enable(RCC_AFIO); // EXTI
0204: rcc_periph_clock_enable(RCC_I2C1); // I2C
0205:
0206: gpio_set_mode(GPIOB,
0207:     GPIO_MODE_OUTPUT_50_MHZ,
0208:     GPIO_CNF_OUTPUT_ALTFN_OPENDRAIN,
0209:     GPIO6|GPIO7); // I2C
0210: gpio_set(GPIOB,GPIO6|GPIO7); // Idle high
```

```
0211:
0212: gpio_set_mode(GPIOC,
0213:     GPIO_MODE_OUTPUT_2_MHZ,
0214:     GPIO_CNF_OUTPUT_PUSHPULL,
0215:     GPIO13); // LED on PC13
0216: gpio_set(GPIOC,GPIO13); // PC13 LED dark
0217:
0218: // AFIO_MAPR_I2C1_REMAP=0, PB6+PB7
0219: gpio_primary_remap(0,0);
0220:
0221: gpio_set_mode(GPIOC, // PCF8574 /INT
0222:     GPIO_MODE_INPUT, // Input
0223:     GPIO_CNF_INPUT_FLOAT,
0224:     GPIO14); // on PC14
0225:
0226: exti_select_source(EXTI14,GPIOC);
0227: exti_set_trigger(EXTI14,EXTI_TRIGGER_FALLING);
0228: exti_enable_request(EXTI14);
0229: nvic_enable_irq(NVIC_EXTI15_10_IRQ); // PC14 <- /INT
```

Строки 201–204 включают тактовые сигналы на выводах GPIOB, необходимые для I2C, GPIOC (для управления светодиодами и PC14). Строка 206 настраивает GPIO PB6 и PB7 для работы с открытым стоком для периферийного устройства I2C. Строка 210, возможно, не является строго необходимой, но до тех пор, пока периферийное устройство I2C не будет настроено, линии шины I2C должны быть подключены к высокому уровню.

Строка 219 настраивает порт I2C1 на использование PB6 и PB7. Функцию `gpio_primary_remap()` библиотеки libopencm3 можно использовать для выбора других вариантов. PB6 и PB7 полезны, поскольку имеют входы, устойчивые к напряжению 5 В.

Строка 221 настраивает GPIO PC14 как вход в плавающем режиме. Эта линия будет физически подтянута к питанию резистором R3 (рис. 11.9).

Строки 226–229 настраивают прерывание EXTI. Функция `exti_select_source()` выбирает GPIO PC14 для добавления в список потенциальных источников прерываний. Затем строка 227 настраивает возникновение прерывания при перепаде сигнала с высокого на низкий уровень. Наконец, строка 228 позволяет периферийному устройству EXTI запрашивать прерывания. Вызов `nvic_enable_irq()` включает вектор прерываний `NVIC_EXTI15_10_IRQ`. Когда произойдет это прерывание, будет вызван обработчик `exti15_10_isr()`.

Чтобы вам не пришлось долго ломать голову над справочным описанием STMicroelectronics (RM0008), здесь представлена табл. 11.2. На мой взгляд, в описании неясно, как поддерживаются прерывания для EXTI. В таблице показано, что строки с 0 по 4 имеют собственный вектор прерывания. Но для портов GPIO с номерами от 5 до 9 или от 10 до 15 векторы прерываний используются более широко.

Таблица 11.2. Список прерываний EXTI STM32F103 и имен процедур ISR[65]

Прерывание	Процедура ISR	Описание
NVIC_EXTI0_IRQ	`exti0_isr()`	Выводы 0: PA0/PB0/PC0
NVIC_EXTI1_IRQ	`exti1_isr()`	Выводы 1: PA1/PB1/PC1
NVIC_EXTI2_IRQ	`exti2_isr()`	Выводы 2: PA2/PB2/PC2
NVIC_EXTI3_IRQ	`exti3_isr()`	Выводы 3: PA3/PB3/PC3
NVIC_EXTI4_IRQ	`exti4_isr()`	Выводы 4: PA4/PB4/PC4
NVIC_EXTI9_5_IRQ	`exti9_5_isr()`	Выводы 5-9: PA5-9/PB5-9/PC5-9
NVIC_EXTI15_10_IRQ	`exti15_10_isr()`	Выводы 10-15: PA10-15/PB10-15 /PC10-15
NVIC_PVD_IRQ	`pvd_isr()`	Линия 16: детектор напряжения питания
NVIC_RTC_ALARM_IRQ	`rtc_alarm_isr()`	Линия 17: событие тревоги RTC
NVIC_USB_WAKEUP_IRQ	`usb_wakeup_isr()`	Линия 18: пробуждение USB

Программное обеспечение I2C

Исходный код для управления периферийным устройством I2C помещен в модуль *i2c.c*. Первая интересующая нас функция – `i2c_configure()`, показанная в листинге 11.2.

Листинг 11.2. Конфигурация I2C

```
0088: void
0089: i2c_configure(I2C_Control *dev,uint32_t i2c,uint32_t ticks) {
0090:
0091:   dev->device = i2c;
0092:   dev->timeout = ticks;
0093:
0094:   i2c_peripheral_disable(dev->device);
0095:   i2c_reset(dev->device);
0096:   I2C_CR1(dev->device) &= ~I2C_CR1_STOP; // Clear stop
0097:   i2c_set_standard_mode(dev->device); // 100 kHz mode
0098:   i2c_set_clock_frequency(dev->device,36);// APB Freq
0099:   i2c_set_trise(dev->device,36); // 1000 ns
0100:   i2c_set_dutycycle(dev->device,I2C_CCR_DUTY_DIV2);
0101:   i2c_set_ccr(dev->device,180); // 100 kHz <= 180 * 1 /36M
0102:   i2c_set_own_7bit_slave_address(dev->device,0x23);
        // Necessary?
0103:   i2c_peripheral_enable(dev->device);
0104: }
```

Структура с именем `I2C_Control` для хранения конфигурации передается в качестве первого аргумента. Периферийный адрес I2C передается в аргументе `i2c`, который в этой демонстрации будет `I2C1`. Последний аргумент определяет используемый тайм-аут, указанный в тиках. Эти значения сохраняются в `I2C_Control` в строках 91 и 92 для дальнейшего использования.

[65] Линии 16–18 (см. нижние строки в таблице) представляют собой внутренние соединения и не соответствуют никакому внешнему выводу (см. также сноску 59 на стр. 185).

Строки 94–96 очищают и сбрасывают периферийное устройство I2C, чтобы, если оно зависло, его можно было «отсоединить». Одним из недостатков I2C является то, что протокол иногда может зависать, если на шине происходят неприятные вещи.

Строки 97–103 настраивают периферийное устройство I2C и включают его. Строка 102 необходима только в том случае, если вы хотите использовать контроллер в ведомом режиме.

В предыдущих версиях libopencm3 была предусмотрена функция `i2c_reset()`. Теперь необходимо предоставить нашу собственную (листинг 11.3), на которую есть ссылка в коде листинга 11.2 в строке 95. Обратите внимание, что функция определена как «статическая», поскольку за пределами модуля i2c.c нет кода, который ссылается на нее.

Листинг 11.3. Функция `i2c_reset()`

```
0062: static void
0063: i2c_reset(uint32_t i2c) {
0064:
0065:     switch (i2c) {
0066:     case I2C1:
0067:          rcc_periph_reset_pulse(RST_I2C1);
0068:          break;
0069: #if defined(I2C2_BASE)
0070:     case I2C2:
0071:          rcc_periph_reset_pulse(RST_I2C2);
0072:          break;
0073: #endif
0074: #if defined(I2C3_BASE)
0075:     case I2C3:
0076:          rcc_periph_reset_pulse(RST_I2C3);
0077:          break;
0078: #endif
0079:     default:
0080:          break;
0081:     }
0082: }
```

Проверка готовности I2C

Прежде чем можно будет инициировать любые операции I2C, вы должны проверить, занято ли устройство. В противном случае ваш запрос, скорее всего, будет проигнорирован или повредит текущей операции. В листинге 11.4 показана используемая процедура.

Листинг 11.4. Проверка I2C на готовность

```
0110: void
0111: i2c_wait_busy(I2C_Control *dev) {
0112:
0113:     while ( I2C_SR2(dev->device) & I2C_SR2_BUSY )
```

```
0114:         taskYIELD(); // I2C Busy
0115:
0116: }
```

Этот код использует процедуры и макроопределения библиотеки libopencm3 для управления периферийным устройством. Однако если устройство занято, управление передается другим задачам FreeRTOS с помощью оператора `TaskYIELD()`.

Запуск I2C

Чтобы инициировать сессию обмена по шине I2C, периферийное устройство должно выполнить операцию «старт». Это может включать арбитраж шины, если используется несколько ведущих. В листинге 11.5 показана процедура, используемая в данной демонстрации.

Листинг 11.5. Функция запуска I2C

```
0122: void
0123: i2c_start_addr(I2C_Control *dev,uint8_t addr,enum I2C_RW rw) {
0124:   TickType_t t0 = systicks();
0125:
0126:   i2c_wait_busy(dev); // Block until not busy
0127:   I2C_SR1(dev->device) &= ~I2C_SR1_AF; // Clear Acknowledge failure
0128:   i2c_clear_stop(dev->device); // Do not generate a Stop
0129:   if ( rw == Read )
0130:       i2c_enable_ack(dev->device);
0131:   i2c_send_start(dev->device); // Generate a Start/Restart
0132:
0133:   // Loop until ready:
0134:   while ( !((I2C_SR1(dev->device) & I2C_SR1_SB)
0135:     && (I2C_SR2(dev->device) & (I2C_SR2_MSL|I2C_SR2_BUSY))) ) {
0136:       if ( diff_ticks(t0,systicks()) > dev->timeout )
0137:           longjmp(i2c_exception,I2C_Addr_Timeout);
0138:       taskYIELD();
0139:   }
0140:
0141:   // Send Address & R/W flag:
0142:   i2c_send_7bit_address(dev->device,addr,
0143:       rw == Read ? I2C_READ : I2C_WRITE);
0144:
0145:   // Wait until completion, NAK or timeout
0146:   t0 = systicks();
0147:   while ( !(I2C_SR1(dev->device) & I2C_SR1_ADDR) ) {
0148:       if ( I2C_SR1(dev->device) & I2C_SR1_AF ) {
0149:           i2c_send_stop(dev->device);
0150:           (void)I2C_SR1(dev->device);
0151:           (void)I2C_SR2(dev->device); // Clear flags
0152:           // NAK Received (no ADDR flag will be set here)
0153:           longjmp(i2c_exception,I2C_Addr_NAK);
0154:       }
```

```
0155:       if ( diff_ticks(t0,systicks()) > dev->timeout )
0156:           longjmp(i2c_exception,I2C_Addr_Timeout);
0157:       taskYIELD();
0158:   }
0159:
0160:   (void)I2C_SR2(dev->device); // Clear flags
0161: }
```

Первым шагом является определение текущего времени тика в строке 124. Это позволяет нам запланировать операцию и тайм-аут при необходимости. Строка 126 ожидает готовности периферийного устройства. Когда все готово, строка 127 очищает ошибку подтверждения, если она была. Строка 128 указывает, что «стоп» не должен генерироваться.

Если операция будет чтением, вызывается `i2c_enable_ack()`, чтобы разрешить получение сигнала ACK от ведомого устройства. Затем периферийному устройству в строке 131 приказывают сгенерировать сигнал «старт».

Строки 134 и 135 проверяют, был ли сгенерирован сигнал «старт». Если он еще не сгенерирован, строки 136 и 137 проверяют и выполняют `longjmp()`, если время ожидания операции истекло. Позже мы поговорим подробнее о функции `longjmp()`, действующей подобно исключению. Если время ожидания не истекло, выполняется оператор FreeRTOS `TaskYIELD()` для совместного использования контроллера, пока протекает ожидание.

После генерации стартового состояния выполнение продолжается в строке 142 отправкой адреса ведомого устройства с битом чтения/записи R/W.

В строке 146 мы снова отмечаем время для еще одного потенциального тайм-аута. Строка 147 ожидает отправки адреса I2C, а строка 148 проверяет, подтвердило ли ведомое устройство запрос. Если ни одно устройство не отвечает на запрошенный адрес, по умолчанию будет получен NACK (благодаря подтягивающему резистору). Если время операции истекло, в строке 156 выполняется функция `longjmp()`.

Если операция успешна, очищается флаг путем вызова `I2C_SR2()` для чтения регистра статуса.

Запись I2C

Если после того, как стартовый бит сгенерирован, при отправке адреса было указано, что затем следует запись, мы должны выполнить это действие. В листинге 11.6 показана используемая функция записи.

Листинг 11.6. Функция записи I2C

```
0167: void
0168: i2c_write(I2C_Control *dev,uint8_t byte) {
0169:     TickType_t t0 = systicks();
0170:
0171:     i2c_send_data(dev->device,byte);
0172:     while ( !(I2C_SR1(dev->device) & (I2C_SR1_BTF)) ) {
0173:       if ( diff_ticks(t0,systicks()) > dev->timeout )
0174:           longjmp(i2c_exception,I2C_Write_Timeout);
```

```
0175:       taskYIELD();
0176:     }
0177: }
```

В строке 169 указано время возможного тайм-аута. Строка 171 отправляет байт данных на периферийное устройство I2C для последовательной отправки по шине. Строка 172 проверяет завершение этой операции и при необходимости организует тайм-аут с помощью `longjmp()` (строка 174). Помимо совместного использования контроллера с помощью `TaskYIELD()`, функция возвращает результат в случае успеха.

Чтение I2C

Если целью было чтение, для чтения байта данных используется процедура чтения. Листинг 11.7 иллюстрирует код соответствующей функции.

Листинг 11.7. Функция чтения I2C

```
0184: uint8_t
0185: i2c_read(I2C_Control *dev,bool lastf) {
0186:     TickType_t t0 = systicks();
0187:
0188:     if ( lastf )
0189:        i2c_disable_ack(dev->device); // Reading last/only byte
0190:
0191:     while ( !(I2C_SR1(dev->device) & I2C_SR1_RxNE) ) {
0192:         if ( diff_ticks(t0,systicks()) > dev->timeout )
0193:             longjmp(i2c_exception,I2C_Read_Timeout);
0194:         taskYIELD();
0195:     }
0196:
0197:     return i2c_get_data(dev->device);
0198: }
```

Одним из необычных аспектов представленной функции `i2c_read()` является то, что она имеет логический флаг `lastf`. Это значение устанавливается вызывающей стороной, если это последний или единственный байт, который нужно прочитать. Это дает ведомому устройству предупреждение о том, что оно может расслабиться (некоторые ведомые устройства должны предварительно выбирать данные, чтобы идти в ногу с главным контроллером). Это цель действия в строке 189.

В противном случае это вопрос проверки состояния в строке 191 и тайм-аута в строке 193, если операция занимает слишком много времени. В противном случае процессор используется совместно с `TaskYIELD()`, и байт возвращается в строке 197.

Перезапуск I2C

Процедура `i2c_write_restart()`, частично показанная в листинге 11.8, обеспечивает возможность перехода от запроса на запись к другому запросу (чтению или записи) без остановки. Вы можете продолжить работу с тем же

ведомым устройством (по тому тот же адресу ведомого) или переключиться на другое. Это важно при наличии нескольких ведущих устройств I2C, поскольку позволяет передавать другое сообщение без повторного согласования доступа к шине.

Листинг 11.8. «Секретный код» выполнения транзакции с перезапуском I2C

```
0205: void
0206: i2c_write_restart(I2C_Control *dev,uint8_t byte,uint8_t addr) {
0207:     TickType_t t0 = systicks();
0208:
0209:     taskENTER_CRITICAL();
0210:     i2c_send_data(dev->device,byte);
0211:     // Must set start before byte has written out
0212:     i2c_send_start(dev->device);
0213:     taskEXIT_CRITICAL();
```

Информацию такого рода трудно получить из справочного руководства STM32 (RM0008). Однако внимательное отношение к мелкому шрифту и сноскам иногда может привести к находке золотых самородков. В руководстве говорится:

> In master mode, setting the START bit causes the interface to generate a ReStart condition at the end of the current byte transfer. (В ведущем режиме установка бита START заставляет интерфейс генерировать условие ReStart в конце передачи текущего байта).

Сделав строки 209–213 критическим разделом, вы гарантируете, что запросите еще один «запуск» до завершения текущей записи I2C.

Демопрограмма

Листинг 11.9 иллюстрирует главный цикл демонстрационной программы. По большей части это просто, но есть несколько вещей, которые заслуживают внимания. После настройки устройства I2C в строке 131 внутренний цикл начинается со строки 134. Пока нет ввода с клавиатуры, этот цикл продолжает запись и чтение из микросхемы PCF8574.

Листинг 11.9. Главный цикл демонстрационной программы

```
0116: static void
0117: task1(void *args __attribute__((unused))) {
0118:   uint8_t addr = PCF8574_ADDR(0); // I2C Address
0119:   volatile unsigned line = 0u; // Print line #
0120:   volatile uint16_t value = 0u; // PCF8574P value
0121:   uint8_t byte = 0xFF; // Read I2C byte
0122:   volatile bool read_flag; // True if Interrupted
0123:   I2C_Fails fc; // I2C fail code
0124:
0125:   for (;;) {
```

```
0126:       wait_start();
0127:       usb_puts("\nI2C Demo Begins "
0128:              "(Press any key to stop)\n\n");
0129:
0130:       // Configure I2C1
0131:       i2c_configure(&i2c,I2C1,1000);
0132:
0133:       // Until a key is pressed:
0134:       while ( usb_peek() <= 0 ) {
0135:           if ( (fc = setjmp(i2c_exception)) != I2C_Ok ) {
0136:               // I2C Exception occurred:
0137:               usb_printf("I2C Fail code %d\n\n",
                                    fc,i2c_error(fc));
0138:               break;
0139:       }
0140:
0141:       read_flag = wait_event(); // Interrupt or timeout
0142:
0143:       // Left four bits for input, are set to 1-bits
0144:       // Right four bits for output:
0145:
0146:       value = (value & 0x0F) | 0xF0;
0147:       usb_printf("Writing $%02X "
                       "I2C @ $%02X\n",value,addr);
0148: #if 0
0149:       /*********************************************
0150:       * This example performs a write transaction,
0151:       * followed by a separate read transaction:
0152:       *********************************************/
0153:       i2c_start_addr(&i2c,addr,Write);
0154:       i2c_write(&i2c,value&0x0FF);
0155:       i2c_stop(&i2c);
0156:
0157:       i2c_start_addr(&i2c,addr,Read);
0158:       byte = i2c_read(&i2c,true);
0159:       i2c_stop(&i2c);
0160: #else
0161:       /*********************************************
0162:        * This example performs a write followed
0163:        * immediately by a read in one I2C transaction,
0164:        * using a "Repeated Start"
0165:       *********************************************/
0166:      i2c_start_addr(&i2c,addr,Write);
0167:      i2c_write_restart(&i2c,value&0x0FF,addr);
0168:      byte = i2c_read(&i2c,true);
0169:      i2c_stop(&i2c);
0170: #endif
0171:      if ( read_flag ) {
0172:          // Received an ISR interrupt:
0173           if ( byte & 0b10000000 )
0174:            usb_printf("%04u: BUTTON RELEASED: "
0175:                "$%02X; wrote $%02X, "
                    "ISR %d\n",
0176:                ++line,byte,
```

```
                    value,isr_count);
0177:           else usb_printf("%04u: BUTTON PRESSED: "
0178:                 "$%02X; wrote $%02X, "
                      "ISR %d\n",
0179:                 ++line,byte,
                      value,isr_count);
0180:           } else {
0181:               // No interrupt(s):
0182:               usb_printf("%04u: "
                        "Read: $%02X, "
0183:                   "wrote $%02X, ISR %d\n",
0184:                   ++line,byte,value,isr_count);
0185:           }
0186:           value = (value + 1) & 0x0F;
0187:       }
0188:
0189:       usb_printf("\nPress any key to restart.\n");
0190:   }
0191: }
```

Особо следует отметить функцию `setjmp()` в строке 135. Поскольку в C отсутствует механизм исключений, которым обладает C++, вместо него использовалась функция `longjmp()`. Выполнение `setjmp()` в начальной части цикла позволяет нам выполнить несколько последующих вызовов I2C, каждый из которых имеет свои точки отказа, включая тайм-ауты. Если произойдет какой-либо сбой, функция `longjmp()` вернет управление к строке 135 и вернет ненулевой код сбоя. Отсюда можно сообщить о проблеме и выйти из внутреннего цикла.

Однако механизм `setjmp/longjmp` требует некоторых затрат. Обратите внимание, что переменные `line`, `value` и `read_flag` помечены как `volatile` (строки 119–122). Это было необходимо, чтобы заставить компилятор проигнорировать изменения этих значений в результате вызова `longjmp()`, если это произойдет. Функция `setjmp` сохраняет кучу регистров, а `longjmp` восстанавливает их, чтобы вернуть управление. Любые переменные, все еще кешированные в регистре, будут уничтожены `longjmp`.

В строках 148, 160 и 170 содержатся условные операторы `#if`, `#else` и `#endif` соответственно. При изменении значения в строке 148 с нуля на ненулевое значение все транзакции станут индивидуальными; т. е. байт будет записан в PCF8574P за одну транзакцию, за которой последует совершенно отдельная транзакция I2C для чтения из него.

Если оставить в строке 148 нулевое значение, вы сможете протестировать операцию перезапуска I2C. Строки 166–169 выполняют запись с последующим чтением в той же транзакции.

Демосессия

Выполните сборку с нуля следующим образом:

```
$ make clobber
$ make
```

```
arm-none-eabi-gcc ... -o main.elf
arm-none-eabi-size main.elf
text  data  bss   dec   hex  filename
13024 28   18200 31252  7a14 main.elf
```

Подготовьте устройство к перепрошивке и выполните следующие действия:

```
$ make flash
arm-none-eabi-objcopy -Obinary main.elf main.bin
/usr/local/bin/st-flash write main.bin 0x8000000
...
2017-12-09T21:32:12 INFO src/common.c: Flash written and verified!
jolly good!
```

Теперь подключите USB-кабель и запустите *minicom*, выполнив следующие действия:

```
$ minicom usb
Welcome to minicom 2.7
OPTIONS:
Compiled on Sep 17 2016, 05:53:15.
Port /dev/cu.usbmodemWGDEM1, 21:33:40
Press Meta-Z for help on special keys
Task1 begun.
Press any key to begin.
```

Еще раз: аргумент «usb» для minicom – это имя файла, в котором вы сохранили настройки *minicom*. В этом примере я использовал файл с именем *usb*.

Когда вы увидите сообщение «Task1 begun» (Задача 1 начата), нажмите любую клавишу. Я нажал <**Enter**>. Как только вы это сделаете, устройство I2C должно быть настроено, и вы начнете видеть сообщения следующего вида:

```
I2C Demo Begins (Press any key to stop)
Writing $F0 I2C @ $20
0001:           Read: $F0, wrote $F0, ISR 0
Writing $F1 I2C @ $20
0002:           Read: $F1, wrote $F1, ISR 0
```

Если нажать клавишу еще раз, управление остановится и затем выпадет во внешний цикл. Еще одно нажатие клавиши перезапустит демонстрацию во внутреннем цикле.

Значения, записанные в PCF8574P, будут увеличиваться в младших четырех битах. Если вы подключили светодиоды к P0 и P1, как показано на схеме, вы должны увидеть их обратный отсчет в двоичном виде. Когда вы нажимаете кнопку, вы должны увидеть несколько сообщений, указывающих на события нажатия и отпускания кнопки.

```
Writing $F5 I2C @ $20
0006:           Read: $F5, wrote $F5, ISR 0
Writing $F6 I2C @ $20
0007:           Read: $F6, wrote $F6, ISR 0
Writing $F7 I2C @ $20
0008: BUTTON PRESSED: $77; wrote $F7, ISR 4
Writing $F8 I2C @ $20
0009: BUTTON PRESSED: $78; wrote $F8, ISR 4
Writing $F9 I2C @ $20
0010: BUTTON PRESSED: $79; wrote $F9, ISR 5
Writing $FA I2C @ $20
0011: BUTTON PRESSED: $7A; wrote $FA, ISR 6
Writing $FB I2C @ $20
0012: BUTTON PRESSED: $7B; wrote $FB, ISR 7
Writing $FC I2C @ $20
0013: BUTTON PRESSED: $7C; wrote $FC, ISR 8
Writing $FD I2C @ $20
0014:           Read: $7D, wrote $FD, ISR 8
Writing $FE I2C @ $20
0015: BUTTON RELEASED: $FE; wrote $FE, ISR 11
```

Значение, отображаемое после «ISR», показывает, сколько раз вызывалась процедура прерывания. Моя кнопка была довольно дребезжащей, и без какого-либо устранения дребезга вы видите несколько событий нажатия кнопки. Обратите внимание, что, пока кнопка удерживалась нажатой, старший бит менялся с 1 на 0 (например, $FX изменился на $7X).

Итоги

Эта глава хорошо подготовит вас к работе с I2C. PCF8574 – очень экономичное решение для добавления большего количества портов GPIO, если у вас нет требований к высокой скорости. В то же время это дает вам опыт работы в мире I2C. PCF8574 продемонстрировал, что он может генерировать прерывания, поэтому вам не нужно постоянно опрашивать изменения во входных портах. Это облегчает нагрузку I2C-трафика на шине.

Упражнения

1. Какое значение байта отправляется при чтении с адреса подчиненного устройства $21 (шестнадцатеричное число)?
2. Когда ведущий запрашивает ответ от несуществующего ведомого устройства на шине, как принимается NACK?
3. В чем преимущество линии $\overline{\text{INT}}$ у PCF8574?
4. Что означает квазидвунаправленность в контексте PCF8574?
5. В чем разница между вытекающим и втекающим током?

Глава 12 • Дисплеи OLED

OLED (organic light-emitting diode, органический светоизлучающий диод) предоставляет любителям интереснейшую разновидность недорогого дисплея. Поскольку OLED-дисплеи основаны на светодиодах, они не требуют ни подсветки, ни цветных фильтров, как ЖК-устройства. Это в некоторых случаях соответствует более низкой стоимости.

Устройство на основе OLED, используемое в этой главе, является монохромным, хотя, помимо черного, оно может отображать два цвета. Это звучит противоречиво, но монохромность означает лишь то, что для основного массива пикселей отображается только один цвет. OLED-дисплеи с двумя цветами будут иметь полосу пикселей одного цвета, а остальные – другого[66]. Фон всегда черный (светодиод не горит).

Доступные сегодня устройства имеют небольшие размеры, обычно 128×32 или 128×64 пикселей. Физические размеры также имеют тенденцию быть небольшими. Тем не менее из-за своей низкой стоимости и ярких цветов из них получаются отличные экраны[67]. В этой главе будет продемонстрировано отображение аналогового вольтметра на OLED.

OLED-дисплей

Устройство, которое было мной приобретено на eBay, рекламировалось как «White/Blue/Yellow Blue 0.96 SPI Serial 128×64 OLED LCD LED Display Module S». Будьте осторожны с «I2C» и «SPI» в этом перечне. Многие продавцы смысла этой надписи не понимают.

Важно то, что OLED должен использовать контроллер SSD1306 для управления программным обеспечением. Сам дисплей производства WiseChip имеет номер детали UG-2864HSWEG01, хотя у продавца это может быть и не указано.

В некоторых предложениях может быть указан номер дисплея UG-2864AMBAG01, который значительно отличается и не может использоваться

---

[66] Во избежание путаницы уточним своими словами: монохромные дисплеи могут отображать только один цвет – цвет собственного свечения пикселей (у цветных OLED-дисплеев используются RGB-пиксели с раздельным управлением для каждого цвета). В данном случае (см. далее) автор использует монохромный дисплей, в котором часть площади занята пикселями одного цвета свечения, остальная часть – другого. Вообще-то, это редкая разновидность для специфического применения, гораздо чаще встречаются именно монохромные устройства.

[67] OLED-дисплеи имеют один крупный недостаток: пиксели со временем (в течение одного-трех лет) имеют свойство «выцветать», теряя яркость, хотя и не теряют способность светиться полностью. В этом отношении OLED напоминают осветительные светодиоды в лампочках, которые также постепенно «выгорают», редко когда «перегорая» полностью. В этом отличие этих разновидностей от обычных сигнальных светодиодов, которые могут без каких-либо видимых изменений работать десятилетиями (при условии, конечно, что не превышены предельно допустимые параметры).

с программным обеспечением, описанным в этой главе. Если вы не против заплатить немного больше, Adafruit продает их как «Монохромный 0.96 графический OLED-дисплей с разрешением 128×64». Покупать у них проще, чем пытаться разыскать нужную деталь на eBay[68].

На рис. 12.1 показан OLED-дисплей, который используется в данном примере. Adafruit OLED похож, но задняя сторона печатной платы отличается. Для программного обеспечения этой главы требуется модуль шириной 128 пикселей и высотой 64 пикселя.

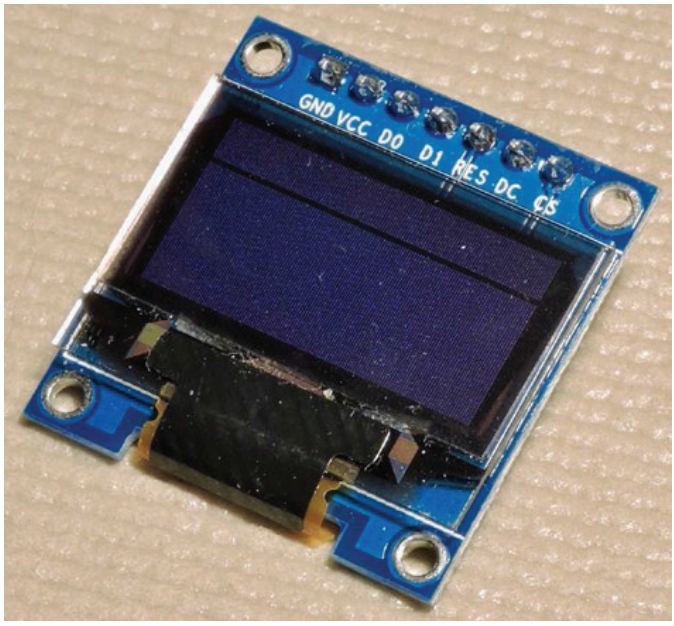


Рис. 12.1. OLED-дисплей с контроллером SSD1306

Конфигурация

Для этой демопрограммы понадобится устройство, настроенное для четырехпроводного SPI. На оборотной стороне устанавливаются резисторы, с помощью которых настраивается устройство (рис. 12.2). Обратите внимание, что R3 и R4 на рисунке установлены, что подтверждает, что это устройство настроено для четырехпроводного SPI. У тех, кто использует устройство Adafruit, должны быть отключены перемычки SJ1 и SJ2.

[68] Встречаются в продаже и другие модели производителя WiseChip (например, UG-2864KSYMG01, UG-2864ASWPG14 и пр.). В отечественных условиях проще разыскать подобный дисплей в местных магазинах или на AliExpress. В любом случае следует ориентироваться на название контроллера дисплея SSD1306, т. е. внимательно смотреть документацию. Хотя, возможно, двухцветного типа именно этого производителя вы и не найдете (наличие двух цветов – все-таки экзотика), но в любом случае совпадение типов контроллера дает гарантию работы от того же программного обеспечения.

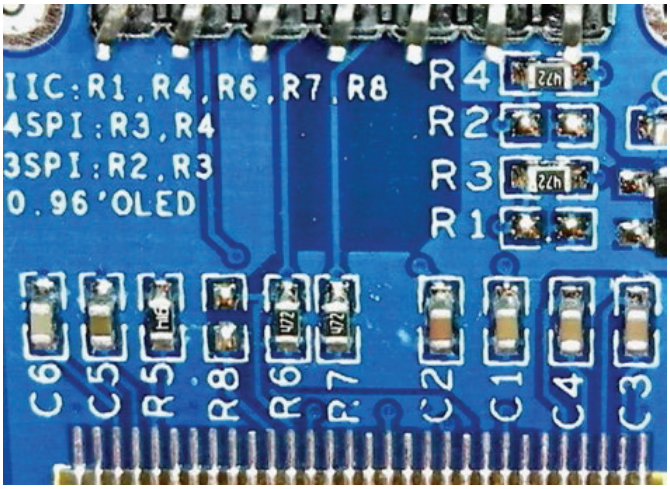


Рис. 12.2. Обратная сторона OLED, иллюстрирующая конфигурационные резисторы от R1 до R8

В табл. 12.1 представлены различные возможные конфигурации. Четырехпроводной SPI подразумевает обычные три сигнала SPI плюс дополнительная линия, указывающая сигнал команды или данных. Эта дополнительная строка имеет низкий уровень, чтобы указать, когда отправляется командный байт, и высокий уровень для посылки данных.

Таблица 12.1. OLED-конфигурации

R1	R2	R3	R4	Конфигурация
+	–	–	+	I2C (не используется в этом примере)
–	–	+	+	Четырехпроводной SPI
–	+	+	–	Трехпроводной SPI (не используется в этом примере)

Подключение дисплея

Плата дисплея имеет семь контактов, перечисленных в табл. 12.2. Соединение Reset является необязательным и должно быть подключено к высокому уровню (неактивно), если оно не используется[69]. Демопрограмма будет использовать GPIO для активации сброса при запуске.

Точное значение потребления тока будет зависеть от нескольких факторов. Adafruit предполагает, что типичный ток может составлять около 20 мА. В ходе собственного тестирования я измерил ток 13.2 мА при всех включенных пикселях, хотя различные варианты конфигурации OLED могут увеличить потребление. Этот уровень достаточно низок, чтобы можно было безопасно питать OLED от стабилизатора +3.3 В на плате контроллера.

[69] Следует отметить, что в большинстве дисплеев (в том числе и OLED-разновидностей) как с последовательными интерфейсами I2C или SPI, так и с параллельным 4/8-битным интерфейсом специальный сигнал сброса отсутствует.

Таблица 12.2. OLED-соединения

OLED-контакт	Функция	Описание
GND	Общий провод	Общий провод (корпус)
VCC	От 3.3 до 5.0 В	Напряжение питания (до 20 мА)
D0 (или SCK)	SCK	Системные такты SPI
D1 (или SDA)	SDIN	SPI MOSI (вход данных OLED)
$\overline{\text{RES}}$	Reset	Сброс (активный низкий уровень)
DC	Данные/Команда	Данные (высокий уровень), команда (низкий уровень)
$\overline{\text{CS}}$	Chip select	Выбор микросхемы (chip select, активный низкий уровень)

Свойства дисплея

Прежде чем изучать демопрограмму, полезно взглянуть на функции OLED-дисплея, которыми она будет управлять. На рис. 12.3 показан OLED автора со всеми включенными пикселями (с использованием команды контроллера 0xA5).



Рис. 12.3. Авторский желтый/синий OLED со всеми включенными пикселями

Хотя это желто-синий OLED-дисплей, он монохромный. Желтый цвет вы получите только в верхних 16 рядах. После разрыва в один ряд ниже него находится 48 рядов синих пикселей. В одноцветных устройствах этот зазор может отсутствовать. Тщательно выбирайте дисплей для своего применения.

Когда все пиксели были включены, мой OLED показал ток 13.3 мА.

Схема демонстрационного примера

Схема демонстрационного примера использует то же соединение SPI, которое мы использовали в проекте Winbond (глава 8), но задействует несколько дополнительных линий управления для устройства OLED. В этой демонстрации по-прежнему используется SPI1 для контроллера SPI, но используется альтернативная конфигурация GPIO, которая будет описана далее. PA15 действует как $\overline{\text{NSS}}$, который будет управлять выводом OLED выбора микросхемы $\overline{\text{CS}}$. PB10 будет сигнализировать OLED о том, отправляются ли команды или данные. Наконец, PB11 можно активировать для сброса OLED-дисплея при запуске демонстрационной программы. Рисунок 12.4 иллюстрирует демосхему.

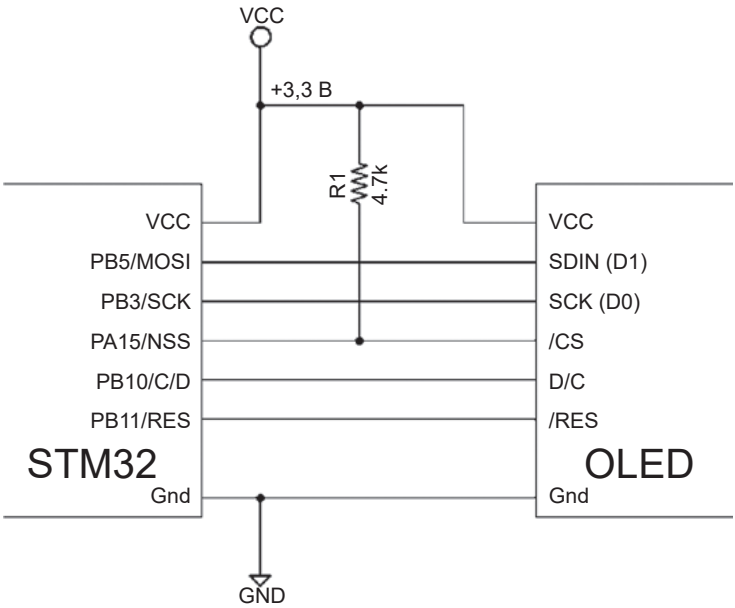


Рис. 12.4. Демонстрационная схема OLED с использованием SPI

Чтобы сброс был эффективным, вывод $\overline{\text{RES}}$ OLED, который подключен к контроллеру SSD1306, должен оставаться в низком уровне в течение как минимум 3 мкс. Сброс устанавливает контроллер в несколько режимов по умолчанию, что позволяет сэкономить часть инициализации.

AFIO

Платформа STM32 поддерживает концепцию переназначения (ремаппинга) функций контактов ввода-вывода GPIO. В их документации это называется «Alternate Function I/O» (Альтернативная функция ввода-вывода). В демонстрационной версии в этой главе возможности AFIO используются для отображения SPI1 на GPIO PA15, PB3, PB4 и PB5. В табл. 12.3 перечислены параметры AFIO для SPI1.

Таблица 12.3. Альтернативные функции ввода-вывода для SPI1

Альтернативная функция	SPI1_REMAP=0	SPI1_REMAP=1
SPI1_NSS	PA4	PA15
SPI1_SCK	PA5	PB3
SPI1_MISO	PA6	PB4
SPI1_MOSI	PA7	PB5

Функция AFIO обеспечивает дополнительную гибкость при планировании ресурсов STM32. Если вам нужны входы, устойчивые к напряжению 5 В, вы можете использовать SPI1_REMAP=1. Иногда AFIO используется, чтобы избежать конфликта с контактами, используемыми другим периферийным устройством.

Чтобы воспользоваться AFIO, вам необходимо выполнить следующее.

1. Включить источник тактовой частоты AFIO.
2. Настроить альтернативную функцию ALTFN.
3. Настроить выходы GPIO для ALTFN. Входы не требуют специальных действий, кроме настройки в качестве входа.

Все эти три шага имеют важное значение. Например, если вы забудете включить такты AFIO – или ничего не произойдет, или периферийное устройство зависнет. С использованием библиотеки libopencm3 такты AFIO включаются следующим образом:

```
rcc_periph_clock_enable(RCC_AFIO);
```

Демопрограмма использует следующий вызов для выбора альтернативной функции SPI1 с помощью функции gpio_primary_remap() библиотеки libopencm3:

```
// Put SPI1 on PB5/PB4/PB3/PA15
gpio_primary_remap(
    AFIO_MAPR_SWJ_CFG_JTAG_OFF_SW_OFF, // Optional
    AFIO_MAPR_SPI1_REMAP); // SPI1_REMAP=1
```

Первый аргумент отключает функциональность JTAG и является второстепенным по отношению к нашей цели переназначения. Второй аргумент указывает, что вы хотите переназначить SPI1 (SPI1_REMAP=1 в табл. 12.3). Естественное сопоставление (SPI1_REMAP=0) используется по умолчанию после перезагрузки системы.

Для выходов GPIO при его настройке необходимо выбрать одну из следующих настроек. В противном случае периферийное устройство не сможет достичь выходных контактов.

- GPIO_CNF_OUTPUT_ALTFN_PUSHPULL.
- GPIO_CNF_OUTPUT_ALTFN_OPENDRAIN.

Например:

```
gpio_set_mode(
    GPIOB,
```

```
    GPIO_MODE_OUTPUT_50_MHZ,
    GPIO_CNF_OUTPUT_ALTFN_PUSHPULL, // Note!
    GPIO5|GPIO3);
```

Обратите внимание на `ALTFN` в имени третьего аргумента. Легко допустить ошибку – вместо этого использовать аналогичное имя, не содержащее `ALTFN`, а затем задаться вопросом, почему ваше периферийное устройство не взаимодействует с выводом.

Графика

Одним из препятствий при работе с графическими устройствами является выполнение таких операций, как рисование линий, кругов и прямоугольников. Это правда, что линии и прямоугольники достаточно просты, если в них используются идеально горизонтальные и вертикальные линии. Но линии, наклоненные под углом, и круги (заполненные и незаполненные) представляют собой проблему. Кроме того, необходимы шрифты для надписей.

Эти программные проблемы настолько велики, что среднестатистический разработчик не хочет тратить время на разработку решений. Ведь это проблемы, которые были решены раньше. Почему мы должны продолжать решать их снова?

Хорошей новостью является то, что проблема уже была решена и что программное обеспечение доступно в форме с открытым исходным кодом. В демонстрационном проекте в этой главе будет использоваться графическое программное обеспечение, написанное Ахимом Дёблером (Achim Döbler) и доступное на GitHub по адресу: https://github.com/achimdoebler/UGUI.

Особенностью этого графического пакета, за который я выражаю благодарность автору, заключается в том, что он предназначен для адаптации к любой графической платформе. Помимо простой настройки в файле *ugui_config.h*, единственным требованием является наличие функции, предоставляемой пользователем:

```
void
local_draw_point(UG_S16 x,UG_S16 y,UG_COLOR c) {
...
}
```

Учитывая координаты x и y и цвет, эта функция вызывается для рисования точки в вашей собственной графической среде. Чтобы это работало, среда uGUI просто инициализируется указателем на функцию:

```
static UG_GUI gui;
...
UG_Init(&gui,local_draw_point,128,64);
```

Аргументы 128 и 64 в этом примере определяют максимальную ширину и высоту холста для рисования. Как только это будет сделано, можно вызвать функции uGUI для заполнения круга, например:

```
UG_FillCircle(x,y,c);
```

Демопроект находится в следующем каталоге:

```
$ cd ~/stm32f103c8t6/rtos/oled
```

Однако наше OLED-устройство является монохромным, поэтому требуется особая обработка цвета. Для перевода цвета в монохромный в файле *meter.c* предусмотрена следующая процедура:

```
0059: static int
0060: ug_to_pen(UG_COLOR c) {
0061:
0062:   switch ( c ) {
0063:   case C_BLACK:
0064:       return 0;
0065:   case C_RED:
0066:       return 2;
0067:   default:
0068:       return 1;
0069:   }
0070: }
```

Эта функция просто преобразует любой цвет, кроме красного и черного, в 1 (белый), при этом черный обозначается как 0. Красный цвет (2) используется демонстрационным программным обеспечением для операции «исключающее ИЛИ».

Операция «исключающее ИЛИ» имеет особое свойство: если пиксель в данный момент имеет значение 0 (черный), он будет окрашен в белый цвет. Если текущий пиксель белый, то он преобразуется в черный. Независимо от текущего состояния графического холста что-то всегда отображается в режиме «исключающее ИЛИ».



В зависимости от вашего компилятора и используемых опций вы можете потратить время на уменьшение объема компилируемого кода. Используйте `#if`, чтобы исключить неиспользуемые функции в модуле *ugui.c*.

Растровое изображение

Для облегчения графического рисования на OLED используется буфер растрового изображения, или, иначе, карты пикселей (pixmap). Это позволяет выполнять обширные операции рисования на полной скорости процессора. В подходящее время растровое изображение копируется на устройство OLED для отображения.

Растровое изображение определяется в файле *meter.c* следующим образом:

```
static uint8_t pixmap[128*64/8];
```

Это определяет 128 раз по 64 пикселя, по восемь пикселей на байт, таким образом, требуется 1024 байта SRAM.

Чтобы облегчить рисование в растровом изображении, используется функция `to_pixel()`, показанная в листинге 12.1. Он вычисляет адрес байта в растровом изображении на основе заданных координат x и y, а затем возвращает номер бита через аргумент указателя `bitno`.

Листинг 12.1. Функция `to_pixel()`

```
0020: static uint8_t dummy;
0021:
0022: static uint8_t *
0023: to_pixel(short x,short y,unsigned *bitno) {
0024:   *bitno = 7 - y % 8; // Inverted
0025:
0026:   if ( x < 0 || x >= 128
0027:     || y < 0 || y >= 64 )
0028:         return &dummy;
0029:
0030:   unsigned inv_y = 63 - y;
0031:   unsigned pageno = inv_y / 8;
0032:   unsigned colno = x % 128;
0033:
0034:   return &pixmap[pageno * 128 + colno];
0035: }
```

Несколько моментов требуют пояснений. Строки 24 и 30 используются для инвертирования изображения. Это было сделано для того, чтобы желтая полоса из 16 рядов отображалась в верхней части дисплея. Чтобы использовать неинвертированные координаты, вы должны изменить строку 24:

вместо

```
0024: *bitno = 7 - y % 8; // Inverted
```

записать

```
0024: *bitno = y % 8; // Non-inverted
```

Аналогично вместо вычисленного значения `inv_y` будет использоваться `y`. Централизация этого сопоставления в одном месте позволяет выводить на дисплей преобразованные картинки. Например, вы можете переработать эту функцию, чтобы преобразовать x и y для отображения на устройстве в портретном режиме, а не в альбомном.

Теоретически не должно быть вызова этой процедуры, если координаты x и y выходят за пределы допустимого диапазона. Но если это произойдет, подпрограмма вернет указатель на фиктивное значение, чтобы вызов можно было игнорировать без фатальных последствий.

Запись растра

После того как номера байтов и битов определены функцией `to_pixel()`, сама функция рисования точек, как показано в листинге 12.2, становится проще. Функция `draw_point()` вызывается более ранней функцией `local_draw_point()`. Процедура `draw_point()` ожидает значение `pen`, равное 2, 1 или 0, а не цвет.

Листинг 12.2. Внутренняя функция `draw_point()`

```
0037: static void
0038: draw_point(short x,short y,short pen) {
0039:
0040:   if ( x < 0 || x >= 128 || y < 0 || y >= 64 )
0041:       return;
0042:
0043:   unsigned bitno;
0044:   uint8_t *byte = to_pixel(x,y,&bitno);
0045:   uint8_t mask = 1 << bitno;
0046:
0047:   switch ( pen ) {
0048:   case 0:
0049:       *byte &= ~mask;
0050:       break;
0051:   case 1:
0052:       *byte |= mask;
0053:       break;
0054:   default:
0055:       *byte ^= mask;
0056:   }
0057: }
```

Строки 40 и 41 завершают функцию, ничего не делая, если координаты x и/или y выходят за пределы допустимого диапазона. В противном случае строки 43 и 44 определяют адрес байта и номер бита для изменяемого пикселя. Строка 45 вычисляет битовую маску на основе значения `bitno` и сохраняет ее в переменной `mask`.

Что произойдет дальше, зависит от значения пера `pen`. Если `pen` было равно 0 (белое), этот бит маскируется, и бит пикселя очищается до нуля. Если пиксель равен 1, значение `mask` объединяется с байтом для получения 1-битного пикселя. Наконец, в строке 55 значение `pen` по умолчанию (обычно 2) вместо этого создаст «исключающее ИЛИ» пикселя.

Программное обеспечение изображения вольтметра

Графическое программное обеспечение, предназначенное для изображения вольтметра на дисплее, находится в файле *meter.c*. Те, кто интересует-

ся дизайном этой программы, могут изучить исходный код для получения подробной информации. Для краткости я просто выделю важные функции внутри него.

meter_init()

```
void meter_init(struct Meter *m,float range);
```

Если бы это был C++, вы могли бы рассматривать функцию `meter_init()` как конструктор. Структура `Meter` инициализируется вызовом, а диапазон аргументов с плавающей запятой настраивает верхний предел диапазона измерений. В демонстрационной программе *main.c* верхний предел указан как 3.5 для питания 3.5 В.

meter_set_value()

```
void meter_set_value(struct Meter *m,float v);
```

Эта функция изменяет значение, хранящееся в объекте измерений `m`, на значение `v`. Она переместит графический указатель (стрелку) в растровом изображении.

meter_redraw()

```
void meter_redraw(struct Meter *m);
```

Эта функция используется внутри во время инициализации для отображения всего вольтметра на растровом изображении. Его можно вызвать еще раз, если программное обеспечение подозревает или знает, что изображение каким-либо образом повреждено. В демоверсии функция вызывается только один раз при инициализации.

meter_update()

Это функция, используемая для передачи растрового изображения из SRAM на OLED с использованием SPI1:

```
void meter_update(void);
```

Код передачи через SPI показан в листинге 12.3.

Листинг 12.3. Функция `meter_update()` передачи по SPI

```
0195: void
0196: meter_update(void) {
0197:   uint8_t *pp = pixmap;
0198:
0199:   oled_command2(0x20,0x02);// Page mode
0200:   oled_command(0x40);
```

```
0201:   oled_command2(0xD3,0x00);
0202:   for ( uint8_t px=0; px<8; ++px ) {
0203:       oled_command(0xB0|px);
0204:       oled_command(0x00); // Lo col
0205:       oled_command(0x10); // Hi col
0206:       for ( unsigned bx=0; bx<128; ++bx )
0207:       oled_data(*pp++);
0208:   }
0209: }
```

Строка 197 получает адрес первого байта растрового изображения. Строка 199 гарантирует, что контроллер SSD1306 находится в «страничном режиме». В этом режиме память OLED разбита на восемь страниц по 128 байт пикселей.

Строка 200 инициализирует SSD1306 для запуска с нулевой строки дисплея, а строка 201 инициализирует SSD1306 для установки смещения горизонтальной координаты дисплея на ноль.

Затем цикл в строках 202–208 обеспечивает постраничную передачу данных на OLED. Строка 203 выбирает OLED-страницу для обновления. Строки 204 и 205 инициализируют индекс горизонтальной координаты равным нулю. Линии 206 и 207 фактически передают данные контроллеру OLED и обновляют указатель `pp` данных пикселей дисплея.

Функции `oled_command()`, `oled_command2()` и `oled_data()` находятся в главном демонстрационном модуле *main.c*.

Главный модуль

Поскольку модуль OLED требует специальной линии данных/команд, давайте рассмотрим функции, используемые программой вольтметра.

oled_command()

Эта функция используется для отправки командных байтов на OLED-контроллер и показана в листинге 12.4.

Листинг 12.4. Функция `oled_command()`

```
0034: void
0035: oled_command(uint8_t byte) {
0036:   gpio_clear(GPIOB,GPIO10);
0037:   spi_enable(SPI1);
0038:   spi_xfer(SPI1,byte);
0039:   spi_disable(SPI1);
0040: }
```

Строка 36 очищает GPIO PB10, так что в строке данных/команды устанавливается низкий уровень, указывая контроллеру OLED, что данные SPI должны интерпретироваться как командные байты. Строки 37–39 передают этот командный байт через SPI1.

Функция `oled_command2()` идентична, за исключением того, что она отправляет два байта команды вместо одного.

oled_data()

Функция `oled_data()` очень похожа на `oled_command()`. Она просто устанавливает высокий уровень на линии GPIO PB10 (строка 53 листинга 12.5), чтобы контроллер OLED принимал данные SPI как значения пикселей.

Листинг 12.5. Функция `oled_data()`

```
0051: void
0052: oled_data(uint8_t byte) {
0053:   gpio_set(GPIOB,GPIO10);
0054:   spi_enable(SPI1);
0055:   spi_xfer(SPI1,byte);
0056:   spi_disable(SPI1);
0057: }
```

oled_reset()

Главный модуль вызывает функцию `oled_reset()` для инициализации OLED-контроллера, как показано в листинге 12-6.

Листинг 12.6. Функция oled_reset()

```
0059: static void
0060: oled_reset(void) {
0061:   gpio_clear(GPIOB,GPIO11);
0062:   vTaskDelay(1);
0063:   gpio_set(GPIOB,GPIO11);
0064: }
```

В строке 61 PB11 устанавливается на низкий уровень. Затем в строке 62 на один такт (около 1 мс) вызывается подпрограмма FreeRTOS vTaskDelay(), чего должно быть более чем достаточно (требуется минимум 3 мкс). Затем после задержки на выводе PB11 снова устанавливается высокий уровень.

oled_init()

Функция `oled_init()` показана в листинге 12.7. Строки 73 и 77 не являются обязательными, они просто активируют встроенный светодиод на PC13. OLED сбрасывается в строке 74, после чего к нему отправляются несколько команд из массива `cmds` (строка 68) в цикле в строках 75 и 76.

Листинг 12.7. Функция `oled_init()`

```
0066: static void
0067: oled_init(void) {
0068:   static uint8_t cmds[] = {
0069:       0xAE, 0x00, 0x10, 0x40, 0x81, 0xCF, 0xA1, 0xA6,
0070:       0xA8, 0x3F, 0xD3, 0x00, 0xD5, 0x80, 0xD9, 0xF1,
```

```
0071:       0xDA, 0x12, 0xDB, 0x40, 0x8D, 0x14, 0xAF, 0xFF };
0072:
0073:   gpio_clear(GPIOC,GPIO13);
0074:   oled_reset();
0075:   for ( unsigned ux=0; cmds[ux] != 0xFF; ++ux )
0076:        oled_command(cmds[ux]);
0077:   gpio_set(GPIOC,GPIO13);
0078: }
```

Демопрограмма

В каталоге проекта выполните следующее:

```
$ make clobber
$ make
$ make flash
```

После того как STM32 прошит и подключен в соответствии со схемой, показанной на рис. 12.4, вы сможете отключить программатор и подключить USB-кабель к устройству STM32. Если все нормально работает, после небольшой паузы вы должны увидеть экран, показанный на рис. 12.5. В зависимости от вашего устройства вы можете видеть разные цвета.



Рис. 12.5. Демонстрационная программа создает графику вольтметра на OLED

Если при разработке нового проекта компоновщик сообщает, что *.bss* не помещается в область оперативной памяти, проверьте значение размера кучи `configTOTAL_HEAP_SIZE` в файле *FreeRTOSConfig.h*. Возможно, вам придется уменьшить размер кучи, чтобы освободить место для хранения вашей программы.

Если дисплей инициализировался неправильно, лучше сразу отключить USB-кабель и перепроверить соединения. В случае успеха запустите *minicom* с настройками запуска USB (мой файл называется «usb»):

```
$ minicom usb
```

После того как *minicom* подключится к вашему USB-устройству и запустится, вы должны увидеть экран сеанса, подобный следующему:

```
Welcome to minicom 2.7
OPTIONS:
Compiled on Sep 17 2016, 05:53:15.
Port /dev/cu.usbmodemWGDEM1, 22:46:21
Press Meta-Z for help on special keys
Press the Return key to prompt a menu display from the demo program:
Test Menu:
0 .. set to 0.0 volts
1 .. set to 1.0 volts
2 .. set to 2.0 volts
3 .. set to 3.0 volts
4 .. set to 3.5 volts
+ .. increase by 0.1 volts
- .. decrease by 0.1 volts
: _
```

Нажатие «1» должно немедленно привести к тому, что индикатор (OLED) отобразит 1.0 В. Аналогично нажатие «3» вызовет сдвиг к положению 3 В. Нажатие клавиши «+» или «–» позволит соответственно увеличить/уменьшить отображаемое напряжение на десятую долю вольта.

Итоги

В этой главе SPI применялся к реальной проблеме управления OLED-дисплеем. При этом было продемонстрировано преимущество использования программного обеспечения с открытым исходным кодом для графических операций. Графика позволила рисовать аналоговый измеритель на OLED, а также использовать шрифт для цифрового отображения напряжения.

Кроме того, было продемонстрировано, как сигналы Data/Command и Reset управляют OLED-дисплеем, помимо обычных сигналов SPI.

Также применялась в этой главе концепция переназначения выводов AFIO для семейства STM32, чтобы продемонстрировать, как SPI1 может перемещать функции ввода-вывода на различные выводы. AFIO обеспечивает большую гибкость в использовании ресурсов микросхемы STM32.

Упражнения

1. Какие макроопределения необходимо использовать для конфигурации выходных контактов AFIO?
2. Какая тактовая частота должна быть задана при изменении AFIO?
3. Какие макроопределения конфигурации следует использовать для входных контактов GPIO?
4. Какова цель входа DC у платы OLED?

Глава 13 • OLED с использованием DMA

В предыдущей главе было разработано программное обеспечение для управления OLED с использованием передачи по SPI в основном режиме. Однако платформа STM32 поддерживает контроллер DMA (direct memory access, прямой доступ к памяти), который можно использовать для выполнения операций ввода-вывода и оставить больше времени доступным для процессора. В этой главе будет описано, как настроить и использовать контроллер DMA для управления OLED-устройством.

Проблемы

Этот проект будет для нас непростым испытанием, поскольку наше OLED-устройство требует особого обращения. Основные проблемы заключаются в следующем:

- контроллер OLED SSD1306 позволяет нам обновлять только одну из восьми страниц одновременно, что требует нескольких передач DMA;
- между отправленными страницами данных необходимо отправить дополнительные команды контроллера SSD1306 для выбора следующей страницы для обновления;
- переключение между командами OLED и данными требует от нас изменения уровня сигнала DC между передачами, что не может быть интегрировано с самой операцией DMA.

В некоторых приложениях есть возможность настроить контроллер DMA и просто запустить его. Затем контроллер DMA опционально прерыванием уведомляет нас о завершении операций. В этом проекте DMA будет запускаться несколько раз для обновления восьми страниц данных памяти OLED. Проект позволит вам узнать, как справиться с этой задачей.

Схема

Схема, используемая в проекте этой главы, идентична схеме, использованной в главе 12 (см. рис. 12.4). Изменения в проекте касаются только используемого программного обеспечения.

Операция прямого доступа к памяти

Контроллер DMA представляет собой простую машину, работающую в три этапа.

1. Первоначальная настройка.
2. Исполнение.
3. Уведомление о завершении или повторном выполнении.

Цель DMA – прочитать данные из источника и записать их в пункт назначения. Как именно это выполняется, зависит от его конфигурации.

Выполнение прямого доступа к памяти

Как контроллер DMA управляет автоматической передачей данных? Проект, представленный в этой главе, настроит контроллер DMA для чтения данных из одной из двух областей памяти:

- массива командных байтов OLED,
- массива байтов данных пикселей.

Таким образом, что касается контроллера DMA, источником данных будет память. При настройке контроллера программное обеспечение настроит адрес и длину байта.

Приемником DMA будет порт данных SPI для передачи на OLED (периферийное устройство GPIO). Таким образом, пунктом назначения DMA при настройке будет отображенный в памяти адрес порта для регистра данных SPI1 (`&SPI1_DR` в терминах языка C).

Еще остается вопрос о том, когда передается данный байт данных. Цикл передачи DMA состоит из следующей серии событий.

1. На контроллер DMA отправляется сигнал запроса.
2. Контроллер DMA выполняет передачу данных (в данном случае из памяти на периферию). Настроенный приоритет будет определять, какая передача произойдет первой.
3. Контроллер DMA отправляет запросчику сигнал подтверждения.
4. Выдается сигнал запроса.
5. Выдается сигнал подтверждения DMA.
6. Контроллер DMA уменьшает счетчик длины данных.
7. При наличии соответствующей настройки увеличивается адрес источника или приемника. Адрес увеличивается на размер (в байтах) переданных данных.

Этот процесс повторяется до тех пор, пока счетчик длины данных не достигнет нуля. В этот момент контроллер DMA достигает состояния завершения, которое при соответствующей настройке будет включать прерывание завершения.

Сигналы запроса DMA

Внутри микроконтроллера STM32 находятся линии сигналов запроса, подключенные к каналам DMA. Микроконтроллер STM32F103C8T6 представляет собой контроллер средней комплектации и имеет только один контроллер DMA (DMA1). DMA1 поддерживает семь каналов DMA, а более крупные микроконтроллеры оснащены вторым контроллером, который поддерживает еще пять каналов.

В табл. 13.1 приведены каналы DMA, поддерживаемые STM32F103C8T6. Наш проект будет использовать канал 3 DMA1, поскольку он поддерживает линию запроса SPI1_TX.

Таблица 13.1. Поддерживаемые каналы DMA1

Источник запроса	Канал	Описание
ADC1 TIM2_CH3 TIM4_CH1	1	DMA1, канал 1 (высший приоритет)
USART3_TX TIM1_CH1 TIM2_UP TIM3_CH3 SPI1_RX	2	DMA1, канал 2
USART3_RX TIM1_CH2 TIM3_CH4 TIM3_UP SPI1_TX	3	DMA1, канал 3
USART1_TX TIM1_CH4 TIM1_TRIG TIM1_COM TIM1_CH2 SPI2/I2S2_RX I2C2_TX	4	DMA1, канал 4
USART1_RX TIM1_UP SPI2/I2C2_TX TIM2_CH1 TIM4_CH3 I2C2_RX	5	DMA1, канал 5
USART2_RX TIM1_CH3 TIM3_CH1 TIM3_TRIG I2C1_TX	6	DMA1, канал 6
USART2_TX TIM2_CH2 TIM2_CH4 TIM4_UP I2C1_RX	7	DMA1, канал 7 (самый низкий приоритет)

Из групп источников запроса в табл. 13.1 видно, что на канал 3, помимо SPI1_TX, могут действовать и другие сигналы:

- USART3_RX,
- TIM1_CH2,
- TIM3_CH4,
- TIM3_UP.

Одновременно может быть активен только один из этих источников запроса. Приложение, которому необходимо использовать SPI1_TX и USART3_RX, должно организовать это так, чтобы в данный момент контроллер DMA был настроен только для одного из них.

Каждый канал имеет настроенный приоритет из четырех уровней. Однако, если конкурирующие каналы имеют одинаковый приоритет, приоритет имеет канал с наименьшим номером.

Таблицу 13.1 можно рассматривать как таблицу соединений между периферийными устройствами и линиями запроса контроллера DMA. Например, SPI1 может запрашивать передачу по каналу 3, в то время как его запросы на прием передаются по каналу 2. SPI2 жестко запрограммирован на запрос по каналам 4 и 5.

Перенос из памяти в память также может выполняться контроллером DMA с источником и пунктом назначения на любом доступном канале.

Запрос SPI1_TX

Наш проект будет использовать запрос SPI1_TX, доступный на канале DMA 3. Эта строка запроса активна, когда выполняются следующие условия:

- флаг `TXE` в регистре состояния SPI1 `SPI_SR` установлен в 1 (буфер передачи пуст);
- флаг `TXDMAEN` регистра управления SPI1 `SPI_CR2` установлен в 1 (DMA включен);
- само периферийное устройство SPI1 включено (бит `SPE` в регистре `SPI_CR1` установлен в 1).

Предполагая, что остальные аспекты конфигурации SPI1 настроены верно, выполнение перечисленных трех условий активирует линию запроса DMA. Чтобы контроллер DMA отреагировал на это, также должны быть выполнены следующие условия:

- установлена одноразовая конфигурация DMA;
- канал DMA включен.

Как только эти условия будут установлены как на периферийном устройстве SPI1, так и на контроллере DMA, работа DMA продолжится без вмешательства программного обеспечения.

Демопрограмма

Знакомство с используемым программным обеспечением поможет сфокусировать внимание на этих концепциях. Исходный код этой главы находится в следующем каталоге:

```
$ cd ~/stm32f103c8t6/rtos/oled_dma
```

Перейдите в этот каталог и перестройте проект с нуля:

```
$ make clobber
$ make
$ make flash
```



Обычно необходимо переставить перемычку Boot0 для прошивки устройства, если предыдущая прошивка настроила AFIO. Установите Boot0 = 1, оставьте Boot1= 0 и затем прошивайте. После этого верните Boot0 = 0.

В листинге 13.1 суммированы некоторые изменения, внесенные в программу `main()` из исходного кода предыдущей главы. Строка 403 влияет на скорость передачи данных ввода-вывода SPI. Если делитель установлен на 64, частота SPI SCLK увеличивается до 1.125 МГц. Если у вас возникли проблемы с работой схемы, увеличьте делитель до 256. Макетная плата может быть очень шумной и ограничивать производительность.

Листинг 13.1. Основные изменения программы

```
0401:   spi_init_master(
0402:           SPI1,
0403:           SPI_CR1_BAUDRATE_FPCLK_DIV_64, // 1.125 MHz
0404:           SPI_CR1_CPOL_CLK_TO_0_WHEN_IDLE,
0405:           SPI_CR1_CPHA_CLK_TRANSITION_1,
0406:           SPI_CR1_DFF_8BIT,
0407:           SPI_CR1_MSBFIRST
0408:   );
...
0412:   // DMA
0413:   rcc_periph_clock_enable(RCC_DMA1);
0414:   nvic_set_priority(NVIC_DMA1_CHANNEL3_IRQ,0);
0415:   nvic_enable_irq(NVIC_DMA1_CHANNEL3_IRQ);
...
0422:   xTaskCreate(spidma_task,"spi_dma",100,NULL,1,&h_spidma);
```

Для работы DMA1 строка 413 включает системные такты. Линии 414 и 415 настраивают NVIC (встроенный контроллер векторов прерываний), чтобы обеспечить генерацию прерывания завершения операции канала 3 DMA1.

Наконец, строка 422 создает еще одну задачу FreeRTOS с именем `spidma_task()` для организации необходимых передач DMA.

Листинг 13.2 иллюстрирует небольшое изменение, внесенное в модуль *meter.c*. Оно просто вызывает модуль *main.c* для выдачи запроса на обновление OLED в строке 199.

Листинг 13.2. Модификация *meter.c*

```
0197: void
0198: meter_update(void) {
0199:   spi_dma_xmit_pixmap();
0200: }
```

В листинге 13.4 (см. далее) показана функция `spi_dma_xmit_pixmap()`, которая запускает ввод-вывод в OLED путем DMA.

Инициализация DMA

Для настройки контроллера DMA перед использованием требуется достаточное количество программного обеспечения. Хорошей новостью является то, что многое из этого нужно сделать только один раз. В листинге 13.3 показана одноразовая конфигурация DMA, используемая демонстрационной программой.

Листинг 13.3. Одноразовая инициализация DMA

```
0189: static void
0190: dma_init(void) {
0191:
0192:   dma_channel_reset(DMA1,DMA_CHANNEL3);
0193:   dma_set_peripheral_address(DMA1,DMA_CHANNEL3,
            (uint32_t)&SPI1_DR);
0194:   dma_set_read_from_memory(DMA1,DMA_CHANNEL3);
0195:   dma_enable_memory_increment_mode(DMA1,DMA_CHANNEL3);
0196:   dma_set_peripheral_size(DMA1,DMA_CHANNEL3,
            DMA_CCR_PSIZE_8BIT);
0197:   dma_set_memory_size(DMA1,DMA_CHANNEL3,
            DMA_CCR_MSIZE_8BIT);
0198:   dma_set_priority(DMA1,DMA_CHANNEL3,DMA_CCR_PL_HIGH);
0199:   dma_enable_transfer_complete_interrupt
            (DMA1,DMA_CHANNEL3);
0200: }
```

Строка 192 сбрасывает контроллер DMA1. Это очищает контроллер от любых неисправностей и устанавливает ряд удобных значений по умолчанию. В строке 193 указывается адрес периферии `SPI1_DR` (регистр данных SPI1). Строка 194 указывает, что контроллер будет выполнять чтение из памяти для канала 3 (таким образом, пунктом назначения будет указанное периферийное устройство). Строка 195 настраивает контроллер DMA на увеличение адреса памяти после передачи каждого байта. Строка 196 указывает, что единица измерения размера – это байт для периферийного устройства, а следующая строка делает то же самое для стороны памяти. Строка 198 присваивает каналу DMA 3 высокий приоритет. Строка 199 включает уведомления о завершении передачи DMA по прерыванию.

На этом этапе контроллер DMA1 готов к действию, и ему нужно всего лишь несколько дополнительных деталей, прежде чем он сможет заработать.

Запуск прямого доступа к памяти

На первом этапе запуска обновления OLED-дисплея с помощью DMA используется процедура `spi_dma_xmit_pixmap()` из модуля *main.c*, в сокращенной форме показанная в листинге 13.4. Когда DMA запускается в первый раз, эта процедура вызывает функцию `start_dma()`. Мы обсудим полную логику этой процедуры позже.

Листинг 13.4. Запуск передачи DMA

```
0156: void
0157: spi_dma_xmit_pixmap(void) {
...
0169:   if ( prime )
0170:      start_dma(); // Start from idle
0171: }
```

Код `start_dma()` представлен в листинге 13.5. Первое, что он делает, – это сбрасывает значение `pageno` OLED обратно в ноль (строка 146) и сохраняет начало буфера растрового изображения OLED в переменной указателя `pixmapp` (строка 147).

Для первого ввода-вывода SPI требуются командные байты, поэтому для GPIO PB10 устанавливается низкий уровень, чтобы указать контроллеру OLED, что следующие данные являются командными байтами (строка 148). Наконец, `spidma_task()` «запускается» в строке 149.

Листинг 13.5. Запуск задачи `spidma` для начала новой передачи DMA

```
0040: static TaskHandle_t h_spidma = NULL;
...
0044: static volatile uint8_t *pixmapp = NULL;
0045: static volatile uint8_t pageno = 0;
...
0142: static void
0143: start_dma(void) {
0144:   extern uint8_t pixmap[128*64/8];
0145:
0146:   pageno = 0;
0147:   pixmapp = &pixmap[0];
0148:   gpio_clear(GPIOB,GPIO10); // Cmd mode
0149:   xTaskNotifyGive(h_spidma);
0150: }
```

Задача управления OLED SPI/DMA

Управление передачей ввода-вывода OLED DMA сложное, поскольку мы должны разбить процедуру обновления на восемь обновлений OLED-страниц, каждое из которых требует своего собственного набора байтов команд и данных. Задача `spidma_task()` показана в листинге 13.6.

Листинг 13.6. Задача управления обновлениями OLED DMA `spidma_task()`

```
0041: static volatile bool dma_busy = false;
0042: static volatile bool dma_idle = true;
0043: static volatile bool dma_more = false;
0044: static volatile uint8_t *pixmapp = NULL;
0045: static volatile uint8_t pageno = 0;
```

```
...
0088: static void
0089: spidma_task(void *arg __attribute((unused))) {
0090:   static uint8_t cmds[] = {
0091:       0x20, 0x02, // 0: Page mode
0092:       0x40, // 2: Display start line
0093:       0xD3, 0x00, // 3: Display offset
0094:       0xB0, // 5: Page #
0095:       0x00, // 6: Lo col
0096:       0x10 // 7: Hi Col
0097:   };
0098:
0099:   for (;;) {
0100:       // Block until ISR notifies
0101:       ulTaskNotifyTake(pdTRUE,portMAX_DELAY);
0102:       if ( dma_busy ) {
0103:           spi_clean_disable(SPI1);
0104:           dma_busy = false;
0105:           if ( gpio_get(GPIOB,GPIO10) ) {
0106:               // Advance data
0107:               pixmapp += 128;
0108:               ++pageno;
0109:           }
0110:           // Toggle between Command/Data
0111:           gpio_toggle(GPIOB,GPIO10);
0112:       }
0113:
0114:       if ( pageno >= 8 ) {
0115:           // All OLED pages sent:
0116:           dma_idle = true;
0117:           if ( dma_more ) {
0118:               // Restart update
0119:               dma_more = false;
0120:               start_dma();
0121:           }
0122:       } else {
0123:           // Another page to send:
0124:           cmds[5] = 0xB0 | pageno;
0125:           if ( !gpio_get(GPIOB,GPIO10) ) {
0126:               // Send commands:
0127:               if ( !pageno )
0128:                   spi_dma_transmit(&cmds[0],8);
0129:               else spi_dma_transmit(&cmds[5],3);
0130:           } else {
0131:               // Send page data:
0132:               spi_dma_transmit(pixmapp,128);
0133:           }
0134:       }
0135:   }
0136: }
```

Задача состоит в бесконечном выполнении цикла, начиная со строки 99. Первым шагом в каждом цикле является вызов `ulTaskNotifyTake()`, что приводит к вечной блокировке до получения уведомления. Прерывание завершения DMA уведомит задачу в процедуре ISR (будет рассмотрена далее). После получения уведомления выполнение возвращается к строке 102.

Флаг `dma_busy` проверяется в строке 102, чтобы определить, выполняется ли в данный момент операция DMA. Однако при первом запуске управление возобновляется в строке 114. При этом проверяется завершение обновления OLED, чего не происходит при первом запуске. Затем управление передается строкам 124 и 125. Строка 125 проверяет состояние GPIO PB10. Если состояние PB10 низкое, то на этот раз мы отправляем командные байты. В строке 124 в последовательность команд добавлено правильное значение `pageno` (`pageno=0` при первом запуске).

Первая последовательность команд длиннее (строка 128), поскольку включены дополнительные команды, позволяющие убедиться, что контроллер OLED SSD1306 находится в правильном режиме обновления. Это делается на тот случай, если ошибка данных привела к тому, что контроллер SSD1306 в какой-то момент выполнил ошибочную команду. После этих команд байты с `cmds[5]` по `cmds[7]` отправляются для определения обновляемой графической страницы. На страницах 1–7 мы просто отправляем только команды настройки страницы для повышения эффективности.

После того как DMA завершил отправку командных байтов, управление снова переходит к строке 102 цикла с параметром `dma_busy = true`. Строка 103 выполняет вызов подпрограммы libopencm3 `spi_clean_disable()`, чтобы дождаться, пока можно будет безопасно манипулировать SPI1 после передачи DMA. DMA отправит байт SPI, но байт данных, возможно, еще не покинул контроллер SPI. Когда управление достигает строки 104, можно безопасно начать новый ввод-вывод.

Строка 105 проверяет состояние GPIO PB10. После того как командные байты были отправлены, это значение все равно будет низким, поэтому на этот раз строки 107–108 будут пропущены. Однако строка 111 будет выполнена, при этом линия DC изменится на высокий уровень для передачи данных.

Управление переходит к строке 125, но в этот момент на GPIO PB10 высокий уровень, поэтому выполняется строка 132. Это запускает передачу по DMA SPI 128 байт данных.

Идем по кругу и снова оказываемся на строке 103. На этот раз GPIO PB10 имеет высокий уровень, поэтому переменная указателя `pixmapp` увеличивается на 128 (строка 107), чтобы указать на следующую графическую страницу. Значение `pageno` также увеличивается (строка 108). Из-за переключения, которое происходит в строке 111, GPIO PB10 возвращается в низкий уровень, указывая, что еще несколько байтов команд должны быть отправлены в строках 124–129.

Этот цикл повторяется еще семь раз, чтобы страницы 1–7 были отправлены на контроллер OLED. В конце концов `pageno` увеличивается до 8, и это видно в строке 114. В строке 116 устанавливается флаг `dma_idle = true`, но начинается еще один раунд обновлений OLED, если переменная флага `dma_more` окажется включенной. Причина этой проверки будет объяснена позже.

Для облегчения этой передачи использовался механизм уведомления о задачах. Чтобы понять почему, теперь будет раскрыта процедура ISR.

Процедура DMA ISR

В листинге 13.7 представлена процедура обработчика прерывания ISR канала 3 DMA1. Строка 57 проверяет наличие флага `DMA_TCIF` канала 3 DMA1 (флаг прерывания завершения передачи) и, если да, очищает его в следующей строке. Как указано в этом примере, это должна быть единственная причина для входа в эту функцию.

Листинг 13.7. Полная процедура ISR DMA

```
0053: void
0054: dma1_channel3_isr(void) {
0055:   BaseType_t woken __attribute__((unused)) = pdFALSE;
0056:
0057:   if ( dma_get_interrupt_flag(DMA1,DMA_CHANNEL3,DMA_TCIF) )
0058:      dma_clear_interrupt_flags(DMA1,DMA_CHANNEL3,DMA_TCIF);
0059:
0060:   spi_disable_tx_dma(SPI1);
0061:
0062:   // Notify spidma_task to start another:
0063:   vTaskNotifyGiveFromISR(h_spidma,&woken);
0064: }
```

Важная операция очистки флага `DMA_TCIF` указана в строке 58. В том же ISR возможны два других источника прерываний. Полный список для данного канала DMA включает следующее:

- DMA_TCIF – флаг прерывания завершения передачи,
- DMA_TEIF – флаг прерывания ошибки передачи,
- DMA_THIF – флаг прерывания передачи полувыполненной задачи.

Последние два не используются в данном демо. Для справки: в табл. 13.2 перечислены все имена прерываний и процедур ISR, доступные для DMA1.

Таблица 13.2. Векторы прерываний для DMA

Канал DMA1	Программа ISR
NVIC_DMA1_CHANNEL1_IRQ	dma1_channel1_isr
NVIC_DMA1_CHANNEL2_IRQ	dma1_channel2_isr
NVIC_DMA1_CHANNEL3_IRQ	dma1_channel3_isr
NVIC_DMA1_CHANNEL4_IRQ	dma1_channel4_isr
NVIC_DMA1_CHANNEL5_IRQ	dma1_channel5_isr
NVIC_DMA1_CHANNEL6_IRQ	dma1_channel6_isr
NVIC_DMA1_CHANNEL7_IRQ	dma1_channel7_isr

Строка 60 процедуры ISR отключает запросы SPI1 на DMA, а строка 63 уведомляет функцию `spidma_task()`, которая блокируется при вызове `ulTaskNotifyTake()`. Обратите внимание, что ISR должен использовать версию «FromISR»

вызова vTaskNotifyGiveFromISR(). Возможности ISR ограничены, поэтому специальные формы вызовов позволяют это сделать.

Перезапуск передачи DMA

Теперь давайте представим процедуру spi_dma_xmit_pixmap() полностью (см. листинг 13.8).

Листинг 13.8. Полный листинг функции spi_dma_xmit_pixmap()

```
0156: void
0157: spi_dma_xmit_pixmap(void) {
0158:   bool prime = false;
0159:
0160:   taskENTER_CRITICAL();
0161:   if ( !dma_idle ) {
0162:      // Restart dma at DMA completion
0163:      dma_more = true;// Restart upon completion
0164:   } else {
0165:      prime = true; // Start from idle
0166:   }
0167:   taskEXIT_CRITICAL();
0168:
0169:   if ( prime )
0170:       start_dma(); // Start from idle
0171: }
```

Если бы обновления изображения вольтметра происходили достаточно часто, они могли бы поступать быстрее, чем OLED-дисплей мог бы обновляться с помощью DMA. Если частота SPI установлена на 1.125 МГц, полное обновление OLED-дисплея занимает около 7.54 мс. В этой демонстрации нет никаких средств для прерывания передачи DMA после ее начала, и в любом случае было бы нежелательно оставлять дисплей частично записанным. Так как же нам справиться с этим кризисом?

Когда обновления происходят часто, мы не хотим прерывать текущее. Однако после завершения текущей передачи DMA мы хотим, чтобы произошло хотя бы еще одно обновление OLED, чтобы отобразить текущее состояние. Этой цели служит переменная типа volatile флага dma_more. Но нам придется бороться с состоянием гонки.

Строка 160 начинает критический раздел, который нельзя прерывать. Прерывания включают в себя вытеснение, позволяющее запускать другие задачи. Отключение прерываний в строке 160 позволяет проверить текущее состояние dma_idle. Если переменная оказывается ложной (false), то это значит, что набор передач DMA выполняется или подходит к концу. В этом случае для dma_more устанавливается значение true, чтобы запросить еще одно обновление OLED после завершения текущего.

Однако если dma_idle имеет значение true, то считается, что механизм DMA простаивает, и его необходимо запустить снова. В этом случае локальная переменная флага prime принимает значение true (строка 165).

Выполнение демопримера

Демопример выполняется так же, как и в главе 12. Однако в интерактивном меню появилась новая опция «p» – команда «разогнать измеритель»[70]:

```
$ minicom usb
```

Когда *minicom* подключится к вашему USB-устройству, нажмите <**Enter**>, чтобы начать работу:

```
Welcome to minicom 2.7
OPTIONS:
Compiled on Sep 17 2016, 05:53:15.
Port /dev/cu.usbmodemWGDEM1, 21:29:29
Press Meta-Z for help on special keys
Monitor Task Started.
Test Menu:
0 .. set to 0.0 volts
1 .. set to 1.0 volts
2 .. set to 2.0 volts
3 .. set to 3.0 volts
4 .. set to 3.5 volts
+ .. increase by 0.1 volts
- .. decrease by 0.1 volts
p .. Meter pummel test
```

Пункты меню работают так же, как и в предыдущей главе, с добавленной опцией меню «p». Этот «тест на разгон» достигает цели благодаря быстрым обновлениям. При активации нажатием «p» измеритель будет быстро перемещаться из конца в конец диапазона. Код теста показан в листинге 13.9.

Листинг 13.9. Процедура испытания на разгон

```
0230: static void
0231: pummel_test(struct Meter *m1) {
0232:   TickType_t t0 = xTaskGetTickCount();
0233:   double v = 0.0;
0234:   double incr = 0.05;
0235:
0236:   meter_set_value(m1,v);
0237:   meter_update();
0238:   while ( (xTaskGetTickCount() - t0) < 5000 ) {
0239:       vTaskDelay(6);
0240:       v += incr;
0241:      if ( v > 3.3 ) {
0242:           incr = -0.05;
```

[70] Использованное автором слово «pummel» (см. название процедуры «pummel_test» далее) наиболее адекватно в данном случае переводится как «пинок».

```
0243:           v = 3.3;
0244:      } else if ( v < 0.0 ) {
0245:           v = 0.0;
0246:           incr = 0.05;
0247:      }
0248:      meter_set_value(m1,v);
0249:      meter_update();
0250:   }
0251: }
```

Процедура тестирования рассчитана на пять секунд (строка 238). Задержка в строке 239 определяет, насколько быстро обновляется картинка измерителя. Здесь установлена задержка в шесть тактов (около 6 мс) между обновлениями. Учитывая, что обновление занимает 7.54 мс, это будет перекрываться с передачей DMA, по крайней мере, некоторое время.

Вы можете обнаружить, что текстовая часть дисплея не обновляется во время теста. Она наверстает упущенное после того, как разгон закончится. Это иллюстрирует природу проблемы.

Дальнейшие проблемы

Хотя демонстрационный код работает так, как задумано, у него есть один недостаток. Если обновления происходят слишком часто, например с интервалом в 1 мс, указатель на OLED-дисплее может потеряться. Почему это происходит?

Фон картинки вольтметра записывается в растровое изображение только один раз. В растровом изображении перерисовываются только указатель и цифровое значение. При каждом измерении исходный указатель рисуется цветом фона, чтобы стереть его, после чего записывается новый указатель цветом переднего плана. Что может случиться – так это то, что растровое изображение, копируемое DMA на OLED, скопирует стертый указатель из-за плохой синхронизации.

Чтобы исправить это, возможно несколько разных подходов. Одним из подходов было бы копирование растрового изображения в другой буфер растрового изображения с использованием передачи из памяти в память DMA. Затем OLED можно обновить с помощью этого буфера растровых изображений, который никогда не модифицируется текущим программным обеспечением. Это, очевидно, требует дополнительного времени для копирования в памяти, а также для переноса другого буфера растровых изображений в SRAM.

Другой подход может заключаться в ограничении максимальной частоты обновлений счетчика с помощью программного обеспечения. В конце концов, стрелка реального аппаратного вольтметра также не может перемещаться мгновенно. Это лишь малая часть проблем, возникающих в вычислениях во встраиваемых системах.

Итоги

Эта глава основана на программном обеспечении, разработанном в главе 12, с добавлением контроллера DMA для управления передачей данных на OLED-

устройство. Демопрограмма помогла вам познакомиться с контроллером DMA и его возможностями. Используя механизм задач FreeRTOS, передача DMA управлялась с помощью команд и передач данных, которые происходили путем манипулирования линией GPIO PB10. Наконец, прерывание завершения передачи DMA использовалось для объединения событий.

Использование DMA не всегда так сложно. Однако эта демонстрация должна подготовить вас к чему-то более сложному, чем обычный пример из учебника.

Упражнения

1. Какие настройки контроллера DMA необходимо изменить в программе перед началом следующей передачи?
2. Имеет ли каждый канал DMA свою собственную процедуру ISR?
3. Откуда приходит запрос DMA при передаче из памяти на периферию, как в демопрограмме?
4. Какие три условия необходимо установить в программе при использовании SPI, прежде чем может начаться передача по DMA?

Глава 14 • Аналого-цифровое преобразование

Встроенным вычислениям для анализа часто необходимо преобразовать уровень аналогового сигнала в цифровую форму. Одним из таких применений является измерение температуры по напряжению, возникающему в полупроводниковом датчике. Поэтому неудивительно, что платформа STM32 имеет как аналого-цифровой преобразователь (АЦП), так и специальный встроенный канал АЦП для измерения температуры.

Демонстрационный проект этой главы покажет, как использовать процедуры библиотеки libopencm3 для доступа к периферийному устройству АЦП для чтения аналоговых каналов PA0 и PA1, а также для считывания температуры чипа и его внутреннего опорного напряжения.

Ресурсы STM32F103C8T6

STM32F103C8T6 оснащен двумя контроллерами АЦП, указанными в libopencm3 следующими именами:

- `АЦП1` – 12-битный аналого-цифровой преобразователь 1 с 18 входными каналами;
- `АЦП2` – 12-битный аналого-цифровой преобразователь 2 с 16 входными каналами.

Каждый из них поддерживает 16 аналоговых входных каналов. АЦП1, кроме того, может получать доступ к внутренним уровням температуры и опорного напряжения V_{ref}.

Периферийное устройство АЦП также включает в себя программируемый предделитель, который устанавливает скорость преобразования. Входом предделителя является тактовый сигнал `PCLK2` (то же, что и `APB2`). Поскольку наша демоверсия инициализируется вызовом

```
rcc_clock_setup_pll(&rcc_hse_configs[RCC_CLOCK_HSE8_72MHZ]);
```

это приводит к тому, что частота APB2 устанавливается равной

```
rcc_apb2_frequency = 72000000;
```

т. е. 72 МГц. Входная тактовая частота АЦП не должна превышать 14 МГц, поэтому это ограничивает нас значением предделителя, равным 6, устанавливая тактовую частоту 72 / 6 = 12 МГц.

Демопрограмма

Схемы для этой демопрограммы нет, поскольку все обеспечивается встроенным периферийным устройством АЦП. Единственными внешними соединениями, которые представляют интерес, являются аналоговые входы PA0 и PA1. Однако позже будет представлена схема подключения потенциометра для генерации измеряемого напряжения. В этой главе в основном рассказывается о том, как настроить программное обеспечение для работы с периферийным устройством АЦП.



Входы GPIO PA0 и PA1 не допускают напряжения 5 В и должны получать напряжения только от нуля до +3.3 В.

Программное обеспечение для этой главы находится в следующем каталоге:

```
$ cd ~/stm32f103c8t6/rtos/adc
```

Перейдите в этот подкаталог и пересоберите проект с нуля:

```
$ make clobber
$ make
$ make flash
```



Для перепрошивки STM32 для этого проекта нет необходимости менять перемычку boot-0, за исключением, возможно, первого раза.

Аналоговые входы PA0 и PA1

В программе `main()` инициализируется периферийное устройство АЦП и его выводы GPIO. На первом этапе настраиваются аналоговые входы следующим образом:

```
0087: rcc_periph_clock_enable(RCC_GPIOA); // Enable GPIOA for ADC
0088: gpio_set_mode(GPIOA,
0089:   GPIO_MODE_INPUT,
0090:   GPIO_CNF_INPUT_ANALOG, // Analog mode
0091:   GPIO0|GPIO1); // PA0 & PA1
```

Тактовый сигнал для GPIO включается в строке 87, как обычно. Строки 88–91 настраивают GPIO PA0 и PA1 для аналогового входа. Обратите внимание, что для настройки входа GPIO используется значение `GPIO_CNF_INPUT_ANALOG`. Это позволяет входному аналоговому напряжению действовать на АЦП вместо цифрового входа.

Конфигурация периферийного устройства АЦП

Основная сложность в этом примере – правильная настройка периферии АЦП. STM32 обладает ошеломляющим набором возможностей в этой обла-

сти. Листинг 14.1 иллюстрирует конфигурацию, используемую в случае этого примера. Весь исходный код, представленный в этом разделе, находится в файле *main.c*.

Сначала необходимо включить тактовый сигнал периферийного устройства АЦП, что выполняется в строке 103. В строке 104 на время инициализации отключается питание периферийного устройства АЦП (а не его тактового источника).

Листинг 14.1. Конфигурация АЦП

```
0102: // Initialize ADC:
0103: rcc_peripheral_enable_clock(&RCC_APB2ENR,RCC_APB2ENR_ADC1EN);
0104: adc_power_off(ADC1);
0105: rcc_peripheral_reset(&RCC_APB2RSTR,RCC_APB2RSTR_ADC1RST);
0106: rcc_peripheral_clear_reset(&RCC_APB2RSTR,RCC_APB2RSTR_ADC1RST) ;
0107: rcc_set_adcpre(RCC_CFGR_ADCPRE_PCLK2_DIV6); // Set. 12MHz, Max. 14MHz
0108:  adc_set_dual_mode(ADC_CR1_DUALMOD_IND); // Independent mode
0109:  adc_disable_scan_mode(ADC1);
0110:  adc_set_right_aligned(ADC1);
0111:  adc_set_single_conversion_mode(ADC1);
0112:  adc_set_sample_time(ADC1,ADC_CHANNEL_TEMP, ADC_SMPR_SMP_239DOT5CYC);
0113: adc_set_sample_time(ADC1,ADC_CHANNEL_VREF, ADC_SMPR_SMP_239DOT5CYC);
0114:  adc_enable_temperature_sensor();
0115:  adc_power_on(ADC1);
0116:  adc_reset_calibration(ADC1);
0117:  adc_calibrate_async(ADC1);
0118:  while (adc_is_calibrating(ADC1));
```

Строки 105 и 106 сбрасывают АЦП. Это разные вызовы, поскольку задействованы разные регистры.

Предварительный делитель АЦП

Строка 107 устанавливает предварительный делитель АЦП для работы на максимальной частоте 12 МГц. Тактовая частота АЦП может составлять до 14 МГц, но делитель, равный 4, дает 18 МГц, что явно превышает максимальное значение. Если вашему приложению требуется максимально возможная скорость преобразования АЦП, единственный вариант – сначала изменить общую тактовую частоту процессора и других устройств. Имейте в виду, что существуют ограничения по тактовой частоте со стороны контроллера USB, которые следует учитывать, если USB задействуется.

Режимы АЦП

Строки 108–111 настраивают ряд различных доступных режимов. Линия 108 позволяет АЦП1 и АЦП2 работать независимо. Строка 109 отключает опцию режима сканирования, а строка 110 настраивает АЦП на сохранение результата с выравниванием по правому краю в его регистре. Наконец, строка 111 настраивает АЦП на остановку после завершения одного преобразования:

```
0108: adc_set_dual_mode(ADC_CR1_DUALMOD_IND); // Independent mode
0109: adc_disable_scan_mode(ADC1);
0110: adc_set_right_aligned(ADC1);
0111: adc_set_single_conversion_mode(ADC1);
```

Время выборки

Строки 112 и 113 устанавливают время выборки, которое будет использоваться в каналах температуры и V_{ref}:

```
0112: adc_set_sample_time(ADC1,ADC_CHANNEL_TEMP, ADC_SMPR_SMP_239DOT5CYC);
0113: adc_set_sample_time(ADC1,ADC_CHANNEL_VREF, ADC_SMPR_SMP_239DOT5CYC);
0114: adc_enable_temperature_sensor();
```

Каждый канал АЦП может быть дискретизирован с разным количеством периодов тактовой частоты. По умолчанию каждое преобразование происходит за 1.5 периода (`ADC_SMPR_SMP_1DOT5CYC`). Общее количество тактов определяется следующим уравнением:

T_{conv} = *SampleRate* + 12.5, где *SampleRate* – частота дискретизации (см. далее).

В случае настройки, согласно строке 112, время преобразования температуры требует следующего количества тактов:

T_{conv} = 239.5 + 12.5 = 252 такта.

Поскольку тактовая частота АЦП равна 12 МГц, общее время преобразования будет следующим:

$$T_{conv} = \frac{252}{12} = 21 \text{ мкс.}$$

Частота дискретизации по умолчанию для данного канала составляет 1.5 такта. В табл. 14.1 перечислены доступные частоты дискретизации.

Таблица 14.1. Частота дискретизации АЦП

Имя libopencm3	Такты	Общее время (тактовая частота АЦП 12 МГц)
ADC_SMPR_SMP_1DOT5CYC	1.5 + 12.5 = 14	1.167 мкс
ADC_SMPR_SMP_7DOT5CYC	7.5 + 12.5 = 20	1.667 мкс
ADC_SMPR_SMP_13DOT5CYC	13.5 + 12.5 = 26	2.167 мкс
ADC_SMPR_SMP_28DOT5CYC	28.5 + 12.5 = 41	3.417 мкс
ADC_SMPR_SMP_41DOT5CYC	41.5 + 12.5 = 54	4.500 мкс
ADC_SMPR_SMP_55DOT5CYC	55.5 + 12.5 = 68	5.667 мкс
ADC_SMPR_SMP_71DOT5CYC	71.5 + 12.5 = 84	7.000 мкс
ADC_SMPR_SMP_239DOT5CYC	239.5 + 12.5 = 252	21.00 мкс

Подготовка АЦП

Прежде чем использовать контроллер АЦП, необходимо выполнить еще три шага (из листинга 14.1):

```
0115: adc_power_on(ADC1);
0116: adc_reset_calibration(ADC1);
0117: adc_calibrate_async(ADC1);
0118: while (adc_is_calibrating(ADC1));
```

Питание включается в строке 115, а калибровочные константы сбрасываются в строке 116. Строка 117 запускает калибровку, а строка 118 ожидает ее завершения. В демонстрационной программе все это выполняется до запуска планировщика FreeRTOS.

Запуск демопрограммы

После того как STM32 будет прошит кодом демонстрационного примера, подключите USB-кабель и запустите *minicom* (опять же, «usb» – это имя файла, которое я использовал для сохранения параметров связи USB):

```
$ minicom usb
Welcome to minicom 2.7
OPTIONS:
Compiled on Sep 17 2016, 05:53:15.
Port /dev/cu.usbmodemWGDEM1, 12:38:28
Press Meta-Z for help on special keys
Temperature 24.72 C, Vref 1.19 Volts, ch0 1.92 V, ch1 0.00 V
Temperature 24.72 C, Vref 1.19 Volts, ch0 1.94 V, ch1 0.00 V
Temperature 24.72 C, Vref 1.19 Volts, ch0 1.97 V, ch1 0.00 V
Temperature 24.72 C, Vref 1.19 Volts, ch0 1.98 V, ch1 0.00 V
```

Каждые 1.5 с будет отображаться новая строка, показывающая следующее:

- внутренняя температура STM32 в градусах Цельсия (в примере 24.72 °C);
- внутреннее значение опорного напряжения V_{ref} STM32 (в примере 1.19 В);
- напряжение канала A0 (1.92 В в первой строке примера);
- напряжение канала A1 (в примере 0.00 В).

Чтобы проверить аналоговые входы A0 и A1, вы можете сначала подключить перемычку к GND, а затем к источнику питания +3.3 В. Не подавайте напряжение выше указанного или отрицательное напряжение – это может привести к необратимому повреждению.

В только что показанном сеансе вывод PA1 был плавающим, а PA0 был заземлен. Если теперь подать +3.3 В на вход PA0, полученное значение должно быть близко к +3.3 В. Когда я это попробовал, у меня получилось 3.29 В. Повторите проверку заземления и +3.3 В на PA1, и программа должна сообщить идентичные результаты.

Чтение АЦП

Если АЦП настроен в программе `main()`, функция `demo_task()` может считывать аналоговые напряжения по каналам следующим образом:

```
0060: int adc0, adc1;
...
```

```
0068: adc0 = read_adc(0) * 330 / 4095;
0069: adc1 = read_adc(1) * 330 / 4095;
```

Чтобы избежать увеличения объема вычислений для чисел с плавающей запятой и уменьшить размер используемой флеш-памяти, здесь для вычисления напряжений в переменных `adc0` и `adc1` используется целочисленная арифметика. Результат 12-битного АЦП имеет 4096 возможных шагов, в результате чего возвращаемый результат находится в диапазоне от 0 до 4095. Результат вычисления при умножении на 330 равен вольтам, умноженным на 100.

Программное обеспечение, отвечающее за чтение с АЦП, представлено в листинге 14.2.

Листинг 14.2. Чтение результатов АЦП

```
0030: static uint16_t
0031: read_adc(uint8_t channel) {
0032:
0033:   adc_set_sample_time(ADC1,channel,ADC_SMPR_SMP_239DOT5CYC);
0034:   adc_set_regular_sequence(ADC1,1,&channel);
0035:   adc_start_conversion_direct(ADC1);
0036:   while ( !adc_eoc(ADC1) )
0037:       taskYIELD();
0038:   return adc_read_regular(ADC1);
0039: }
```

В строке 33 время выборки устанавливается равным 21 мкс, а канал выборки устанавливается в строке 34. В ней указывается, что необходимо выбрать один канал (аргумент 2), причем канал задан списком, начиная с адреса аргумента &channel (это одноэлементный список).

Строка 35 запускает процедуру преобразования АЦП и ожидает завершения в строках 36 и 37. И снова вызывается `TaskYIELD()`, чтобы позволить другим задачам эффективно распределять время процессора. Наконец, строка 38 извлекает результат преобразования и возвращает его вызывающей стороне.

Вычисление температуры

Функция `demo_task()` выполняет следующий вызов для определения внутренней температуры:

```
0066: temp100 = degrees_C100();
```

В листинге 14.3 приведена функция `degrees_C100()`.

Листинг 14.3. Функция `degrees_C100()`

```
0044: static int
0045: degrees_C100(void) {
0046:   static const int v25 = 143;
0047:   int vtemp;
```

```
0048:
0049:   vtemp = (int)read_adc(ADC_CHANNEL_TEMP) * 3300 / 4095;
0050:
0051:   return (v25 - vtemp) / 45 + 2500;
          // temp = (1.43 - Vtemp) / 4.5 + 25.00
0052: }
```

Документация STM32F103C8T6 очень схематична в отношении этого расчета. Поскольку справочный документ PDF (RM0008) применим ко всему семейству устройств STM32, сложно разобраться в расчетах, необходимых именно для устройства Blue Pill.

Необходимая информация доступна в PDF-документе DS5319 (раздел «Дополнительные источники», [2] в конце этой главы).

Для микросхем серий STM32F103x8 и STM32F103xB см. табл. 51 в этом PDF-файле. Для вашего удобства эта таблица представлена в этой главе как табл. 14.2.

Как изложено в PDF-документе и указано в исходном коде, температура устройства STM32F103C8T6 рассчитывается следующим образом:

$$Temp = \frac{V_{25} - V_{\text{sense}}}{4.5} + 25.$$

Таблица 14.2. Характеристики датчика температуры для устройств STM32F103x8 и STM32F103xB

Символ	Параметр	Мин.	Тип.	Макс.	Единицы
T_L	Линейность V_{sense} по температуре	–	±1	±2	°C
Avg_Slope	Средняя крутизна преобразования	4.0	4.3	4.6	мВ/°C
V_{25}	Напряжение при 25 °C	1.34	1.43	1.52	V
t_{START}	Время запуска	4	–	10	мкс
T_{S_temp}	Время выборки ADC при считывании температуры	–	–	17.1	мкс

В листинге 14.3 видно, что использовалось типичное значение V_{25} = 1.43 (×100) из табл. 14.2. Значение 45 происходит от величины 4.5 для `Avg_Slope` (значение, умноженное на 10). Кажется, это хорошо соответствует устройству, которое я использовал, и находится в указанном диапазоне. Но если вы обнаружите, что вычисленное значение велико, попробуйте уменьшить значение с 45 до 43 (что соответствует наклону 4.3 мВ/°C).

Еще одно интересное значение – T_{S_temp}, равное 17.1 мкс. Это время выборки, проведенное при тестировании, в результате которого были представлены результаты в таблице. Это также рекомендуемое время выборки, указанное в RM0008.

Если вам не нужны показания температуры, можете сэкономить энергопотребление, отключив датчик следующим образом:

```
adc_disable_temperature_sensor();
```

В техническом описании также есть примечание о температуре:

The temperature sensor output voltage changes linearly with temperature. The offset of this line varies from chip to chip due to process variation (up to 45 °C from one chip to another).

The internal temperature sensor is more suited to applications that detect temperature variations instead of absolute temperatures. If accurate temperature readings are needed, an external temperature sensor part should be used.

(Выходное напряжение датчика температуры изменяется линейно с температурой. Смещение этой линии варьируется от чипа к чипу из-за различий в процессе (до 45 °C от одного чипа к другому[71]).

Внутренний датчик температуры больше подходит для приложений, которые обнаруживают изменения температуры, а не абсолютные температуры. Если необходимы точные показания температуры, следует использовать внешний датчик температуры.)

Опорное напряжение

АЦП1 также позволяет считывать внутреннее опорное напряжение V_{ref}. Это значение можно использовать для калибровки АЦП с целью повышения точности. Типичное значение составляет 1.2 В. Ориентируйтесь на руководство по применению AN2834, содержащее очень полезную информацию для тех, кто ищет максимальную точность от платформы STM32[72].

Аналоговые напряжения

Измерение 0 или +3.3 В может показаться не слишком интересным, поэтому давайте улучшим этот процесс. Вы можете создать любое напряжение в этом диапазоне с помощью потенциометра. Хотя диапазон значений от 1 до 15 кОм должен быть подходящим, для стабильных показаний, лучше всего использовать нижний предел этого диапазона[73].

На рис. 14.1 показан потенциометр[74] на 10 кОм, который был использован для этого эксперимента. Если вы его покупаете, приобретите переменный

[71] Величина разброса абсолютно нереальная для любого датчика температуры. Однако так действительно написано на стр. 235 руководства RM0008, так что оставим эту явную несуразицу на совести авторов технического описания.

[72] Документ AN2834 «Application note. How to optimize the ADC accuracy in the STM32 MCUs» можно найти по адресу https://www.st.com/resource/en/application_note/cd00211314-how-to-get-the-best-adc-accuracy-in-stm32-microcontrollers-stmicroelectronics.pdf.

[73] Максимально допустимое сопротивление внешнего источника напряжения зависит от частоты дискретизации, см. табл. 47 на стр. 75 руководства DS5319 «RM0008. Reference manual» (раздел «Дополнительные источники», [1] к главе 7).

[74] Переменные резисторы в англоязычной литературе именуются потенциометрами. В настоящее время этот термин постепенно становится привычным и в отечественных источниках, поэтому мы далее употребляем эти названия вперемешку как синонимы. Однако при поиске в каталогах отечественных интернет-магазинов надежнее употреблять привычное русское наименование «переменный резистор». Оно более верное и с технической точки зрения, так как вообще-то потенциометр – лишь одна из двух возможных конфигураций при подключении переменного резистора к схеме.

резистор с линейной характеристикой, а не логарифмической. Потенциометры для аудиоприменений изменяются логарифмически, чтобы громкость при регулировании менялась пропорционально зависимости для слуха. Поставщики из Северной Америки будут использовать префикс «B», например «B10K», для обозначения линейности, как показано на рис. 14.1[75].

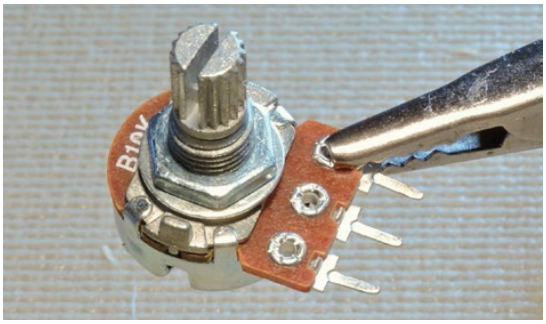


Рис. 14.1. Линейный потенциометр 10 кОм

Схема показана на рис. 14.2. Вы можете использовать только один переменный резистор, если вам не хватает второго. Обязательно подключите клеммы на противоположных концах потенциометра к источнику питания и заземлению. Центральный контакт подключен к движку внутри потенциометра и должен быть подключен к входу АЦП PA0 или PA1.

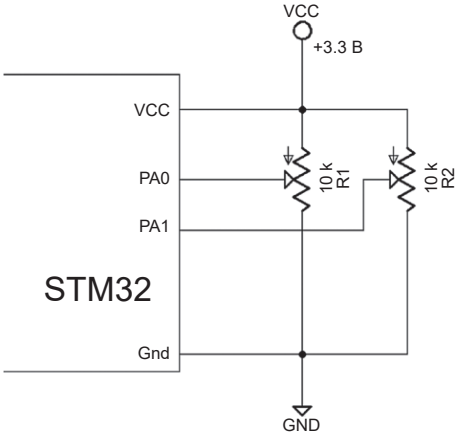


Рис. 14.2. Пара потенциометров, подключенных к входам АЦП PA0 и PA1

[75] Все импортные переменные резисторы с линейным характером изменения сопротивления от угла поворота имеют тип характеристики «B» (английская буква; обязательно указано в перечне характеристик, даже если не написано на корпусе). Для старых отечественных переменников (типа СП-1, СП-3 и пр.) линейная характеристика обозначалась русской буквой «А».

При подключенном *minicom* можно поворачивать движки против часовой стрелки. Если при повороте против часовой стрелки напряжение растет до +3.3 В, поменяйте местами соединения с крайними выводами потенциометра. В крайнем левом положении оно должно быть около нуля. Когда вы поворачиваете потенциометр до середины шкалы, вы должны увидеть около +1.5 В, а при полном повороте по часовой стрелке показания должны быть около +3.3 В.

Итоги

Представленный демонстрационный пример лишь поверхностно коснулся того, что обеспечивает периферийное устройство АЦП STM32 в плане гибкости. Помимо одиночных преобразований, периферийное устройство АЦП можно настроить на использование групп каналов и выполнение сканирования. При сканировании любой последовательности каналов в результаты можно выводить значения АЦП. Наконец, сканирование и группы могут включать в себя каналы обоих периферийных устройств ADC1 и ADC2.

Эта глава дает вам простые основы использования АЦП. Прочтите главу 11 справочного руководства STM32 RM0008 (см. раздел «Дополнительные источники», [1] к главе 7). чтобы получить полную информацию о возможностях измерений с помощью АЦП.

Упражнения

1. Как отображается внутренняя температура STM32?
2. Чем значение `GPIO_CNF_INPUT_ANALOG` отличается от значений `GPIO_CNF_INPUT_PULL_UPDOWN` или `GPIO_CNF_INPUT_FLOAT`?
3. Если `PCLK` имеет частоту 36 МГц, какова будет тактовая частота АЦП при настройке с предварительным делителем, равным 4?
4. Назовите три параметра конфигурации, которые влияют на общую мощность, потребляемую АЦП.
5. Если предположить, что тактовая частота АЦП после предделителя равна 12 МГц, сколько времени займет сконфигурированная выборка `ADC_SMPR_SMP_41DOT5CYC`?

Дополнительные источники

1. STMicroelectronics. «RM0008. Reference manual». «RM0008. Reference manual» (раздел «Дополнительные источники», [1] к главе 7).
2. STMicroelectronics. Руководство DS5319 «STM32F103x8, STM32F103xB» https://www.st.com/resource/en/datasheet/stm32f103c8.pdf (дата доступа 10.09.2024).

Глава 15 • Дерево тактовых сигналов

До этого момента в комнате находился слон, которого мы почти не замечали. Мы настраивали и использовали различные тактовые сигналы, не говоря о них слишком много. Сейчас самое время раскрыть некоторые компоненты тактовой системы, которые скрывались в тени.

Хотя существуют конструкции с использованием асинхронных логических схем, большинство микропроцессоров используют один или несколько источников тактовых импульсов. Серия STM32 не является исключением. Эта серия обладает широкими возможностями настройки, что несколько усложняет ее программное обеспечение. Но эта дополнительная гибкость позволяет разработчику снизить требования к питанию за счет отключения ненужных периферийных устройств и тактовых сигналов.

В этой главе будет рассмотрена система генерации тактовых импульсов, которую поддерживает STM32F103C8T6, и способы ее настройки. Эта информация позволит вам рассчитать правильные значения предварительного делителя, необходимые для получения правильной скорости передачи данных и тактовой частоты SPI, а также для правильной работы таймеров. Мы также предоставим вам внутреннюю информацию, необходимую для использования специальных функций тактовой системы и предотвращения ошибок.

Основы

Многие импульсные последовательности могут быть получены из других последовательностей, но в начале любой цепочки должен быть один или несколько источников. В STM32F103C8T6 имеется четыре независимых источника тактовой частоты, а именно:

1) RC-генератор 8 МГц (HSI),
2) генератор с кварцевым или керамическим резонатором (HSE) 4–16 МГц,
3) кварцевый генератор с резонатором 32.768 кГц (LSE),
4) RC-генератор 40 кГц (LSI).

В табл. 15.1 приведены обозначения, использованные для перечисленных генераторов, а также некоторые из их основных характеристик. Например, показано, что генератор HSE управляется резонатором и обладает хорошей стабильностью, тогда как генератор HSI управляется RC-цепочкой (резистором и конденсатором) и имеет относительно низкую стабильность.

Таблица 15.1. Обозначения тактовых генераторов STM32[76]

Обозначение	Низкая/высокая частота	Внутренний/внешний	Управляется	Стабильность
LSI (40 кГц)	Низкая	Внутренний	Резистор и конденсатор	Плохая
LSE (32768 Гц)	Низкая	Внешний резонатор	Кристаллический резонатор	Хорошая
HSI (8 МГц)	Высокая	Внутренний	Резистор и конденсатор	Плохая
HSE (4–16 МГц)	Высокая	Внешний резонатор	Кристаллический резонатор	Хорошая

RC-генераторы

Хороший вопрос: зачем городить RC-генераторы, если они нестабильные? Для некоторых приложений может быть достаточно, чтобы у микроконтроллера была какая-то тактовая частота для выполнения инструкций. Это избавляет проектировщика от необходимости устанавливать кристаллический резонатор и, таким образом, уменьшает количество деталей.

На рис. 15.1 показаны два кристалла резонаторов, поставляемые с платой Blue Pill. Обратите внимание на размер резонатора 8000 МГц. Прямо под ним на фотографии находится кристалл 32.768 кГц, заключенный в прямоугольный пластиковый шарик. По отношению к микросхеме самого контроллера (выше резонатора 8 МГц) это крупные компоненты.

Работа RC-генератора, как известно специалистам по электронике, состоит из зарядки и разрядки конденсатора через резистор. Комбинация емкости и сопротивления определяет общую частоту. Создание конденсаторов внутри интегральной схемы представляет собой сложную задачу, но ее стоит решить для тех пользователей, которые хотят уменьшить количество внешних компонентов.

Обратите внимание, что все RC-генераторы STM32 – это внутренние генераторы. В противном случае резисторы и конденсаторы пришлось бы подключать извне.

Кварцевые генераторы

Кварцевые генераторы гораздо более точны и стабильны, чем RC-генераторы. Однако у них есть тот недостаток, что необходимо подключить внешний кристалл к микросхеме MCU. На рис. 15.1 показаны два кристалла, расположенные на печатной плате Blue Pill, причем кристалл 8 МГц используется генератором HSE. Кристалл с частотой 32.768 кГц управляет генератором LSE.

Питание генераторов

Более высокая скорость переключения генератора между низким и высоким уровнями сигнала означает дополнительное потребление тока. Каждый раз,

[76] Обозначения расшифровываются следующим образом: LSI – Low Speed Internal (*низкоскоростной внутренний*); HSE – High Speed External (*высокоскоростной внешний*); остальные два аналогично.

когда генератор переключается с низкого уровня на высокий или с высокого уровня на низкий, электроны в CMOS-элементе попадают в цепь сквозного протекания тока; кроме того, идут процессы зарядки и разрядки емкостей затворов CMOS-транзисторов, задействованных в схеме генератора и емкости входа его нагрузки. Все это требует энергии.

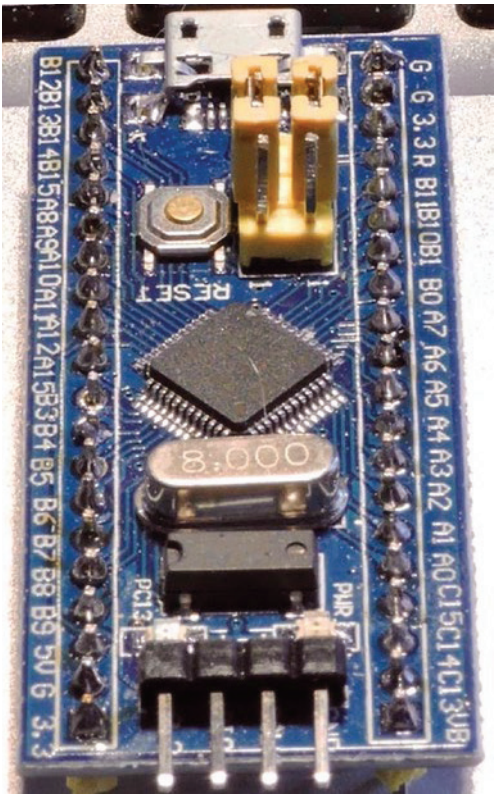


Рис. 15.1. Кристаллические резонаторы 8 МГц и 32.768 кГц (под ним)

Понятно, что если вы будете выполнять переключение чаще за тот же промежуток времени, то общий ток будет потребляться больше. Именно поэтому конструкции тактовых генераторов на платформе STM32 уделяется так много внимания.

Для некоторых приложений, где система питается от батареи и время выполнения менее важно, имеет смысл использовать низкоскоростной генератор. Если, с другой стороны, приложение питается от настольного компьютера через USB и скорость является доминирующим требованием, то более высокие частоты генератора являются предпочтительными.

Еще один критерий выбора – точность. Если вы реализуете последовательную связь между различными устройствами, вам необходимо иметь точное представление о скорости передачи данных. Наличие выбора предлагает разработчикам различные компромиссы.

Часы реального времени (RTCCLK)

В качестве источника тактовых сигналов для часов реального времени RTCCLK можно выбрать генераторы HSE, LSE или LSI. В табл. 15.2 приведены доступные конфигурации RTCCLK[77]. Обратите внимание, что при выборе тактовой частоты HSE предделитель жестко запрограммирован равным 128.

Таблица 15.2. Источники синхронизации RTC при частоте HSE, равной 8 МГц

Источник	Частота	Делитель	Результирующая частота
HSE	8.000 МГц	128	62.5 кГц
LSE	32.768 кГц	1	32.768 кГц
LSI	40 кГц	1	40 кГц

Сторожевой таймер

Независимый сторожевой таймер (IWDG) жестко подключен к тактовому генератору LSI 40 кГц.

Системный источник (SYSCLK)

Наиболее интересной категорией базовой конфигурации источников тактовых сигналов являются системные источники, на основе которых формируются другие важные тактовые сигналы. SYSCLK может быть получен только от двух из четырех источников синхронизации[78]:

- HSI (RC), 8 МГц,
- HSE (кристаллический резонатор), 4–16 МГц (8 МГц на Blue Pill).

Существует еще один дополнительный источник, полученный с помощью системы фазовой автоподстройки частоты (phase-locked loop, PLL, ФАПЧ), которая умножает частоту входного тактового сигнала HSI или HSE. Когда источником для ФАПЧ является тактовый сигнал HSI, входной сигнал сначала делится на 2. В табл. 15.3 представлена удобная таблица значений выхода ФАПЧ при использовании HSI.

Таблица 15.3. Системные частоты, полученные из HSI и ФАПЧ

Источник	Частота	Множитель ФАПЧ	Результирующая частота
HSI	8 МГц	Без ФАПЧ	8 МГц
HSI	8 МГц / 2	2	8 МГц

[77] См. также раздел главы 10 «Синхронизация RTC».

[78] Следует добавить, что один из выводов для подключения внешнего резонатора к генератору HSE (вывод OSC_IN) может использоваться для подключения внешнего источника тактовой частоты до 25 МГц; см. по этому поводу стр. 95 справочного руководства STM32 RM0008 (ссылка в разделе «Дополнительные источники», [1] главы 7).

Таблица 15.3 (окончание)

Источник	Частота	Множитель ФАПЧ	Результирующая частота
HSI	8 МГц / 2	3	12 МГц
HSI	8 МГц / 2	4	16 МГц
HSI	8 МГц / 2	5	20 МГц
HSI	8 МГц / 2	6	24 МГц
HSI	8 МГц / 2	7	28 МГц
HSI	8 МГц / 2	8	32 МГц
HSI	8 МГц / 2	9	36 МГц
HSI	8 МГц / 2	10	40 МГц
HSI	8 МГц / 2	11	44 МГц
HSI	8 МГц / 2	12	48 МГц
HSI	8 МГц / 2	13	52 МГц
HSI	8 МГц / 2	14	56 МГц
HSI	8 МГц / 2	15	60 МГц
HSI	8 МГц / 2	16	64 МГц

Если в качестве источника синхронизации выбран HSE, частоты изменятся на значения, показанные в табл. 15.4. На входе ФАПЧ может использоваться либо HSE, разделенный на 2 (HSE / 2), либо не разделенный (HSE / 1). Максимальная используемая системная частота составляет 72 МГц.

Таблица 15.4. Системные частоты, полученные из HSE и ФАПЧ

Источник	Частота	Множитель ФАПЧ	HSE / 2	HSE / 1
HSE	8 МГц	Без ФАПЧ	8 МГц	8 МГц
HSE	8 МГц	2	8 МГц	16
HSE	8 МГц	3	12 МГц	24
HSE	8 МГц	4	16 МГц	32
HSE	8 МГц	5	20 МГц	40
HSE	8 МГц	6	24 МГц	48
HSE	8 МГц	7	28 МГц	56
HSE	8 МГц	8	32 МГц	64
HSE	8 МГц	9	36 МГц	72
HSE	8 МГц	10	40 МГц	–
HSE	8 МГц	11	44 МГц	–
HSE	8 МГц	12	48 МГц	–
HSE	8 МГц	13	52 МГц	–
HSE	8 МГц	14	56 МГц	–
HSE	8 МГц	15	60 МГц	–
HSE	8 МГц	16	64 МГц	–

На рис. 15.2 представлена немного упрощенная схема дерева системных тактовых сигналов до точки SYSCLK. Звездочки обозначают то, что обычно настроено по умолчанию для платы Blue Pill STM32F103C8T6.

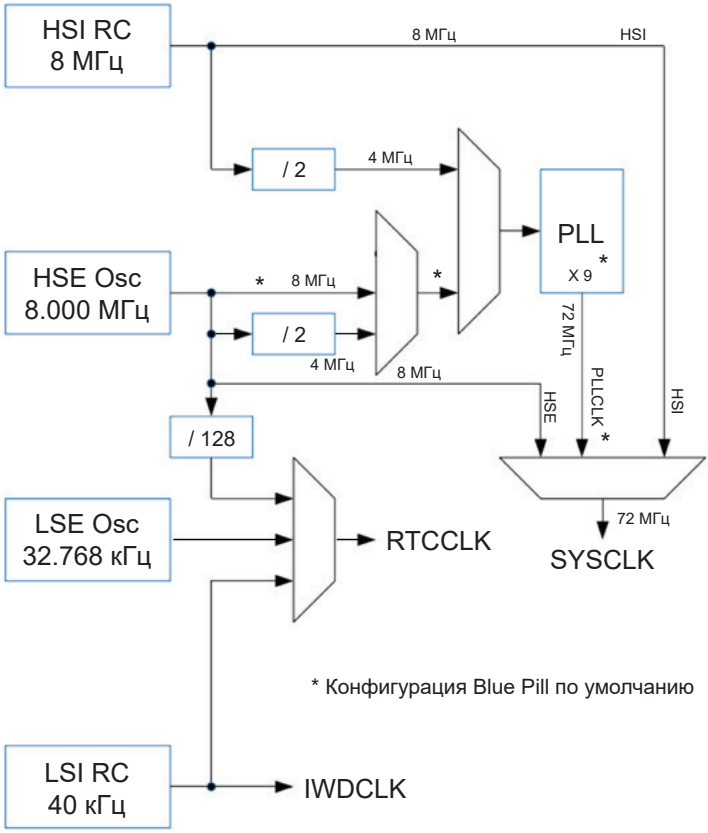


Рис. 15.2. Упрощенное изображение дерева тактовых сигналов до точки SYSCLK

SYSCLK и USB

Если вы не используете USB, можете игнорировать эту проблему. Но когда требуется поддержка USB, ваш выбор ограничен, как показано в табл. 15.5. Предварительный делитель USB должен быть настроен так, чтобы USBCLK составлял 48 МГц.

Таблица 15.5. Допустимые значения тактовых частот при использовании USB

Частота SYSCLK	Предделитель USB	Итоговая тактовая частота USB
72 МГц	÷ 1.5	48 МГц
48 МГц	÷ 1	48 МГц

Шина AHB

В документе STMicroelectronics RM0008, описывающем серию STM32, содержатся ссылки на AHB без объяснения того, что это такое. Так что же такое AHB? В этом поможет «Википедия» («Дополнительные источники», [1]):

«Расширенная архитектура шины микроконтроллера (AMBA) – это открытый стандарт требований внутрикристалльных межсоединений для связи и управления функциональными блоками в разработках system-on-a-chip (SoC). <...> AMBA была представлена ARM в 1996. Первыми шинами AMBA были Advanced System Bus (ASB) и Advanced Peripheral Bus (APB). В ее второй разновидности, AMBA 2 в 1999, ARM добавила AMBA High-performance Bus (AHB) с протоколом по одному тактовому фронту»[79].

Иными словами, AHB – это высокопроизводительная шина AMBA. В семействе STM32 AHB имеет предварительный делитель, который использует SYSCLK в качестве входного сигнала. Предполагая, что SYSCLK составляет 72 МГц, в табл. 15.6 приведены варианты выбора для AHB.

Таблица 15.6. Частоты шины AHB STM32F103C8T6 при SYSCLK = 72 МГц

Битовое значение	Предделитель	Результирующая частота
0xxx	SYSCLK без деления	72 МГц
1000	SYSCLK / 2	36 МГц
1001	SYSCLK / 4	18 МГц
1010	SYSCLK / 8	9 МГц
1011	SYSCLK / 16	4.5 МГц
1100	SYSCLK / 64	1.125 МГц
1101	SYSCLK / 128	562.5 кГц
1110	SYSCLK / 256	281.25 кГц
1111	SYSCLK / 512	140.625 кГц

Начиная с SYSCLK, на рис. 15.3 показано, почему все это называется деревом тактовых сигналов. Из сигнала SYSCLK на основе настроенных предделителей и установок получаются многие другие тактовые сигналы.

rcc_clock_setup_in_hse_8mhz_out_72mhz()

В большинстве примеров, представленных в этой книге, в начале основной программы изначально использовалась следующая процедура libopencm3:

```
rcc_clock_setup_in_hse_8mhz_out_72mhz();
```

Проект libopencm3 теперь требует, чтобы это было в следующем виде:

```
rcc_clock_setup_pll(&rcc_hse_configs[RCC_CLOCK_HSE8_72MHZ]);
```

Чтобы помочь нам понять, что именно делается, в листинге 15.1 показан исходный код функции libopencm3 (исходный код наглядно демонстрирует то, что нам нужно знать).

[79] Цитата (с орфографической правкой) из русскоязычной версии статьи «Википедии» «Advanced_Microcontroller_Bus_Architecture».

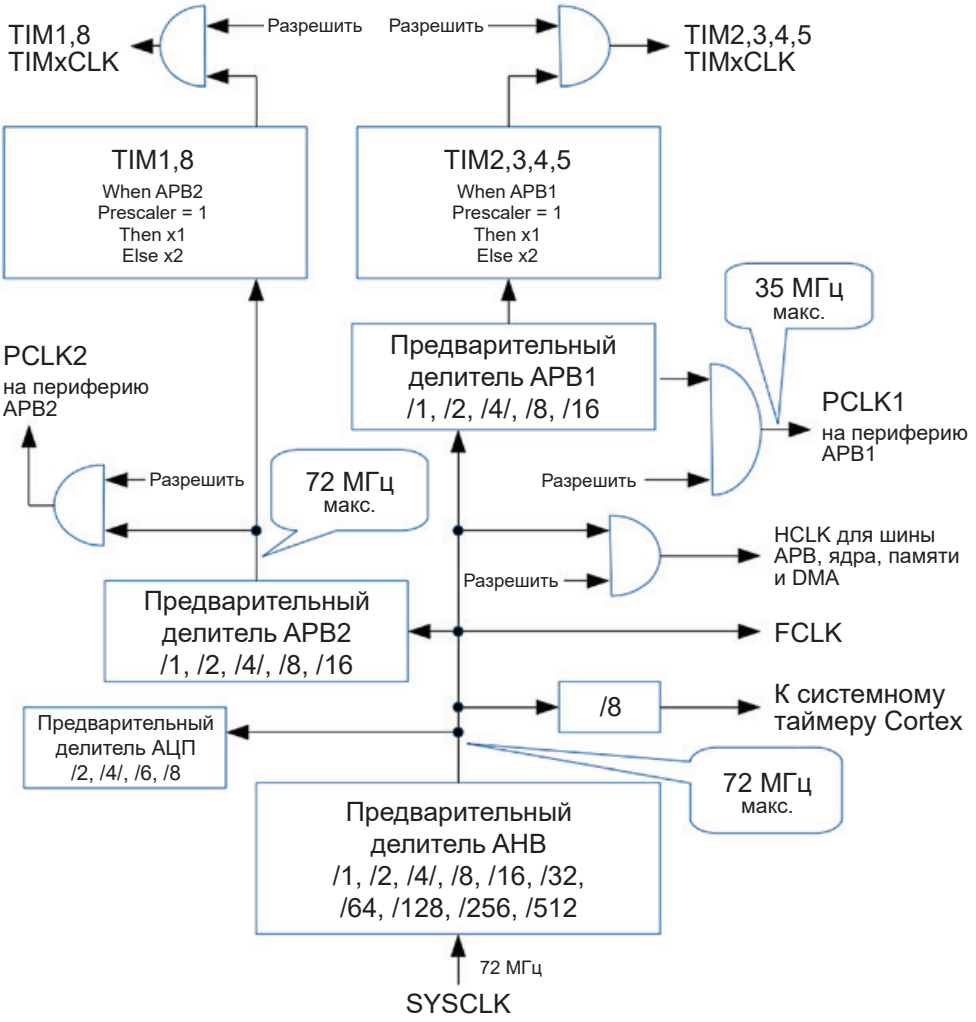


Рис. 15.3. Дерево тактовых сигналов, начиная с SYSCLK

Листинг 15.1. Функция `rcc_clock_setup_in_hse_8mhz_out_72mhz()`

```
0911: void rcc_clock_setup_in_hse_8mhz_out_72mhz(void)
0912: {
0913:   /* Enable internal high-speed oscillator. */
0914:   rcc_osc_on(RCC_HSI);
0915:   rcc_wait_for_osc_ready(RCC_HSI);
0916:
0917:   /* Select HSI as SYSCLK source. */
0918:   rcc_set_sysclk_source(RCC_CFGR_SW_SYSCLKSEL_HSICLK);
0919:
0920:   /* Enable external high-speed oscillator 8MHz. */
0921:   rcc_osc_on(RCC_HSE);
0922:   rcc_wait_for_osc_ready(RCC_HSE);
```

```
0923:   rcc_set_sysclk_source(RCC_CFGR_SW_SYSCLKSEL_HSECLK);
0924:
0925:   /*
0926:    * Set prescalers for AHB, ADC, ABP1, ABP2.
0927:    * Do this before touching the PLL (TODO: why?).
0928:   */
0929:   rcc_set_hpre(RCC_CFGR_HPRE_SYSCLK_NODIV); /*Set. 72MHz Max. 72MHz */
0930:   rcc_set_adcpre(RCC_CFGR_ADCPRE_PCLK2_DIV8); /*Set. 9MHz Max. 14MHz */
0931: rcc_set_ppre1(RCC_CFGR_PPRE1_HCLK_DIV2); /*Set. 36MHz Max. 36MHz */
0932: rcc_set_ppre2(RCC_CFGR_PPRE2_HCLK_NODIV); /*Set. 72MHz Max. 72MHz */
0933:
0934:   /*
0935:    * Sysclk runs with 72MHz -> 2 waitstates.
0936:    * 0WS from 0-24MHz
0937:    * 1WS from 24-48MHz
0938:    * 2WS from 48-72MHz
0939:   */
0940:   flash_set_ws(FLASH_ACR_LATENCY_2WS);
0941:
0942:   /*
0943:    * Set the PLL multiplication factor to 9.
0944:    * 8MHz (external) * 9 (multiplier) = 72MHz
0945:   */
0946:   rcc_set_pll_multiplication_factor(RCC_CFGR_PLLMUL_PLL_CLK_MUL9);
0947:
0948:   /* Select HSE as PLL source. */
0949:   rcc_set_pll_source(RCC_CFGR_PLLSRC_HSE_CLK);
0950:
0951:   /*
0952:    * External frequency undivided before entering PLL
0953:    * (only valid/needed for HSE).
0954:   */
0955:   rcc_set_pllxtpre(RCC_CFGR_PLLXTPRE_HSE_CLK);
0956:
0957:   /* Enable PLL oscillator and wait for it to stabilize. */
0958:   rcc_osc_on(RCC_PLL);
0959:   rcc_wait_for_osc_ready(RCC_PLL);
0960:
0961:   /* Select PLL as SYSCLK source. */
0962:   rcc_set_sysclk_source(RCC_CFGR_SW_SYSCLKSEL_PLLCLK);
0963:
0964:   /* Set the peripheral clock frequencies used */
0965:   rcc_ahb_frequency = 72000000;
0966:   rcc_apb1_frequency = 36000000;
0967:   rcc_apb2_frequency = 72000000;
0968: }
```

Основные используемые шаги следующие.

1. Генератор HSI включается и ожидает готовности (строки 914–915).
2. Выбирается генератор HSI в качестве источника SYSCLK (строка 918).
3. HSE (кварцевый генератор 8 МГц) включается в строке 921, и программа ожидает его готовности (строка 922).

4. Затем SYSCLK переключается на использование генератора HSE (строка 923). Обратите внимание, что это еще не ФАПЧ, поэтому на данный момент SYSCLK составляет 8000 МГц, как определено кристаллом.
5. Шина AHB настраивается на использование предварительного делителя (строка 929), в результате чего входная тактовая частота AHB составляет 72 МГц после того, как ФАПЧ будет выбрана (позже) в качестве источника тактовой частоты.
6. Предварительный делитель АЦП настроен на значение 8 (строка 930), что дает частоту 9 МГц (после переключения на ФАПЧ). Как указано в комментарии, она не должна превышать 14 МГц.
7. Предварительный делитель для шины периферии APB1 настроен на значение 2, в результате чего тактовая частота APB1 после переключения на ФАПЧ (строка 931) составляет 36 МГц. Это максимальная частота для этой шины.
8. Предварительный делитель для шины APB2 настроен на неиспользование деления, в результате чего частота APB2 равна 72 МГц при последующем переключении на использование ФАПЧ (строка 932). Это также максимальная частота для APB2.
9. Поскольку SYSCLK работает на частоте 72 МГц, для каждого доступа к флеш-памяти должно быть вставлено два цикла ожидания (строка 940).
10. Теперь для ФАПЧ установлен множитель 9, чтобы установить выходную тактовую частоту 72 МГц (строка 946).
11. Строка 955 удаляет любые настройки ФАПЧ для HSE, которые могли быть установлены.
12. Наконец, строка 962 выбирает ФАПЧ в качестве источника SYSCLK. Это увеличивает частоту SYSCLK с 8 до 72 МГц, при этом шина AHB теперь работает на частоте 72 МГц, APB1 – на частоте 36 МГц, а APB2 – на частоте 72 МГц.
13. Строки 965–967 устанавливают глобальные значения `rcc_ahb_frequency`, `rcc_apb1_frequency` и `rcc_apb2_frequency` для использования приложением.

Глобальные переменные определены в *rcc.h* и определяются следующим образом:

```
#include <libopencm3/stm32/rcc.h>
extern uint32_t rcc_ahb_frequency;
extern uint32_t rcc_apb1_frequency;
extern uint32_t rcc_apb2_frequency;
```

Из этого видно, что при запуске необходимо сделать немало, чтобы убедиться, что тактовый сигнал не сбивается и не выходит из строя.

Периферийные устройства APB1

Каждое периферийное устройство, подключенное к шине APB1 на плате Blue Pill, получает тактовую частоту 36 МГц (если не настроено иное). Однако каж-

дое периферийное устройство имеет собственный логический элемент «И» для включения/отключения разрешения этих тактовых импульсов в целях экономии энергии.

Чтобы обеспечить получение тактового сигнала, периферийное устройство должно его разрешить. Например, для периферийного устройства CAN необходимо разрешить синхронизацию самого периферийного устройства. То же самое относится и к периферийным устройствам таймеров на APB1.

Периферийные устройства APB2

Как и периферийные устройства APB1, каждое периферийное устройство, подключенное к шине APB2, должно включать/отключать разрешение получения собственной тактовой частоты 72 МГц. Это также относится к периферийным устройствам таймеров APB2.

Таймеры

Здесь особое внимание уделяется таймерам, поскольку в их настройке есть не столь очевидная особенность. Таймеры APB1 и APB2 имеют предварительный делитель, позволяющий делить тактовую частоту шины для получения более низкой частоты. Исключением, однако, является то, что, когда делитель APB1/APB2 установлен на 1, частота шины умножается на 2!



Повторим: когда предварительный делитель таймера установлен на 1, выходной сигнал предделителя равен частоте шины, умноженной на 2!

Функция rcc_set_mco()

Библиотека libopencm3 предоставляет функцию с именем `rcc_set_mco()` для настройки вывода тактовой частоты (microcontroller clock output, MCO) на GPIO PA8. Допустимые значения макроопределения, передаваемые в качестве аргумента, описаны в табл. 15.7.

```
rcc_set_mco(macro);
```

Таблица 15.7. Действительные значения аргумента для `rcc_set_mco()`

Имя	Значение	Тактовый сигнал для вывода MCO
RCC_CFGR_MCO_NOCLK	0x0	Нет тактового сигнала (отключен)
RCC_CFGR_MCO_SYSCLK	0x4	SYSCLK
RCC_CFGR_MCO_HSI	0x5	HSI
RCC_CFGR_MCO_HSE	0x6	HSE
RCC_CFGR_MCO_PLL_DIV2	0x7	ФАПЧ / 2

Вызова процедуры `rcc_set_mco()` самого по себе недостаточно. GPIO PA8 должен быть настроен для альтернативной функции ввода-вывода:

```
rcc_periph_clock_enable(RCC_GPIOA);
gpio_set_mode(GPIOA,
```

```
  GPIO_MODE_OUTPUT_50_MHZ,
  GPIO_CNF_OUTPUT_ALTFN_PUSHPULL, // ALTFN
  GPIO8); // PA8=MCO
```

Демопример HSI

Пример кода для демонстрации тактовых сигналов HSI находится здесь:

```
$ cd ~/stm32f103c8t6/hsi
$ make clobber
$ make
$ make flash
```

Обратите внимание, что в этом примере не используется FreeRTOS. Код примера просто обеспечивает вывод тактового сигнала HSI на вывод PA8 GPIO. В листинге 15.2 показана ответственная за это основная программа.

Листинг 15.2. Демонстрационная программа *hsi.c*

```
0010: int
0011: main(void) {
0012:
0013:   // LED Configuration:
0014:   rcc_periph_clock_enable(RCC_GPIOC);
0015:   gpio_set_mode(GPIOC,GPIO_MODE_OUTPUT_2_MHZ,
0016:              GPIO_CNF_OUTPUT_PUSHPULL,GPIO13);
0017:   gpio_clear(GPIOC,GPIO13); // LED Off
0018:
0019:   // MCO Configuration:
0020:   rcc_periph_clock_enable(RCC_GPIOA);
0021:   gpio_set_mode(GPIOA,
0022:       GPIO_MODE_OUTPUT_50_MHZ,
0023:       GPIO_CNF_OUTPUT_ALTFN_PUSHPULL,
0024:       GPIO8); // PA8=MCO
0025:
0026:   rcc_set_mco(RCC_CFGR_MCO_HSI);
0027:
0028:   gpio_set(GPIOC,GPIO13); // LED On
0029:   for (;;);
0030:   return 0;
0031: }
```

Помимо настройки LED PC13, основными элементами кода являются следующие:

1) тактовый сигнал для GPIOA включается в строке 20;
2) у GPIOA вывод PA8 настраивается на выход (макс. 50 МГц, строка 22) и на альтернативную функцию (строка 23) в двухтактном режиме (push/pull);
3) в строке 26 на PA8 направляется тактовый сигнал HSI.

После прошивки STM32 вы сможете увидеть тактовый сигнал HSI на PA8 с помощью осциллографа сразу после подачи питания или после сброса (рис. 15.4). На рисунке видно, что тактовая частота HSI составляет около 8 МГц.



Рис. 15.4. Осциллограмма HSI MCO

Демопример HSE

Пример кода демонстрации тактовых сигналов HSE находится здесь:

```
$ cd ~/stm32f103c8t6/hse
$ make clobber
$ make
$ make flash
```

Обратите внимание, что этот пример также не использует FreeRTOS. Его код просто обеспечивает вывод тактового сигнала HSE на контакт PA8 GPIO. Единственная разница между этой демонстрационной программой и предыдущей – одна строка:

```
rcc_set_mco(RCC_CFGR_MCO_HSE);
```

После прошивки STM32 вы сможете увидеть тактовые сигналы HSE на PA8 с помощью осциллографа сразу после подачи питания (рис. 15.5). На рисунке видно, что частота равна 8 МГц более точно.



Рис. 15.5. Осциллограмма HSE на выводе MCO
(обратите внимание, насколько она похожа на рис. 15.4)

Демонстрация частоты ФАПЧ/2

Пример кода для демонстрации тактовой частоты ФАПЧ, поделенной на 2, находится здесь:

```
$ cd ~/stm32f103c8t6/mco_pll2
$ make clobber
$ make
$ make flash
```

Еще раз обратите внимание, что этот пример также не использует FreeRTOS. Его код просто обеспечивает вывод тактового сигнала PLL×2 на вывод PA8 GPIO. Единственная разница между этой демопрограммой и примером HSE – одна строчка:

```
rcc_set_mco(RCC_CFGR_MCO_PLL_DIV2);
```

Отправка на PA8 тактового сигнала ФАПЧ/2 (вместо реальной тактовой частоты ФАПЧ 72 МГц) обусловлена тем, что вывод GPIO ограничен частотой 50 МГц. Вы можете попытаться отправить сигнал на частоте 72 МГц, но сигнал будет сильно искажен и, возможно, вызовет нагрузку на задействованные активные компоненты. 36 МГц вполне в пределах приемлемого диапазона характеристик вывода.

После прошивки STM32 вы сможете увидеть тактовую частоту ФАПЧ/2 на PA8 с помощью осциллографа, как только будет подано питание. Обратите внимание на рис. 15.6, что показанная частота составляет около 36 МГц, как и ожидалось (72 МГц / 2).

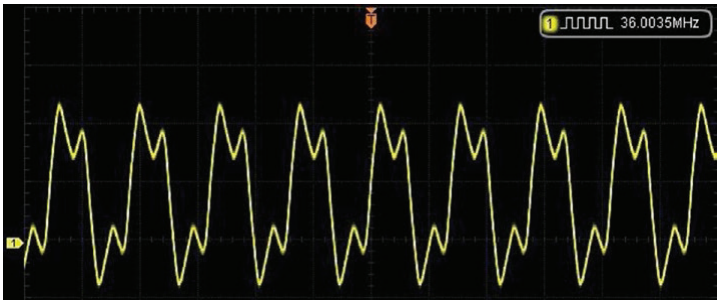


Рис. 15.6. Осциллограмма ФАПЧ/2 на выводе MCO

Итоги

В этой главе был представлен обзор системы дерева тактовых сигналов, начиная с основных источников синхронизации: HSI и HSE для системных тактов и HSE, LSE и LSI для часов реального времени. Генератор LSI также использовался сторожевым таймером.

Основная категория источников связана с системными тактовыми сигналами. Системная тактовая частота может использовать ФАПЧ, которая способна увеличивать входную тактовую частоту до 72 МГц. На основе системных тактов получаются тактовые сигналы шины AHB. Затем из тактового сигнала AHB получаются тактовые сигналы для шин APB1 и APB2.

Наконец, было отмечено, что каждое периферийное устройство, которому требуется тактовый сигнал, имеет свой собственный логический элемент, который позволяет включить или отключить тактовый сигнал. Такая конструкция экономит электроэнергию, оставляя ненужные сигналы отключенными.

Демонстрационные примеры HSI, HSE и PLL/2 иллюстрируют, как проверить тактовые сигналы, которые в противном случае окажутся невидимыми. Также возможно, что тактовый сигнал, выведенный на вывод MCO PA8, может быть применен к внешним периферийным устройствам, если им требуется отдельный тактовый сигнал.

Дополнительные источники

1. «Усовершенствованная архитектура шины микроконтроллера». «Википедия». Оригинал на английском языке: https://en.wikipedia.org/wiki/Advanced_Microcontroller_Bus_Architecture (дата доступа 12.09.2024).

Упражнения

1. В чем преимущество RC-генераторов?
2. В чем недостаток RC-генераторов?
3. В чем преимущество кварцевых генераторов?
4. Для чего используется ФАПЧ?
5. Что означает AHB?
6. Почему GPIO PA8 должен быть настроен с `GPIO_CNF_OUTPUT_ALTFN_PUSHPULL`?

Глава 16 • Режим PWM таймера 2

Семейство STM32 имеет сложный набор опций работы таймеров. STM32F103 имеет таймеры общего назначения TIM2, 3, 4 и 5, а также расширенные таймеры TIM1 и TIM8. Таймеры общего назначения имеют множество функций, поэтому вам не придется использовать функции расширенных таймеров.

В этой главе будет продемонстрировано одно из часто востребованных применений таймера – управление сервоприводом с помощью PWM (широтно-импульсной модуляции, ШИМ). На рис. 16.1 показан пример типичного сервопривода, отключенного от управляемых механизмов.

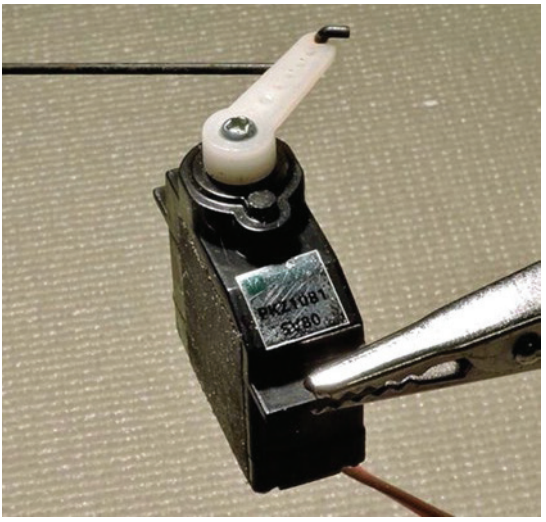


Рис. 16.1. Типичный RC-сервопривод[80] *PKZ1081 SV80*

PWM-сигналы

Как выглядит PWM-сигнал? Рисунок 16.2 иллюстрирует один из них. Показанный образец имеет импульс высокого уровня длительностью 2.6 мс (обратите внимание на курсоры и показанное значение BX-AX). Измеренная частота составила около 147 Гц. Это означает, что весь период сигнала составляет около 6.8 мс.

[80] Английский термин servomotor (пишется как слитно, так и раздельно: servo motor) следует переводить как «сервопривод» (так как он только однократно передвигает подвижную деталь на определенный угол или расстояние). В англоязычном ритейле сервоприводы часто называют радиоуправляемыми приводами (RC-servo – Radio Controlled servomotor), потому что в большинстве случаев они предназначены для установки в автомодели или дроны, дистанционно управляемые по радио. Это, конечно, не значит, что радиосигнал непосредственно управляет двигателем (см. следующую главу), но здесь мы следуем за авторской терминологией.

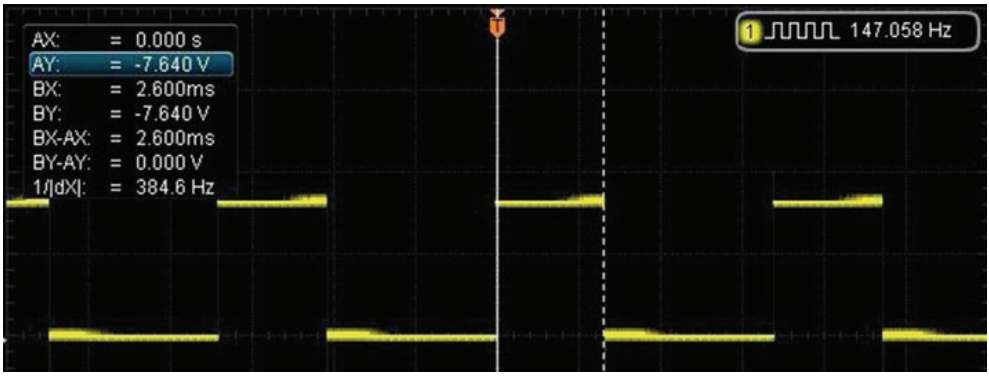


Рис. 16.2. Сигнал PWM с периодом 147 Гц и длительностью импульса 2.6 мс

Большинство PWM-элементов управления серводвигателями основаны на длительности времени, в течение которого сигнал остается высоким, а не на коэффициенте заполнения (duty cycle), но системы управления различаются[81].

Таймер 2

Таймеры со 2-го по 5-й в STM32F103C8T6 являются таймерами общего назначения. Несмотря на общее назначение, они достаточно гибкие. Их общие особенности включают в себя следующее:

- 16-битный счетчик суммирующий, вычитающий, суммирующий/вычитающий с автоматической перезагрузкой;
- 16-битный предварительный делитель для деления входной частоты счетчика от 1 до 65 536;
- предварительный делитель можно менять на лету;
- до четырех независимых каналов для:
  - захвата входа;
  - захвата выхода;
  - генерации PWM (режимы с выравниванием по краю и центру периода);
  - выхода в одноимпульсном режиме;

[81] Большинство систем PWM-управления двигателями постоянного тока, яркостью светильников или, например, температурой электронагревателей основаны на изменении коэффициента заполнения – относительной длительности высокого уровня в течение периода, что приводит к изменению среднего значения напряжения. При достаточно высокой частоте PWM-сигнала (сотни-тысячи герц, т. е. много выше собственной инерции управляемого элемента или человеческого глаза, как в случае светодиодных ламп) собственно частота и абсолютная длительность импульса при этом не имеют значения. Это и есть настоящая широтно-импульсная модуляция, PWM. Сервоприводы устроены иначе – они управляются именно абсолютной продолжительностью импульса в миллисекундах. Как справедливо замечено в следующей главе, такой способ, вообще-то, широтно-импульсной модуляцией называть не следует, хотя в данном случае используется функция PWM таймера (см. далее).

- схема синхронизации управления таймером с внешними сигналами и для взаимодействия с другими таймерами;
- генерация прерываний/DMA:
  - обновление счетчика переполнения/недополнения, инициализация счетчика программным обеспечением или пороговым триггером;
  - пороговое событие (запуск счетчика, остановка, инициализация или подсчет по внутреннему/внешнему триггеру);
  - захват входа;
  - захват выхода;
- триггерный вход.

Все это обсуждается в разделе 15 справочного руководства RM0008 («Дополнительные источники», [1] к главе 7), но в этой главе мы сосредоточимся на генерации сигнала ШИМ. Программное обеспечение для примеров этой главы находится в следующем каталоге:

```
$ cd ~/stm32f103c8t6/rtos/tim2_pwm
```

Исходный код полностью находится в файле *main.c*. Части задачи `task1()`, относящиеся к таймеру, будут перечислены далее в отдельных небольших разделах, чтобы облегчить обсуждение. Некоторая необходимая начальная конфигурация показана в листинге 16.1.

Листинг 16.1. Конфигурация вывода PA1 для выхода таймера 2

```
0029: rcc_periph_clock_enable(RCC_TIM2);
0030: rcc_periph_clock_enable(RCC_AFIO);
0031:
0032: // PA1 == TIM2.CH2
0033: rcc_periph_clock_enable(RCC_GPIOA);
0034: gpio_primary_remap(
0035:   AFIO_MAPR_SWJ_CFG_JTAG_OFF_SW_OFF, // Optional
0036:   AFIO_MAPR_TIM2_REMAP_NO_REMAP); // default: TIM2.CH2=GPIOA1
0037: gpio_set_mode(GPIOA,GPIO_MODE_OUTPUT_50_MHZ, // High speed
0038:   GPIO_CNF_OUTPUT_ALTFN_PUSHPULL,GPIO1); // GPIOA1=TIM2.CH2
```

Строка 29 включает тактовый сигнал для таймера 2, а тактовый сигнал AFIO[82] включаются в строке 30. Это необходимо сделать до строки 35.

Строка 33 включает тактовую частоту порта GPIOA для использования PA1. Строки 34–36 представляют собой вызов AFIO, который говорит: используйте сопоставление по умолчанию, при котором канал 2 таймера 2 выходит на PA1 (в случае по умолчанию это можно опустить). Измените этот вызов, если вам нужно, чтобы он вместо этого перешел на PB3. Наконец, строки 37 и 38 используют макроопределение ALTFN для подключения вывода GPIO к таймеру PA1. Это очень важно.

[82] Напомним, что AFIO означает Alternate Function Input Output, альтернативная функция ввода-вывода (см. раздел «AFIO» главы 12).

В листинге 16.2 показан код, который инициализирует таймер и устанавливает его рабочий режим.

Листинг 16.2. Инициализация таймера 2 и установка его режима

```
0042: // TIM2:
0043: timer_disable_counter(TIM2);
0044: timer_reset(TIM2);
0045:
0046: timer_set_mode(TIM2,
0047:     TIM_CR1_CKD_CK_INT,
0048:     TIM_CR1_CMS_EDGE,
0049:     TIM_CR1_DIR_UP);
```

Строки 43 и 44 отключают счетчик и сбрасывают таймер 2. Затем строка 46 устанавливает рабочий режим для таймера следующим образом:

- `TIM_CR1_CKD_CK_INT` настраивает коэффициент деления между частотой таймера (`CLK_INT`) и тактовой частотой выборки, используемой цифровыми фильтрами. Здесь мы не используем цифровые фильтры, поэтому это макроопределение устанавливает частоту цифрового фильтра, равную тактовой частоте (подробнее см. описание `TIMx_CR1.CKD` для таймера 2 на сайте https://libopencm3.org);
- `TIM_CR1_CMS_EDGE` указывает, что должен использоваться режим с выравниванием по краю (а не с выравниванием по центру);
- `TIM_CR1_DIR_UP` указывает, что счетчик будет суммировать.

В листинге 16.3 продолжается настройка таймера 2, а затем начинается демонстрационный пример запуска таймера.

Листинг 16.3. Оставшаяся часть конфигурации и запуск таймера

```
0026: static const int ms[4] = { 500, 1200, 2500, 1200 };
0027: int msx = 0;
...
0050: timer_set_prescaler(TIM2,72);
0051: // Only needed for advanced timers:
0052: // timer_set_repetition_counter(TIM2,0);
0053: timer_enable_preload(TIM2);
0054: timer_continuous_mode(TIM2);
0055: timer_set_period(TIM2,33333);
0056:
0057: timer_disable_oc_output(TIM2,TIM_OC2);
0058: timer_set_oc_mode(TIM2,TIM_OC2,TIM_OCM_PWM1);
0059: timer_enable_oc_output(TIM2,TIM_OC2);
0060:
0061: timer_set_oc_value(TIM2,TIM_OC2,ms[msx=0]);
0062: timer_enable_counter(TIM2);
```

Строка 50 устанавливает частоту таймера, устанавливая для нее предварительный делитель. Этот момент будет описан полностью позже. Строка 52

необходима только для расширенных таймеров (таймеры 1 и 8) и поэтому закомментирована. Этот вызов игнорируется libopencm3 для таймеров 2–5.

Строки 53 и 54 настраивают еще две опции таймера. Вызов функции `timer_enable_preload()` указывает, что регистр `TIM2_ARR` буферизован (для перезагрузки). Функция `timer_continous_mode()` настраивает таймер так, чтобы он продолжал работать, а не останавливался после одного импульса. В строке 55 задается максимальное значение таймера для установления его периода. Подробнее об этом будет также сказано позже.

Строки 57–59 настраивают канал OC2 (канал 2 сравнения выходов) таймера 2 для работы в режиме PWM1. Настройка режима PWM1 происходит в строке 58. Значение `TIM_OCM_PWM1`, согласно документации, определяет следующее:

> При прямом счете выходной канал активен (высокий уровень), если счетчик таймера меньше, чем регистр захвата/сравнения, иначе канал перейдет в низкий уровень.

В строке 61 выходному регистру сравнения присваивается значение, найденное в массиве `ms[]`. Это устанавливает начальную ширину импульса (в микросекундах). Таймер наконец запускается в строке 62.

В строке 26 в начале листинга объявляется массив `ms[]`, содержащий четыре значения. Это ширина импульса в микросекундах, центральное значение – 1200 (1.2 мс). При установленном в строке 58 режиме произойдет следующее:

- значения счетчика от 0 до `ms[msx]-1` приведут к тому, что GPIO PA1 перейдет в высокий уровень (пока он считается активным);
- как только счетчик превысит это значение, уровень PA1 станет низким.

При такой конфигурации выход PA1 первоначально будет находиться в состоянии высокого уровня на 500 мкс (0.5 мс), в общей сложности 33 333 отсчета (период, настроенный в строке 55).

PWM-цикл

Демонстрационная программа выполняет цикл, показанный в листинге 16.4.

Листинг 16.4. Демонстрационный цикл PWM

```
0064: for (;;) {
0065:     vTaskDelay(1000);
0066:     gpio_toggle(GPIOC,GPIO13);
0067:     msx = (msx+1) % 4;
0068:     timer_set_oc_value(TIM2,TIM_OC2,ms[msx]);
0069: }
```

В начале цикла строка 65 задерживает выполнение примерно на одну секунду (1000 мс), после чего переключается светодиод GPIO PC13 (строка 66). В строке 67 обновляется индексная переменная массива `msx`, чтобы она увеличивалась, но опять начиналась с нуля, если превышает значение 3. Следующее значение массива с этим индексом используется для изменения регистра сравнения выхода в строке 68. Как только сервопривод получит этот

импульс, его позиция изменится (или вы можете просмотреть изменение ширины импульса на осциллографе).

Вычисление предварительного делителя таймера

В листинге 16.3 строка 50 представляла собой вызов функции, устанавливающей предварительный делитель счетчика. Давайте разберем этот расчет. Для вашего удобства настройка предделителя была такая:

```
timer_set_prescaler(TIM2,72)
```

Вход счетчика составляет 72 МГц, поскольку при настройке Blue Pill делитель APB1 обычно устанавливается на 2, и, таким образом, частота шины делится до 36 МГц (ее максимум). Что говорится в примечании о предделителе для TIM2, 3, 4 и 5?

Когда предварительный делитель APB1 = 1, частота умножается на 1, иначе – на 2.

Вас можно извинить, если вы запутались. Вкратце: у нас есть частота SYSCLCK 72 МГц, деленная на 2, чтобы соответствовать максимальной частоте 36 МГц для шины APB1. После этого есть еще один делитель, который применяется к таймерам со 2-го по 5-й. Я буду называть его глобальным предварительным делителем, поскольку он применим ко всем таймерам от 2-го по 5-й. Выходные данные этого предделителя поступают в собственные предделители таймеров. Неудивительно, что со всеми этими предварительными делителями возникает путаница!

Цитата о предварительном делителе APB1 напоминает нам о том, что частота, выходящая из глобального делителя для таймеров, на самом деле является частотой шины APB1, умноженной на 2[83]! Следовательно, на входе в собственный предделитель таймера 2 получается 72 МГц, а не 36. Теперь мы можем объяснить числитель формулы: 72 000 000 / 72 = 1 000 000.

Числитель представляет частоту 72 МГц, поступающую в частный предварительный делитель таймера 2. Установка значения этого предделителя таймера 2, равного 72, приводит к обновлению таймера с частотой 1 МГц (строка 50 в листинге 16.3).

Нам не пришлось использовать это соотношение, но оно оказалось удобным. Каждый отсчет происходит за 1 мкс, что позволяет нам указать длительность импульса в микросекундах.

Цикл 30 Гц

Я предположил, что RC-сервоприводу требуется частота цикла 30 Гц. Это определяется конфигурацией, показанной в листинге 16.3, строка 55:

$$\frac{\frac{f_{ARB1} \times 2}{\text{делитель}}}{f_{period}} = \frac{36\,000\,000 \times 2}{\frac{72}{30}} = 33\,333.3.$$

[83] См. также раздел «Таймеры» на стр. 360.

Чтобы запрограммировать эту формулу, мы могли бы написать код:

```
timer_set_prescaler(TIM2,36000000*2/72/30);
```

или проще:

```
timer_set_prescaler(TIM2,33333);
```

Чтобы уменьшить период (увеличить частоту) до 50 Гц, просто замените в расчете 30 на 50.

Помните, что частота, поступающая в частный предделитель таймера, удваивается, если предделитель APBx равен 1.

Подключение сервопривода

К сожалению, сервоприводы обычно работают при напряжении около 6 В. STM32 для устранения этого разрыва требуется небольшая схема драйвера.

Хорошей новостью является то, что интерфейс довольно прост. Вы можете использовать CMOS-микросхему CD4050BE для приема сигнала напряжением 3.3 В на входе и создания уровня почти 6 В на выходе (рис. 16.3). Обратите внимание, что контакт питания микросхемы (выв. 1 IC1A) подключен к питанию сервопривода. Конструкция CD4050 такова, что вход (контакт 3 IC1A) можно безопасно подключить к STM32.

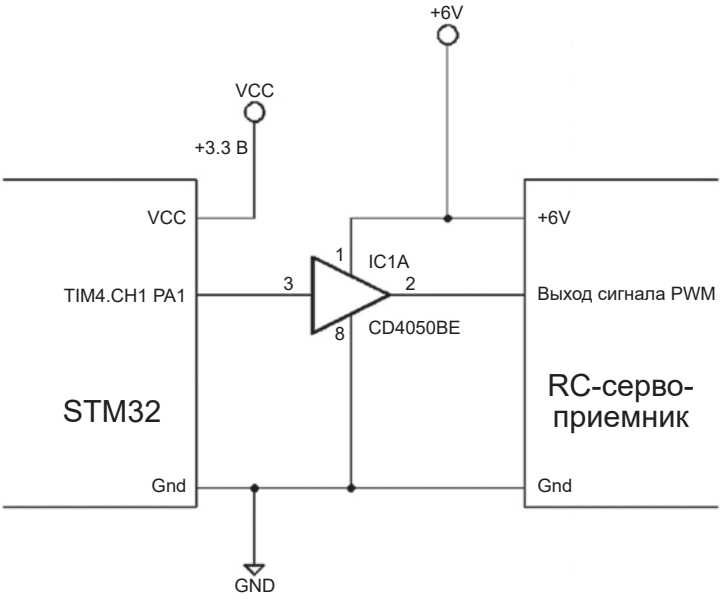


Рис. 16.3. Схема интерфейса 6 В между микроконтроллером STM32 и управляющим выводом сервопривода

Другой заменой CD4050 могут быть 74HCT244 или 74HCT245 (с другой разводкой выводов). Крайне важно, чтобы эти специальные чипы использовались для устранения разрыва в напряжении. Подробнее об этом см. в статье «Выбор правильного преобразователя логического уровня» (http://www.gaw.ru/html.cgi/txt/publ/powersuply/converter_levels.htm). Хотя другие CMOS-чипы могут работать при напряжении 6 В, они могут не воспринимать входной сигнал STM32 как высокий (это зависит от порогового значения VIH)[84].

При подключении схемы убедитесь, что вы подключили общий провод 6-вольтовой системы к выводу GND STM32. Это обеспечивает общую точку опорного напряжения.

Запуск демопрограммы

После сборки и прошивки программного обеспечения выполните следующие действия:

```
$ make clobber
$ make
$ make flash
```

Все, что вам нужно сделать, – это убедиться в правильности подключений и подключить питание (или подключить STM32 от USB). В этом примере не используется связь USB или UART.

При подключенном сервоприводе он должен каждую секунду подергиваться в одно крайнее, затем среднее положение, в другое крайнее, снова в среднее и обратно в первое крайнее положение. Сервоприводы различаются по требованиям к ШИМ, поэтому вам может потребоваться изменить следующее:

- длительность импульсов в листинге 16.3, строка 26 (указаны в микросекундах),
- период в листинге 16.3, строка 55.

Для развлечения прикрепите лазерную указку к сервоприводу.

PWM на PB3

Выход сравнения таймера 2 можно перенаправить на PB3. Это можно использовать, если вам требуется 5-вольтовый ШИМ-сигнал. PB3 – это GPIO,

[84] Указанные автором 74HCT244 или 74HCT245, во-первых, решительно избыточны для целей, подобных разбираемому примеру (содержат по 8 элементов, из которых требуется только один), во-вторых, у них напряжение питания ограничено величиной 5.5 В. Для преобразования уровня управляющего сигнала удобнее и надежнее использовать специальные преобразователи уровня с двумя напряжениями питания, например CD40109BE; или решения с открытым стоком (например, очень удобную микросхему CD40107BE, содержащую всего два логических инвертора, которые можно включить последовательно, чтобы получить неинвертированный сигнал). В качестве выхода с открытым стоком можно использовать и непосредственно выход STM32, как рассказывается в разделе «PWM на PB3» далее. Тогда промежуточный преобразователь уровня не понадобится вовсе.

устойчивый к напряжению 5 В, хотя он не может напрямую генерировать сигнал высокого напряжения 5 В. Однако при работе в качестве GPIO с открытым стоком подтягивающий резистор может повысить напряжение сигнала до 5 В[85].

Исходный код этой версии проекта находится здесь:

```
$ cd stm32f103c8t6/rtos/tim2_pwm_pb3
```

Модуль *main.c* практически идентичен приведенному примеру, за исключением следующих различий:

```
$ diff -c ../tim2_pwm/main.c main.c
...
! // PA1 == TIM2.CH2
! rcc_periph_clock_enable(RCC_GPIOA); // Need GPIOA clock
    gpio_primary_remap(
        AFIO_MAPR_SWJ_CFG_JTAG_OFF_SW_OFF, // Optional
!       AFIO_MAPR_TIM2_REMAP_NO_REMAP); // default: TIM2.CH2=GPIOA1
!   gpio_set_mode(GPIOA,GPIO_MODE_OUTPUT_50_MHZ, // High speed
!       GPIO_CNF_OUTPUT_ALTFN_PUSHPULL,GPIO1); // GPIOA1=TIM2.CH2
--- 29,41 ----
!   // PB3 == TIM2.CH2
!   rcc_periph_clock_enable(RCC_GPIOB); // Need GPIOB clock
    gpio_primary_remap(
        AFIO_MAPR_SWJ_CFG_JTAG_OFF_SW_OFF, // Optional
!       AFIO_MAPR_TIM2_REMAP_PARTIAL_REMAP1); // TIM2.CH2=PB3
!   gpio_set_mode(GPIOB,GPIO_MODE_OUTPUT_50_MHZ, // High speed
!       GPIO_CNF_OUTPUT_ALTFN_OPENDRAIN,GPIO3); // PB3=TIM2.CH2
```

Первое изменение активирует GPIOB вместо GPIOA. После этого вызов `gpio_primary_remap()` использует аргумент `AFIO_MAPR_TIM2_REMAP_PARTIAL_REMAP1` для направления вывода сравнения 2 таймера 2 в PB3.

Последнее изменение в `gpio_set_mode()` настраивает PB3 для использования вывода с открытым стоком. Это необходимо, поскольку PB3 не может напрямую выдавать сигнал 5 В (он может подтягивать его только до +3.3 В). Однако PB3 может позволить подтягивать его до +5 В с помощью резистора, когда он работает с открытым стоком. Это изменение и добавление подтягивающего резистора в диапазоне от 2 до 10 кОм позволит генерировать выходной сигнал напряжением 5 В.

Другие таймеры

Когда дело доходит до сигналов PWM для сервоприводов, люди часто хотят знать, сколько каналов PWM можно сделать доступными. В табл. 16.1

[85] Напоминаем (см. сноску 22 на стр. 66), что 5-вольтовые выводы STM32 выдерживают до V_{cc} + 4 В, т. е. максимум около 7 В.

приведены таймеры, доступные на STM32F103C8T6, и назначения GPIO по умолчанию. Некоторые таймеры также имеют альтернативные назначения.

Таблица 16.1. Таймеры, доступные в STM32F103C8T6

Таймер	Тип	Канал 1	Канал 2	Канал 3	Канал 4
TIM1	Расширенный	PA12 PB13=CH1N	PA8 PB14=CH2N	PA9 PB15=CH3N	PA10 PB12=BKIN
TIM2	Общего назначения	PA0	PA1	PA2	PA3
TIM3	Общего назначения	PA6	PA7	PB0	PB1
TIM4	Общего назначения	PB6	PB7	PB8	PB9
TIM5	Общего назначения	None	None	None	PA3
TIM8	Расширенный	None	None	None	None

Таймеры имеют по четыре канала, которые можно настроить как входы или выходы GPIO. Таймер TIM8 вообще не имеет соединений GPIO, но может быть внутренне связан с другими таймерами. TIM5 связывает только 4-й канал с PA3. Остальные таймеры общего назначения с TIM2 по TIM4 имеют полный набор GPIO.

Расширенный таймер TIM1 имеет наиболее полный набор входов/выходов, допускающий до восьми назначений. Записи, отмеченные CHxN, представляют собой GPIO, использующие противоположную полярность. Наконец, сигнал BKIN служит специальным входом «разрыва».

Ответ на вопрос «Сколько таймеров имеют PWM-режим?» – пять. Чтобы использовать все пять, вам необходимо принять GPIO PA3 для TIM5. TIM1–TIM4 могут генерировать выходные сигналы ШИМ на четырех каналах, подключенных к GPIO. В общей сложности эти пять таймеров обеспечивают 21 выходной канал.

Больше PWM-каналов

Для получения большего количества выходных каналов PWM требуется небольшая настройка и программное обеспечение. Каждый таймер имеет четыре канала, поэтому TIM8, например, можно использовать для генерации до четырех различных прерываний на основе регистра сравнения выходов каждого канала. Несмотря на то что ни один из каналов таймера 8 не подключен к GPIO, сама процедура прерывания может управлять выходами GPIO с помощью программного обеспечения без потери точности.

Итоги

В этой главе аппаратные таймеры применялись к задаче генерации выходных сигналов PWM, подходящих для управления RC-сервоприводами. Конечно, PWM не ограничивается только сервоприводами. Сигналы ШИМ могут применяться другими способами, чтобы воспользоваться преимуществами изменений в рабочем периоде.

Прелесть использования аппаратных таймеров заключается в том, что после настройки для работы они практически не требуют программной под-

держки. Чтобы изменить ширину импульса или рабочий цикл, требуется одно небольшое обновление таймера, а затем таймер продолжает свою работу автоматически. Аппаратные таймеры также обеспечивают бóльшую точность, поскольку они не подвержены программным задержкам.

Дополнительные источники

1. STMicroelectronics. «RM0008. Reference manual». См. раздел «Дополнительные источники» [1] к главе 7.

Упражнения

1. Что такое период сигнала RC-сервопривода?
2. Почему на плате Blue Pill тактовая частота входа таймера составляет 72 МГц? Почему не 36 МГц?
3. Что следует поменять в таймере для изменения ширины импульса?

Глава 17 • PWM-вход с таймером 4

Небольшой размер STM32 делает его естественным применением для летающих моделей с дистанционным управлением. Используя существующие радиоконтроллеры, STM32 может взаимодействовать с приемниками RC-сервосигналов и выполнять функции управления согласно вашему собственному воображению.

В эту главу включена демонстрационная программа, которая использует таймер 4 для измерения входящего RC-сервосигнала. Поскольку всю работу выполняет периферийное устройство таймера, процессор может свободно успевать реагировать на показания сервопривода, увеличивая задействованную вычислительную мощность.

Сервосигнал

Для RC-сервосигналов не существует стандарта, но большинство из них, похоже, используют длительность импульса около 0.9 мс в одном крайнем случае и около 2.1 мс в другом. Частота повторения часто составляет около 50 Гц, но может достигать 300 Гц, в зависимости от производителя. Рисунок 17.1 иллюстрирует предполагаемый сигнал, который будет декодироваться в примере этой главы.

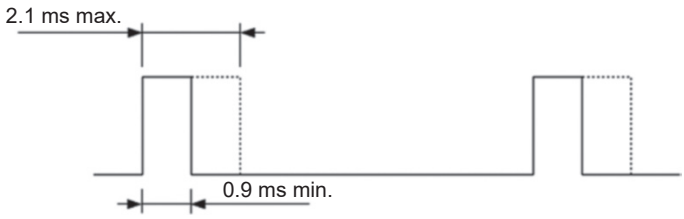


Рис. 17.1. Типичный RC-сервосигнал

Позиционирование сервопривода определяется длительностью импульса, а не коэффициентом заполнения периода. Из-за этого некоторые утверждают, что это вообще не следует называть широтно-импульсной модуляцией. Среднее положение сервопривода, скорее всего, будет импульсом шириной 1.5 мс.

Напряжение сигнала

Демопрограмма использует таймер 4, канал 2 которого подключен к GPIO PB6. Этот GPIO устойчив к напряжению 5 В. Напряжение большинства источников сервосигналов может варьироваться от 4.5 до 6 В. Используйте

резистор сопротивлением 2 кОм между приемником сигналов и PB6, чтобы безопасно ограничить ток. В случае конфликта сигналов или если сигнал поднимается чуть выше 5 В[86], резистор ограничит ток до безопасного значения 0.5 мА.

Демопроект

Исходный код этого проекта находится здесь:

```
$ cd ~/stm32f103c8t6/rtos/tim4_pwm_in
```

Теперь давайте рассмотрим демонстрационное программное обеспечение.

Конфигурация GPIO

Конфигурация вывода PB6 довольно рутинная. Строка 45 включает тактовый сигнал GPIOB, а остальные строки настраивают PB6 в качестве входа. Несмотря на то что этот вход поступает в таймер 4, его не нужно объявлять как альтернативный GPIO.

```
0045: // PB6 == TIM4.CH1
0046: rcc_periph_clock_enable(RCC_GPIOB); // Need GPIOB clock
0047: gpio_set_mode(GPIOB,GPIO_MODE_INPUT, // Input
0048:     GPIO_CNF_INPUT_FLOAT,GPIO6); // PB6=TIM4.CH1
```

Конфигурация таймера 4

Как и для GPIO, должен быть включен тактовый сигнал для таймера 4:

```
0043:   rcc_periph_clock_enable(RCC_TIM4); // Need TIM4 clock
```

Далее следует настройка самого таймера, показанная в листинге 17.1.

Листинг 17.1. Конфигурация таймера 4

```
0050:   // TIM4:
0051:   timer_disable_counter(TIM4);
0052:   timer_reset(TIM4);
0053:   nvic_set_priority(NVIC_DMA1_CHANNEL3_IRQ,2);
0054:   nvic_enable_irq(NVIC_TIM4_IRQ);
0055:   timer_set_mode(TIM4,
0056:       TIM_CR1_CKD_CK_INT,
0057:       TIM_CR1_CMS_EDGE,
0058:       TIM_CR1_DIR_UP);
```

[86] Напоминаем, что для вывода STM32, допускающего напряжение +5 В, максимально допустимое значение составляет около 7 В (V_{cc} + 4 В, см. сноску 85 на стр. 272).

```
0059:   timer_set_prescaler(TIM4,72);
0060:   timer_ic_set_input(TIM4,TIM_IC1,TIM_IC_IN_TI1);
0061:   timer_ic_set_input(TIM4,TIM_IC2,TIM_IC_IN_TI1);
0062:   timer_ic_set_filter(TIM4,TIM_IC1,TIM_IC_CK_INT_N_2);
0063:   timer_ic_set_prescaler(TIM4,TIM_IC1,TIM_IC_PSC_OFF);
0064:   timer_slave_set_mode(TIM4,TIM_SMCR_SMS_RM);
0065:   timer_slave_set_trigger(TIM4,TIM_SMCR_TS_TI1FP1);
0066:   TIM_CCER(TIM4) &= ~(TIM_CCER_CC2P|TIM_CCER_CC2E
0067:       |TIM_CCER_CC1P|TIM_CCER_CC1E);
0068:   TIM_CCER(TIM4) |= TIM_CCER_CC2P|TIM_CCER_CC2E|TIM_CCER_CC1E;
0069:   timer_ic_enable(TIM4,TIM_IC1);
0070:   timer_ic_enable(TIM4,TIM_IC2);
0071:   timer_enable_irq(TIM4,TIM_DIER_CC1IE|TIM_DIER_CC2IE);
0072:   timer_enable_counter(TIM4);
```

Счетчик отключается и сбрасывается в строках 51 и 52. Многие элементы конфигурации таймера не могут быть изменены, когда он активен. Строки 53 и 54 – просто подготовка прерывания таймера 4.

Вызов строки 55 устанавливает основные элементы TIM4.

- Входом для прескалера таймера будут внутренние такты.
- События внутри таймера будут управляться фронтом.
- Счетчик будет считать на увеличение.

В строке 59 для предделителя таймера устанавливается значение 72, так что один тактовый импульс будет возникать каждую микросекунду. Напомним, что частота шины APB1 ограничена 36 МГц; поэтому прескалер APB1 делит на 2 (SYSCLK составляет 72 МГц). Но поскольку прескалер APB1 не равен 1, вход глобального прескалера таймера равен частоте шины APB1, умноженной на 2, или 72 МГц.

Строки 60 и 61 показывают, что входы таймера IC1 и IC2 оба направлены на первый вход таймера (TI1). Эта причудливая конфигурация означает, что мы можем считывать сигнал сервопривода с помощью одного GPIO (PB6), но использовать для таймера два входа с разной обработкой.

Строка 62 устанавливает цифровой входной фильтр, который производит выборку внутреннего тактового сигнала (после частного предделителя таймера), деленного на 2. В строке 63 говорится, что тактовый сигнал цифрового фильтра не будет иметь предварительного масштабирования.

В строке 64 указано, что при повышении входного сигнала PB6 (TI1) счетчик должен быть очищен. Очистка происходит после того, как в регистр `TIM4_CCR1` загружено захваченное значение счетчика. В этом примере это будет мера продолжительности цикла повторения.

В строке 65 устанавливается второй триггер для таймера 2, в результате чего текущий счетчик таймера копируется в регистр захвата `TIM4_CCR2`. Это происходит, когда входной сигнал на PB6 снова становится низким, и, таким образом, будет измеряться длительность импульса в тактах счетчика. Это обнаружение изменения сигнала основано на цифровом фильтре сигнала от TI1.

Строки 66–68 настраивают две конфигурации захвата.

- Вход захвата 1 включен (`TIM_CCER_CC1E`).
- Вход захвата 1 имеет активный высокий уровень (по умолчанию).
- Вход захвата 2 включен (`TIM_CCER_CC2E`).
- Вход захвата 2 имеет активный низкий уровень (`TIM_CCER_CC2P`).

К сожалению, в настоящее время для этого не существует подпрограмм libopencm3, поэтому использовались имена макроопределений.

Строки 69 и 70 активируют два входа таймера 4, а строка 71 разрешает прерывания таймера 4 для входов 1 и 2. Наконец, строка 72 запускает счетчик таймера 4.

Цикл task1

Когда таймер работает, наша задача входит в цикл, который показан в листинге 17.2. Цикл выполняется медленно, останавливаясь примерно на секунду в строке 75. Затем он включает светодиод на PC13 (в знак того, что работает).

Листинг 17.2. Цикл демонстрационной задачи `task1`

```
0019: static volatile uint32_t cc1if = 0, cc2if = 0,
0020: c1count = 0, c2count = 0;
...
0074: for (;;) {
0075:    vTaskDelay(1000);
0076:    gpio_toggle(GPIOC,GPIO13);
0077:
0078:    std_printf("cc1if=%u (%u), cc2if=%u (%u)\n",
0079:        (unsigned)cc1if,(unsigned)c1count,
0080:        (unsigned)cc2if,(unsigned)c2count);
0081: }
```

В строках 78–80 выводятся некоторые интересующие значения:

- `CC1IF` – значение счетчика в конце цикла, которое поступает из регистра `TIM4_CCR1`. Это говорит нам о том, как долго длился цикл в тактах счетчика. Значение, отображаемое в скобках после него, представляет собой просто количество возникновений процедуры обработки прерываний ISR на данный момент;
- `CC2IF` – это значение счетчика, фиксируемое при падении входного сигнала с высокого уровня на низкий. Это представляет ширину импульса в тактах счетчика. Значение, указанное в скобках, представляет собой количество прерываний ISR на данный момент.

Процедура обработки прерывания ISR

Значения, используемые основным циклом, обновляются в процедуре обработки прерывания таймера ISR, что показано в листинге 17.3.

Листинг 17.3. Процедура ISR таймера

```
0022: void
0023: tim4_isr(void) {
```

```
0024:   uint32_t sr = TIM_SR(TIM4);
0025:
0026:   if ( sr & TIM_SR_CC1IF ) {
0027:       cc1if = TIM_CCR1(TIM4);
0028:       ++c1count;
0029:       timer_clear_flag(TIM4,TIM_SR_CC1IF);
0030:   }
0031:   if ( sr & TIM_SR_CC2IF ) {
0032:       cc2if = TIM_CCR2(TIM4);
0033:       ++c2count;
0034:       timer_clear_flag(TIM4,TIM_SR_CC2IF);
0035:   }
0036: }
```

Прерывание включено для захвата входа 1 и захвата входа 2 (строки 69 и 70 в листинге 17.1). При входе в процедуру регистр состояния (status register) таймера считывается в строке 24. Если прерывание вызвано событием захвата 1, то окажется установлен флаг `TIM_SR_CC1IF` (строка 26). Если это так, счетчик фиксируется в строке 27, а прерывание сбрасывается в строке 29. Строка 28 просто увеличивает счетчик прерываний для вывода в основном цикле.

Если прерывание возникло при захвате входа 2, то код выполняется аналогичным образом в строках 32–34. Значения `cc1if`, `c1count`, `cc2if` и `c2count` – это значения, захваченные и переданные основному циклу (строки 78–80 листинга 17.2). Обратите внимание, что эти переменные там объявлены с атрибутом `volatile`, поскольку разные потоки управления обновляют и читают эти значения.

Запуск демопрограммы

Демопрограмма состоит из подключения приемника дистанционного управления сервоприводом к входу GPIO PB6, который выдерживает +5 В, прошивки кода и запуска *minicom* через USB.

Сначала подготовьте программное обеспечение:

```
$ make clobber
$ make
$ make flash
```

Когда программное обеспечение во флеш-памяти MCU готово, можно подключать RC-сервоприемник. Рисунок 17.2 иллюстрирует подключение.

Резистор R1 настоятельно рекомендуется для защиты. Если по какой-либо причине возник конфликт сигналов, резистор ограничит ток до безопасного значения. Вход GPIO чувствителен к напряжению, поэтому резистор не ухудшит сигнал.

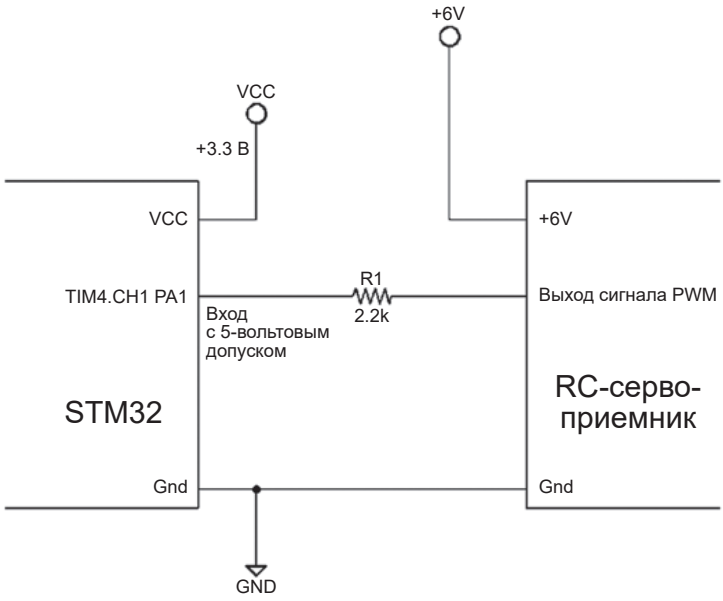


Рис. 17.2. Подключение RC-сервоприемника к STM32

Когда будете готовы к работе, подключите USB-кабель и запустите *minicom*. Я сохранил настройки USB в файле с именем «usb» (ваши могут отличаться):

```
$ minicom usb
Welcome to minicom 2.7
OPTIONS:
Compiled on Sep 17 2016, 05:53:15.
Port /dev/cu.usbmodemWGDEM1, 19:54:45
Press Meta-Z for help on special keys
cc1if=25174 (176), cc2if=985 (176)
cc1if=25119 (215), cc2if=989 (215)
cc1if=25125 (255), cc2if=974 (255)
cc1if=25172 (294), cc2if=990 (294)
cc1if=25134 (333), cc2if=985 (333)
cc1if=25183 (372), cc2if=981 (372)
cc1if=25200 (411), cc2if=992 (411)
cc1if=25149 (450), cc2if=990 (450)
cc1if=25339 (489), cc2if=990 (489)
cc1if=24513 (528), cc2if=442 (528)
cc1if=24180 (569), cc2if=209 (569)
cc1if=24135 (610), cc2if=219 (610)
cc1if=24283 (650), cc2if=217 (650)
cc1if=24320 (691), cc2if=208 (691)
cc1if=24265 (732), cc2if=258 (732)
cc1if=25344 (771), cc2if=1027 (771)
cc1if=26232 (809), cc2if=1698 (809)
cc1if=26354 (847), cc2if=1800 (847)
cc1if=26403 (884), cc2if=1871 (884)
```

```
cc1if=26495 (921), cc2if=1869 (921)
cc1if=26640 (959), cc2if=1887 (959)
cc1if=26464 (996), cc2if=1896 (996)
cc1if=26489 (1033), cc2if=1868 (1033)
cc1if=26432 (1070), cc2if=1878 (1070)
cc1if=26648 (1107), cc2if=1900 (1107)
cc1if=26431 (1144), cc2if=1883 (1144)
cc1if=26654 (1181), cc2if=1891 (1181)
cc1if=26571 (1219), cc2if=1880 (1218)
cc1if=26566 (1256), cc2if=1889 (1256)
cc1if=26621 (1293), cc2if=1880 (1293)
cc1if=26739 (1330), cc2if=1897 (1330)
```

Вывод результатов сессии

Выходные данные сессии представляют собой односекундное обновление считанных значений таймера. Например, первая строка:

```
cc1if=25174 (176), cc2if=985 (176)
```

Это представляет собой время:

t = 25 174 / 1 000 000 = 25.2 мс.

Из этого значения мы можем вычислить частоту сигнала следующим образом:

f = 1 / 0.0252 = 39.7 Гц.

Значение (176) – это значение счетчика прерываний, которое пригодится при отладке. Это говорит нам о том, что прерывание возникало 176 раз для события захвата таймера 1.

Второе значение (985) дает нам длительность импульса:

t = 985 / 1 000 000 = 0.985 мс.

В дальнейшем, когда позиция изменится, мы получим

```
cc1if=26739 (1330), cc2if=1897 (1330)
```

Это представляет длительность импульса следующим образом:

t = 1897 / 1 000 000 = 1.9 мс.

Входы таймера

Демопример иллюстрировал, как выполнить считывание данных с сервоприемника, но как на самом деле это сделал таймер? Давайте проясним презентацию и рассмотрим внутреннюю работу входных каналов.

Найдите минутку и изучите рис. 17.3. Он представляет собой несколько упрощенное представление об используемых входах таймера 4. Таймер имеет всего четыре входа, но в таком режиме работы можно использовать только входы TI1 и TI2.

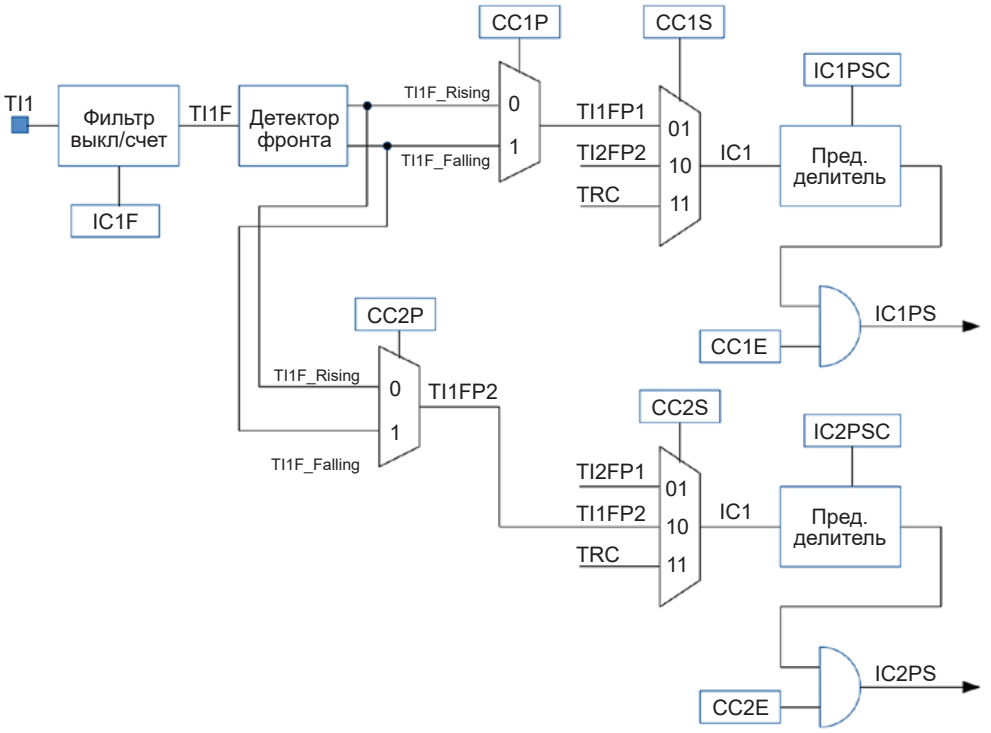


Рис. 17.3. Конфигурация входа таймера

Входной сигнал в демопрограмме поступает на TI1 из GPIO PB6. Это первый входной канал таймера.

TI1 опционально обрабатывается цифровым фильтром, управляемым битом конфигурации `IC1F`. Значение `IC1F` представляет элемент конфигурации, который является частью набора регистров таймера STM32. В данном случае это 4-битное значение в регистре `TIM4_CCMR1`. В строке 62 листинга 17.1 это значение устанавливается так, чтобы счетчик фильтра таймера был равен 2. Поскольку тактовая частота `fCK_INT` = 1 МГц, это означает, что дискретизированный сигнал должен быть одинаковым для двух последовательных выборок, прежде чем появится изменение после фильтра на `TI1F`. Это не позволяет ложному шумовому импульсу вызвать срабатывание таймера и испортить показания.

Сигнал `TI1F` поступает в детектор фронта, генерируя внутренние сигналы `TI1F_Rising` и `TI1F_Falling`. Элемент конфигурации `CC1P` выбирает, какую полярность сигнала использовать. Строка 60 настраивает `CC1S` так, чтобы сигнал `TI1FP1` (по нарастающему фронту) использовался в качестве входного сигнала

захвата IC1. Когда этот сигнал срабатывает (через IC1PS), счетчик таймера копируется в регистр захвата 1. Строка 64 настраивает таймер таким образом, что счетчик таймера сбрасывается после захвата.

Входной сигнал IC1 может быть предварительно масштабирован, а затем настроен с помощью IC1PSC. В строке 63 эта дополнительная функция отключена, чтобы не было предварительного масштабирования. Строки 68 и 69 включают CC1E, позволяя сигналу IC1PS активировать событие захвата 1. Этот конкретный захват измеряет период сигнала ШИМ в демонстрации.

Но мы еще не закончили.

В конфигурации используется входной канал 2 (IC2), полученный из того же сигнала, дискретизированного каналом 1. Элемент конфигурации CC2S в этом режиме приводит к использованию сигнала TI1F_Rising или TI1F_Falling (канал 1) вместо обычного входа TI2. Это полезно при измерении входного сигнала PWM, поскольку нам нужно фиксировать разные события из одного и того же сигнала. Остальная часть цепочки I2CPS в основном такая же, как и для I1CPS, за исключением того, что она управляет событием захвата 2. Поскольку IC2 имеет противоположную полярность (спадающий фронт), организованную CC2S, I2CPS может вызвать захват счетчика при падении сигнала. Это дает нам счетчик в момент, когда ширина импульса возвращается к низкому значению.

Итоги

В этой главе показано, как можно использовать таймер STM32 для измерения ширины и периода импульса сигнала. В демоверсии было выполнено только 39 × 2 прерываний для захвата периода и ширины импульса каждую секунду. Код ISR достаточно минимален, поэтому ценное время процессора доступно для выполнения другой полезной работы.

Есть много других функций у входов таймеров, которые здесь остаются неисследованными. Читателю рекомендуется просмотреть справочное руководство STM32 RM0008 (см. «Дополнительные источники», [1] к главе 7) для получения дополнительных идей.

Упражнения

1. Почему на входе таймера имеется цифровой фильтр?
2. Когда происходит сброс таймера в режиме поступления PWM на вход?
3. Откуда поступает входной сигнал IC2 в режиме поступления PWM на вход?

Глава 18 • Шина CAN

Разработка шины CAN (controller area network, локальная сеть контроллера) началась в 1983 году в компании Robert Bosch GmbH как способ стандартизации связи между компонентами. До этого каждый автомобильный блок управления имел свои собственные системы, которые требовали большого количества двухточечных проводов. В 1986 году компания Bosch представила разработанный протокол CAN на Конгрессе общества инженеров автомобильной промышленности SAE (www.csselectronics.com/screen/page/simple-intro-to-can-bus/language/en). В 1991 году протокол CAN 2.0 был опубликован компанией Bosch и принят в 1993 году в качестве международного стандарта (ISO 11898). С тех пор автомобили используют этот протокол для связи с целью уменьшения размеров жгутов проводов за счет использования шины.

Наличие CAN-шины в устройстве STM32 делает его привлекательным для автомобильных или управляющих приложений. Несмотря на то что демопрограмма в этой главе будет моделировать систему управления автомобилем, станет очевидно, что шина CAN не обязательно должна ограничиваться автомобильными приложениями. Модели самолетов, дронов и модели железнодорожных систем – это лишь некоторые из потенциальных приложений для хобби[87].

Шина CAN

Представьте, что вам нужно уменьшить объем жгута проводов для производства новой модели автомобиля. Этот автомобиль имеет несколько электронных блоков управления в передней, центральной и задней части корпуса, каждый из них требует связи с другими, включая главный блок управления, который, возможно, расположен за приборной панелью. Как уменьшить количество необходимых проводов?

Почти с самого начала производители сократили количество проводов, используя металлический корпус автомобиля в качестве общего провода, соединив его с отрицательной клеммой аккумулятора. Однако управление нагрузкой традиционно осуществляется путем подачи или разрыва питания +12 В, например, на стоп-сигнал или лампы поворотников. Для этого требуется отдельный провод в жгуте для каждой управляемой нагрузки.

---

[87] Как показали последние веяния в области конструирования беспилотных летательных аппаратов (дронов), резкая граница между любительскими конструкциями, сделанными для развлечения, и профессиональными аппаратами размывается. Любительский дрон, построенный в домашней мастерской из деталей, приобретенных на Ali Express, вполне может применяться для разведки местности или даже для еще более серьезных целей.

Кроме того, электронные блоки управления теперь необходимо снабжать и линиями связи. Если устройства взаимодействуют с большинством или со всеми другими устройствами, количество проводных соединений резко возрастает.

Решением проблемы со жгутами является внедрение шинной системы. Каждый блок управления, который осуществляет связь, подключен к одной и той же шине и отправляет сообщения по ней. Для этой конфигурации вам понадобится следующее:

- линия питания (+12 В),
- одна или пара сигнальных линий шины,
- отрицательный общий провод для возвратного пути тока (металлический кузов автомобиля).

Если предположить пару сигнальных линий шины, то для питания и связи со всеми подключенными к ней устройствами нам понадобится всего три провода. На рис. 18.1 показана гипотетическая шинная система и соединения питания.

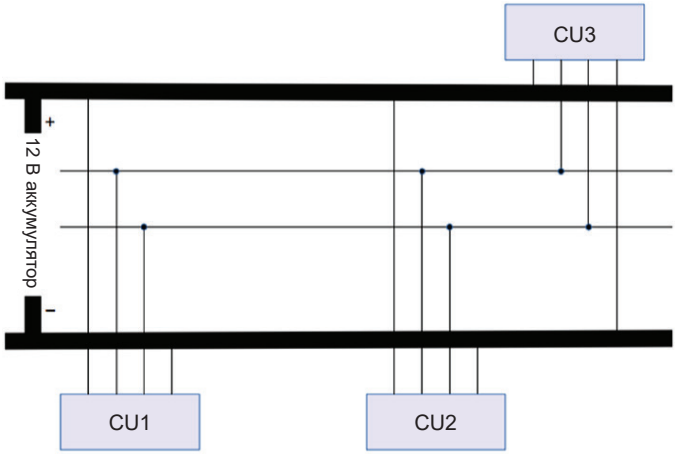


Рис. 18.1. Гипотетическая автомобильная шинная система

При таком расположении любой блок управления (Control Unit) CUx может обмениваться данными с любым другим блоком управления. Если блок управления CU3 расположен в задней части автомобиля, то он может управлять лампочками, расположенными сзади, на основе сообщений, передаваемых по шине. CU3 может переключать близлежащие лампы, переключая питание +12 В с помощью коротких отрезков провода на сам блок. Хотя реально автомобили не всегда управляют стоп-сигналами и поворотниками именно так, технически это возможно. Иногда следует учитывать проблемы с безопасностью – важно, чтобы стоп-сигналы не выходили из строя, например, из-за ошибок программного обеспечения. Однако в целом конструкция шины обеспечивает широкую связь и управление с помощью более компактной проводки.

Дифференциальные сигналы

Высокоскоростная линейная шина CAN определяется форматом сигнала ISO 11898-2, пример сигнала показан в верхней части рис. 18.2. Сигнальная пара состоит из линии CAN-H (высокий уровень) и линии CAN-L (низкий уровень), обе находятся в режиме ожидания около уровня 2.5 В. Состояние ожидания известно как *рецессивное* логическое состояние и приравнивается к логической единице. При этом разница напряжений между проводами составляет 0 В.

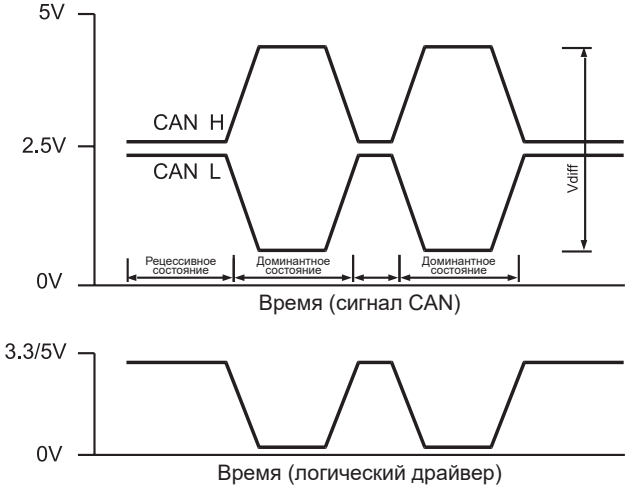


Рис. 18.2. Линейная CAN-шина и форматы сигналов

Активное состояние сигнала известно в стандарте как *доминантное* логическое состояние и приравнивается к логическому нулю. Доминантное логическое состояние соответствует CAN-H не менее +3.5 В и CAN-L не более 1.5 В. Разница напряжений между проводами составит не менее 2 В. Дифференциальный сигнал необходим для высокоскоростной помехоустойчивой связи. Состояние логики определяется тем, насколько CAN-H отличается от CAN-L (V_{diff} на рисунке). Для предотвращения отражений сигнала высокоскоростная линейная шина на каждом конце обрывается сопротивлением 120 Ом, подключенным между линиями CAN-H и CAN-L.

В нижней половине рис. 18.2 показан несимметричный сигнал обычного логического драйвера. Для наших целей это сигнал, который выходит из STM32 и поступает в чип драйвера. С точки зрения драйверного сигнала доминантный логический уровень низкий, а рецессивный – высокий. Высокий уровень может составлять 5 или 3.3 В, в зависимости от используемых логических уровней.

Для схемы драйвера требуется только один провод, но это ограничивает его короткими участками из-за отражений сигнала и шума. С другой стороны, дифференциальная конструкция шины позволяет использовать трассы длиной в несколько метров в условиях сильной зашумленности.

Доминантный/Рецессивный

Для CAN-шины уже описаны рецессивное и доминантное состояния сигналов. Но каково значение этих терминов?

Рецессивный сигнал – это «слабая» форма сигнала. При дифференциальной передаче сигналы CAN-H и CAN-L достигают состояния ожидания через резистивную сеть. Это естественное состояние покоящегося сигнала. Для несимметричного сигнала драйвера «слабое» состояние представляет собой высокий уровень, который достигается с помощью подтягивающего резистора. Это тоже состояние покоя сигнала драйвера.

Во время доминирующего перехода линия переводится из состояния ожидания в активное состояние посредством работы транзисторов («сильное» состояние). Для сигналов дифференциальной шины линия CAN-H подтягивается к высокому уровню транзистором верхнего плеча. Одновременно с этим на линии CAN-L устанавливается низкий уровень включением транзистора нижнего плеча. Сигнал драйвера также переходит из состояния с высоким уровнем в состояние с низким уровнем с помощью включения транзистора нижнего плеча. Другими словами, сигнал драйвера переводится на низкий уровень включением транзистора с открытым стоком.

Дифференциальная шина аналогична несимметричному сигналу драйвера, за исключением того, что сигнал на каждой линии имеет зеркальную копию. Они оба бездействуют около середины, когда находятся в состоянии покоя (рецессивное состояние), но отодвигаются друг от друга, когда становятся активными (доминантное).

А теперь представьте себе, что два драйвера объединены общей шиной. Проще всего думать о несимметричном сигнале драйвера, но помните, что этот принцип применим и к дифференциальной шине. Таблица 18.1 иллюстрирует таблицу истинности для двух драйверов, подключенных к шине.

Таблица 18.1. Таблица истинности для арбитража шины

Драйвер 1	Драйвер 2	Результат на шине	Описание
Рецессивное (лог. 1)	Рецессивное (лог. 1)	Рецессивное (лог. 1)	Ожидание на шине
Доминантное (лог. 0)	Рецессивное (лог. 1)	Доминантное (лог. 0)	Доминантное состояние
Рецессивное (лог. 1)	Доминантное (лог. 0)	Доминантное (лог. 0)	Доминантное состояние
Доминантное (лог. 0)	Доминантное (лог. 0)	Доминантное (лог. 0)	Доминантное состояние

По сути, состояние шины становится логическим ИЛИ[88] всех драйверов, подключенных к шине. Когда какой-либо драйвер применяет состояние доминантного бита (логического нуля), вся шина принимает доминантное состояние. Только когда шиной не управляет ни один драйвер, шина остается в рецессивном состоянии. Теперь должно быть очевидно, почему состояния называют рецессивными и доминантными.

[88] Если быть точным, то с точки зрения положительной логики «логическим И». Однако подключенную таким образом шину обычно действительно именуют «проводное ИЛИ».

Шинный арбитраж

В шинных системах, таких как I2C, имеется один ведущий и несколько ведомых устройств. Ведомому не разрешено передавать до тех пор, пока этого не потребует ведущее устройство. В I2C с несколькими ведущими устройствами должна существовать процедура арбитража, которая определяет, какому ведущему устройству разрешено продолжить работу в случае, если два или более устройств сталкиваются при попытке одновременной передачи. Это, как правило, сложная проблема, которая может привести к блокировке шины.

В отличие от этого случая шина CAN позволяет выступить каждому подключенному устройству. Однако предотвращение столкновений здесь также требует арбитража. Способ, которым это делается для CAN-шины, уникален и основан на принципе рецессивного и доминантного состояний. Рисунок 18.3 иллюстрирует, как взаимодействуют рецессивный и доминантный биты.

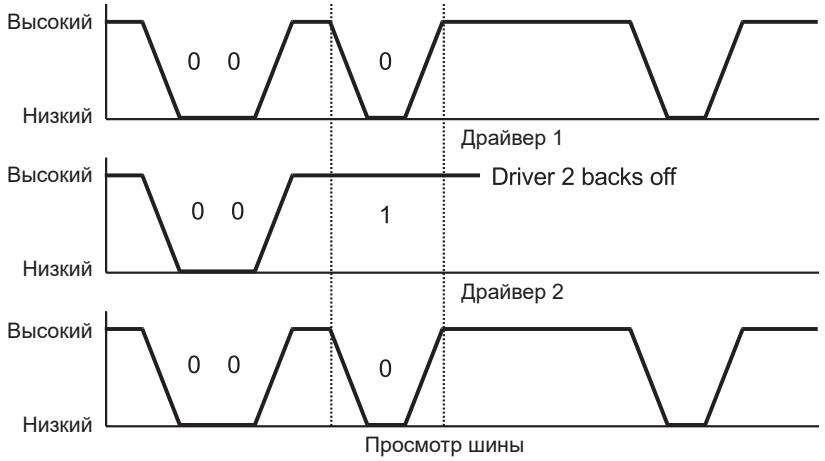


Рис. 18.3. Арбитраж CAN-шины

Арбитраж происходит следующим образом.

1. Драйвер 1 и драйвер 2 одновременно начинают связь, отправляя идентификатор сообщения.
2. Первые два бита являются доминантными (нулевыми), поэтому все устройства, подключенные к шине, видят два нуля.
3. Драйвер 1 отправляет третий бит как доминантный (нулевой), а драйвер 2 пытается отправить рецессивный (единичный) бит. Шина видит только доминантный бит (ноль).
4. Драйвер 2, понимая, что его рецессивный бит «затоптан», отступает, так как проиграл арбитражный процесс. Сообщение драйвера 1 не пострадало и может продолжаться.

Арбитраж иллюстрирует назначение доминантных битов. Каждое устройство продолжает передачу, прослушивая шину. Если какое-либо устройство

отправляет рецессивный бит, но считывает его обратно как доминантный бит, это означает, что арбитраж для шины был потерян. Проигравшие устройства затем отменяют передачу, позволяя победителю продолжить передачу сообщения невредимым.

Синхронизация

Представленная только что арбитражная процедура представляет собой упрощенное объяснение более сложной реальности. Как несколько передатчиков взаимодействуют друг с другом?

Перед передачей идентификатора сообщения отправляется бит SOF (start of frame, начало кадра). Это доминантный (нулевой) бит, поэтому ни одно прослушивающее устройство не пропустит указание начала сообщения. Таким образом, при получении бита SOF отправляющее устройство на той же шине может отменить попытку отправить сообщение, если оно не готово. Если устройство готово, оно синхронизирует свои тактовые сигналы и пытается отправить идентификатор сообщения.

После бита SOF каждое сообщение начинается с идентификатора сообщения. В зависимости от версии протокола идентификатор сообщения имеет длину 11 или 29 бит. Процедура арбитража определяет, кто победит, на основе идентификатора отправляемого сообщения. Напомним, что доминантные биты побеждают рецессивные. Следовательно, значение идентификатора сообщения, состоящее из всех нулевых битов, является сообщением с наивысшим приоритетом.

Формат сообщения

Основной формат сообщения CAN представлен в табл. 18.2. Существует также немного отличающийся расширенный формат, который читателю рекомендуется изучить.

Поле кадра RTR (Remote Transmission Request, запрос удаленной передачи) имеет возможность запроса передачи от удаленного устройства. Когда бит RTR рецессивный, это указывает на ответ устройства. В этих сообщениях DLC (Data Length Code, код длины данных) не используется, и данные не передаются.

Поле IDE (Identifier extension, расширенный идентификатор) определяет, используется ли формат сообщения с расширенным идентификатором. В этой главе демонстрационный пример использует 11-битный формат, используя для индикации этого установку доминирующего бита.

Поле DLC указывает длину данных (в ответе, отличном от RTR) в байтах. Затем за полем DLC следует указанное количество байтов данных.

В конце кадра имеется поле CRC (cyclic redundancy check, контрольный избыточный код), поэтому искаженные сообщения можно игнорировать.

Наконец, в конце кадра находится битовое поле ACK. Этот бит передается как рецессивный бит (логическая 1), поэтому, если какое-либо устройство получит сообщение с неповрежденной CRC, принимающее устройство зафиксирует шину в течение длительности этого бита, используя доминантное состояние. Это позволяет передатчику увидеть, что хотя бы одно устройство

приняло сообщение нормально. Вообще говоря, если одно устройство получает сообщение нормально, значит, все устройства сделали это. Обратите внимание, что всем принимающим устройствам разрешено отвечать подтверждением одновременно.

Если, с другой стороны, ни одно устройство не получило сообщение, бит ACK останется в рецессивном состоянии в том виде, в каком он был передан. Это указывает передатчику, что передача не удалась. Эта часть протокола может немного затруднить разработку драйвера CAN-шины, поскольку вам нужно как минимум еще одно устройство на шине для подтверждения отправленного сообщения. Единственный другой способ проверить отправку сообщения CAN – это использовать внутреннюю функцию обратной связи периферийного устройства STM32.

Таблица 18.2. Формат сообщения CAN-шины (нерасширенный)

Имя поля	Длина в битах	Описание
SOF	1	Start of frame (начало кадра)
ID	11	ID сообщения/приоритет
RTR	1	Remote Transmission Request (запрос удаленной передачи): 0 для кадров данных, 1 для удаленного запроса
IDE	1	Расширенный идентификатор: 0 для 11-битного формата, 1 для 29-битного формата
Зарезервировано	1	Должно быть 0
DLC	4	Код длины данных: от 0 до 8 байт
Data	0–64	Передаваемые данные (DLC определяет длину)
CRC	15	Cyclic redundancy check (контрольный избыточный код)
CRC разделитель	1	Должна быть 1 (рецессивный бит)
ACK	1	Поле ответа: передатчик отправляет 1 (рецессивный), получатель(и) отвечает 0 (доминантный)
ACK разделитель	1	Должна быть 1 (рецессивный бит)
EOF	1	End of frame (конец кадра): 1 (рецессивный бит)

Ограничение STM32

Периферийное устройство шины CAN STM32 использует SRAM для хранения сообщений. К сожалению, конструкция микроконтроллера STM32F103 такова, что CAN и USB используют одну и ту же область памяти. По этой причине CAN и USB нельзя использовать одновременно.

В противном случае можно отключить одно устройство и использовать другое, и наоборот, но это не представляется практичным. По этой причине в демонстрационном примере для связи будет использоваться UART.

Демопрограмма

О протоколе шины CAN и его расширениях можно рассказать гораздо больше. Заинтересованному читателю рекомендуется поискать другие книги по этой теме. Однако цель этой главы – проиллюстрировать, как использовать

периферийное устройство шины CAN STM32 достаточно простыми для начала способами.

В этом примере мы собираемся реализовать три гипотетических EU (electronic unit, электронных блока). Я буду называть их от EU1 до EU3 и избегать аббревиатуры ECU, которая обычно применяется к блоку управления двигателем. Блоки следующие:

1) EU1, блок управления приборной панелью (имеет интерфейс UART). Он обеспечивает управление сигнальными лампами и считывает температуру с заднего EU3;
2) EU2, задний блок управления, отвечающий за стояночные огни, сигналы поворота и стоп-сигналы;
3) EU3, передний блок управления, отвечающий за передние стояночные и поворотные сигнальные огни.

Таким образом, для полной демонстрации требуется три микроконтроллера STM32. Однако, если у вас их два, вы можете хотя бы частично продемонстрировать работу, хотя лучше всего работает три. При необходимости удалите передний EU3.

Сборка программы

Каталог программного обеспечения для демонстрационного примера находится по следующему адресу:

```
$ cd stm32f103c8t6/rtos/can
```

Найдите минутку и перекомпилируйте его:

```
$ make clobber
$ make
```

Это скомпилирует три исполняемых файла:

```
$ ls *.elf
front.elf main.elf rear.elf
```

UART-интерфейс

EU1 – это основной блок управления, который будет моделировать то, что будет находиться за приборной панелью нашего гипотетического автомобиля (модуль *main.c*). Для этого примера настроены следующие параметры последовательного порта, которые должны согласовываться с *minicom* или другой терминальной программой по вашему выбору:

- скорость передачи данных: 115 200 бод,
- биты данных: 8,
- паритет: No,
- стоповый бит: 1,
- аппаратное управление потоком данных: RTS/CTS.

Подробности подключения см. в табл. 10.1.

Обратите внимание, что аппаратное управление потоком требует подключения проводов RTS и CTS к адаптеру последовательного порта USB-TTL и STM32. После подключения ваша терминальная программа также должна быть настроена на использование аппаратного управления потоком данных. Если какая-либо деталь неверна, связь не произойдет. Если управление потоком по какой-то причине не работает, то вы можете увидеть потерянные данные и мусор.

Если при аппаратном управлении потоком возникают проблемы или в вашем последовательном адаптере USB-TTL отсутствуют необходимые сигналы RTS/CTS, измените следующую строку исходного кода в файле *main.c*

```
open_uart(1,115200,"8N1","rw",1,1);
```

на следующую:

```
open_uart(1,9600,"8N1","rw",0,0);
```

При большой скорости обмена может оказаться, что данные потеряны, потому здесь уменьшена скорость передачи данных. Более низкая скорость передачи данных, равная 9600 бод, вполне подойдет для наших целей[89].

Прошивка микроконтроллеров

Для этого примера нужно прошить три микроконтроллера:

1) EU1: `main.c`,
2) EU2: `front.c`,
3) EU3: `rear.c`.

Прошьем устройства (после сборки проекта). Очевидно, вам придется подключить программатор к каждому устройству по очереди. Если нужно их перепрошить, вам, вероятно, придется временно изменить перемычку boot-0.

```
$ make flash
$ make flash_front
$ make flash_rear
```

Шина для демонстрации

Чтобы упростить задачу, в этом примере используется короткая, но однопроводная шина CAN (SWCAN) с подтягивающим резистором сопротивлением 4.7 кОм. В этой конфигурации каждый микроконтроллер STM32 имеет свои линии CAN_RX и CAN_TX, соединенные вместе и подключенные к общей шине. Обычно эти соединения подключаются к микросхеме драйвера шины CAN, такой как PCA82C251, с отдельными соединениями RX и TX. Для получения более подробной информации о микросхеме драйвера выполните поиск «PCA82C251 datasheet PDF».

[89] По этому поводу см. сноски 44 на стр. 95, а также 46 на стр. 104

При подключении демопримера обратите особое внимание на тот факт, что MCU *main.c* использует соединения CAN, отличные от других. Это было сделано для того, чтобы можно было использовать вводы UART, устойчивые к напряжению 5 В, что позволило использовать последовательные адаптеры USB-TTL с напряжением 5 В (см. рис. 18.4).

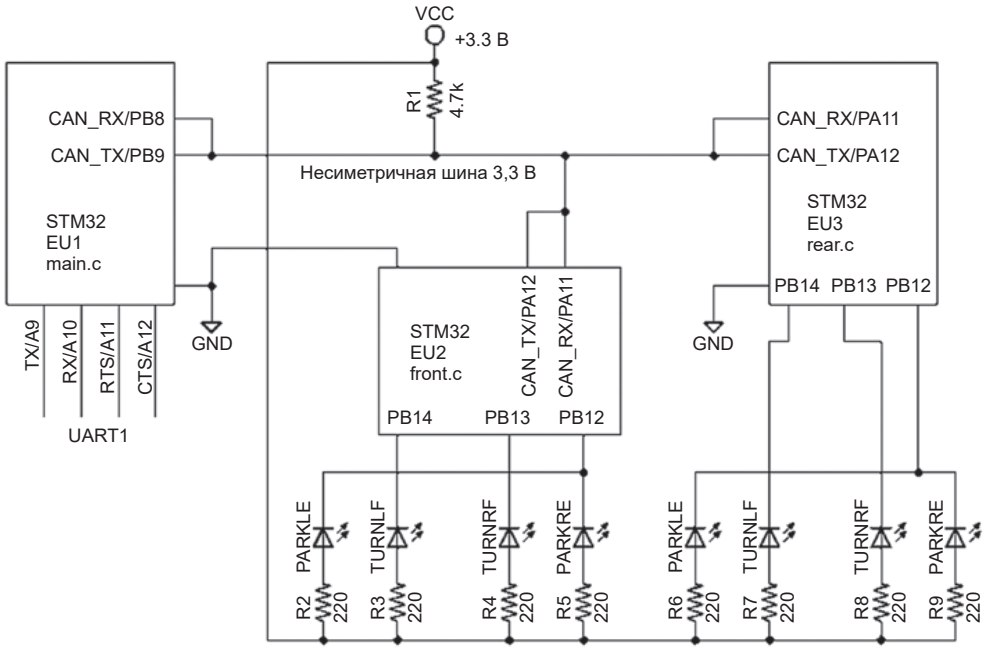


Рис. 18.4. Демопроект подключения шины CAN

Резистор R1 представляет собой подтягивающее сопротивление, необходимое для установления рецессивного состояния. Поскольку микроконтроллер представляет собой устройство с напряжением 3.3 В, наше рецессивное состояние будет около +3.3 В. Из-за подтягивающего резистора мы устанавливаем вывод GPIO для CAN_TX с настройкой `GPIO_CNF_OUTPUT_ALTFN_OPENDRAIN` (акцент на открытый сток).

Другие контроллеры разделяют шину и общий провод. Не забудьте связать вместе выводы GND. Любое сообщение, отправленное одним контроллером, принимается всеми остальными.

Светодиоды заменяют типовые автомобильные сигналы на панели, стоп-сигналы и стояночные огни. Задний контроллер управляет стоп-сигналами и сигналами поворота.

Запуск демопрограммы

Демопример можно настроить на макетной плате. На рис. 18.5 показана собственная схема автора с питанием от платы MB102 (см. раздел «Источник

питания» в главе 1). Это обеспечивает подачу +5 В на каждый из входов +5 В платы Blue Pill, в результате чего встроенный стабилизатор каждой платы контроллера подает +3.3 В на остальную часть системы.

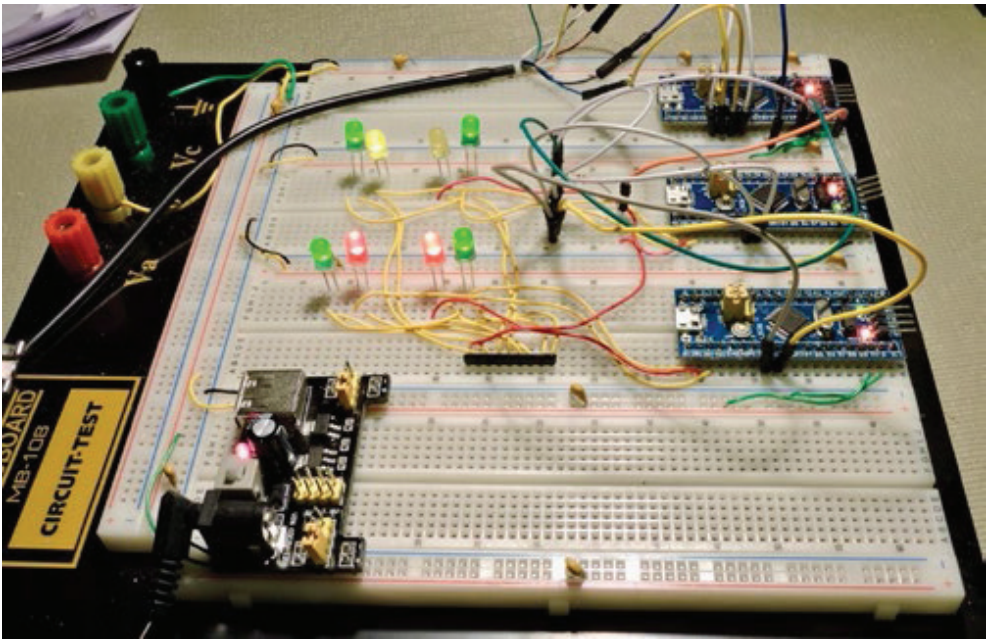


Рис. 18.5. Настройка макета демопримера CAN

В центре фотографии вы можете увидеть резисторную сборку SIP9 сопротивлением 220 Ом, которую я использовал для резисторов светодиодов. Отдельные встроенные резисторы сборки заменяют до восьми отдельных резисторов. Вывод 1 резистора SIP9 общий и идет на линию питания +3.3 В. Чтобы включить светодиод, подключенные выводы GPIO пропускают ток на землю. Если вы не используете компонент SIP9, просто используйте отдельные резисторы, как показано на схеме рис. 18.4 (от R2 до R9).

В верхней части фотографии показан кабель устройства USB-UART и соединения с главным микроконтроллером, расположенным на верхнем по фото краю макетной платы. Он представляет встроенный контроллер EU1 (с прошивкой *main.c*). Нижний по фото контроллер – это задний блок EU3 с прошивкой *back.c*.

Светодиоды расположены так, чтобы передние автомобильные огни были направлены вверх, а задние – вниз по фотографии. Внешние зеленые светодиоды обозначают стояночные огни. Желтые светодиоды обозначают передние указатели поворота, а красные светодиоды обозначают задние указатели поворота и стоп-сигналы.

Несимметричная шина CAN расположена справа от светодиодов, в основном белые провода со штеккерами поверх плат соединяют выводы CAN_RX

и CAN_TX от каждого контроллера. В этой путанице проводов есть один резистор сопротивлением 4.7 кОм, подключенный к +3.3 В.

Когда все будет готово, подключите последовательное устройство USB-TTL к компьютеру и запустите *minicom* (или аналогичную программу). Как только все будет готово, включите питание макетной платы. Основной контроллер должен откликнуться через последовательный порт следующим образом:

```
Welcome to minicom 2.7
OPTIONS:
Compiled on Sep 17 2016, 05:53:15.
Port /dev/cu.usbserial-A703CYQ5, 20:38:01
Press Meta-Z for help on special keys
Car simulation begun.
Menu:
L - Turn on left signals
R - Turn on right signals
P - Turn on parking lights
B - Activate brake lights
Lower case the above to turn OFF
V - Verbose mode (show received messages)
CAN Console Ready:
> _
```

В этом выводе через последовательный порт показывается меню, позволяющее управлять различными функциями вашего гипотетического автомобиля. Если все работает правильно, встроенные светодиоды плат (PC13) переднего и заднего блоков должны мигать примерно раз в секунду. Если вы видите, что они дважды мигнули и остановились, это указывает на проблему с подключением к шине. Я столкнулся с этой проблемой, когда забыл, что выводы GPIO CAN отличаются в главном контроллере от переднего и заднего блоков. Перепроверьте проводку.

Нажмите заглавную букву «P», чтобы включить габаритные огни. Если все прошло хорошо, габаритные огни теперь горят, а консоль также отображает температуру:

```
CAN Console Ready:
> P
> Temperature: +24.73 C
>
```

Температура передавалась на основной блок с заднего блока. Чтобы выключить габаритные огни, нажмите строчную букву «p». Стояночные огни должны немедленно погаснуть.

Если вы теперь нажмете заглавную букву «B», должны загореться стоп-сигналы (в моей схеме красные). Нажатие строчной буквы «b» снова выключает их.

Нажмите заглавную «L» или «R», чтобы включить левый или правый поворотник соответственно. Обратите внимание, что передний и задний сигналы

мигают синхронно, хотя они управляются двумя отдельными микроконтроллерами. Нажмите строчную букву «l» или «r», чтобы снова выключить сигнал. Вы также можете включить четырехстороннее мигание, включив левый и правый поворотники вместе.

И наконец, включите указатель поворота, скажем левый, нажав заглавную букву «L». Затем нажмите заглавную букву «B», чтобы включить стоп-сигналы. Теперь левый указатель поворота мигает, но правый задний продолжает гореть, обозначая стоп-сигнал. При выключении указателя поворота оба задних стоп-сигнала должны гореть.

CAN-сообщения

Сообщения в основном отправляются из основного EU1 на передние и задние блоки. После каждого запроса сигналов основной блок также отправляет сообщение на заднюю панель с запросом температуры (он устанавливает бит RTR для запроса ответа). Когда задний блок получает сообщение с истинным флагом RTR, он измеряет температуру и передает ее на шину. Все остальные могут прочитать это сообщение, но информацию использует только основной блок.

Остальные сообщения отправляются для включения/выключения данного светодиода. Чтобы сигнальные лампы мигали одновременно, также отправляется мигающее сообщение.

Синхронизация

Когда сигнальные лампы мигают, для человеческого глаза это выглядит совершенно одновременно. Но насколько они синхронизированы? С помощью осциллографа можно увидеть, что включение и выключение сигнальной лампы «плавало» примерно в пределах ±850 мкс и изменялось во времени. Все микроконтроллеры получают сообщение на свои периферийные устройства одновременно, но FreeRTOS выполняет упреждающее планирование. Поскольку используются такты длительностью 1 мс, время может быть смещено на величину до 1 мс.

Что делать, если для приложения управления нужна более высокая точность? Одним из подходов может быть увеличение частоты тактов таймера (уменьшение временного интервала). Возможны и другие улучшения прикладного программного обеспечения. Например, процедура прерывания может уведомить об ожидающей задаче. На эту проблему нет однозначного ответа. Часто все сводится к тому, насколько важен вопрос и сколько усилий вы готовы потратить, чтобы его получить.

Итоги

В этой главе основное внимание уделяется представлению некоторых концепций шины CAN и демонстрационной схемы. Запуск демонстрационного примера доказал, что можно контролировать другие микроконтроллеры практически в реальном времени с помощью коротких сообщений CAN. Кроме того, это подтверждает концепцию общей шины, в которой нет главных

и подчиненных устройств. Наконец, видно, что STM32 способен применять связь CAN как в однопроводном, так и в режимах дифференциальной шины (дифференциальный – с помощью микросхемы драйвера).

Из-за размера и сложности этого проекта программное обеспечение для показанной демонстрации будет описано в следующей главе.

Дополнительные источники

1. CSS Electronics. «CAN Bus Explained – A Simple Intro» www.csselectronics.com/screen/page/simple-intro-to-can-bus/language/en (дата доступа 15.09.2024). Русскоязычный аналог: «CAN-шина. Просто и понятно»: https://elm3.ru/wiki/can-shina (дата доступа 15.09.2024).

Глава 19 • Программное обеспечение CAN-шины

Демонстрационный пример шины CAN в предыдущей главе проиллюстрировал три микроконтроллера STM32, передающих сообщения на общей шине. Никто не был ведущим, и никто не был ведомым. Все это было организовано с помощью периферийного устройства CAN-шины STM32 и драйвера устройства из библиотеки libopencm3.

В этой главе будет обсуждаться использование API драйвера libopencm3, позволяющее создавать собственные приложения шины CAN. В сочетании с использованием FreeRTOS у вас появится удобная среда для программирования более сложных проектов.

Инициализация

Исходные модули проекта расположены в следующем каталоге:

```
$ cd ~/stm32f103c8t6/rtos/can
```

Наиболее сложной частью настройки периферийного устройства CAN-шины является его настройка и инициализация. В листинге 19.1 показана функция `initialize_can()`, которая была предоставлена в исходном модуле *canmsgs.c*.

Листинг 19.1. Код инициализации CAN

```
0090: void
0091: initialize_can(bool nart,bool locked,bool altcfg) {
0092:
0093:   rcc_periph_clock_enable(RCC_AFIO);
0094:   rcc_peripheral_enable_clock(&RCC_APB1ENR,
                                    RCC_APB1ENR_CAN1EN);
0095:
0096:   /*****************************************************
0097:   * When:
0098:   * altcfg CAN_RX=PB8, CAN_TX=PB9
0099:   * !altcfg CAN_RX=PA11, CAN_TX=PA12
0100:   *****************************************************/
0101:   if ( altcfg ) {
0102:       rcc_periph_clock_enable(RCC_GPIOB);
0103:       gpio_set_mode(GPIOB,GPIO_MODE_OUTPUT_50_MHZ,
            GPIO_CNF_OUTPUT_ALTFN_OPENDRAIN,
            GPIO_CAN_PB_TX);
```

```
0104:       gpio_set_mode(GPIOB,GPIO_MODE_INPUT,
                GPIO_CNF_INPUT_FLOAT,
                GPIO_CAN_PB_RX);
0105:
0106:       gpio_primary_remap( // Map CAN1 to use PB8/PB9
0107:           AFIO_MAPR_SWJ_CFG_JTAG_OFF_SW_OFF, // Optional
0108:           AFIO_MAPR_CAN1_REMAP_PORTB);
0109:   } else {
0110:       rcc_periph_clock_enable(RCC_GPIOA);
0111:       gpio_set_mode(GPIOA,GPIO_MODE_OUTPUT_50_MHZ,
                GPIO_CNF_OUTPUT_ALTFN_OPENDRAIN,GPIO_CAN_TX);
0112:       gpio_set_mode(GPIOA,GPIO_MODE_INPUT,
                GPIO_CNF_INPUT_FLOAT,GPIO_CAN_RX);
0113:
0114:       gpio_primary_remap( // Map CAN1 to use PA11/PA12
0115:       AFIO_MAPR_SWJ_CFG_JTAG_OFF_SW_OFF, // Optional
0116:       AFIO_MAPR_CAN1_REMAP_PORTA);
0117:   }
0118:
0119:   can_reset(CAN1);
0120:   can_init(
0121:     CAN1,
0122:     false, // ttcm=off
0123:     false, // auto bus off management
0124:     true, // Automatic wakeup mode.
0125:     nart, // No automatic retransmission.
0126:     locked, // Receive FIFO locked mode
0127:     false, // Transmit FIFO priority (msg id)
0128:     PARM_SJW, // Resynch time quanta jump width (0..3)
0129:     PARM_TS1, // segment 1 time quanta width
0130:     PARM_TS2, // Time segment 2 time quanta width
0131:     PARM_BRP, // Baud rate prescaler for 33.333 kbs
0132:     false, // Loopback
0133:     false); // Silent
0134:
0135: can_filter_id_mask_16bit_init(
0137:     0, // Filter bank 0
0138:     0x000 << 5, 0x001 << 5, // LSB == 0
0139:     0x000 << 5, 0x001 << 5, // Not used
0140:     0, // FIFO 0
0141:     true);
0142:
0143: can_filter_id_mask_16bit_init(
0145:     1, // Filter bank 1
0146:     0x010 << 5, 0x001 << 5, // LSB == 1 (no match)
0147:     0x001 << 5, 0x001 << 5, // Match when odd
0148:     1, // FIFO 1
0149:     true);
0150:
0151:   canrxq = xQueueCreate(33,sizeof(struct s_canmsg));
0152:
```

```
0153:   nvic_enable_irq(NVIC_USB_LP_CAN_RX0_IRQ);
0154:   nvic_enable_irq(NVIC_CAN_RX1_IRQ);
0155:   can_enable_irq(CAN1,CAN_IER_FMPIE0|CAN_IER_FMPIE1);
0156:
0157:   xTaskCreate(can_rx_task,"canrx",400,NULL,
                    configMAX_PRIORITIES-1,NULL);
0158: }
```

Эта функция обеспечивает инициализацию в следующие основные этапы.

1. Включение тактовой подсистемы AFIO (строка 93). Это необходимо для того, чтобы мы могли выбирать, какие выводы GPIO использовать для портов CAN-шины.
2. Включение тактовой подсистемы периферийного устройства CAN-шины (строка 94). Это необходимо для работы периферийного устройства.
3. Включение тактовой подсистемы для соответствующего порта GPIO (строка 102 или 110, в зависимости от конфигурации, выбранной логическим аргументом `altcfg`).
4. Режим вывода GPIO выбирается для CAN_TX (строка 103 или 111).
5. Режим входа GPIO выбирается для CAN_RX (строка 104 или 112).
6. Сопоставление AFIO выбирается для линий CAN_TX и CAN_RX (строки 106–108 или строки 114–116).
7. Для инициализации и настройки периферийного устройства шины CAN вызывается процедура `can_reset()` из libopencm3 (строки 119–133).
8. В строках 135–141 настраивается банк фильтров CAN 0 для определения того, куда должны идти определенные сообщения.
9. В строках 143–149 настраивается банк фильтров CAN 1 для определения того, куда должны идти другие сообщения.
10. В строке 151 создается очередь приема FreeRTOS с именем `canrxq`.
11. В строках 153 и 154 включаются имеющиеся в STM32 два канала прерываний NVIC.
12. Включаются FIFO 0 и FIFO 1, ожидающие прерывания от сообщений буферы FIFO периферийного устройства шины CAN (строка 155).
13. Наконец, в строке 157 создается задача приема.

Очевидно, что в этой процедуре довольно много деталей. Давайте разберем некоторые этапы.

Функция can_init()

Функция `can_init()` предоставляется библиотекой libopencm3 и требует нескольких аргументов для настройки устройства. Давайте рассмотрим аргументы вызова более подробно. Приведенные аргументы по порядку заключаются в следующем.

1. Аргумент `CAN1` указывает, какое периферийное устройство использовать. Для STM32F103C8T6 доступно только одно.
2. Значение `false` в этом аргументе указывает, что мы не используем режим связи с синхронизацией по времени.

3. Значение `false` в следующем аргументе указывает, что мы не используем режим автоматического отключения шины (если происходит слишком много ошибок, шина может быть автоматически отключена).
4. Значение `true` в следующем аргументе указывает, что нам нужен режим автоматического пробуждения, если контроллер будет переведен в спящий режим.
5. Значение `nart` в следующем аргументе означает, что мы не хотим, чтобы периферийное устройство CAN выполняло автоматическую повторную передачу при обнаружении ошибки.
6. Значение `locked` в следующем аргументе означает, что, когда приемный буфер FIFO заполняется, новое сообщение не заменит существующее сообщение (FIFO заблокирован). Если значение `false`, новые сообщения могут заменить существующие сообщения, когда FIFO заполнен.
7. Значение `false` в следующем аргументе указывает, что исходящие сообщения имеют приоритет в соответствии с их идентификатором сообщения. В противном случае сообщения передаются в хронологическом порядке.
8. `PARM_SJW`.
9. `PARM_TC1`.
10. `PARM_TS2` определяют параметры синхронизации CAN.
11. `PARM_BRP` объявлен в соответствии со строкой 78 файла *canmsgs.h*, так что эффективная скорость передачи данных составляет 33.333 Кбит/с.
12. Предпоследнему аргументу присваивается значение `false`, чтобы отключить возможность обратной связи периферийного устройства.
13. Значение `false` в последнем аргументе указывает, что периферийное устройство работает в «нормальном» режиме. В «бесшумном» режиме периферийное устройство может получать удаленные данные, но не может инициировать сообщения.

Приемные фильтры CAN

Периферийное устройство шины CAN имеет возможность фильтровать входные сообщения. Если представить себе большой набор коммуникаторов, подключенных к шине, становится очевидным, что будет получен большой трафик сообщений. Обычно не каждый узел заинтересован во всех сообщениях. Обработка каждого сообщения съедала бы доступные ресурсы контроллера.

Для приема сообщений периферийное устройство CAN поддерживает две очереди FIFO (очередь типа «первым пришел – первым обслужен»). С помощью фильтрации в этом демонстрационном примере идентификаторы сообщений с четными номерами помещаются в FIFO 0, с нечетными номерами – в FIFO 1. Это организовано с помощью конфигурации банков фильтров 0 и 1.

Вызов функции `can_filter_id_mask_16bit_init()` в строках 135–141 организует набор сообщений для размещения в FIFO 0 (строка 140). Аргумент 2 в этом примере объявляет конфигурацию банка фильтров 0 (строка 137). Последний аргумент (`true`) просто включает фильтр.

Аргументы 3 (строка 138) и 4 (строка 139) определяют фактическое значение идентификатора фильтра и используемую битовую маску. Это 16-битные

фильтры, но ширина фильтра составляет 32 бита. По этой причине используются два одинаковых фильтра:

- 0x000 – это результирующий идентификатор, с которым необходимо сопоставить после применения маски,
- 0x001 – это битовая маска, которая применяется к идентификатору перед сравнением.

Оба этих аргумента необходимо сдвинуть на 5 бит влево, чтобы выровнять по левому краю 11-битные идентификаторы в 16-битном поле.

Во втором настроенном фильтре (строки 143–149) мы имеем то же значение маски (0x001), но сравниваем два разных значения идентификатора:

- 0x010 – это идентификатор «нет совпадения»,
- 0x001 – нечетное значение после маскировки.

Если к идентификатору применяется маска 0x001, результат соответствует 0x001, если идентификатор нечетный. Однако независимо от того, какой идентификатор получается после маскировки с помощью 0x001, он никогда не будет соответствовать заданному здесь значению 0x010. Это просто еще один способ отключения второго неиспользуемого фильтра.

Согласно настройке сообщение всегда будет нечетным или четным и попадет в одну из очередей приема FIFO периферийного устройства CAN.

Существует несколько других возможностей для указания фильтров, включая использование 32-битных значений сравнения для расширенных значений идентификаторов. Читателю рекомендуется просмотреть документацию по API libopencm3 и справочное руководство STMicroelectronics RM0008 (см. «Дополнительные источники» [1] к главе 7), раздел 24.7.4, для получения дополнительной информации.

Прерывания приема CAN

Каждая очередь CAN FIFO имеет свое собственное прерывание. Это позволяет разработчику назначать разные приоритеты прерываний каждой очереди FIFO. В демонстрационном примере мы обрабатываем обе функции одинаково, поэтому в листинге 19.2 показаны промежуточные подпрограммы, используемые для перенаправления кода на одну общую функцию с именем `can_rx_isr()`.

Листинг 19.2. Промежуточные обработчики приемного прерывания CAN

```
0058: void
0059: usb_lp_can_rx0_isr(void) {
0060:     can_rx_isr(0,CAN_RF0R(CAN1)&3);
0061: }
0067: void
0068: can_rx1_isr(void) {
0069:     can_rx_isr(1,CAN_RF1R(CAN1)&3);
0070: }
```

Макроопределения `CAN_RF0R()` и `CAN_RF1R()` позволяют коду определять длину очередей FIFO. Общий код обработчика прерывания приема CAN показан в листинге 19.3.

Листинг 19.3. Общая ISR приема CAN

```
0029: static void
0030: can_rx_isr(uint8_t fifo,unsigned msgcount) {
0031:     struct s_canmsg cmsg;
0032:     bool xmsgidf, rtrf;
0033:
0034:     while ( msgcount-- > 0 ) {
0035:         can_receive(
0036:             CAN1,
0037:             fifo,           // FIFO # 1
0038:             true,           // Release
0039:             &cmsg.msgid,
0040:             &xmsgidf,       // true if msgid is extended
0041:             &rtrf,          // true if requested transmission
0042:             (uint8_t *)&cmsg.fmi, // Matched filter index
0043:             &cmsg.length,   // Returned length
0044:             cmsg.data,
0045:             NULL);          // Unused timestamp
0046:         cmsg.xmsgidf = xmsgidf;
0047:         cmsg.rtrf = rtrf;
0048:         cmsg.fifo = fifo;
0049:         // If the queue is full, the message is lost
0050:         xQueueSendToBackFromISR(canrxq,&cmsg,NULL);
0051:     }
0052: }
```

Общий порядок кода следующий.

1. Получение сообщений (строки 35–45).
2. Постановка сообщений в очередь FreeRTOS `canrxq` (строка 50).
3. Повтор, пока сообщений не останется (строка 34).

Чтобы понять другие детали, нам нужно знать о задействованных структурах. Они проиллюстрированы в листинге 19.4.

Листинг 19.4. Структуры сообщений из *canmsgs.h*

```
0020: struct s_canmsg {
0021:   uint32_t msgid;       // Message ID
0022:   uint32_t fmi;         // Filter index
0023:   uint8_t  length;      // Data length
0024:   uint8_t  data[8];     // Received data
0025:   uint8_t  xmsgidf : 1; // Extended message flag
0026:   uint8_t  rtrf : 1;    // RTR flag
0027:   uint8_t  fifo : 1;    // RX Fifo 0 or 1
0028: };
0029:
0030: enum MsgID {
0031:   ID_LeftEn = 100,     // Left signals on/off (s_lamp_en)
0032:   ID_RightEn,          // Right signals on/off (s_lamp_en)
0033:   ID_ParkEn,           // Parking lights on/off (s_lamp_en)
```

```
0034:   ID_BrakeEn,          // Brake lights on/off (s_lamp_en)
0035:   ID_Flash,            // Inverts signal bulb flash
0036:   ID_Temp,             // Temperature
0037:   ID_HeartBeat = 200,  // Heartbeat signal (s_lamp_status)
0038:   ID_HeartBeat2        // Rear unit heartbeat
0039: };
0040:
0041: struct s_lamp_en {
0042:   uint8_t  enable : 1;  // 1==on, 0==off
0043:   uint8_t  reserved : 1;
0044: };
0045:
0046: struct s_temp100 {
0047:   int      celciusx100; // Degrees Celcius x 100
0048: };
0049:
0050: struct s_lamp_status {
0051:   uint8_t   left : 1;   // Left signal on
0052:   uint8_t   right : 1;  // Right signal on
0053:   uint8_t   park : 1;   // Parking lights on
0054:   uint8_t   brake : 1;  // Brake lines on
0055:   uint8_t   flash : 1;  // True for signal flash
0056:   uint8_t   reserved : 4;
0057: };
```

По сути, сообщение поступает в структуру `s_canmsg` (см. строки 35–45 листинга 19.3). Некоторые части необходимо загрузить, а затем скопировать в структуру. Например, член структуры `xmsgidf` имеет размер 1 бит, поэтому он принимается в локальной переменной с именем `xmsgidf` (строка 40), а затем копируется в `cmsg.xmsgidf` в строке 46. Остальные члены копируются в структуру в строках 47 и 48. К моменту продолжения выполнения в строке 50 структура полностью заполняется и затем копируется в очередь. Обратите внимание, что вызываемая подпрограмма FreeRTOS называется `xQueueSendToBackFromISR()`, т. е. имя заканчивается на «FromISR». Это необходимо, поскольку в обработчиках прерывания ISR часто необходимо принимать специальные меры из-за их асинхронной природы.

Основная полезная нагрузка хранится в массиве `data[8]`, а его активная длина определяется длиной элемента в этой программе. Наше приложение использует действительно небольшие сообщения.

Сообщение обозначается идентификатором сообщения `MsgID`. Это документировано следующим образом:

```
0030: enum MsgID {
0031:   ID_LeftEn = 100,    // Left signals on/off (s_lamp_en)
0032:   ID_RightEn,         // Right signals on/off (s_lamp_en)
0033:   ID_ParkEn,          // Parking lights on/off (s_lamp_en)
0034:   ID_BrakeEn,         // Brake lights on/off (s_lamp_en)
0035:   ID_Flash,           // Inverts signal bulb flash
0036:   ID_Temp,            // Temperature
```

```
0037:   ID_HeartBeat = 200, // Heartbeat signal (s_lamp_status)
0038:   ID_HeartBeat2       // Rear unit heartbeat
0039: };
```

Вопрос для популярной викторины: какое сообщение в этом наборе имеет наивысший приоритет?

Ответ: `ID_LeftEn` (со значением 100).

Почему? Потому что это наименьший (определенный) идентификатор сообщения в наборе. Напомним, что из-за особенностей доминирующих битов CAN в арбитражном состязании всегда выигрывает наименьший идентификатор сообщения.

Это типы сообщений, используемые нашей демонстрационной программой. Значение идентификатора сообщения `ID_HeartBeat` – это сообщение «я жив» от переднего контроллера, а `ID_HeartBeat2` – аналогичное сообщение от заднего контроллера. Наша демонстрационная версия ничего не делает с этими сообщениями при получении, но при наличии большего количества кода главный контроллер может предупреждать, если передний или задний контроллер не отправляет сообщение через регулярные промежутки времени.

Значения идентификатора сообщения `ID_LeftEn`, `ID_RightEn`, `ID_ParkEn` и `ID_BrakeEn` указывают на сообщение управления фонарями. Структура `s_lamp_en` выполняет запланированное действие. Ее член `enable` указывает, должно ли сообщение включать или выключать фонарь. Эти данные передаются в элементе массива `data[]` `s_canmsg`:

```
0041: struct s_lamp_en {
0042:   uint8_t    enable : 1; // 1==on, 0==off
0043:   uint8_t    reserved : 1;
0044: };
```

Сообщение `ID_Temp` используется как для запроса температуры, так и для ее получения. Главный блок управления запросит показания температуры, отправив `ID_Temp`, при этом для члена `rtrf` установлено значение `true`. Когда задний блок управления получит это сообщение, он прочитает его и ответит, установив для флага `rtrf` значение `false`. Возвращенное значение температуры передается в элементе `data[]` как структура `s_temp100`:

```
0046: struct s_temp100 {
0047:   int    celciusx100; // Degrees Celcius x 100
0048: };
```

Прием в приложении

Как только ISR помещает сообщение с данными в очередь `s_canmsg`, что-то должно извлечь сообщения из очереди. В модуле *canmsgs.c* для этого определена задача:

```
0076: static void
0077: can_rx_task(void *arg __attribute((unused))) {
```

```
0078:     struct s_canmsg cmsg;
0079:
0080:     for (;;) {
0081:       if (xQueueReceive(canrxq,&cmsg,portMAX_DELAY) == pdPASS)
0082:           can_recv(&cmsg);
0083:     }
0084: }
```

Эта небольшая задача просто извлекает из `canrxq` сообщения, поставленные в очередь ISR. Если сообщений в очереди нет, задача блокируется из-за аргумента тайм-аута `portMAX_DELAY` в строке 81. Если сообщение успешно извлекается из очереди, вместе с ним вызывается прикладная функция `can_recv()` (не путать с процедурой libopencm3 с именем `can_receive()`).

Обработка сообщения

Приложение уведомляется о входящих сообщениях CAN, когда функция `can_recv()` вызывается модулем *canmsgs.c*. Это выполняется вне вызова ISR, поэтому использование большинства функций программирования должно быть безопасным. Листинг 19.5 иллюстрирует функцию, объявленную в модуле *back.c*.

Листинг 19.5. Обработка полученного сообщения CAN в приложении (из файла *back.c*)

```
0119: void
0120: can_recv(struct s_canmsg *msg) {
0121:     union u_msg {
0122:         struct s_lamp_en lamp;
0123:     } *msgp = (union u_msg *)msg->data;
0124:     struct s_temp100 temp_msg;
0125:
0126:     gpio_toggle(GPIO_PORT_LED,GPIO_LED);
0127:
0128:     if ( !msg->rtrf ) {
0129:         // Received commands:
0130:         switch ( msg->msgid ) {
0131:         case ID_LeftEn:
0132:         case ID_RightEn:
0133:         case ID_ParkEn:
0134:         case ID_BrakeEn:
0135:         case ID_Flash:
0136:             lamp_enable((enum MsgID)msg->msgid,msgp->lamp.enable);
0137:             break;
0138:         default:
0139:             break;
0140:     }
0141:     } else {
0142:         // Requests:
0143:         switch ( msg->msgid ) {
```

```
0144:         case ID_Temp:
0145:             temp_msg.celciusx100 = degrees_C100();
0146:             can_xmit(ID_Temp,false,false,sizeof temp_msg,&temp_msg);
0147:             break;
0148:         default:
0149:             break;
0150:         }
0151:     }
0152: }
```

Сообщение передается в `can_recv()` с указателем на структуру `s_canmsg`, которая заполняется полученными данными. Поскольку элемент данных `msg->data` интерпретируется на основе его идентификатора сообщения, объединение `u_msg` объявляется в строках 121–123. Это позволяет программисту получать доступ к массиву `msg->data` как к структуре `s_lamp_en`, когда это необходимо.

Чтобы подать признаки рабочего состояния, строка 126 включает встроенный светодиод (PC13). Обычные сообщения не будут иметь установленного флага `msg->rtrf` (строка 128). В этом случае нас ожидают обычные команды для сигнальных огней (строки 130–137).

В противном случае, когда `msg->rtrf` имеет значение `true`, это представляет собой запрос к заднему модулю считать температуру и ответить ею (строки 143–150). Функция `degrees_C100()` возвращает температуру в градусах Цельсия, умноженную на 100. Это просто передается в строке 146. Обратите внимание, что третьим аргументом вызова является флаг RTR, который отправляется как `false` (что означает ответ). Функция `can_xmit()` объявлена в *canmsgs.c*, не путать с процедурой `can_transmit()` из libopencm3.

Помните, что `can_recv()` вызывается как часть другой задачи. Этого требует безопасное взаимодействие между задачами.

Отправка CAN-сообщений

Отправлять сообщения легко, поскольку нам просто нужно вызвать подпрограмму libopencm3 `can_transmit()`:

```
0018: void
0019: can_xmit(uint32_t id,bool ext,bool rtr,uint8_t length,void *data) {
0020:
0021:   while ( can_transmit(CAN1,id,ext,rtr,length,(uint8_t*)data) == -1 )
0022:       taskYIELD();
0023: }
```

В аппаратном обеспечении платы Blue Pill имеется только один контроллер CAN-шины, поэтому CAN1 здесь жестко запрограммирован как аргумент 1. Идентификатор сообщения передается через `id`, флаг расширенного адреса передается через `ext`, а флаг запроса `rtr` передается в качестве четвертого аргумента. Наконец, передается длина данных и указатель на данные. Если вызов завершается неудачей и возвращает –1, вызывается `TaskYIELD()` для

разделения времени процессора. Если очередь отправляющих периферийных устройств CAN-шины заполнена, вызов завершится неудачей.

Это поднимает важный момент, о котором следует помнить. После успешного возврата из `can_transmit()` вызывающая сторона никак не может узнать, что сообщение уже отправлено. Напомним, что в нашей конфигурации мы указали, будут ли сообщения отправляться в приоритетной или хронологической последовательности. Но шина также может быть занята, что приводит к задержке фактической отправки наших сообщений. Кроме того, если наше сообщение(я) имеет более низкий приоритет (более высокие значения идентификатора сообщения), чем другие сообщения на шине, наше периферийное устройство CAN должно ждать, пока оно не выиграет арбитраж шины.

Итоги

Оставшаяся часть демонстрационного примера представляет собой обычный код на языке C. Читателю предлагается ознакомиться с ним. Вследствие упаковки API приложения CAN-шины в модули `canmsgs.c` и `canmsgs.h` задача написания приложения упрощается. Это также экономит время за счет использования общего проверенного кода.

Этот пример лишь поверхностно показал, что можно сделать на шине CAN. Некоторые люди, возможно, захотят проверить работу своих автомобилей, но следует соблюдать осторожность. Некоторые конструкции CAN-шины, такие как GMLAN (локальная сеть General Motors), могут включать в себя всплески сигнала напряжением 12 В в качестве сигнала пробуждения. Вероятно, существует ряд других вариаций этой темы.

Шина CAN применяется в ряде других приложений, таких как управление производством или лифтами. Поработав с этим демонстрационным проектом, вы сможете воплотить в жизнь новые конструкторские идеи.

Упражнения

1. Сколько буферов FIFO поддерживается периферийным устройством CAN ST M32F103?
2. Сколько наборов фильтров поддерживается периферийным устройством CAN?
3. Если необходимо поставить пару фильтров, но нужен только один, какие есть два способа сделать это?
4. Что такое флаг RTR и каково его назначение?

Глава 20 • Прерывания UART

Хотя последовательная периферия UART/USART была рассмотрена в главе 6, существует значительное улучшение, которое можно получить, если добавить управление прерываниями приемника. В этой главе рассматриваются причины этого положения и способы это сделать с помощью библиотеки libopencm3.

Основы

Периферийное устройство STM32 USART содержит один регистр данных, в котором хранится последнее полученное 8- или 9-битное слово. Это состояние остается действительным до тех пор, пока следующее последовательное слово не будет полностью получено в сдвиговом регистре приемника. Если эти данные не считываются процессором до того, как будет получено следующее слово, помечается состояние переполнения, и хранящиеся данные теряются. Последние полученные данные перезаписывают регистр данных.

При низких скоростях передачи программа, выполняемая контроллером, обычно может получать данные быстрее, чем они поступают. Но если работа программного обеспечения задерживается из-за других событий или приоритета задачи, данные могут быть потеряны. При более высоких скоростях передачи данных вашему программному обеспечению может быть сложно потреблять данные достаточно быстро[90].

Решение этой проблемы состоит в том, чтобы периферийное устройство вызывало прерывание в тот самый момент, когда данные загружаются в регистр хранения данных. Выполняемый в данный момент код приостанавливается, и немедленно вызывается подпрограмма обработки прерываний (ISR). Эта процедура считывает полученный символ и помещает его в память до прибытия следующего слова данных. Когда ISR возвращает управление основной программе, она возобновляет выполнение кода с того места, где оно было прервано.

Подпрограммы обработки прерываний (ISR)

Существуют некоторые ограничения на использование ISR. Они вызываются асинхронно, т. е. могут сработать в середине критически важных функций, таких как `malloc()`. По этой причине необходимо, чтобы ISR был написан так, чтобы избежать вызова нерекурсивных функций. Это одна из причин, почему подпрограммы FreeRTOS для использования ISR имеют имена, заканчивающиеся на «FromISR», например `xQueueSendFromISR()`.

[90] По поводу скорости передачи данных через UART см. также сноску 44 на стр. 95.

ISR также должны вернуть управление как можно скорее, чтобы избежать проблем с другими прерываниями и системой в целом[91]. Вложенные прерывания требуют больше места в стеке. По этой причине они не должны блокировать выполнение.

ISR также использует стек текущей задачи при вызове. По этой причине ISR должен использовать минимальное пространство стека. Аналогичным образом все задачи должны выделять достаточно дополнительного пространства стека для использования подпрограммами обслуживания прерываний.

Учитывая, что прерывания могут произойти практически в любой момент, ISR должен позаботиться о доступе к глобальным переменным. ISR также должен избегать критических секций, поскольку последние обычно включают в себя отключение прерываний. Демонстрационная программа этой главы использует очередь FreeRTOS для решения проблемы доступа к общим данным. Это удовлетворяет требованиям, поскольку операции с очередью являются атомарными операциями.

Демопрограмма

Демонстрационная программа находится в каталоге `~/stm32f103c8t6/rtos/uart_rxint`. Ее функциональность заключается в простом чтении последовательных данных из minicom (или его эквивалента) и передаче их обратно на терминал. При желании программу можно модифицировать для замены строчных букв на прописные и наоборот в качестве доказательства того, что устройство STM32 обработало данные. Подключения демопроекта можно выполнить согласно рис. 20.1.

Разработка программы

Демонстрационная программа имеет знакомую форму: основная функция выполняет всю настройку и запускает планировщик ядра FreeRTOS (листинг 20.1).

Листинг 20.1. Функция настройки `main()`

```
0205: int
0206: main(void) {
0207:
```

[91] Это замечание в полной мере касается функционирования в среде ОС реального времени, где общее управление осуществляется с помощью переключения задач в основной программе, причем в предположении, что контроллер максимально загружен этими задачами. Если мы имеем рядовую линейную программу, то в ней совершенно иная ситуация: всю программу можно безболезненно упаковать в прерывания, оставив основную программу в виде пустого цикла. В огромном большинстве программ для контроллеров внешние события (включая обмен по USB, UART, SPI и пр.) происходят в сравнении со скоростью работы процессора настолько редко, что прерывания просто не успевают помешать друг другу: вероятность их пересечения близка к нулю. А если пересечения и случаются, то ничего страшного не произойдет, если одно из прерываний ненадолго отложится.

```
0208:   rcc_clock_setup_pll(&rcc_hse_configs[RCC_CLOCK_HSE8_72MHZ]);
0209:
0210:   init_clock();
0211:   init_gpio();
0212:   init_usart();
0213:
0214:   xTaskCreate(main_task,"MAIN",100,NULL,configMAX_PRIORITIES-1,NULL);
0215:   xTaskCreate(tx_task,"UARTTX",100,NULL,configMAX_PRIORITIES-1,NULL);
0216:   vTaskStartScheduler();
0217:
0218:   for (;;); // Don't go past here
0219:   return 0;
0220: }
```

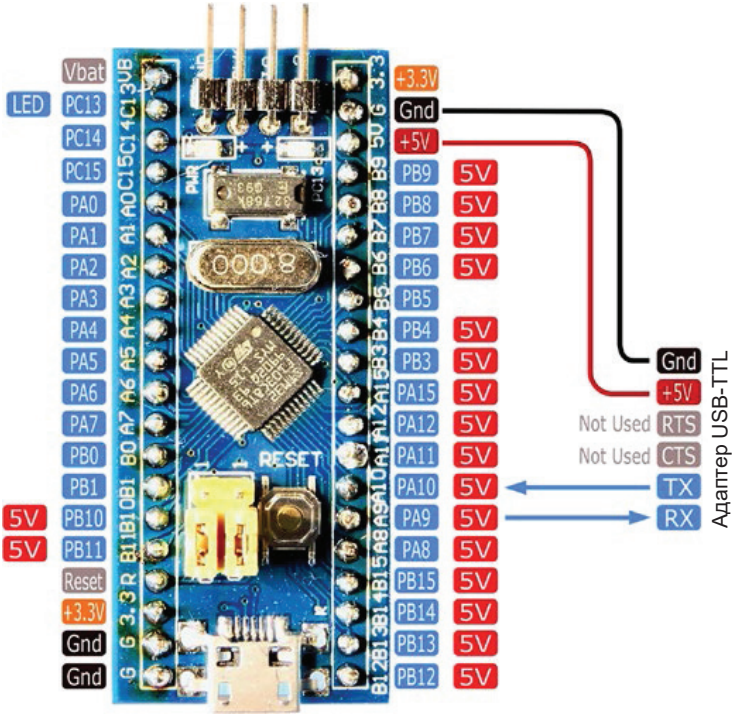


Рис. 20.1. Схема подключения UART

Строка 208 устанавливает тактовую частоту процессора. Строка 210 инициализирует все остальные тактовые частоты для этой программы, а строка 211 инициализирует порт GPIO для светодиода. Наконец, периферийное устройство USART инициализируется в строке 212.

Основная задача и задача передачи создаются в строках 214 и 215 перед запуском планировщика FreeRTOS в строке 216. Обычно выполнение оттуда не должно возвращаться, но бесконечный цикл в строке 218 предусмотрен в целях безопасности.

Инициализация тактовых сигналов

Листинг 20.2 иллюстрирует инициализацию источников тактовых сигналов для программы. Строка 57 инициализирует тактовый сигнал для GPIOA, поскольку наше периферийное устройство UART будет использовать этот порт. Встроенный светодиод управляется от GPIOC; строка 58 включает такты для него. Наконец, строка 59 включает тактовую частоту периферийного устройства UART.

Листинг 20.2. Инициализация тактовых сигналов

```
0053: static void
0054: init_clock(void) {
0055:
0056:     // Clock for GPIO port A: GPIO_USART1_TX
0057:     rcc_periph_clock_enable(RCC_GPIOA);
0058:     rcc_periph_clock_enable(RCC_GPIOC);
0059:     rcc_periph_clock_enable(RCC_USART1);
0060: }
```

Инициализация GPIO

Процедура `init_gpio()` представлена в листинге 20.3. Строки 122–126 настраивают вывод порта GPIO13 на выход. Затем в строке 127 в него записывается высокий уровень, чтобы выключить светодиод (напомним, что светодиод горит, когда выход этого порта подключается к нулевому потенциалу).

Листинг 20.3. Инициализация GPIO

```
0118: static void
0119: init_gpio(void) {
0120:
0121:   // gpio for LED
0122:   gpio_set_mode(
0123:       GPIOC,
0124:       GPIO_MODE_OUTPUT_2_MHZ,
0125:       GPIO_CNF_OUTPUT_PUSHPULL,
0126:       GPIO13);
0127:   gpio_set(GPIOC,GPIO13); // LED off
0128: }
```

Инициализация UART

Основная часть инициализации UART в этой программе происходит в процедуре `init_usart()`, показанной в листинге 20.4. Код UART будет использовать две очереди FreeRTOS, дескрипторы которых определены в строках 26 и 27.

Очередь передачи создается в строке 89, а очередь приема – в строке 90. Размеры этих очередей были выбраны равными 256 байтам, но это не обязательно. Если используются очень высокие скорости передачи, вы можете выбрать больший размер очереди приема и уменьшенный размер отправки.

Поскольку мы используем прерывания, мы должны включить контроллер прерываний в строке 93. Строки 95–98 настраивают вывод PA9 порта GPIOA в соответствии с `GPIO_USART1_TX` в качестве выхода. Аналогичным образом строки 99–102 настраивают вывод PA10 порта GPIOA в соответствии с `GPIO_USART1_RX` в качестве входа. Скорость передачи данных и остальная конфигурация UART задаются в строках 104–108.

Важно отметить, что строка 110 разрешает прерывания периферийному устройству UART. Код обработчика, который получит управление, мы увидим в листинге 20.5. Строка 111 запускает периферийное устройство UART.

Листинг 20.4. Инициализация UART

```
0026: static QueueHandle_t uart_txq, // TX queue
0027: uart_rxq; // RX queue
0085: static void
0086: init_usart(void) {
0087:
0088:     // Create queues
0089:     uart_txq = xQueueCreate(256,sizeof(char));
0090:     uart_rxq = xQueueCreate(256,sizeof(char));
0091:
0092:     // Enable Interrupt controller
0093:     nvic_enable_irq(NVIC_USART1_IRQ);
0094:
0095:     gpio_set_mode(GPIOA,
0096:         GPIO_MODE_OUTPUT_50_MHZ,
0097:         GPIO_CNF_OUTPUT_ALTFN_PUSHPULL,
0098:         GPIO_USART1_TX);
0099:     gpio_set_mode(GPIOA,
0100:         GPIO_MODE_INPUT,
0101:         GPIO_CNF_INPUT_FLOAT,
0102:         GPIO_USART1_RX);
0103:
0104:     usart_set_baudrate(USART1,115200);
0105:     usart_set_databits(USART1,8);
0106:     usart_set_stopbits(USART1,USART_STOPBITS_1);
0107:     usart_set_mode(USART1,USART_MODE_TX_RX);
0108:     usart_set_parity(USART1,USART_PARITY_NONE);
0109:
0110:     usart_enable_rx_interrupt(USART1);
0111:     usart_enable(USART1);
0112: }
```

Процедура обработки прерываний

Библиотека libopencm3 требует, чтобы подпрограмма обработки прерываний UART называлась `usart1_isr()`, как показано в листинге 20.5. В строке 72 проверяется, были ли получены какие-либо данные для UART. Если да, то выполняется код между строками 73 и 77.

Строка 74 копирует полученное слово данных в переменную `ch` (строка 68). В этой демонстрационной программе мы ожидаем только 8 бит, поэтому

использовался тип char. Как только этот байт будет получен, мы по возможности помещаем его в uart_rxq в строке 77. Если по какой-либо причине очередь заполнена, статус сбоя, возвращаемый функцией xQueueSendFromISR(), игнорируется. Поскольку этот код выполняется внутри прерывания, мы не можем блокировать его и ждать, пока в очереди освободится место. Также обратите внимание, что имя функции отправки очереди заканчивается суффиксом «FromISR».

Блок кода находится внутри цикла while (начиная со строки 72). При чрезвычайно высоких скоростях передачи данных в регистр данных может быть принят еще один байт данных сразу после того, как мы извлекли последний. Вместо того чтобы ждать накладных расходов следующего прерывания, мы обрабатываем его немедленно. При более низких скоростях передачи этот цикл никогда не повторяется.

Листинг 20.5. Процедура обработки прерываний UART

```
0066: void
0067: usart1_isr(void) {
0068:     char ch;
0069:     BaseType_t hptask=pdFALSE;
0070:
0072:     while ( ((USART_SR(USART1) & USART_SR_RXNE) != 0) ) {
0073:         // Have received data:
0074:         ch = usart_recv(USART1);
0075:
0076:         // Use best effort to send byte to queue
0077:         xQueueSendFromISR(uart_rxq,&ch,&hptask);
0078:     }
0079: }
```

Функция uart_getc()

Доступ к символьным данным, помещенным в очередь в строке 77, осуществляется через функцию uart_getc() в листинге 20.6. Когда задача хочет получить данные, она вызывает функцию uart_getc() для получения байта данных в строке 156. Если ожидающих данных нет, вызов передается планировщику задач в строке 157. В противном случае выполнение переходит к строке 158, где мы переключаем состояние светодиода, чтобы продемонстрировать, что мы получили данные, и возвращаем их в строке 159.

Листинг 20.6. Функция uart_getc()

```
0152: static char
0153: uart_getc(void) {
0154:     char ch;
0155:
0156:     while ( xQueueReceive(uart_rxq,&ch,0) != pdPASS )
0157:         taskYIELD();
0158:     gpio_toggle(GPIOC,GPIO13);
```

```
0159:     return ch;
0160: }
```

Задача main_task()

Одной из запущенных задач является функция `main_task()`, показанная в листинге 20.7. Ее назначение – просто получить символ в строке 190 и блокироваться до тех пор, пока не поступят данные. Как только данные поступают, они отправляются обратно через UART в строке 197.

При желании код в строках 192–195 можно добавить в исполняемый файл, чтобы инвертировать регистр полученных данных. Это обеспечивает визуальное подтверждение того, что STM32 действительно обработал данные. Это можно сделать, изменив значение в строке 29 на 1.

Листинг 20.7. Функция `main_task()`

```
0029: #define INVERT_CASE 0
0185: static void
0186: main_task(void *args __attribute((unused))) {
0187:
0188:     for (;;) {
0189:         // Block until char received
0190:         char ch = uart_getc();
0191: #if INVERT_CASE
0192:         if ( islower(ch) )
0193:             ch = toupper(ch);
0194:         else if ( isupper(ch) )
0195:             ch = tolower(ch);
0196: #endif
0197:         uart_putc(ch);
0198:     }
0199: }
```

Функция uart_putc()

Листинг 20.8 иллюстрирует функцию `uart_putc()`, которую вызывает `main_task()` для отправки данных обратно на терминал. Каждый байт помещается в очередь `uart_txq` в строке 138, если очередь не заполнена. При заполнении задача передается планировщику в строке 139. Если байт представляет собой перевод строки, то возврат каретки также ставится в очередь в строках 140–145. Опять же, если очередь заполнена, выполнение останавливается на строке 144 и управление переходит планировщику. Только когда 1 или 2 байта успешно поставлены в очередь для отправки, выполнение возвращается из функции `uart_putc()`.

Листинг 20.8. Функция `uart_putc()`

```
0134: static inline void
0135: uart_putc(char ch) {
```

```
0136:
0137:     // Queue the byte to send
0138:     while ( xQueueSend(uart_txq,&ch,0) != pdPASS )
0139:         taskYIELD();
0140:     if ( ch == '\n' ) {
0141:         // When we see LF, also send CR
0142:         ch = '\r';
0143:         while ( xQueueSend(uart_txq,&ch,0) != pdPASS )
0144:             taskYIELD();
0145:     }
0146: }
```

Задача tx_task()

Последним важным компонентом этой системы задач является функция `tx_task()`, приведенная в листинге 20.9. Байты, которые должны быть переданы обратно на терминал, принимаются в строке 171 в переменную `ch`. Когда нет данных для получения из очереди, аргумент `portMAX_DELAY` вызывает блокировку вызова функции `xQueueReceive()`. Это, в свою очередь, приводит к тому, что выполнение передается планировщику задач. Однако, если вызов по какой-либо причине завершится неудачно, выход из задачи будет принудительно выполнен в строке 172.

Как только у нас есть байт для отправки в переменную `ch`, цикл в строках 175 и 176 крутится до тех пор, пока периферийное устройство UART не будет готово принять новые данные. Только когда периферийное устройство готово принять новые данные, в строке 177 вызывается функция `usart_send()` из libopencm3. У вас может возникнуть соблазн вызвать вместо этого функцию `usart_send_blocking()` из libopencm3, но это было бы ошибкой. Проблема в том, что `usart_send_blocking()` будет крутиться в ожидании готовности, но не перестанет занимать процессор при вызове `TaskYIELD()`. Это связано с тем, что библиотека libopencm3 «не знает», что мы используем ядро FreeRTOS.

Листинг 20.9. Функция `tx_task()`

```
0166: static void
0167: tx_task(void *args __attribute((unused))) {
0168:    char ch;
0169:
0170:    for (;;) {
0171:    while ( xQueueReceive(uart_txq,&ch, portMAX_DELAY) != pdPASS )
0172:          taskYIELD();
0173:
0174:         // Received a char to transmit:
0175:         while ( !usart_get_flag(USART1,USART_SR_TXE) )
0176:              taskYIELD(); // Not ready to send
0177:         usart_send(USART1,ch);
0178: }
0179: }
```

Запуск демопрограммы

Настройте демопрограмму перед ее запуском, при необходимости изменив строку 29. Чтобы инвертировать регистр отображаемых данных, измените значение макроса на 1. Если оставить для него нулевое значение, данные просто будут отображаться как есть. Возможно, вам захочется инвертировать, чтобы обеспечить четкое доказательство того, что устройство STM32 обрабатывает данные.

```
0029: #define INVERT_CASE 0
```

Пересоберите проект с нуля, особенно если были внесены изменения:

```
$ make clobber
$ make
```

Подключив программатор, прошиваем программу:

```
$ make flash
```

Конфигурация

Теперь запустите *minicom* (или эквивалентный эмулятор терминала) после подключения адаптера последовательного порта к stm32 и подключения его к USB-порту. Укажите имя последовательного USB-устройства после опции -D. У меня на Mac имя было `/dev/cu.usbserial-A50285BI`. Опция -s заставляет *minicom* войти в меню настройки:

```
$ minicom -D /dev/cu.usbserial-A50285BI -s
```

Для этой демопрограммы убедитесь, что в *minicom* отключено аппаратное управление потоком:

```
F - Hardware Flow Control : No
```

Также убедитесь, что вы настроили скорость передачи данных и другие параметры последовательного порта так, чтобы они соответствовали 115 200 бод, режим 8N1.

Перейдите в опцию конфигурации *minicom*:

```
File transfer protocols
```

При необходимости добавим новый протокол, который назовем «cat»[92] (в моем примере это строка опции «J»):

[92] cat (от англ. concatenate) – утилита/команда UNIX/Linux, выводящая последовательно указанные файлы в едином потоке без изменений.

```
┌──────────────────────────────────────────────────┐
| Name      Program   Name U/D FullScr IO-Red. Multi |
| A zmodem  sz -vv -b  Y   U   N       Y       Y     |
| B ymodem  sb -vv     Y   U   N       Y       Y     |
...
| J cat     cat        Y   U   Y       Y       N     |
| K -                                               |
| L -                                               |
| M Zmodem download string activates... D           |
| N Use filename selection window..... Yes          |
| O Prompt for download directory..... No           |
|                                                   |
| Change which setting? (SPACE to delete)           |
└──────────────────────────────────────────────────┘
```

Обратите внимание, что используемые настройки должны соответствовать значениям, указанным в табл. 20.1.

Параметр	Значение
Name	Y
U/D	U
FullScr	Y
IO-Red.	Y
Multi	N

Таблица 20.1. Настройки minicom для протокола передачи файлов «cat»

После того как вы это настроите, можете сохранить эти настройки для будущего использования. Затем выйдите обратно или выберите **Exit** (Выход) в окне конфигурации. На этом этапе то, что вы вводите в терминал, должно быть отражено обратно. Нажатие <**Ctrl+J**> (перевод строки) также должно вернуть курсор влево.

Запуск демопрограммы

Теперь протестируем протокол передачи «cat». Введите <**Meta+S**>[93], чтобы вызвать меню **Upload** (Загрузить) minicom, и выберите **cat**. На этом этапе вы можете либо выбрать текстовый файл из меню **Select a file for upload** (Выбрать файл для загрузки), либо просто нажать клавишу <**Enter**>, чтобы открыть диалоговое окно ввода имени файла:

```
No file selected - enter filename:
>
```

[93] «Meta» – клавиша на клавиатуре компьютеров, ориентированных на Linux. На большинстве современных неспециализированных клавиатур клавиша «Meta» обычно обозначена как «Win» (ОС Windows) или как «Command» (⌘) на клавиатурах, ориентированных на macOS.

Обычно я использую файл */etc/resolv.conf*, который должен быть у всех экземпляров Linux/Mac. Вы можете выбрать любой текстовый файл, который вам нравится:

```
No file selected - enter filename:
> /etc/resolv.conf
```

Как только вы нажмете <**Enter**>, команда `cat` должна скопировать этот текстовый файл на ваше устройство STM32. Как это произошло? На моем Mac я получаю что-то вроде следующего:

```
#
# Thisanism used by most
# processes on this system.
#
# To view the DNS configuration used by this system, use:
# scutil --dns
#
# SEE ALSO
# dns-sd(1), scutil(8)
#
# This file is automatically generated.
#
search mangled.net
nameserver 2001:18c0:ffd0:28::1
nameserver 2001:18c0:ffd0:28::2
nameserver 209.13.165.152
nameserver 209.13.165.153
```

Внимательно изучив выходные данные, вы можете увидеть, что в начале есть некоторая потеря данных. Мы поговорим об этом в следующем разделе. На моей машине с Linux проблем с потерей данных не было. Если вы включили инверсию регистра, вы должны видеть прописные буквы там, где были символы нижнего регистра, и наоборот.

Потеря данных

Следует отметить, что эта демонстрационная программа запускалась без включения управления потоком. Следовательно, при высоких скоростях передачи может произойти некоторая потеря данных. Если вы повторите тест на более низкой скорости передачи данных, возможно, вам удастся устранить потерю.

При любом сообщении от отправителя к получателю существует вероятность того, что данные могут прийти быстрее, чем получатель сможет их получить. Кто-нибудь когда-нибудь давал вам информацию об адресе или номере телефона, когда вы пытались ее записать? Если они говорили слишком быстро, вам, вероятно, пришлось попросить их повторить эту информацию или замедлить темп. Это разные формы восстановления и управления

потоком. Знаменитый эпизод сериала «Я люблю Люси» [1] иллюстрирует проблему с конвейерной лентой на заводе: когда конвейерная лента движется слишком быстро, Люси и Этель в конечном итоге едят и прячут шоколадные конфеты при себе, чтобы не отставать.

Вы должны знать, что устройства USART STM32 и библиотека libopencm3 имеют возможность поддерживать аппаратное управление потоком данных. Включение этого в такие программы, как этот демопример, выходит за рамки этой главы. Однако продвинутый студент может захотеть выполнить это задание, чтобы получить полное представление о связанных с ним трудностях.

Итоги

В этой главе вы узнали, как можно улучшить приемник UART за счет включения прерывания приема. Это расширило лимит периферийного устройства на один регистр данных за счет использования оперативной памяти с помощью очереди ядра FreeRTOS. ISR требовал фиксированное имя `usart1_isr()` для USART1. Для других периферийных устройств USART можно определить разные ISR.

Контроллер прерываний должен быть включен с помощью `nvic_enable_irq()`, а само периферийное устройство должно быть настроено на вызов необходимых прерываний.

Другие прерывания, такие как прерывание передачи, также могут поддерживаться. Но это прерывание возникает только тогда, когда текущее отправленное слово завершено. Следовательно, для отправки первого сообщения в потоке данных должны существовать специальные операции начальной загрузки.

Дополнительные источники

1. «Я люблю Люси», www.youtube.com/watch?v=K3axU2b0dDk&t=4s (дата доступа 16.09.2024).

Упражнения

1. Как обработка прерываний приема помогает предотвратить потерю данных?
2. Почему вместо буфера массива памяти используется очередь FreeRTOS?
3. Почему ISR не должен блокировать выполнение?
4. Почему необходимо использовать функцию `usart_send()` в libopencm3 вместо `usart_send_blocking()`?
5. Каковы необходимые условия перед вызовом `usart_send()`?

Глава 21 • Новые проекты

Запуск нового проекта с нуля может оказаться трудоемким занятием. Вот почему эта глава направлена на то, чтобы помочь вам начать работу с минимумом рутины. Я также собираюсь указать вам на несколько деталей, которые были проигнорированы ради простоты и которые могут быть важны для вашего проекта. Это должно внушить вам чувство уверенности.

Создание проекта

Первым шагом в новом проекте является создание его подкаталога, файла *Makefile*[94], а затем импорт исходных модулей FreeRTOS и начального файла конфигурации с именем *FreeRTOSConfig.h*. Вы можете сделать это вручную или с помощью сценария, но предоставленный *Makefile* сделает это за вас.

Сначала найдите правильный стартовый каталог:

```
$ cd ~/stm32f103c8t6/rtos
```

Придумайте хорошее имя подкаталога для вашего проекта, которого еще нет. В этом примере мы назовем его *myproj*. Чтобы создать проект с именем myproj, выполните следующую команду `make`:

```
$ make -f Project.mk PROJECT=myproj
...bunch of copies etc...
****************************************************************
Your project in subdirectory myproj is now ready.
1. Edit FreeRTOSConfig.h per project requirements.
2. Edit Makefile SRCFILES as required. This also
   chooses which heap_*.c to use.
3. Edit stm32f103c8t6.ld if necessary.
4. make
5. make flash
6. make clean or make clobber as required
****************************************************************
```

Если вы теперь создадите рекурсивный список вашего подкаталога, то увидите, что он заполнен несколькими файлами:

```
$ ls -R ./myproj
FreeRTOSConfig.h   Makefile   main.c        rtos
```

[94] Makefile – файл с инструкциями для утилиты make, которая нужна для автоматической сборки проекта.

```
stm32f103c8t6.ld
./myproj/rtos:
FreeRTOS.h         heap_1.c   list.h        portmacro.h
task.h             LICENSE    heap_2.c      mpu_prototypes.h
projdefs.h         tasks.c    StackMacros.h heap_3.c
mpu_wrappers.h     queue.c    timers.h      croutine.h
heap_4.c           opencm3.c  queue.h       deprecated_definitions.h
heap_5.c           port.c     semphr.h      event_groups.h
list.c             portable.h stdint.readme
```

Makefile

В листинге 21.1 показано содержимое *Makefile*, который будет создан для вас. Обычно этот файл следует слегка отредактировать, чтобы отразить исходные модули, которые вы будете использовать. Этот *Makefile* использует несколько макроопределений для всего проекта. Давайте рассмотрим их сейчас.

Листинг 21.1. *Makefile* проекта по умолчанию

```
0001: ##########################################################
0002: # Project Makefile
0003: ##########################################################
0004:
0005: BINARY = main
0006: SRCFILES = main.c rtos/heap_4.c rtos/list.c rtos/port.c \
rtos/queue.c rtos/tasks.c rtos/opencm3.c
0007: LDSCRIPT = stm32f103c8t6.ld
0008:
0009: # DEPS = # Any additional dependencies for your build
0010: # CLOBBER += # Any additional files to be removed with \
"make clobber"
0011:
0012: include ../../Makefile.incl
0013: include ../Makefile.rtos
0014:
0015: ##########################################################
0016: # NOTES:
0017: # 1. remove any modules you don't need from SRCFILES
0018: # 2. "make clean" will remove *.o etc., but leaves *.elf, *.bin
0019: # 3. "make clobber" will "clean" and remove *.elf, *.bin etc.
0020: # 4. "make flash" will perform:
0021: # st-flash write main.bin 0x8000000
0022: ##########################################################
```

Макроопределение BINARY

Этот макроопределение устанавливает имя вашего скомпилированного исполняемого файла. По умолчанию для него установлено значение *main*, чтобы при сборке проекта создавался файл *main.elf*. Вы можете изменить это название на что-нибудь более захватывающее.

Макроопределение SRFILES

Это макроопределение устанавливает имена исходных файлов, которые будут скомпилированы в окончательный исполняемый файл *main.elf*. По умолчанию включены следующие исходные файлы:

- *main.c* (должно соответствовать имени, установленному в `BINARY`),
- *rtos/heap_4.c*,
- *rtos/list.c*,
- *rtos/port.c*,
- *rtos/queue.c*,
- *rtos/tasks.c*,
- *rtos/opencm3.c*.

Модуль *main.c* (или названный иначе) – это модуль, который вы будете писать и разрабатывать. Остальные модули являются вспомогательными, о них речь пойдет позже. Некоторые из них являются необязательными. Например, если вы не используете очереди сообщений FreeRTOS, вы можете исключить модуль *rtos/queue.c* (с соответствующими изменениями в *FreeRTOSConfig.h*).

Если у вас есть дополнительные исходные файлы (помимо *main.c*), добавьте их в список `SRCFILES`. Они также будут скомпилированы и включены в окончательную сборку.

Макроопределение LDSCRIPT

В предоставленном *Makefile* для этого параметра установлено значение `stm32f103c8t6.ld`. Это указывает на файл в каталоге вашего проекта, который вы уже видели в главе 9. Многие проекты могут использовать этот файл без изменений. Если у проекта есть особенности, такие как оверлеи, файл можно изменить.

Макроопределение DEPS

Если у вас есть особые зависимости, можете определить их с помощью этого макроопределения. Например, если у вас есть текстовый файл типа *mysettings.xml*, который влияет на сборку *main.elf*, то для принудительной перестройки *main.elf* добавьте следующее:

```
DEPS = mysettings.xml
```

Макроопределение CLOBBER

Файлы *make* были написаны для поддержки некоторых основных целей, включая стирание[95], например:

```
$ make clobber
```

Эта команда удаляет файлы, которые были созданы и которые не нужно сохранять. Например, все объектные файлы (**.o*) и исполняемые файлы (**.elf*)

[95] Clobber в переводе означает «затереть», «разгромить».

будут удалены в ходе операции очистки. Это также гарантирует, что при следующем выполнении сборки все будет построено с нуля.

Иногда процедура сборки создает другие объекты, которые можно удалить после завершения сборки. Если файл *file.dat* создается в процессе сборки и вы хотите очистить его после удаления, добавьте его в макроопределение:

```
CLOBBER = file.dat
```

Дополнительные Makefile

Чтобы уменьшить размер *Makefile* проекта и централизовать другие определения, в файл проекта включены еще два *Makefile*:

- *../../Makefile.incl* (*~/stm32f103c8t6/Makefile.incl*),
- *../Makefile.rtos* (*~/stm32f103c8t6/rtos/Makefile.rtos*).

Первый из них определяет макроопределения и правила построения проектов. Если вам нужно внести улучшения в правила `make` в рамках всего проекта, это то, с чего стоит начать.

Второй из них просто добавляет подкаталог *./rtos* для поиска включаемых файлов при сборке FreeRTOS:

```
TGT_CFLAGS += -I./rtos -I.
TGT_CXXFLAGS += -I./rtos -I.
```

Заголовки зависимостей

Макроопределение DEPS, описанное ранее, добавляет зависимости для сборки *main.elf*. Что, если у вас есть другой заголовочный файл с именем *myproj.h*, изменение которого приведет к перекомпиляции *main.o*? Это можно сделать, добавив обычное правило зависимостей *Makefile*:

```
main.o: myproj.h
```

Это сообщает команде `make`, что файл *main.c* следует перекомпилировать в *main.o*, если дата файла *myproj.h* более новая. Это может уберечь вас от поиска ошибок, связанных с изменением заголовочного файла, если *main.o* не был перестроен, когда это необходимо.

Параметры компиляции

Иногда для определенных модулей требуется специальный параметр компиляции. В проекте OLED это использовалось для подавления некоторых предупреждений компилятора от стороннего исходного модуля:

```
ugui.o: CFLAGS += -Wno-parentheses
```

Эта опция компиляции добавляется только в компиляцию *ugui.o* для подавления предупреждений о скобках, которые следует добавить для ясности.

Прошивка 128к

Если вы еще этого не сделали, перейдите в подкаталог вашего проекта. После того как вы создадите свой проект с помощью команд

```
$ cd ./myproj
$ make
```

вам необходимо запрограммировать флеш-память STM32. По умолчанию команда `make` настроена на это следующим образом:

```
$ make flash
arm-none-eabi-objcopy -Obinary main.elf main.bin
/usr/local/bin/st-flash write main.bin 0x8000000
...
```

Первым шагом является преобразование elf-файла (*main.elf*) в двоичный образ (*main.bin*). Эту задачу выполняет ARM-версия утилиты *objcopy*. После этого для программирования STM32 используется утилита *st-flash*.

Большинство, если не все, контроллеров STM32F103C8T6 поддерживают прошивку до 128 Кбайт. Вам потребуется установить новую версию утилиты *st-flash*. Чтобы прошить более 64 Кбайт, выполните следующее:

```
$ make bigflash
arm-none-eabi-objcopy -Obinary main.elf main.bin
/usr/local/bin/st-flash --flash=128k write main.bin 0x8000000
...
```

Новая опция `--flash=128k` предоставляется утилитой *st-flash*, чтобы игнорировать идентификатор устройства и прошивать память объемом до 128 Кбайт.

Реальный вопрос заключается в том, действительно ли все чипы STM32F103C8T6 поддерживают 128 Кбайт? Все четыре моих устройства были куплены в разных источниках на eBay. Фактически в интернете нашелся только один случай, когда это не работает. Была ли это ошибка программиста? Или заводские испытания и гарантия прошли только младшие 64 Кбайта? Если кто-то знает ответ, я хотел бы знать.

FreeRTOS

Важная часть вашего проекта – компоненты FreeRTOS. Некоторые из них являются необязательными, другие, наоборот, обязательными. Давайте рассмотрим каждый по очереди.

Модуль rtos/opencm3.c

Этот модуль был написан автором и на самом деле не является частью FreeRTOS. Требуется подключить библиотеку libopencm3 к FreeRTOS. Модуль показан в листинге 21.2.

Листинг 21.2. Исходный модуль *~/stm32f103c8t6/rtos/opencm3.c*

```
0001: /* Warren W. Gay VE3WWG
0002:  *
0003:  * To use libopencm3 with FreeRTOS on Cortex-M3 platform, we must
0004:  * define three interlude routines.
0005: */
0006: #include "FreeRTOS.h"
0007: #include "task.h"
0008: #include <libopencm3/stm32/rcc.h>
0009: #include <libopencm3/stm32/gpio.h>
0010: #include <libopencm3/cm3/nvic.h>
0011:
0012: extern void vPortSVCHandler( void ) __attribute__ (( naked ));
0013: extern void xPortPendSVHandler( void ) __attribute__ (( naked ));
0014: extern void xPortSysTickHandler( void );
0015:
0016: void sv_call_handler(void) {
0017:   vPortSVCHandler();
0018: }
0019:
0020: void pend_sv_handler(void) {
0021:   xPortPendSVHandler();
0022: }
0023:
0024: void sys_tick_handler(void) {
0025:   xPortSysTickHandler();
0026: }
0027:
0028: /* end opncm3.c */
```

Как указано в исходном коде, эти функции libopencm3 вызывают FreeRTOS. Например, функция `sys_tick_handler()` вызывает libopencm3, когда происходит прерывание по такту системного таймера. Но вызов `xPortSysTickHandler()` – это функция FreeRTOS, которая обрабатывает системные таймерные операции.

Модуль rtos/heap_4.c

Это модуль FreeRTOS, используемый в этой книге. Однако на самом деле вариантов может быть несколько. Цитата с веб-сайта www.freertos.org (www.freertos.org/a00111.html):

- heap_1 – Самый простой модуль; не позволяет освободить память;
- heap_2 – Разрешает освобождение памяти, но не объединяет соседние свободные блоки;
- heap_3 – Просто оборачивает стандартные функции malloc() и free() для обеспечения безопасности потоков;
- heap_4 – Объединение соседних свободных блоков во избежание фрагментации; включает возможность размещения абсолютного адреса;
- heap_5 – Согласно heap_4, с возможностью охвата кучей нескольких несмежных областей памяти.

На некоторые из этих исходных модулей может влиять параметр макроопределения `configAPPLICATION_ALLOCATED_HEAP` из FreeRTOSConfig.h. Просто замените *rtos/heap_4.c*, упомянутый в *Makefile*, на модуль по вашему выбору.

Необходимые модули

В дополнение к модулю динамической памяти *rtos/heap_*.c* для FreeRTOS обычно требуются следующие модули:

- *rtos/list.c* (поддержка внутреннего списка),
- *rtos/port.c* (поддержка переносимости),
- *rtos/queue.c* (поддержка очередей и семафоров),
- *rtos/tasks.c* (поддержка задач).

В зависимости от параметров, выбранных в файле *FreeRTOSConfig.h* в каталоге вашего проекта, вы можете исключить один или два модуля для сборки меньшего размера. Например, если вы не используете поддержку очередей, мьютексов или семафоров, вы можете избежать добавления *rtos/queue.c*.

FreeRTOSConfig.h

Это файл конфигурации FreeRTOS вашего проекта. Этот включаемый файл содержит макроопределения для настройки, которые влияют на способ компиляции модулей FreeRTOS в проект. Они также могут влиять на любые вызовы макроопределений, выполняемые вашим приложением. Если какие-либо значения в этом файле изменены, перед перестройкой вам следует его закрыть. Помните, что вы создаете и операционную систему, и приложение вместе.

В листинге 21.3 представлен частичный список того, что содержится в файле *FreeRTOSConfig.h*. В первом разделе настраиваются такие элементы, как тактовая частота процессора (`configCPU_CLOCK_HZ`). Другие определяют такие функции, как приоритетное использование (`configUSE_PREEMPTION`).

Пара важных настроек макроса – `configCPU_CLOCK_HZ` и `configSYSTICK_CLOCK_HZ`. Для использования libopencm3 с тактовой частотой 72 МГц обычно требуется настроить `configSYSTICK_CLOCK_HZ` следующим образом:

```
#define configSYSTICK_CLOCK_HZ ( configCPU_CLOCK_HZ / 8 )
```

Если вы ошиблись, программа, использующая `vTaskDelay()` или другие функции, зависимые от времени, будут неправильными. Вы можете проверить это, запустив демоверсию в `stm32f103c8t6/rtos/blinky2`. При неправильной настройке мигание не будет составлять полсекунды.

Листинг 21.3. Частичный список содержимого файла *FreeRTOSConfig.h*, используемого в проекте RTC

```
/*-----------------------------------------------------------
 * Application-specific definitions.
 *
```

```
 * These definitions should be adjusted for your particular hardware and
 * application requirements.
 *
 * THESE PARAMETERS ARE DESCRIBED WITHIN THE 'CONFIGURATION' SECTION OF THE
 * FreeRTOS API DOCUMENTATION AVAILABLE ON THE FreeRTOS.org WEB SITE.
 *
 * See http://www.freertos.org/a00110.html.
*----------------------------------------------------------*/
#define configUSE_PREEMPTION          1
#define configUSE_IDLE_HOOK           0
#define configUSE_TICK_HOOK           0
#define configCPU_CLOCK_HZ ( ( unsigned long ) 72000000 )
#define configSYSTICK_CLOCK_HZ ( configCPU_CLOCK_HZ / 8 )
#define configTICK_RATE_HZ ( ( TickType_t ) 1000 )
#define configMAX_PRIORITIES ( 5 )
#define configMINIMAL_STACK_SIZE ( ( unsigned short ) 128 )
#define configTOTAL_HEAP_SIZE ( ( size_t ) ( 17 * 1024 ) )
#define configMAX_TASK_NAME_LEN ( 16 )
#define configUSE_TRACE_FACILITY      0
#define configUSE_16_BIT_TICKS        0
#define configIDLE_SHOULD_YIELD       0
#define configUSE_MUTEXES 1
#define configUSE_TASK_NOTIFICATIONS  1
#define configUSE_TIME_SLICING        1
#define configUSE_RECURSIVE_MUTEXES   0
/* Co-routine definitions. */
#define configUSE_CO_ROUTINES         0
#define configMAX_CO_ROUTINE_PRIORITIES ( 2 )
/* Set the following definitions to 1 to include the API function, or zero to exclude
the API function. */
#define INCLUDE_vTaskPrioritySet      1
#define INCLUDE_uxTaskPriorityGet     1
#define INCLUDE_vTaskDelete           0
#define INCLUDE_vTaskCleanUpResources 0
#define INCLUDE_vTaskSuspend          1
#define INCLUDE_vTaskDelayUntil       1
#define INCLUDE_vTaskDelay            1
```

Макроопределение `configTOTAL_HEAP_SIZE` важно настроить, если вы столкнулись со следующей ошибкой:

```
section '.bss' will not fit in region 'ram'
As defined:
#define configTOTAL_HEAP_SIZE ( ( size_t )
( 17 * 1024 ) )
```

FreeRTOS выделит в кучу 17 Кбайт. Но, по мере того как вы разрабатываете свое потрясающее приложение и используете больше статических областей памяти, оставшийся объем SRAM может сжаться до такой степени, что куча

не поместится. Это помешает завершению этапа установления связи. В этом случае вы можете уменьшить размер кучи до завершения сборки приложения. Опытные пользователи затем могут просмотреть созданную карту памяти, чтобы узнать, насколько можно повторно увеличить кучу. Или вы можете просто гадать, увеличивая кучу до тех пор, пока она не перестанет собираться.

Другие конфигурационные макроопределения, такие как `INCLUDE_vTaskDelete`, просто определяют, следует ли скомпилировать этот уровень поддержки FreeRTOS для вашего приложения. Если вы никогда не удаляете задачу, зачем включать код для этой цели?

Все эти параметры конфигурации описаны на веб-сайте FreeRTOS (www.freertos.org) и в их прекрасном бесплатном руководстве.

Пользовательские библиотеки

Обычной практикой является размещение часто используемых процедур, таких как драйверы USB или UART, в библиотеке. Разработав их однажды, вы захотите использовать эти драйверы повторно. Возможно, вы заметили, что это было сделано в некоторых демонстрационных проектах в этой книге. По умолчанию все программы включают заголовки из *~/stm32f103c8t6/rtos/libwwg/include* и ссылку на каталог библиотеки *~/stm32f103c8t6/rtos/libwwg*, связанный с *libwwg.a*.

В каталоге *~/stm32f103c8t6/rtos/libwwg/src* находятся некоторые исходные модули, которые входят в эту статическую библиотеку. Они компилируются, а объектные модули помещаются в *libwwg.a*. Но здесь есть проблема, о которой вам следует знать.

Все эти модули скомпилированы со следующим заголовочным файлом: *~/stm32f103c8t6/rtos/libwwg/src/rtos/FreeRTOSConfig.h*.

Вероятно, он отличается от вашего файла уровня проекта *FreeRTOSConfig.h*. Если конфигурации существенно различаются, лучший подход – скопировать необходимые исходные файлы в каталог вашего проекта и добавить их в ваш *Makefile* (`SRCFILES`). Сделав это, вы гарантируете, что подпрограммы используют тот *FreeRTOSConfig.h*, с которым компилируется остальная часть вашего приложения.

Опять же, это связано с тем, что каждая сборка вашего приложения включает в себя и сборку операционной системы. Большая часть этого процесса управляется макроопределениями, поэтому заголовочные файлы играют важную роль.

Ошибки новичков

Одна ошибка новичков, с которой время от времени сталкиваются все мы, – это внесение изменений в структуру заголовочного файла, влияющих на модули, которые не перекомпилируются. При изменении структуры зависимости членов изменяются. Ранее скомпилированный модуль продолжит использовать старые зависимости.

В идеале *Makefile* должен перечислять все зависимости или генерировать их и затем включать в проект. Однако это не всегда делается или делается недостаточно тщательно, особенно во время лихорадочной разработки проекта.

Если вы изменили структуру (или класс в C++), рекомендуется сначала выполнить команду `make clobber`, чтобы все было перекомпилировано с нуля. В огромных проектах такой подход непрактичен. Но для небольших проектов это гарантирует, что все модули будут скомпилированы из одних и тех же заголовочных файлов.

У вас есть ошибка, которая не имеет смысла? Невозможное происходит? Возможно, вам нужно перестроить свой проект с нуля, чтобы убедиться, что вы не имеете проблем со своим набором инструментов.

Итоги

Эта глава подготовила вас к созданию собственных проектов STM32 с использованием FreeRTOS. Вы рассмотрели модули FreeRTOS, входящие в вашу сборку, а также связующий модуль, связывающий libopencm3 с FreeRTOS. Благодаря возможности настройки *FreeRTOSConfig.h* вы можете управлять тем, как создается проект.

Дополнительные источники

1. «FreeRTOS Memory Management». www.freertos.org/a00111.html (дата доступа 17.09.2024).

Упражнения

1. Что определяет макроопределение make `BINARY`?
2. Каково назначение заголовочного файла *FreeRTOSConfig.h*?
3. Как добавить опцию компилятора -O3 только для компиляции модуля *Speedy.c*?
4. В чем основной недостаток использования *heap_1*?

Глава 22 • Поиск неисправностей

Даже в самых простых и тривиальных проектах программист неизбежно сталкивается с необходимостью устранения неполадок. Необходимость встроенных вычислений в проектах на микроконтроллерах часто возрастает, потому что у вас нет такой роскоши, как дамп основного файла для анализа, как в Linux. У вас также может не быть механизма вывода ошибки в момент ее возникновения.

В этой главе мы сначала рассмотрим средства отладки, доступные на платформе STM32. Затем будут рассмотрены некоторые методы устранения неполадок, а также другие ресурсы по этим темам.

Отладчик GNU (GDB)

Отладчик GNU (GNU debugger, GDB) достаточно мощный, и на его изучение стоит потратить время. Используя USB-программатор ST-Link V2, можно получить доступ к STM32 со своего рабочего стола и выполнить код пошагово, контролируя память и регистры с возможностью установления точек останова. Первым шагом является получение и запуск сервера GDB.

GDB-сервер

Откройте еще один сеанс терминала, в котором вы сможете запустить сервер GDB. Вместе с установкой программного обеспечения *st-flash* будет установлена команда `st-util`. Если вы запустите `st-util` без подключения программатора к USB-порту, ваш сеанс будет выглядеть примерно так:

```
$ st-util
st-util 1.3.1-4-g9d08810
2018-02-08T21:09:22 WARN src/usb.c: Couldn't find any ST-Link/V2 devices
```

Если вы видите такое, убедитесь, что программатор ST-Link V2 подключен к компьютеру.

Если программатор подключен, но не видит подключенного к нему устройства STM32, ваш сеанс будет выглядеть примерно так:

```
$ st-util
st-util 1.3.1-4-g9d08810
2018-02-08T21:10:52 INFO src/usb.c: -- exit_dfu_mode
2018-02-08T21:10:52 INFO src/common.c: Loading device parameters....
2018-02-08T21:10:52 WARN src/common.c: unknown chip id! 0xe0042000
```

Если ваше устройство все-таки подключено, немедленно отключите программатор, чтобы избежать повреждений, и еще раз проверьте соединения.

Легко ошибиться, если вы используете отдельные перемычки DuPont между программатором и STM32. Чтобы этого избежать, я рекомендую вам изготовить для этой цели специальный кабель.

Если все идет хорошо, у вас должен быть сеанс, подобный следующему:

```
$ st-util
st-util 1.3.1-4-g9d08810
2018-02-08T21:07:18 INFO src/usb.c: -- exit_dfu_mode
2018-02-08T21:07:18 INFO src/common.c: Loading device parameters....
2018-02-08T21:07:18 INFO src/common.c: Device connected is: F1 \
Medium-density device, id 0x20036410
2018-02-08T21:07:18 INFO src/common.c: SRAM size: 0x5000 bytes (20 KiB), \
    Flash: 0x20000 bytes (128 KiB) in pages of 1024 bytes
2018-02-08T21:07:18 INFO src/gdbserver/gdb-server.c: Chip ID is 00000410,\
    Core ID is 1ba01477.
2018-02-08T21:07:18 INFO src/gdbserver/gdb-server.c: Listening at *:4242..
```

Отсюда мы видим, что мы подключены к устройству F1 (это и есть STM32F103) и что оно имеет 20 Кбайт статической оперативной памяти SRAM. В зависимости от вашего устройства и установленной версии команды `st-util` она может отображать только 64 Кбайт флеш-памяти или, как здесь, вместо этого может показывать 128 Кбайт. И наконец, обратите внимание, что утилита «прослушивает» (`Listening`) порт *:4242. Звездочка указывает, что прослушиваются все интерфейсы порта 4242.

Если вы хотите завершить работу этого сервера, нажмите <**Ctrl+C**>.

Удаленный GDB

При работающем сервере `st-util` можно использовать GDB для подключения к этому серверу и запуска сеанса отладки. Давайте используем проект RTC в качестве рабочего примера:

```
$ cd ~/stm32f103c8t6/rtos/rtc
```

Очень важно, чтобы использовалась версия GDB, соответствующая вашим инструментам компилятора. Большинство из вас, скорее всего, будут использовать *arm-none-eabi-gdb*, хотя это может отличаться в зависимости от вашей установки. Поскольку утомительно вводить каждый раз такое длинное словосочетание, вы можете использовать для этой цели псевдоним оболочки:

```
$ alias g='arm-none-eabi-gdb'
```

Это позволяет вам просто ввести «g», чтобы вызвать этот вариант. В этой главе я приведу команду полностью, но при желании используйте псевдоним, чтобы не вводить целиком. Просто запустите команду, чтобы начать:

```
$ arm-none-eabi-gdb
GNU gdb (GNU Tools for ARM Embedded Processors 6-2017-q2-update) 7.12.1.20170417-git
```

```
Copyright (C) 2017 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later
...
For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb)
```

На данный момент мы еще не подключены к серверу `st-util`. Следующий шаг не является обязательным, но он необходим, если вы хотите использовать в сеансе символические обозначения и отладку уровня исходного кода:

```
(gdb) file main.elf
Reading symbols from main.elf...done.
```

Обратите внимание: это подтверждает, что необходимые обозначения (`symbols`) были извлечены из файла *main.elf* в текущем каталоге. Затем подключитесь к серверу `st-util`:

```
(gdb) target extended-remote :4242
Remote debugging using :4242
0x08003060 in ?? ()
(gdb)
```

IP-адрес перед портом :4242 в качестве ярлыка не вводится. Это означает, что мы подключаемся через локальный адрес, указывающий на отправителя 127.0.0.1:4242. Соединение подтверждается сообщением «Remote debugging using :4242» (Удаленная отладка с использованием 4242).

Теперь загрузим программу во флеш-память:

```
(gdb) load main.elf
Loading section .text, size 0x30e8 lma 0x8000000
Loading section .data, size 0x858 lma 0x80030e8
Start address 0x8002550, load size 14656
Transfer rate: 8 KB/sec, 7328 bytes/write.
```

Сервер `st-util` автоматически запишет образ файла во флеш-память (обратите внимание, что он загружает его, начиная с адреса 0x8000000). Существуют также области данных, запрограммированные во флеш-памяти, начиная с адреса `0x80030e8`. Код запуска найдет это и скопирует эти данные в нужное место в SRAM перед вызовом функции `main()`.

Далее мы устанавливаем точку останова для функции `main()` в строке 252:

```
(gdb) b main
Breakpoint 1 at 0x800046c: file main.c, line 252.
(gdb)
```

Если мы не установим точку останова, программное обеспечение запустится и заработает. Установка точки останова позволяет выполнить всю инициализацию, но остановить выполнение на первом операторе в программе `main()`. Обратите внимание, что команда точки останова завершится неудачей, если вы в ней пропустите указание `file`, поскольку она не будет знать обозначение кода, к которому относится вся команда.

Теперь запустим программу:

```
(gdb) c
Continuing.
Breakpoint 1, main () at main.c:252
252 main(void) {
(gdb)
```

Программа запустилась, а затем остановилась на установленной нами точке останова. Теперь мы можем пройти через исходные операторы с помощью команды GDB `n` (от *next* – следующая):

```
(gdb) n
254 rcc_clock_setup_in_hse_8mhz_out_72mhz(); // Use this for "blue pill"
(gdb) n
256 rcc_periph_clock_enable(RCC_GPIOC);
(gdb) n
257 gpio_set_mode(GPIOC,GPIO_MODE_OUTPUT_50_MHZ,
GPIO_CNF_OUTPUT_PUSHPULL,GPIO13);
(gdb)
```

Это позволило нам пройти пошагово эти три оператора. Если вы хотите выполнить трассировку внутри какой-либо функции в составе встреченного оператора, вместо этого введите команду GDB `s` (от *step* – шаг).

Чтобы просто запустить программу с этого момента, используйте команду GDB `c` (от *continue* – продолжить):

```
(gdb) c
Continuing.
```

Чтобы прервать программу и восстановить управление, нажмите <**Ctrl+C**>:

```
^C
Program received signal SIGTRAP, Trace/breakpoint trap.
0x08000842 in xPortPendSVHandler () at rtos/port.c:403
403 __asm volatile
(gdb)
```

Точка прерывания программы может различаться. Чтобы просмотреть стек вызовов, используйте команду GDB `bt` (от *backtrace* – обратная трассировка):

```
(gdb) bt
#0 0x08000842 in xPortPendSVHandler () at rtos/port.c:403
#1 <signal handler called>
#2 0x08000798 in prvPortStartFirstTask () at rtos/port.c:270
#3 0x080008d6 in xPortStartScheduler () at rtos/port.c:350
Backtrace stopped: Cannot access memory at address 0x20005004
(gdb)
```

Это говорит нам о том, что мы прервали выполнение внутри кода планировщика FreeRTOS.

Чтобы выйти из GDB, введите команду `quit`.

Текстовый пользовательский интерфейс GDB

Чтобы сделать сеансы отладки более удобными, GDB поддерживает несколько различных форм выкладки. На рис. 22.1 показана выкладка, полученная при вводе команды `layout split` (*разделенный макет*). Это дает вам и исходный код, и просмотр инструкций на ассемблерном уровне.

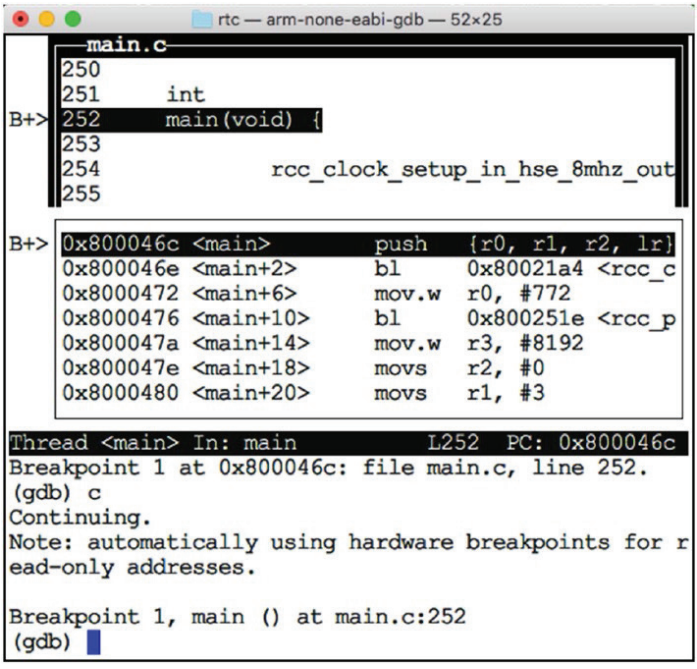


Рис. 22.1. Отображение разделенного макета GDB

Возможны и другие формы представления. Например, чтобы отслеживать, что происходит в программах на языке ассемблера, вам нужно использовать представление по команде `layout regs` (*макет регистров*), показанное на рис. 22.2. В этом представлении показаны инструкции языка ассемблера, а также содержимое регистров. Измененные регистры подсвечиваются. К со-

жалению, небольшой размер окна терминала, который я использовал на рис. 22.2, не совсем удобен для просмотра. Если вы используете увеличенное окно терминала, вы увидите все регистры.

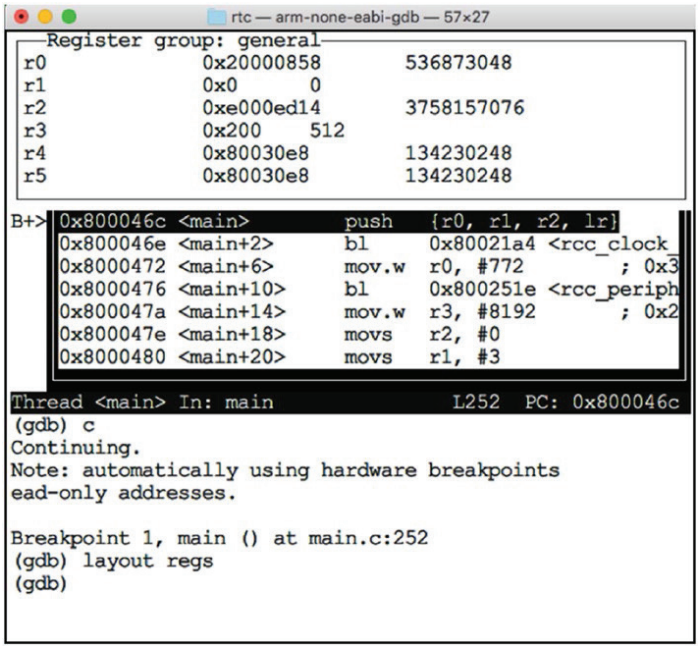


Рис. 22.2. Просмотр GDB в варианте layout regs (полный набор регистров отображается в окне терминала большего размера)

GDB – это нечто большее, чем можно здесь описать. Затраты на чтение руководства по GDB или онлайн-публикаций по этой теме могут сэкономить вам время в долгосрочной перспективе.

Проблемы с выводами GPIO периферии

Вы пишете новую программу с участием периферийного устройства UART, которое требует использования вывода GPIO. Вы настраиваете его, но вывод не работает. Надеюсь, эта книга уже подготовила вас к ответу. Что не так с этим фрагментом кода?

```
rcc_periph_clock_enable(RCC_GPIOA);
rcc_periph_clock_enable(RCC_USART1);
// UART TX on PA9 (GPIO_USART1_TX)
gpio_set_mode(GPIOA,
    GPIO_MODE_OUTPUT_50_MHZ,
    GPIO_CNF_OUTPUT_PUSHPULL,
    GPIO_USART1_TX);
```

Я повторяюсь, потому что здесь очень легко совершить ошибку. Да, код настроил вывод GPIO PA9 для выхода. Но это не то же самое, что периферийный выход. Для этого вы должны настроить его как выход альтернативной функции (обратите внимание на аргумент 3):

```
// UART TX on PA9 (GPIO_USART1_TX)
gpio_set_mode(GPIOA,
    GPIO_MODE_OUTPUT_50_MHZ,
    GPIO_CNF_OUTPUT_ALTFN_PUSHPULL, // NOTE!!
    GPIO_USART1_TX);
```

Это приводит к подключению периферийного устройства к выводу GPIO и отключению обычной функции GPIO. Если вы ошибетесь, вы можете сойти с ума, пытаясь найти ошибку. Код будет выглядеть правильно, но подглядывающие из-за спины будут смеяться. Запишите это в свое сознание заранее.

Альтернативная функция не работает

Ваш код выполняет некоторую инициализацию периферийного устройства для использования вывода GPIO, и вы даже используете правильное макроопределение `GPIO_CNF_OUTPUT_ALTFN_OPENDRAIN` для шины CAN, но все равно никакой радости. Где ошибка?

```
rcc_peripheral_enable_clock(&RCC_APB1ENR,RCC_APB1ENR_CAN1EN);
rcc_periph_clock_enable(RCC_GPIOB);
gpio_set_mode(GPIOB,
    GPIO_MODE_OUTPUT_50_MHZ,
    GPIO_CNF_OUTPUT_ALTFN_OPENDRAIN,
    GPIO_CAN_PB_TX);
gpio_set_mode(GPIOB,
    GPIO_MODE_INPUT,
    GPIO_CNF_INPUT_FLOAT,
    GPIO_CAN_PB_RX);
gpio_primary_remap(
    AFIO_MAPR_SWJ_CFG_JTAG_OFF_SW_OFF,
    AFIO_MAPR_CAN1_REMAP_PORTB); // CAN_RX=PB8, CAN_TX=PB9
```

Похлопайте себя по плечу, если вспомнили, что необходимо включить тактовый сигнал AFIO:

```
rcc_periph_clock_enable(RCC_AFIO);
```

Если вы пропустите эту установку, переназначение GPIO не будет работать. Ему нужны тактовые сигналы. Подобное упущение может оказаться коварным.

Сбой периферии

Это должно быть очевидно, но периферийным устройствам необходимо включить собственные тактовые сигналы. Для CAN-шины потребовался этот вызов (см. код выше):

```
rcc_peripheral_enable_clock(&RCC_APB1ENR,RCC_APB1ENR_CAN1EN);
```

Для других периферийных устройств, таких как UART, вызов может быть проще:

```
rcc_periph_clock_enable(RCC_USART1);
```

Очевидно, что если такты периферийного устройства отключены, как это происходит после перезагрузки, то оно будет вести себя как мертвый кусок кремния.

Сбой прерывания во FreeRTOS

Во FreeRTOS есть правило о том, что можно и что нельзя вызывать из обработчика прерывания ISR. Хотя задача может вызвать `xQueueSend()` в любое время, ISR должен вместо этого использовать функцию `xQueueSendFromISR()` (обратите внимание на `FromISR` в конце имени функции). Причины могут различаться в зависимости от платформы, но ISR обычно работают в очень строгих условиях.

Прерывания по своей природе асинхронны, поэтому любой вызов функции является подозрительным, если точно не известно, что он является реентерабельным[96]. FreeRTOS принимает специальные меры, чтобы гарантировать безопасность вызываемой функции. Нарушьте это правило, и вы можете столкнуться со спорадическими сбоями.

Переполнение стека

К сожалению, размеры стека необходимо определять заранее при создании задачи, например:

```
xTaskCreate(monitor_task,"monitor",350,NULL,1,NULL);
```

Аргумент 3 в вызове выделяет 350 слов памяти для стека этой задачи (каждое слово имеет длину 4 байта). Функция `xTaskCreate()` выделяет стек из кучи. Если размер стека недостаточен, произойдет сбой памяти с непредсказуемыми результатами.

[96] Реентерабельный (от англ. *reentrant* – повторно входимый) – компьютерная программа или процедура, разработанная таким образом, что одна и та же копия инструкций программы в памяти может быть совместно использована несколькими пользователями или процессами.

Улучшением было бы проверять состояние переполнения и что-то с ним сделать. FreeRTOS предлагает три способа решения этой проблемы. Они определяются макроопределением `configCHECK_FOR_STACK_OVERFLOW`, как задано в файле *FreeRTOSConfig.h*.

1. `configCHECK_FOR_STACK_OVERFLOW` определен как 0. FreeRTOS не будет проверять переполнение; это наиболее эффективно для работы.
2. `configCHECK_FOR_STACK_OVERFLOW` определен как 1, чтобы FreeRTOS выполнила быструю проверку стека. Менее эффективен, чем подход 1, но более эффективен, чем 3.
3. `configCHECK_FOR_STACK_OVERFLOW` определен как 2, чтобы FreeRTOS выполняла более тщательную проверку стека. Это наименее эффективная операция.

Если макроопределение задано с ненулевым значением, вы должны указать функцию, которая будет вызываться при переполнении стека:

```
void
vApplicationStackOverflowHook(
  xTaskHandle *pxTask,
  signed portCHAR *pcTaskName
) {
    // do something, perhaps
    // flash an LED
}
```

Когда обнаруживается переполнение стека, вызывается функция перехвата. Некоторое повреждение памяти, вероятно, уже произошло к моменту вызова этой функции, поэтому лучше всего сигнализировать об этом нарушении, используя самые простые методы, такие как включение светодиода или его многократное мигание.

Оценка размера стека

Для функций, вызывающих библиотечные подпрограммы, особенно сторонние, оценка требуемого размера стека может быть затруднена. Так как же оценить, сколько места необходимо? FreeRTOS предоставляет функцию, которая помогает в этом вопросе:

```
#include "FreeRTOS.h"
#include "task.h"
// Returns # of words:
UBaseType_t uxTaskGetStackHighWaterMark(TaskHandle_t task);
```

В документации FreeRTOS не указано, что эта функция возвращает, но возвращаемое значение измеряется в словах (words). Функция, получив дескриптор задачи, вернет количество неиспользованных слов стека. Если задача была создана со стеком из 350 слов и на данный момент использовалось максимум 100 слов, то возвращаемое значение будет 250. Другими словами,

чем ближе это возвращаемое значение к нулю, тем больше вероятность того, что задача в конце концов переполнит стек.

В документации FreeRTOS указано, что вызов этой функции может быть дорогостоящим в плане используемых ресурсов, и поэтому ее следует использовать только при отладке. Но даже в этом случае будьте осторожны, поскольку востребованность стека может меняться в зависимости от шаблонов использования. И все-таки при попытке получить оценку это лучше, чем ничего.

Когда отладчик не помогает

Бывают случаи, когда отладчик непрактичен. Например, при отладке драйверов устройств могут возникать прерывания и тайм-ауты, когда пошаговое выполнение кода не поможет. В этой ситуации вы можете собрать подсказки, например где происходит сбой или какое событие было успешно обработано последним. Во многих из этих сложных ситуаций доступ к светодиоду или GPIO может дать более ценную информацию.

В рамках обработчика прерывания ISR вы часто не можете позволить себе роскошь отправить сообщение на ЖК-дисплей или последовательное сообщение на терминал. Вместо этого вам нужно найти простые способы передачи событий, например включение светодиодов. Если у вас есть цифровой осциллограф или логический анализатор, выдача сигналов на несколько выводов GPIO может быть очень информативной, особенно при определении того, сколько времени затрачивается на обработку прерывания.

В крайних случаях вам может потребоваться выделить буфер трассировки, который может заполнить ISR-программа. Затем, используя GDB, вы можете прервать выполнение и проверить этот буфер.

Push/Pull (двухтактный выход) или открытый сток

Некоторые проблемы связаны с использованием выходного вывода GPIO. Например, во многих сообщениях на форумах утверждается, что для SPI не работает аппаратный выбор ведомого устройства. Однако все прекрасно работает, если вы используете конфигурацию с открытым стоком и подтягивающий резистор. Это имеет смысл, если учесть, что STM32 поддерживает SPI в режиме нескольких ведущих.

Любой элемент шины с несколькими ведущими должен использовать подтягивающий резистор, потому что, если один контроллер удерживает сигнал шины на высоком уровне, другому придется бороться, чтобы понизить тот же сигнал. Подтягивающий резистор позволяет сигналу повышаться до высокого уровня автоматически, когда ни один контроллер не активен на сигнале. Это также позволяет любому участнику шины без борьбы снизить уровень сигнала.

Эта ситуация подчеркивает еще одну проблему. При чтении технических характеристик STMicroelectronics полезно обращать внимание на мелкий шрифт и сноски. Там таится много непростых вещей.

Дефекты периферии

В редких случаях вы можете столкнуться с неправильным поведением периферийных устройств. Периферийные устройства STM32 представляют собой сложные конечные автоматы, и в определенных ситуациях или в определенных конфигурациях иногда всплывают недоработки. Найдите и загрузите PDF-файл «STM32F1 Errata Sheet», чтобы узнать, что может пойти не так. Обычно для таких ситуаций предусмотрен обходной путь.

Читая список ошибок, вы можете заметить, что многие проблемы относятся к отладке. Это предупреждение о том, что не все, что вы можете увидеть в удаленном сеансе GDB, соответствует реальности. Удаленная отладка очень полезна, но в особых ситуациях может столкнуться с трудностями.

Ресурсы

Большая часть вашего времени, вероятно, будет потрачена на то, чтобы периферийные устройства STM32 работали так, как вы хотите. Чем более продвинутое ваше приложение, тем больше вероятность того, что вы потратите время на решение проблем с периферией.

Самый лучший источник информации о периферийных устройствах содержится в справочном руководстве STMicroelectronics RM0008 (ссылка в разделе «Дополнительные источники» к главе 7). Как минимум вам потребуется загрузить этот PDF-файл и использовать его для решения сложных проблем. Но это не единственный ресурс, который вам нужен.

Найдите и загрузите PDF-документ DS5319 «STM32F103x8 STM32F103xB» (см. «Дополнительные источники», [2] к главе 14). Очень важная таблица, содержащаяся в этом документе, – это табл. 5 «Medium-density STM32F103xx pin definitions» (Определения выводов STM32F103xx средней плотности). Возможно, вас не беспокоят определения контактов, но вы найдете это золотой жилой для обобщения того, чем может стать каждый контакт GPIO при правильной конфигурации. Чтобы использовать табл. 5, просмотрите столбец корпуса LQFP48 для STM32F103C8T6. Пройдя вниз по столбцу, вы найдете номера контактов. Например, контакт 11 после сброса является GPIO PA1 и может быть настроен на одно из следующих значений:

- `USART2_RTS`,
- `ADC12_IN1`,
- `TIM2_CH2`.

И он не выдерживает 5-вольтового напряжения.

Вся эта важная информация, кажется, должна быть в справочном руководстве, но STMicroelectronics поместила все такие характеристики в отдельном файле. Для удобства читателя мы поместили выборку для STM32F103C8T6 в корпусе LQFP48 из табл. 5 в приложении Б.

Раздел «Electrical Characteristics» (Электрические характеристики) будет интересен тем, кто ищет оценки потребляемой мощности. Например, в табл. 17, посвященной типичному потреблению тока, указано, что микроконтроллер, работающий на частоте 72 МГц, будет потреблять около 27 мА

при отключенной периферии. В этом документе доступно несколько других таблиц и диаграмм такого рода.

Библиотека libopencm3

Несмотря на то что у libopencm3 есть вики по API, иногда требуются сведения, которые не предоставлены или неочевидны. Я подозреваю, что вы испытаете то же самое при разработке новых приложений. Возникают такие вопросы:

- когда я могу комбинировать разные значения в аргументе вызова функции?
- когда их необходимо предоставлять отдельными вызовами?

Это две стороны одного и того же вопроса. Во-первых, вот прямая ссылка на вики-страницы по API: http://libopencm3.org/docs/latest/html.

В левой части документация разделена по членам семейства STM32. Для STM32F103 вам нужно детализировать «STM32F1». В некоторых случаях детали оговариваются. Например, функция

```
void gpio_set(uint32_t gpioport,uint16_t gpios);
```

сопровождается следующим комментарием:

> *Set a Group of Pins Atomic. Set one or more pins of the given GPIO port to 1 in an atomic operation.*
>
> (Атомарная установка группы выводов. Установка одного или нескольких выводов данного порта GPIO в единицу в атомарной операции).

Это ясно говорит нам о том, что вы можете объединить несколько выводов GPIO, используя оператор «ИЛИ» языка C (|) в аргументе 2, например:

```
gpio_set(GPIOB,GPIO5|GPIO5);
```

Вероятно, само собой разумеется, что вы не можете комбинировать значения для переменной `gpioport`.

Существуют и другие типы вызовов функций, подобные этому:

```
bool usart_get_flag(uint32_t usart,uint32_t flag);
```

Имя `flag` в единственном числе и описание «USART Read a Status Flag» (флаг состояния чтения USART) обозначают единственное число. Что произойдет, если объединить флаги? Хотя это может быть небезопасно и ненормально, единственный способ ответить на этот вопрос – посмотреть исходный код. Внизу описания вы найдете ссылку «Definition at line 107 of file usart_common_f124.c» (Определение в строке 107 файла usart_common_f124.c). Если вы нажмете на нее, откроется список исходных файлов модуля, содержащего функцию. Оттуда вы можете выполнить поиск или прокрутить вниз до определения функции и увидеть, что оно выглядит следующим образом:

```
bool usart_get_flag(uint32_t usart, uint32_t flag)
{
    return ((USART_SR(usart) & flag) != 0);
}
```

Это говорит о нескольких вещах.

1. Если вы укажете несколько флагов, вы получите только логический результат (любой из флагов может привести к возврату `true`). Это вряд ли то, что вы хотите.
2. Здесь рассказывается, как самостоятельно получить флаги состояния с помощью макроопределения `USART_SR(usart)`. Однако вам может потребоваться включить еще один файл заголовка, чтобы сделать макроопределение доступным.

Цель этого раздела – напомнить, что необходимо внимательно читать описания API libopencm3. Если тип аргумента является перечислимым типом, это почти гарантирует, что вам не следует объединять аргументы. Если тип аргумента представляет собой целое число со знаком или без знака, вы можете объединить их. Прежде чем предполагать, проверьте документацию. Если вы не найдете нужных ответов, используйте первоисточник.

Приоритеты задач FreeRTOS

FreeRTOS обеспечивает многозадачность с несколькими уровнями приоритета. Имейте в виду, что механизм приоритетов может не соответствовать вашим ожиданиям. Приоритеты задач расположены таким образом, что нулевой уровень является самым низким приоритетом. Уровень `configMAX_PRIORITIES-1` – это наивысший приоритет задачи. Макроопределение `configMAX_PRIORITIES` определено в файле *FreeRTOSConfig.h*.

Задача ожидания имеет нулевой приоритет. Она запускается, когда никакая другая задача не готова к запуску. Существует несколько настраиваемых параметров FreeRTOS для задания того, что происходит во время простоя, о чем вы можете прочитать в руководстве. По умолчанию процессор просто выполняет пустой цикл до тех пор, пока задача с более высоким приоритетом не перейдет в состояние «Ready» (Готово).

Планировщик FreeRTOS предназначен для предоставления ресурсов процессора задачам, которые находятся в состоянии «Ready» или «Running» (Выполняется). Если у вас есть одна или несколько задач в этих состояниях с более высоким приоритетом, задачи с более низким приоритетом не будут выполняться. Это отличается, например, от Linux, где процессор используется совместно менее приоритетными задачами. В FreeRTOS процессы с более низким приоритетом требуют, чтобы все задачи с более высоким приоритетом находились в одном из следующих состояний:

- «Suspended» (Приостановлено) вызовами типа `vTaskSuspend();`
- «Blocked» (Заблокировано) блокирующим вызовом типа `xTaskNotifyWait()`.

Это имеет последствия для задач, ожидающих периферийного события. Если драйвер внутри задачи выполняет цикл занятости, то процессор не отключается до тех пор, пока не произойдет вытесняющее прерывание. Даже когда цикл занятости (busy loop) вызывает функцию `TaskYIELD()`, процессор передается другой готовой задаче с тем же приоритетом в циклической последовательности. Опять же, единственный способ получить доступ к процессору задаче с более низким приоритетом – это приостановить или заблокировать все задачи с более высоким приоритетом.

Это требует корректировки вашего образа мышления из Linux/Unix, где процессор используется совместно каждым процессом, готовым к запуску. Если вы хотите такого, то во FreeRTOS вы должны запускать все свои задачи с одним и тем же уровнем приоритета. Все задачи одного уровня планируются по циклическому принципу.

Эта проблема проявляется в том, что задачи с более низким приоритетом кажутся периодически зависающими. Такая ситуация – явный признак того, что ваша схема приоритетов нуждается в корректировке или что задачи не блокируются или не приостанавливаются должным образом.

Планирование в libopencm3

Библиотека libopencm3 была разработана независимо от FreeRTOS. Следовательно, когда драйвер периферийного устройства ожидает периферийного события, он часто включает цикл занятости. Давайте посмотрим на один пример того, что я имею в виду:

```
void usart_wait_send_ready(uint32_t usart)
{
    /* Wait until the data has been transferred into the shift register. */
    while ((USART_SR(usart) & USART_SR_TXE) == 0);
}
```

Функция `usart_wait_send_ready()` вызывается перед отправкой байта данных в USART. Но обратите внимание на цикл `while` – он просто крутит процессор, ожидая, пока флаг `USART_SR_TXE` станет истинным. Результатом этого является то, что вызывающая задача израсходует весь свой интервал времени, прежде чем вытеснение передаст процессорное время другой задаче. Этот принцип выполняет свою работу, но неоптимально и может привести к слишком большой задержке.

Чтобы лучше использовать процессор, задаче следовало бы немедленно отдавать свой временной интервал другой задаче для выполнения другой полезной работы. К сожалению, в libopencm3 для этого нет средств перехвата. Это оставляет вам следующие варианты выбора.

1. Оставить все как есть (возможно, для вашего приложения это не критично).
2. Скопировать функцию в свой код и добавить вызов `TaskYIELD()`.
3. Обновить свою копию библиотеки libopencm3.
4. Реализовать функцию перехвата и отправить ее в сообщество libopencm3.

Самым простым решением является второй подход. Скопируйте код функции в свое приложение и слегка измените его, чтобы вызвать `TaskYIELD()`:

```
void usart_wait_send_ready(uint32_t usart)
{
    /* Wait until the data has been transferred into the shift register. */
    while ((USART_SR(usart) & USART_SR_TXE) == 0)
        taskYIELD(); // Make FreeRTOS friendly
}
```

Итоги

В этой книге мы сосредоточились на STM32F103C8T6, члене семейства STM32. Это позволило нам сконцентрироваться на фиксированном количестве функций. Конечно, есть и другие устройства семейства с дополнительными периферийными устройствами, такими как DAC (цифроаналоговый преобразователь, ЦАП), и это только один пример. Если у вас теперь есть аппетит к более сложным задачам, рассмотрите устройства семейства STM32F407, например плату Discovery. Если у вас ограниченный бюджет, в продаже есть и другие варианты, например Core407V. Семейство STM32F4 предоставляет гораздо больше возможностей SRAM, флеш-памяти и периферийных устройств, чем мы рассмотрели в этой книге. Оно также включает аппаратные средства с плавающей запятой, что может быть критично для производительности некоторых приложений.

Я надеюсь, что эта книга дала вам хорошую информацию о платформе STM32 и подарила вам интересные задачи для решения. В честь этого в этой главе не будет упражнений! Спасибо, что позволили мне быть вашим гидом.

Приложение А • Ответы на упражнения

Глава 4

1. Какой порт GPIO использует встроенный светодиод на плате Blue Pill? Укажите имя libopencm3 для этого порта.
   Ответ: PORTC.
2. Какой вывод GPIO использует встроенный светодиод на плате Blue Pill? Укажите имя libopencm3 для этого вывода.
   Ответ: GPIO13.
3. Какой уровень на выводе необходим для включения встроенного светодиода на плате Blue Pill?
   Ответ: низкий логический уровень (нулевой потенциал).
4. Какие два фактора влияют на выбранное количество циклов при программируемой задержке в однозадачных средах?
   Ответ:
   а) тактовая частота процессора,
   б) время выполнения инструкции.
5. Почему программируемые задержки не используются в многозадачной среде?
   Ответ: потому что время выполнения других задач в вашей системе будет влиять на прошедшее время запрограммированной задержки.
6. Какие три фактора влияют на время цикла задержки?
   Ответ:
   а) выбранная платформа,
   б) тактовая частота процессора,
   в) контекст выполнения (выполняющийся код во флеш-памяти или SRAM).
7. Перечислите три режима входного порта GPIO.
   Ответ:
   а) аналоговый,
   б) цифровой, плавающий,
   в) цифровой, поднимите и опустите.
8. Участвуют ли подтягивающие и заземляющие резисторы в работе аналогового входа?
   Ответ: нет.
9. Когда для входных портов включается триггер Шмитта?
   Ответ: GPIO или цифровой вход периферии.
10. Участвуют ли подтягивающие и заземляющие резисторы в выходных портах GPIO?
    Ответ: нет. Они применяются только к входам.

11. Какое название режима следует использовать при настройке выхода USART TX (передача) для работы вывода в двухтактном режиме?
Ответ: `GPIO_CNF_OUTPUT_ALTFN_PUSHPULL`.

24. При настройке вывода для использования светодиодов какой режим GPIO предпочтителен для снижения электромагнитных помех?
Ответ: `GPIO_MODE_OUTPUT_2_MHZ` (варианты с более высокой частотой, такие как `GPIO_MODE_OUTPUT_10_MHZ`, используют больший ток и генерируют дополнительные электромагнитные помехи).

Глава 5

1. Сколько задач выполняется в программе blinky2?
Ответ: 1.
2. Сколько потоков управления работает в blinky2?
Ответ: два потока: основной поток и задача `task1`.
3. Что произойдет с частотой мигания blinky2, если значение `configCPU_CLOCK_HZ` установить равным 36 000 000?
Ответ: частота мигания удвоится, поскольку планировщик FreeRTOS ожидает, что процессор будет работать вдвое быстрее.
4. Откуда берется стек задачи `task1`?
Ответ: стек задачи `task1` выделяется из кучи.
5. Когда именно начинается задача `task1()`?
Ответ: при вызове функции `vTaskStartScheduler()`.
6. Зачем нужна очередь сообщений?
Ответ: для безопасного взаимодействия между различными потоками управления.

Перейдите к проекту в *stm32/rtos/blinky*, соберите его и запустите. Затем ответьте на следующее (п. 7–11).

7. Несмотря на то что здесь используется цикл задержки выполнения, почему программа работает с коэффициентом заполнения почти 50 %?
Ответ: поскольку выполняется только одна задача, время остается достаточно постоянным.
8. Насколько сложно оценить, как долго горит светодиод на PC13? Почему?
Ответ: сложно из-за синхронизации инструкций, предварительной выборки флеш-памяти и т. д.
9. С помощью осциллографа измерьте время включения и выключения PC13 (или посчитайте количество миганий в секунду и вычислите обратное значение). Сколько миллисекунд горит светодиод?
Ответ: 84 мс.
10. Если бы в этот проект была добавлена еще одна задача, которая потребляла большую часть ресурсов ЦП, как бы это повлияло на частоту мигания?
Ответ: скорость мигания значительно замедлится.
11. Добавьте в файл `main.c` задачу `task2`, которая ничего не делает, кроме выполнения в цикле команды `__asm__("nop")`. Создайте эту задачу

в `main()` перед запуском планировщика. Как это повлияло на частоту мигания? Почему?
Ответ: скорость значительно замедлилась, потому что вторая задача отнимает больше процессорного время по сравнению с одной задачей.

Глава 6

1. Какой уровень TTL сигнала покоя USART?
   Ответ: высокий (около 5 В).
2. Данные USART имеют последовательность с прямым или обратным порядком битов?
   Ответ: с прямым порядком (сначала младший бит).
3. Какие тактовые частоты должны быть включены для использования UART?
   Ответ: `RCC_GPIOx` и `RCC_USARTn`.
4. Что означает аббревиатура 8n1?
   Ответ: 8 бит данных, без проверки четности и 1 стоповый бит.
5. Что произойдет, если вы предоставите UART-данные для отправки, когда приемное устройство еще не освободилось?
   Ответ: данные потеряются.
6. Можно ли создавать задачи до, после или до и после вызова `vTaskStartScheduler()`?
   Ответ: до и после.
7. Каков минимальный размер буфера, определяемый функцией `xQueueReceive()`?
   Ответ: размер приемного буфера должен соответствовать размеру элемента, указанному при создании очереди, или превышать его.
8. Как указать, что функция `xQueueSend()` должна немедленно вернуть управление, если очередь заполнена?
   Ответ: укажите аргумент xTicksToWait со значением 0.
9. Как указать, что `xQueueReceive()` должен блокироваться, если очередь пуста?
   Ответ: укажите аргумент `xTicksToWait` с помощью макроопределения `portMAX_DELAY`.
10. Что произойдет с задачей, если `xQueueReceive()` обнаружит, что очередь пуста, и придется ждать ее наполнения?
    Ответ: задача уступит место другой задаче.

Глава 7

1. Какая подготовка GPIO необходима перед включением периферийного USB-устройства?
   Ответ: тактовый сигнал GPIOA должен быть включен, в противном случае периферийное устройство USB автоматически возьмет на себя управление выводами PA11 и PA12.
2. Какие альтернативные конфигурации GPIO доступны для USB?
   Ответ: альтернативные конфигурации для USB отсутствуют. Используются только PA11 и PA12.

3. Какую процедуру libopencm3 необходимо регулярно вызывать для обработки событий USB?
   Ответ: необходимо часто вызывать подпрограмму `usbd_poll()` для обработки событий, требующих действий.

Глава 8

1. Сколько линий данных используется SPI в двунаправленных каналах? Каковы названия сигналов в этих линиях?
   Ответ: в двунаправленных каналах SPI используются две линии передачи данных: MOSI и MISO.
2. Откуда берутся тактовые сигналы?
   Ответ: тактовый сигнал (SCK) всегда предоставляется ведущим на шине SPI.
3. Какие уровни напряжения используются для сигналов SPI?
   Ответ: уровни напряжения SPI обычно составляют 5 или 3.3 В в зависимости от требований конструкции системы. Устройство STM32 использует напряжение 3.3 В.
4. Почему для вывода STM32 $\overline{\text{NSS}}$ необходимо использовать подтягивающий резистор?
   Ответ: необходимо использовать подтягивающий резистор для вывода $\overline{\text{NSS}}$, поскольку микроконтроллер STM32 настраивает выход как выход с открытым стоком, независимо от того, как он был настроен изначально. Без подтягивающего резистора линия выбора микросхемы никогда не поднимется на высокий уровень.
5. Почему в некоторых транзакциях SPI необходимо отправлять фиктивное значение?
   Ответ: фиктивное значение оправляется, чтобы заставить ведущее периферийное устройство генерировать тактовые импульсы, необходимые ведомому устройству для отправки своих данных. Ведомое устройство всегда зависит от ведущего устройства SPI в обеспечении тактовой частоты.

Глава 9

1. Почему в структуре, тип которой определен как `s_overlay`, члены определяются как указатели на имена, а не как длинные целые числа?
   Ответ: для расчета размера в байтах нужны указатели имен. Если бы типом был `long int`, то вычисляемый размер был бы в словах, а не в байтах.
2. Почему в сценарий компоновщика была добавлена область памяти `xflash`?
   Ответ: область xflash была создана для хранения всего флеш-кода на W25QXX, который не появится во флеш-памяти контроллера. Кроме того, этот код загружается в xflash по начальному адресу, равному нулю, тогда как флеш-память контроллера вместо этого начинается с 0x08000000.
3. Какова цель оверлея-заглушки?
   Ответ: функция-заглушка вызывает диспетчер оверлеев, чтобы убедиться, что необходимый код загружен в область оверлея в SRAM. Как только указатель функции известен, он должен передать вызывающие аргументы и возвращаемые значения, если таковые имеются.

4. Какова цель `noinline` в декларации GNU `__attribute__((noinline,section(".ov_fee")))`? Зачем это нужно?
   Ответ: атрибут `noinline` не позволяет компилятору рассматривать функцию как «встроенный» код. Это особенно важно для небольших функций, которые может оптимизировать компилятор.
5. Где находится объявление `__attribute((section("..."))`?
   Ответ: объявление `__attribute__((section("…"))` может присутствовать только в прототипе функции.

Глава 10

1. Каковы три возможных источника прерываний от RTC?
   Ответ: тремя источниками прерываний являются RTC (тик), сигнал тревоги и переполнение.
2. Какова цель вызовов `TaskENTER_CRITICAL_FROM_ISR` и `TaskEXIT_CRITICAL_FROM_ISR`?
   Ответ: вызовы `TaskENTER_CRITICAL_FROM_ISR()` и `TaskEXIT_CRITICAL_FROM_ISR()` блокируют возникновение других прерываний при выполнении критической операции.
3. Сколько битов имеет счетчик RTC?
   Ответ: счетчик RTC имеет размер в 32 бита.
4. Какой источник синхронизации продолжает работать, когда контроллер STM32 отключается?
   Ответ: тактовый генератор LSE (кварцевый генератор 32.768 кГц), который продолжает работать даже при отключении напряжения питания при условии сохранения напряжения питания резервной батареи VBAT.
5. Какой источник синхронизации является наиболее точным?
   Ответ: наиболее точным источником синхронизации является тактовый генератор HSE, поскольку он управляется кварцевым генератором с частотой 8 МГц, но только при подаче основного питания на контроллер.

Глава 11

1. Какое значение байта отправляется при чтении с адреса подчиненного устройства $21 (шестнадцатеричное число)?
   Ответ: шестнадцатеричный адрес $21 соответствует $42, если его сдвинуть влево на 1 бит. Для операции чтения требуется единичный бит в младшей позиции, в результате чего значение байта равно $43.
2. Когда ведущий запрашивает ответ от несуществующего ведомого устройства на шине, как принимается NACK?
   Ответ: чтобы подтвердить ответ, ведомое устройство должно перевести линию SDA на низкий уровень. Если подтверждение от ведомого устройства отсутствует, нагрузочный резистор поддерживает высокий уровень линии, в результате чего NACK принимается по умолчанию.
3. В чем преимущество линии $\overline{\text{INT}}$ у PCF8574?
   Ответ: линия $\overline{\text{INT}}$ позволяет ведомому устройству напрямую уведомлять управляющий контроллер, если входная линия меняет состояние. В про-

тивном случае контроллеру пришлось бы загружать шину I2C запросами на чтение, чтобы увидеть, когда произошло событие.

4. Что означает квазидвунаправленность в контексте PCF8574?
   Ответ: чтобы получить входной сигнал, порт GPIO должен позволять установку входным драйвером на низкий уровень. Такая ситуация фактически делает его входным или выходным портом. Однако если порт GPIO уже установлен на низкий уровень, его нельзя использовать для входа. По этой причине его считают квазидвунаправленным.
5. В чем разница между вытекающим и втекающим током?
   Ответ: когда ток вытекающий, он течет с положительной стороны через нагрузку, подключенную к земле (отрицательную). При втекающем токе нагрузка подключается к положительной шине, а ток включается и выключается подключением или разрывом соединения с общим проводом.

Глава 12

1. Какие макроопределения необходимо использовать для конфигурации выходных контактов AFIO?
   Ответ: выходы GPIO должны использовать макросы `GPIO_CNF_OUTPUT_ALTFN_PUSHPULL` или `GPIO_CNF_OUTPUT_ALTFN_OPENDRAIN`, иначе вывод останется неподключенным к периферийному устройству (он будет действовать как обычный GPIO).
2. Какая тактовая частота должна быть задана при изменении AFIO?
   Ответ: должен быть включен источник тактов RCC_AFIO с помощью `rcc_periph_clock_enable()`.
3. Какие макроопределения конфигурации следует использовать для входных контактов GPIO?
   Ответ: входы не требуют какой-либо специальной обработки, кроме включения тактовой частоты периферийных устройств AFIO и тактовой частоты GPIO, а также необходимости инициализации входа периферийного устройства.
4. Какова цель входа DC у платы OLED?
   Ответ: входная линия DC (к OLED) позволяет различать байты команд OLED (при установленном низком уровне) и графические данные OLED (при высоком уровне).

Глава 13

1. Какие настройки контроллера DMA необходимо изменить в программе перед началом следующей передачи?
   Ответ: начальный адрес и длина были изменены после отключения канала DMA.
2. Имеет ли каждый канал DMA свою собственную процедуру ISR?
   Ответ: да, каждый канал DMA имеет свою процедуру ISR.
3. Откуда приходит запрос DMA при передаче из памяти на периферию, как в демопрограмме?

Ответ: запрос DMA поступает от периферийного устройства, за исключением передачи из памяти в память. В демонстрационной программе флаг пустого буфера передачи SPI сигнализировал о необходимости передачи.

4. Какие три условия необходимо установить в программе при использовании SPI, прежде чем может начаться передача по DMA?
   Ответ: в демонстрационной программе, где использовался SPI, для запуска DMA необходимо было выполнить следующие действия:
   а) канал DMA должен быть включен;
   б) включение DMA TX для SPI должно быть включено;
   в) SPI должен быть включен.

Глава 14

1. Как отображается внутренняя температура STM32?
   Ответ: величиной напряжения.
2. Чем значение `GPIO_CNF_INPUT_ANALOG` отличается от значений `GPIO_CNF_INPUT_PULL_UPDOWN` или `GPIO_CNF_INPUT_FLOAT`?
   Ответ: значение конфигурации `GPIO_CNF_INPUT_ANALOG` позволяет переменному напряжению поступать на вход АЦП. В противном случае будет восприниматься только низкий или высокий уровень сигнала.
3. Если `PCLK` имеет частоту 36 МГц, какова будет тактовая частота АЦП при настройке с предварительным делителем, равным 4?
   Ответ: тактовая частота АЦП будет 36 МГц / 4, что составляет 9 МГц.
4. Назовите три параметра конфигурации, которые влияют на общую мощность, потребляемую АЦП.
   Ответ: три фактора, которые влияют на энергопотребление АЦП, следующие:
   а) `adc_power_on(adc)` (или `off`),
   б) `adc_enable_temperature_sensor()` (или `disable`),
   в) `adc_start_conversion_direct(adc)`.
5. Если предположить, что тактовая частота АЦП после предделителя равна 12 МГц, сколько времени займет сконфигурированная выборка `ADC_SMPR_SMP_41DOT5CYC`?
   Ответ: (41.5 + 12.5) / 12 МГц = 54 / $12 \cdot 10^6$ = 4.5 мкс.

Глава 15

1. В чем преимущество RC-генераторов?
   Ответ: RC-генераторы не требуют внешнего резонатора (резонаторы слишком велики, чтобы их можно было включить в микросхему[97]).
2. В чем недостаток RC-генераторов?

[97] Смотря какие микросхемы: существуют генераторы (например, под маркой Epson) и часы реального времени (например, DS3231), в которых «кварц» интегрирован в микросхему. В отношении контроллеров речь идет о том, что интегрирование резонатора удорожает производство и действительно раздувает габариты, причем это не оправданно функциональностью – существует много применений, в которых «кварцевая» точность не требуется.

Ответ: RC-генераторы склонны к дрейфу и джиттеру[98] и менее стабильны. Кроме того, они менее точны, что приводит к проблемам с определением скорости передачи данных или промежутков времени и т. д.

3. В чем преимущество кварцевых генераторов?
   Ответ: генераторы с кварцевым резонатором более стабильны и могут точно соответствовать внешнему оборудованию. Это делает их более подходящими для определения скорости передачи данных и т. д.
4. Для чего используется ФАПЧ?
   Ответ: ФАПЧ используется для умножения тактовой частоты до значений, превышающих входную тактовую частоту.
5. Что означает AHB?
   Ответ: AHB означает высокопроизводительный вариант расширенной архитектуры шины микроконтроллера (AMBA High-Performance Bus).
6. Почему GPIO PA8 должен быть настроен с `GPIO_CNF_OUTPUT_ALTFN_PUSHPULL`?
   Ответ: без `ALTFN` в имени макроопределения GPIO останется отключенным и будет обычным GPIO, не имеющим ничего общего с выходом тактовой частоты контроллера.

Глава 16

1. Что такое период сигнала RC-сервопривода?
   Ответ: период сигнала ШИМ – это время между началом импульса и началом следующего.
2. Почему на плате Blue Pill тактовая частота входа таймера составляет 72 МГц? Почему не 36 МГц?
   Ответ: входная частота таймера составляет 72 МГц (для Blue Pill STM32), поскольку, когда предварительный делитель AHB1 не равен единице, частота таймера равна удвоенной частоте шины AHB1.
3. Что следует поменять в таймере для изменения ширины импульса?
   Ответ: значение регистра сравнения.

Глава 17

1. Почему на входе таймера имеется цифровой фильтр?
   Ответ: цифровой фильтр исключает ложное срабатывание от случайных шумовых импульсов.
2. Когда происходит сброс таймера в режиме поступления PWM на вход?
   Ответ: как указано в демопрограмме главы 17, счетчик сбрасывается после возникновения первого события захвата.
3. Откуда поступает входной сигнал IC2 в режиме поступления PWM на вход?
   Ответ: в режиме входа PWM на вход IC2 поступает сигнал из входного канала 1.

Глава 19

1. Сколько буферов FIFO поддерживается периферийным устройством CAN STM32F103?

[98] Джиттер – непредсказуемые колебания фронтов сигналов от периода к периоду.

Ответ: в периферийном устройстве CAN имеется два FIFO (FIFO0 и FIFO1).

2. Сколько наборов фильтров поддерживается периферийным устройством CAN?
   Ответ: в периферийном устройстве CAN имеется два банка фильтров (банки 0 и 1).
3. Если имеется для установки пара фильтров, но нужен только один, какие есть два способа сделать это?
   Ответ: вы можете поставить один фильтр, если требуется пара, одним из двух способов:
   а) объявите два одинаковых фильтра (сработает только один);
   б) объявите один нормальный фильтр и один неработающий – с маской, соответствующей несуществующему значению.
4. Что такое флаг RTR и каково его назначение?
   Ответ: флаг RTR (Remote Transmission Request, запрос удаленной передачи) используется для запроса передачи, когда она отправляется в рецессивном состоянии. Ответ всегда отправляется с флагом RTR в доминантном состоянии.

Глава 20

1. Как обработка прерываний приема помогает предотвратить потерю данных?
   Ответ: обработка прерываний обеспечивает более быстрое чтение регистра данных периферийного устройства, сохраняя его в памяти до того, как регистр будет перезаписан.
2. Почему вместо буфера массива памяти используется очередь FreeRTOS?
   Ответ: очередь FreeRTOS предоставляет тщательно протестированный код для сохранения данных в памяти с использованием минимальной критической секции, чтобы сделать операцию атомарной.
3. Почему обработчик прерывания не должен блокировать выполнение основного кода?
   Ответ: прерывание не должно блокировать, чтобы не заблокировать другие прерывания. Быстрый возврат позволяет продолжить прерванный код.
4. Почему необходимо использовать функцию `usart_send()` в libopencm3 вместо `usart_send_blocking()`?
   Ответ: `usart_send_blocking()` «не знает», что она используется с ядром FreeRTOS, и будет запускать пустые циклы процессора до тех пор, пока USART не сможет принять данные. Это приводит к перегрузке контроллера до тех пор, пока не произойдет вытеснение. Предпочтителен вызов `TaskYIELD()`, когда устройство еще не готово.
5. Каковы необходимые условия перед вызовом `usart_send()`?
   Ответ: подпрограмма `usart_send()` предполагает, что устройство готово. Таким образом, необходимо проверить готовность перед вызовом.

Глава 21

1. Что определяет макроопределение make `BINARY`?
   Ответ: макроопределение BINARY определяет имя исполняемого файла приложения с добавленным суффиксом *.elf*.

2. Каково назначение заголовочного файла *FreeRTOSConfig.h*?
   Ответ: заголовочный файл *FreeRTOSConfig.h* настраивает некоторые аспекты системы FreeRTOS.
3. Как добавить опцию компилятора -O3 только для компиляции модуля *Speedy.c*?
   Ответ: в Makefile добавьте следующее правило: `Speedy.o: CFLAGS += -O3`
4. В чем основной недостаток использования *heap_1*?
   Ответ: основным недостатком использования *heap_1.c* в проекте FreeRTOS является то, что функция `free()` не поддерживается. Никакая динамически выделенная память не может быть освобождена и повторно использована.

Приложение Б • Выводы GPIO STM32F103C8T6

Это приложение предоставлено для удобства. Эта информация взята из PDF-документа STMicroelectronics DS5319, который можно загрузить по адресу https://www.st.com/resource/en/datasheet/stm32f103c8.pdf. Информация здесь взята из табл. 5 на стр. 28 только для STM32F103C8T6 в корпусе LQFP48.

Условные обозначения: I – вход; O – выход; S – питание; FT – допускается уровень 5 В.

Вывод	Название	Тип	I/O-уровень	После включения	По умолчанию	Переназначение
1	V_{BAT}	S	–	V_{BAT}	–	–
2	PC13-TAMPER-RTC(1)	I/O	–	PC13(2)	TAMPER-RTC	–
3	PC14-OSC32_IN(1)	I/O	–	PC14(2)	OSC32-IN	–
4	PC15-OSC32_OUT(1)	I/O	–	PC15(2)	OSC32-OUT	–
5	OSC_IN	I	–	OSC_IN	–	PD0(3)
6	OSC_OUT	O	–	OSC_OUT	–	PD1(3)
7	NRST	I	–	NRST	–	–
8	V_{SSA}	S	–	V_{SSA}	–	–
9	V_{DDA}	S	–	V_{DDA}	–	–
10	PA0-WKUP	I/O	–	PA0	WKUP USARTRT2_CTS(4) ADC12_IN 0 TIM2_CH1_E$^{TR(4)}$	–
11	PA1	I/O	–	PA1	USARTRT2_RTS(4) ADC 12_IN 1 TIM2_CH2(4)	–
12	PA2	I/O	–	PA2	USART2_TX(4) ADC12_IN2 TIM2_CH3(4)	–
13	PA3	I/O	–	PA3	USART2_RX(4) ADC12_IN3 TIM2_CH4(4)	–
14	PA4	I/O	–	PA4	SPI1_NSS(4) USART2_CK(4) ADC12_IN4	–
15	PA5	I/O	–	PA5	SPI1_SCK(4) ADC12_IN5	–
16	PA6	I/O	–	PA6	SPI1_MISO(4) ADC12_IN6 TIM3_CH1(4)	TIM1_BKIN
17	PA7	I/O	–	PA7	SPI1_MOSI(4) ADC12_IN7 TIM3_CH2(4)	TIM1_CH1N
18	PB0	I/O	–	PB0	ADC12_IN8 TIM3_CH3(4)	TIM1_CH2N
19	PB1	I/O	–	PB1	ADC12_IN9 TIM3_CH4(4)	TIM1_CH3N

Вывод	Название	Тип	I/O-уровень	После включения	По умолчанию	Переназначение
20	PB2	I/O	FT	PB2/BOOT1	–	–
21	PB10	I/O	FT	PB10	I2C2_SCL USART3_TX(4)	TIM2_CH3
22	PB11	I/O	FT	PB11	I2C2_SDA USART3_RX(4)	TIM2_CH4
23	V_{SS_1}	S	–	V_{SS_1}	–	–
24	V_{DD_1}	S	–	V_{DD_1}	–	–
25	PB12	I/O	FT	PB12	SPI2_NSS I2C2_SMBAl USART3_CK(4) TIM1_BKIN(4)	–
26	PB13	I/O	FT	PB13	SPI2_SCK USART3_CTS(4) TIM1_CH1N(4)	–
27	PB14	I/O	FT	PB14	SPI2_MISO USART3_RTS(4) TIM1_CH2N(4)	–
28	PB15	I/O	FT	PB15	SPI2_MOSI TIM1_CH3N(4)	–
29	PA8	I/O	FT	PA8	USART1_CK TIM1_CH1(4) MCO	–
30	PA9	I/O	FT	PA9	USART1_TX(4) TIM1_CH2(4)	–
31	PA10	I/O	FT	PA10	USART1_RX(4) TIM1_CH3(4)	–
32	PA11	I/O	FT	PA11	USART1_CTS CANRX(4) USBDM TIM1_CH4(4)	–
33	PA12	I/O	FT	PA12	USART1_RTS CANTX(4) USBDP TIM1_ETR(4)	–
34	PA13	I/O	FT	JTMS/SWDIO	–	PA13
35	VSS_2	S	–	VSS_2	–	–
36	VDD_2	S	–	VDD_2	–	–
37	PA14	I/O	FT	JTCK/SWCLK	–	PA14
38	PA15	I/O	FT	JTDI	–	TIM2_CH1_ETR PA15 SPI1_NSS
39	PB3	I/O	FT	JTDO	–	TIM2_CH2 PB3 TRACESWO SPI1_SCK
40	PB4	I/O	FT	JNTRST	–	TIM3_CH1 PB4 SPI1_MISO
41	PB5	I/O	FT	PB5	I2C1_SMBAl	TIM3_CH2 SPI1_MOSI
42	PB6	I/O	FT	PB6	I2C1_SCL(4) TIM4_CH1(4)	USART1_TX

Вывод	Название	Тип	I/O-уровень	После включения	По умолчанию	Переназначение
43	PB7	I/O	FT	PB7	I2C1_SDA[4] TIM4_CH2[4]	USART1_RX
44	BOOT0	I	–	BOOT0	–	–
45	PB8	I/O	FT	PB8	TIM4_CH3[4]	I2C1_SCL CANRX
46	PB9	I/O	FT	PB9	TIM4_CH4[4]	I2C1_SDA CANTX
47	V_{SS_3}	S	–	V_{SS_3}	–	–
48	V_{DD_3}	S	–	V_{DD_3}	–	–

(1) Питание PC13, PC14 и PC15 осуществляется через специальный выключатель питания. Поскольку он пропускает только ограниченное количество тока (3 мА), использование GPIO от PC13 до PC15 в режиме вывода ограничено: скорость не должна превышать 2 МГц при максимальной нагрузка 30 пФ, и эти входы/выходы нельзя использовать в качестве источника тока (например, для управления светодиодом).

(2) Основная функция после первого включения. В дальнейшем это зависит от содержимого резервных регистров даже после сброса (поскольку эти регистры не сбрасываются при основном сбросе). Подробную информацию об управлении этими вводами-выводами см. разделы описания домена резервных регистров и регистра BKP в справочном руководстве STM32F10xxx, доступном на сайте STMicroelectronics www.st.com.

(3) Выводы номер 5 и 6 в корпусе LQFP48 после сброса настраиваются как `OSC_IN/OSC_OUT`, однако функциональность PD0 и PD1 можно переназначить на эти контакты с помощью программного обеспечения. Более подробную информацию см. в разделе «Alternate function I/O and debug configuration» (Альтернативные функции ввода-вывода и конфигурация отладки) справочного руководства STM32F10xxx. Использование PD0 и PD1 в режиме вывода ограничено, поскольку в этом режиме их можно использовать только на частоте 50 МГц.

(4) Эта альтернативная функция может быть переназначена программным обеспечением на некоторые другие контакты порта (если они доступны в используемом корпусе). Подробности см. в разделе «Alternate function I/O and debug configuration» (Альтернативные функции ввода-вывода и конфигурация отладки) справочного руководства STM32F10xxx, доступного на сайте STMicroelectronics www.st.com (см. ссылку в разделе «Дополнительные источники» к главе 7).

Предметный указатель

A

AFIO, 214
AHB, частоты для STM32F103C8T6, 255
AMBA, 255

B

Blue Pill, 46

D

DMA, 105
DuPont, 25

F

FIFO, 301
FreeRTOS, 21, 77, 177
  возможности, 77
  прерывания RTC, 175
  программа blinky2, 80
  FreeRTOSConfig.h, 84

G

GPIO, 20, 28, 46, 59
  альтернативные функции, 97, 214
  выводы STM32F103C8T6, 356
  конфигурация, 64
  обозначения, 64
  расширитель PCF8574, 192
  режим, 65
  режимы входа, 66, 71
  режимы выхода, 67, 71
  характеристики, 72
  API, 59, 63

I

I2C, запуск, 202
IDE, 22

L

libopencm3. См. *Библиотека libopencm3*, 21
  API USART, 103

M

MCO, 259

O

OLED, 210
OLED-дисплей, 210
  контроллер SSD1306, 211
  схема подключения, 214

U

UART, 89
  выводы управления потоком, 104
  проект, 93, 96
  проект uart2, 99
  сигнал, 89
  USART, 89
USB
  демопрограмма, 120
  отправка, 119
  прием, 119

A

Адаптер USB-TTL, 28, 90
  подключение, 92
Альтернативная функция ввода-вывода, 214
Аналого-цифровой преобразователь (АЦП), 239
  аналоговые входы PA0/PA1, 240
  время выборки, 242
  конфигурация периферии, 240

опорное напряжение, 246
чтение, 243

Б

Библиотека libopencm3, 21, 33, 342
GPIO API, 63
RTC, 172
Буфер FIFO, 301

В

Вывод тактовой частоты MCO, 259

Г

Графические устройства, 216
Графическое ПО, 216

Д

Датчик температуры, 244

И

Интегрированная среда разработки (IDE), 22
Интерфейс прикладного программирования (API), 59, 63, 103, 105
Интерфейс SPI, 124

М

Макетная плата, 25
Мьютекс, 79, 179

О

Оверлей, 149
демопрограмма, 163
диспетчер, 159, 162
определение, 156
Отладчик GNU (GDB), 331

П

Перемычки DuPont, 25
Периферийное устройство, 20, 21, 41
настройка, 67
на шине APB1, 258
на шине APB2, 259
USART, 89
Питание, 49
внешний источник , 48, 51
встроенный стабилизатор, 47
источник питания, 29
USB-питание, 48
Питание , 46
Плата Blue Pill, 46
Порт USB, 109
выводы GPIO, 114
инициализация, 115
подключение к компьютеру, 112
хост, 111
Прерывания
обработчик (ISR), 172, 175, 177, 309, 313
UART, 309
Программатор ST-Link V2, 24, 51
подключение, 52
Программные задержки, 74
Прямой доступ к памяти (DMA), 225
выполнение, 226
запрос, 226
каналы, 226

Р

Развязывающий конденсатор, 27
Растровое изображение, 217

С

Сброс (Reset), 50
Семафор, 79
Сервопривод, 264

Т

Тактовый генератор, 249
ФАПЧ, 252
HSE, 173, 252
HSI, 252
LSE, 173, 252
LSI, 173, 252
RTCCLK, 252
SYSCLK, 252
USBCLK, 254

У

Утилита st-flash, 53
Утилита STM32 ST-LINK, 43, 55

Ф

ФАПЧ, 252

Ц

Цифро-аналоговый преобразователь (DAC), 345

Ч

Часы реального времени (RTC)
встроенные, 172

Ш

Шина AHB, 254
Шина APB1, 258
Шина CAN, 284
  демопрограмма, 290
  дифференциальная линия, 286
  однопроводная линия, 286
  формат сообщения, 289
Шина I2C, 188
  адрес, 190
  биты данных, 189
  ведущий и ведомый, 188
  запись и чтение, 191, 203
  старт и стоп, 189
Широтно-импульсная модуляция (PWM), 264
  сигнал, 264

Э

Электромагнитные помехи (EMI), 65, 70
Электронный блок (EU), 291

Уоррен Гей

Основы STM32

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com
Перевод *Ревич Ю. В.*
Корректор *Абросимова Л. А.*
Верстка *Чаннова А. А.*
Дизайн обложки *Мовчан А. Г.*

Гарнитура PT Serif. Печать цифровая.
Усл. печ. л. 29,41. Тираж 100 экз.

Веб-сайт издательства: **www.dmkpress.com**

В данной книге детально рассмотрены аппаратные возможности микроконтроллера STM32, включая порты GPIO и другие периферийные устройства, такие как USB и контроллеры шины CAN. Приведена настройка программного обеспечения для Windows.

Используя FreeRTOS и библиотеку libopencm3 вместо программной среды Arduino, вы научитесь разрабатывать многозадачные приложения, выходящие за рамки возможностей Arduino.

Вы узнаете, как:

- загрузить и настроить среду разработки libopencm3 + FreeRTOS с помощью компилятора GCC;
- инициализировать и использовать драйверы libopencm3 и обрабатывать прерывания;
- использовать DMA для управления OLED-дисплеем на основе SPI, изображающим аналоговый вольтметр;
- читать ШИМ-сигнал из пульта дистанционного управления с помощью аппаратных таймеров;
- работать с шиной I2C для добавления выводов GPIO с помощью микросхемы PCF8574;
- создать выход с широтно-импульсной модуляцией для управления сервоприводом с помощью аппаратных таймеров

и многое другое.

Издание адресовано инженерам встраиваемых систем, разработчикам, желающим изучить архитектуру ARM, а также будет полезно студентам и радиолюбителям.

Apress

ДМК Пресс
Издательство
www.дмк.рф

ISBN 978-5-93700-294-5
9 785937 002945 >